The Cryptographic Atlas

A practical map of modern cryptographic primitives, protocols, guarantees, and design patterns.

Content was generated and revised with AI agents. Treat this PDF as educational draft material and verify technical claims against cited sources before relying on them.

Generated 2026-10-27. Source: https://christophelebrun.github.io/cryptographic-atlas/

Table of Contents

INTRODUCTION

Welcome to The Cryptographic Atlas 7

TAXONOMY

Taxonomy Overview 10

Assumptions 13

Primitives vs Protocols 15

Composability 16

SECURITY GOALS

Security Goals 18

Confidentiality 21

Integrity 24

Authenticity 27

Anonymity 30

Unlinkability 33

Forward Secrecy 36

Verifiability 39

Auditability 42

Accountability 45

Deniability 48

Non-Repudiation 51

Availability 54

Censorship Resistance 57

Additional Security Goals 60

DISCRETE LOGARITHM

Discrete Logarithm 63

Elliptic Curves 66

Factoring and RSA 69

Code-Based Assumptions 72

Pairings 75

Lattices 78

Random Oracle Model 81

TRUSTED SETUP

Trusted Setup 84

Common Reference Strings 87

Additional Assumptions and Substrates 90

BASIC PRIMITIVES

Basic Primitives Overview 95

Primitive Engineering Concepts 96

Hash Functions 100

SYMMETRIC ENCRYPTION

Symmetric Encryption 103

AUTHENTICATED ENCRYPTION

Authenticated Encryption 106

Key-Committing Encryption 109

Message Authentication Codes 112

KEY DERIVATION FUNCTIONS

Key Derivation Functions 115

Password Hashing 118

Randomness and Nonces 121

COMMITMENTS

Commitments 124

Pedersen Commitments 128

Digital Signatures 131

Public-Key Encryption 134

Key Encapsulation and Exchange 137

Secret Sharing 140

STRUCTURED PRIMITIVES

Structured Primitives Overview 143

ADVANCED STRUCTURED PRIMITIVES

Advanced Structured Primitives 144

Polynomial Commitments 149

Vector Commitments 152

Blind Signatures 155

Verifiable Random Functions 158

Verifiable Encryption 161

E-Cash Primitives 164

Homomorphic Commitments 167

Homomorphic Encryption 170

Threshold Cryptography 173

Functional Encryption 176

Timelock and VDFs 179

Accumulators and Merkle Trees 182

PROOF SYSTEMS

Proof Systems Overview 185

PROOF-SYSTEM COMPONENTS

Proof-System Components 186

FRI 191

Folding Schemes 194

ARITHMETIZATION

Arithmetization 197

Lookup Arguments 200

Sumcheck 203

Recursive Proofs 206

Zero-Knowledge Proofs 209

SNARKs, STARKs, and Bulletproofs 213

Range Proofs 217

Membership Proofs 220

PROTOCOLS

Protocols Overview 223

ADDITIONAL PROTOCOL FAMILIES

Additional Protocol Families 224

Secure Channels 229

Password-Authenticated Key Exchange 233

Oblivious Transfer 236

Private Information Retrieval 239

Anonymous Tokens 242

Blind-Signature Credentials 245

Anonymous Credentials 248

Nullifiers 251

Oblivious Pseudorandom Functions 255

Private Set Intersection 258

Secure Aggregation 261

Multi-Party Computation 264

Mixnets 267

Electronic Voting 270

SYSTEMS AND APPLICATIONS

Systems and Applications Overview 273

Private Payments 274
ZK Rollups 277
Secure Messaging 280
Identity Wallets 282
Encrypted Mempools 284
Private Machine-Learning Analytics 286
Private DAO Voting 288
Coercion-Resistant Voting 291
Anonymous Airdrop 293

DESIGN PATTERNS

Design Patterns Overview 295

OPERATIONAL DESIGN PATTERNS

Operational Design Patterns 296
Domain Separation 302
Transcript Binding 305
Privacy-Preserving Revocation 308
Encrypt-Then-Prove 311
Threshold Issuance 314
Key Rotation and Migration 317
Anonymous Membership 320
Anti-Double-Use Nullifiers 323
Private Aggregation 326
Delayed Reveal 330
Reveal Only a Function 333
Make Receipts Useless 336

GLOSSARY

Glossary 339

APPENDICES

Source Policy and Topic References 343
Notation 344
Maturity Scale 345
Editorial State, Coverage, and Backlog 346

COMPARISON MATRICES

Comparison Matrices 356
Generated Comparison Matrices 358
Generated Relationship Graph 363

Concrete Algorithms and Schemes 373
Diagram Authoring 379
Post-Quantum Posture 381
Confidence Models 384
Metadata Leakage 386
Threat-Model Checklist 387

Introduction

Welcome to The Cryptographic Atlas

The Cryptographic Atlas is a living map of cryptographic concepts: what they do, what they do not do, what assumptions they rely on, and how they are commonly composed into larger systems.

This book is written for technical readers who need practical conceptual clarity before they evaluate designs, read protocol documentation, or talk with cryptographers.

Mission

The mission is to explain cryptographic building blocks as parts of a system design vocabulary. A reader should leave with sharper questions about threat models, assumptions, failure modes, maturity, and composition risks.

What this book is

This book is:

- an educational atlas of cryptographic ideas;
- a guide to categories, guarantees, and trade-offs;
- a place to compare primitives, protocols, proof systems, and design patterns;
- a reminder that real systems depend on more than mathematical primitives.

What this book is not

This book is not production cryptography guidance. It is not a source of copy-paste implementations, audited protocols, or deployment recipes.

Do not design or deploy custom cryptographic protocols without expert review.

The atlas is also not complete. See Editorial State, Coverage, and Backlog for the current coverage assessment, promotion candidates, source-depth priorities, and changelog.

AI generation disclosure

The content in this book was generated and iteratively revised with AI agents. Treat it as educational draft material: verify technical claims against the cited sources, and do not rely on it as a substitute for expert cryptographic review.

How to read it

Start with the taxonomy, then move from goals to building blocks:

1. Identify the security goal.

2. Identify the assumptions and trust model.
3. Pick the primitive, proof system, protocol, or pattern being discussed.
4. Check what it does not provide.
5. Look for failure modes and composition risks.
6. Check its post-quantum posture and confidence model.

Core taxonomy

The atlas uses eight levels:

Level	What it describes	Examples
Security goals	Desired properties	confidentiality, integrity, anonymity
Assumptions and substrates	Mathematical or operational foundations	discrete logarithm, random oracle model, trusted setup
Basic primitives	Small cryptographic building blocks	hashes, commitments, signatures
Structured primitives	Building blocks with richer structure	homomorphic encryption, threshold signatures
Proof systems	Ways to prove statements under constraints	zero-knowledge proofs, range proofs
Protocols	Multi-step interactions between parties	MPC, secure aggregation, anonymous credentials
Systems	End-to-end applications	e-voting, private payments, anonymous airdrops
Design patterns	Reusable composition ideas	anonymous membership, delayed reveal, private aggregation

Concrete algorithms, named schemes, parameter families, and protocol suites are treated as instances of these concepts, not as a ninth top-level category. For example, Ed25519 is a concrete instance of digital signatures, Groth16 is a concrete instance of a proof system, and TLS 1.3 is a concrete protocol suite. See Concrete Algorithms and Schemes.

Motivating example: private voting

A private voting system may require anonymous eligibility, anti-double-vote protection, ballot secrecy, valid ballot proofs, private tallying, delayed reveal, and coercion mitigation. No single primitive provides all of these. The system must compose several tools, each with different assumptions and failure modes.

For example:

- anonymous credentials can help prove eligibility without directly revealing identity;
- nullifiers can help detect double use without naming the voter;
- commitments can bind a voter to a ballot before it is opened or tallied;
- zero-knowledge proofs can show that a hidden ballot is valid;
- homomorphic encryption or multi-party computation can support private tallying;
- coercion resistance requires protocol and user-experience properties beyond ballot secrecy.

The central lesson is composability: each tool contributes a narrow guarantee, and the system must make the gaps explicit.

Cross-cutting questions

For every major concept, ask:

- Is the construction quantum-vulnerable, plausibly post-quantum, or dependent on instantiation?
- Does confidence come from a mathematical assumption, public verification, one honest party, a `t-of-n` threshold, an honest majority, non-collusion, or a trusted issuer?
- Which part of the system has the weakest posture or most centralized confidence model?

Safety note

Cryptography is easy to misuse because guarantees are precise and conditional. This atlas tries to make those conditions visible.

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.

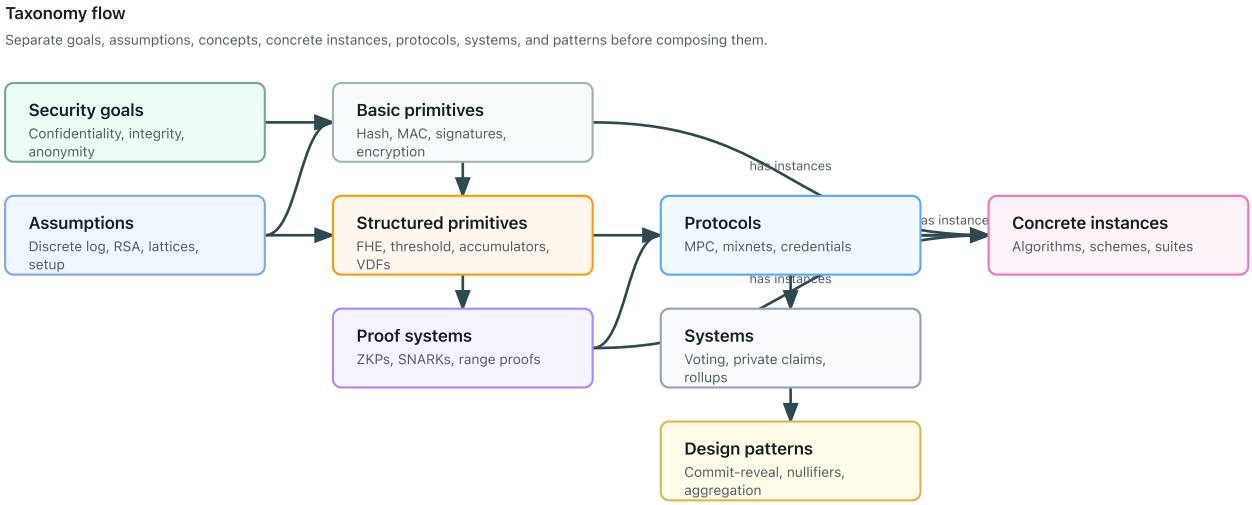
Taxonomy

Taxonomy Overview

The atlas organizes cryptographic ideas from desired outcomes to system patterns:

```
Security goals -> assumptions -> primitives -> structured primitives -> proof systems ->
protocols -> systems -> design patterns
```

Named algorithms, concrete schemes, parameter sets, curves, and protocol suites sit as an instance layer under the concept they instantiate.



This ordering prevents a common mistake: starting with a fashionable tool before stating the problem, the adversary, and the assumptions.

Taxonomy table

Level	Question it answers	Examples
Security goal	What property do we want?	confidentiality, integrity, anonymity, unlinkability
Assumption or substrate	What must be hard, trusted, or available?	discrete logarithm, lattices, pairings, random oracle model
Basic primitive	What small building block provides a narrow guarantee?	hash function, commitment, digital signature
Structured primitive	What extra algebraic or access structure is useful?	homomorphic encryption, threshold cryptography, VDF
Proof system	How can a party prove a statement without revealing too much?	zero-knowledge proof, range proof, membership proof
Protocol	How do parties interact to achieve a goal?	MPC, secure aggregation, mixnet
System or application	What user-facing application combines many tools?	e-voting, private DAO voting, anonymous airdrop

Level	Question it answers	Examples
Design pattern	What reusable composition pattern appears across systems?	anonymous membership, delayed reveal, anti-double-use nullifiers

Concrete instances

A concrete instance is a named algorithm, scheme, parameter family, curve, proof-system construction, or protocol suite that realizes a broader concept.

Instance	Instantiates	Why this placement matters
SHA-256	Hash functions	The page for hash functions explains the general goal; the instance adds concrete deployment and parameter details.
Ed25519	Digital signatures	The scheme inherits the signature concept but has specific curve, encoding, and implementation rules.
ML-KEM	Key encapsulation mechanisms	The instance changes post-quantum posture and parameter choices.
Groth16	Proof systems / SNARKs	The instance has specific trusted-setup and pairing assumptions.
TLS 1.3	Secure-channel protocols	The suite combines primitives, key exchange, authentication, and transcript binding.

Use the parent concept page to explain the security goal, non-goals, assumptions, and failure modes. Use an instance entry or comparison table when the concrete name changes assumptions, maturity, deployment risk, parameter sensitivity, or common misuse patterns. See Concrete Algorithms and Schemes.

Cross-cutting classifications

Two classifications cut across the taxonomy:

- Post-quantum posture: whether a construction is vulnerable, plausibly post-quantum, dependent on instantiation, unknown, or not applicable.
- Confidence models: where confidence comes from, such as public verification, one honest party, `t-of-n` threshold assumptions, honest majority, non-collusion, or a trusted issuer.

Why the levels matter

Each level has different failure modes. A primitive can be mathematically sound and still be used in a protocol that leaks metadata. A protocol can satisfy a narrow model and still fail inside a product because enrollment, timing, recovery, or coercion was ignored.

Practical workflow

When evaluating a design, ask:

1. What security goals are claimed?
2. What assumptions make those goals plausible?

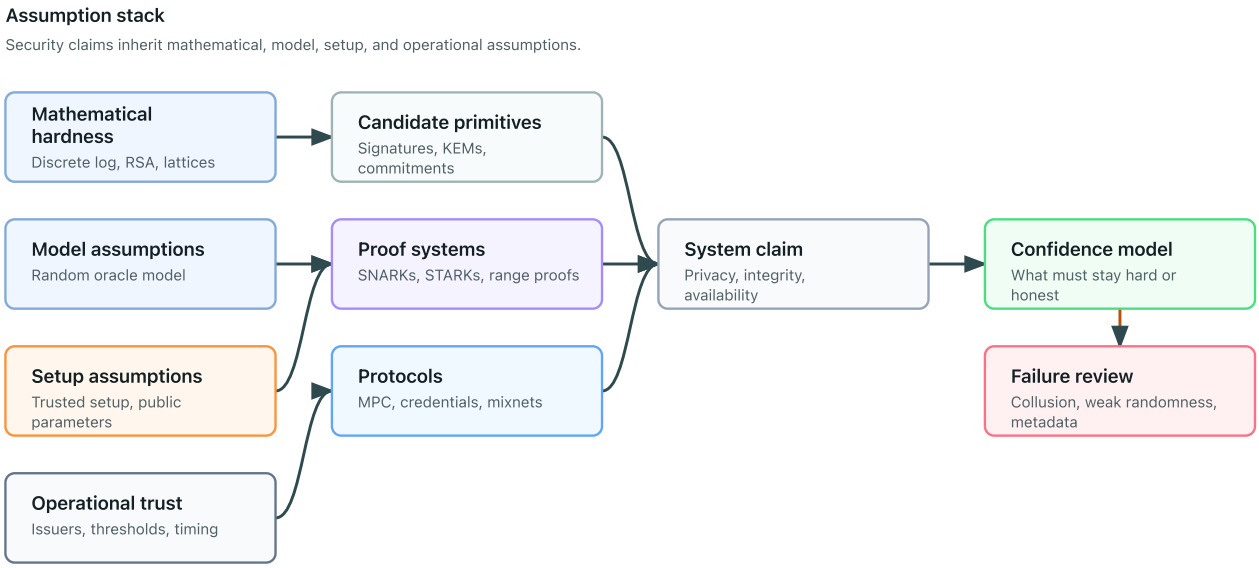
3. Which primitives and protocols provide the claimed properties?
4. Which properties are missing?
5. What happens when the components are composed?

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.

Assumptions

Cryptographic guarantees are conditional. An assumption is something that must hold for a claim to be meaningful.



Types of assumptions

Type	Examples
Mathematical	discrete logarithm hardness, lattice assumptions
Model	random oracle model, algebraic group model
Setup	trusted setup, common reference string, public parameters
Trust	honest majority, non-collusion, threshold of honest parties
Network	reliable broadcast, authenticated channels, timing bounds
Implementation	secure randomness, constant-time code, safe key storage

Post-quantum posture

Post-quantum posture is a classification of assumptions under a quantum adversary. For example, discrete-logarithm assumptions are quantum-vulnerable, while some hash-based and lattice-based constructions are commonly treated as plausible post-quantum candidates when parameters and implementations are appropriate.

See Post-Quantum Posture.

Confidence model

A confidence model states who or what must remain honest, independent, hard, available, or verifiable. Examples include one-honest-party mixnets, `t-of-n` threshold trustees, honest-majority MPC, public-

verifiable proof systems, trusted issuers, and transparent setup.

See Confidence Models.

Practical checklist

- Who can break the system if they collude?
- What setup must be generated correctly?
- What happens if randomness is weak?
- What metadata remains public?
- Which assumptions are mature and which are research-stage?

Assumption pages

- Discrete logarithm
- Elliptic curves
- Factoring and RSA
- Code-based assumptions
- Pairings
- Lattices
- Random oracle model
- Trusted setup
- Common reference strings
- Additional assumptions and substrates

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography".
- Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring".

Primitives vs Protocols

A primitive is a small building block with a narrow interface. A protocol is a multi-step construction, often involving several parties and several primitives.

A concrete algorithm, scheme, parameter family, or protocol suite is an instance of a concept. It should point back to the parent concept rather than replace it.

Comparison

Aspect	Primitive	Protocol
Scope	Narrow operation	End-to-end interaction
Examples	hash, signature, commitment	MPC, mixnet, anonymous credential issuance
Main question	What property does this operation provide?	What happens across participants, messages, and failure cases?
Common risk	Misstating the guarantee	Ignoring metadata, aborts, or trust assumptions

Where named schemes fit

Named constructions belong under the concept they instantiate:

Named item	Parent concept	Notes
SHA-256	Hash functions	A concrete hash algorithm, not the whole concept of hashing.
Ed25519	Digital signatures	A concrete signature scheme with specific curve and encoding choices.
Groth16	SNARKs / proof systems	A concrete proof system with trusted-setup and pairing assumptions.
TLS 1.3	Secure-channel protocol suite	A concrete protocol suite composed from several primitives and transcript rules.

This keeps the taxonomy concept-first while still letting the book discuss concrete deployment names where they matter.

Example

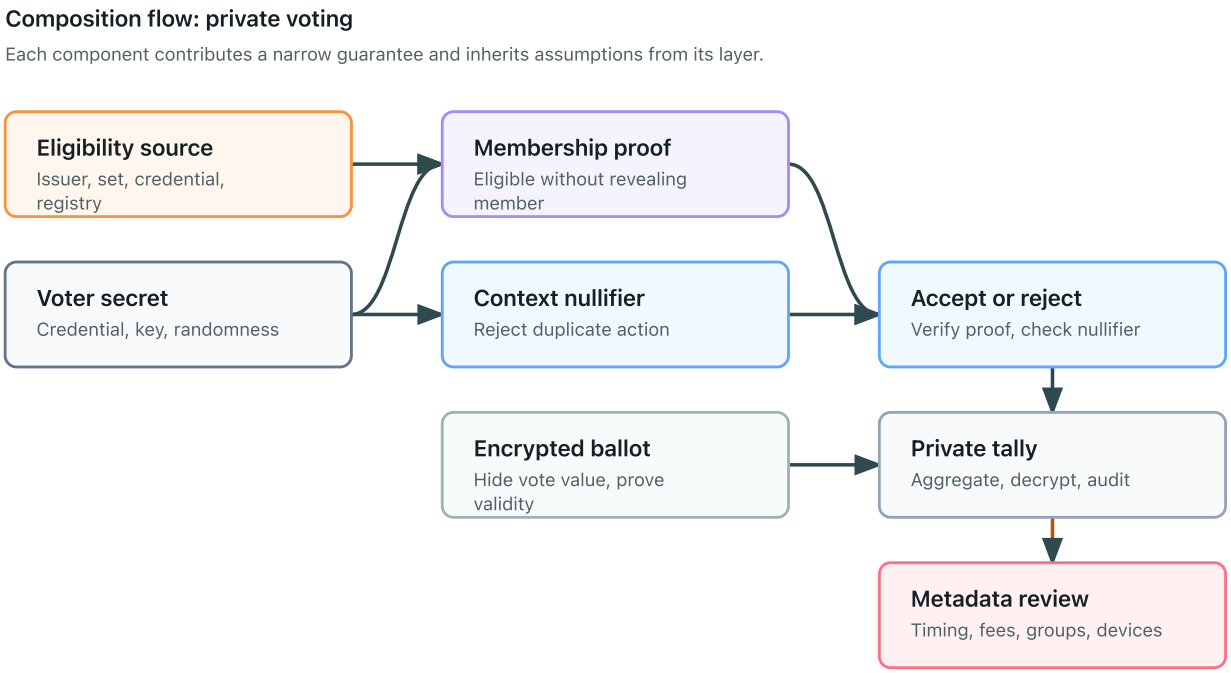
A commitment can bind a value. A voting protocol may use commitments, signatures, zero-knowledge proofs, nullifiers, and tallying rules to achieve a larger goal.

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.

Composability

Cryptographic tools do not compose automatically. Each primitive or protocol comes with a specific interface, adversary model, setup assumption, and leakage profile.



One-sentence intuition

A component can keep its promise and still leave the larger system insecure if the system needs a different promise.

Common composition mistakes

Mistake	Why it fails
Encryption does not prove validity	A ciphertext can hide an invalid value unless the system also proves the plaintext is well formed.
Commitments do not authenticate users	A commitment binds someone to a value, but it does not say who that someone is.
Zero-knowledge proofs do not hide metadata	A proof can hide a witness while network timing, account history, or reused identifiers remain linkable.
Threshold cryptography is not trustlessness	Threshold schemes replace one trusted party with assumptions about a group, quorum, setup, and key management.
Anonymous credentials do not automatically prevent coercion	A user may still be pressured to reveal secrets, show a transcript, or vote under observation.

What to check

- Are all security goals stated separately?
- Does each component provide exactly the property being claimed?
- Are identities, timestamps, network addresses, and repeated identifiers handled?
- Does setup require a trusted party, ceremony, or honest majority?
- Are invalid inputs rejected without exposing private values?
- Can a malicious participant force aborts, denial of service, or selective failures?

Failure pattern

A system often fails at the boundary between components. For example, a private voting design may encrypt ballots but forget to prove that each encrypted ballot encodes a valid choice. The tally remains private, but a malicious voter may submit malformed data.

Safer framing

Instead of saying "this system is secure," say:

> Under these assumptions, against this adversary, this component contributes this property, while these leaks and non-goals remain.

Further reading

- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.
- Katz and Lindell, Introduction to Modern Cryptography.

Security Goals

Security Goals

A security goal describes an outcome you want: keep a message secret, detect a change, or let a user submit a valid transaction. A primitive is a tool used to pursue that outcome. Start with the outcome, then choose the tools and state the conditions they need. This chapter separates these goals using the terminology of RFC 4949 and the definitions in Boneh and Shoup.

Start with one ordinary action

Imagine Alice sends Bob a payment instruction. Asking whether it is “secure” leaves several different questions unanswered:

Reader's question	Security goal	A limit to remember
Can someone else read the instruction?	Confidentiality	Bob can still disclose what he receives.
Can someone change the amount without detection?	Integrity	An unchanged instruction might still be old or fraudulent.
Did it come from the expected key or party?	Authenticity	A valid signature does not establish human intent or permission.
Can an observer identify Alice?	Anonymity	Hiding the amount does not hide the sender.
Can an observer connect this payment to another?	Unlinkability	A pseudonym can hide a name while linking every action.
Would a later key theft expose today's conversation?	Forward secrecy	It does not protect plaintext already copied.

The first three distinctions follow RFC 4949. Anonymity and unlinkability depend on the observer's information, as formalized by Pfitzmann and Hansen, §§3-5. Forward secrecy depends on the protocol and key lifecycle; see TLS 1.3, §2 and Appendix E.

Checking results and keeping a service usable

Other questions concern the whole process rather than a single message:

Reader's question	Security goal	A limit to remember
Can I check that the result follows the stated rules?	Verifiability	Checking a claim does not establish the truth of its external inputs.
Can someone review what happened later?	Auditability	A protected log can still omit events.
Can evidence identify a participant that broke a rule?	Accountability	Attribution does not automatically impose consequences.
Can a recipient turn the conversation into proof for outsiders?	Deniability aims to prevent this.	It cannot stop screenshots or disclosure.

Reader's question	Security goal	A limit to remember
Can a recipient retain evidence for a later dispute?	Non-repudiation aims to support this.	Evidence still needs a key-control and dispute model.
Can authorized users complete the operation when needed?	Availability	A service may work for most users while excluding Alice.
Can an operator selectively block Alice's valid submission?	Censorship resistance	Inclusion needs reachable paths and explicit network assumptions.

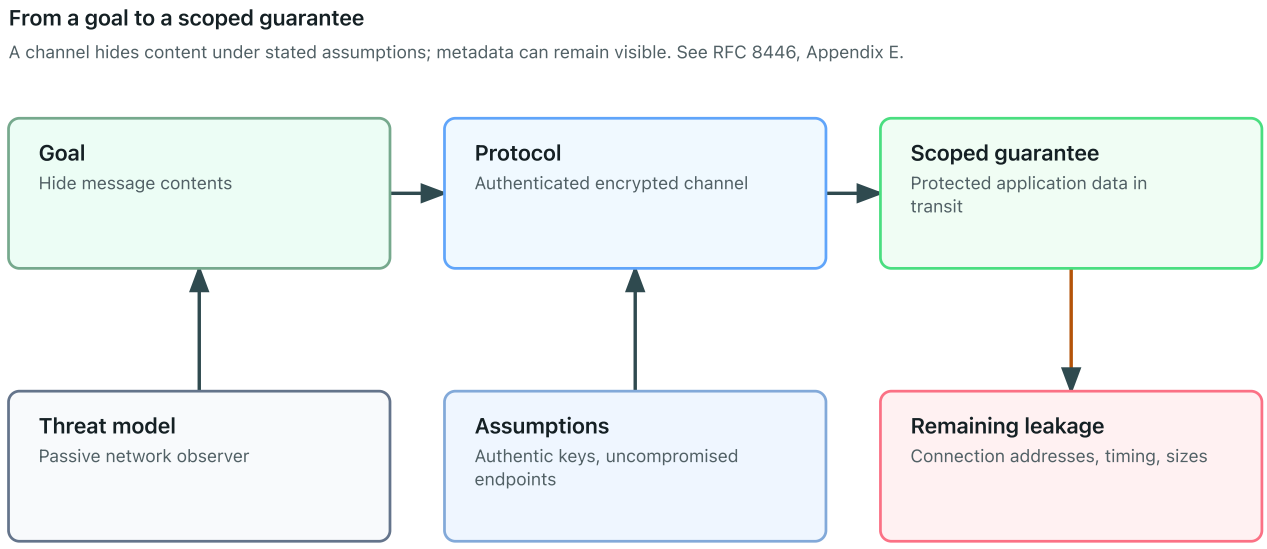
These distinctions are developed with sources on each linked page. For voting, also read receipt-freeness and coercion resistance: ballot secrecy alone does not address a voter who is pressured to cooperate with an attacker.

Turn a goal into a checkable requirement

Use this sentence as a design worksheet:

> Protect **which information or action**, from **which adversary**, for **how long**, provided **which assumptions hold**, while allowing **which leakage or failures**?

For example: “Hide payment amounts from a passive network observer during delivery, assuming uncompromised endpoints and authenticated key establishment; connection addresses, timing, and approximate sizes remain visible.” This is a deliberately limited confidentiality requirement, consistent with TLS 1.3's security analysis. It makes no claim about anonymity or the recipient's behavior.



The diagram is a reading guide. **All goals in this chapter are taxonomy level 1.** Encryption and signatures are primitives; secure channels are protocols; a payment application is a system. A goal can apply to any of those scopes without changing its taxonomy level.

Goals can support or pull against each other

- **Confidentiality and integrity:** authenticated encryption combines content protection with tamper detection. It still needs key management and replay handling. RFC 5116, §§1-3.
- **Anonymity and authenticity:** a credential can establish eligibility without exposing the holder's identity, if its presentation protocol provides that property. Camenisch and Lysyanskaya.
- **Deniability and non-repudiation:** decide who should be able to verify evidence. Publicly verifiable signatures on a message work against deniability for that message. Borisov, Goldberg, and Brewer.
- **Auditability and privacy:** evidence helps review, but logs can expose identities, relationships, and secrets. Collect what the audit requires and define access and retention. NIST SP 800-92, §§2-3.

> ⚠ **A NAMED BUILDING BLOCK IS NOT AN END-TO-END GUARANTEE**
>
> A zero-knowledge proof (ZKP) addresses disclosure from a specified proof interaction; it does not erase network metadata. A signature authenticates a statement under a key; it does not make the statement true. Evaluate the complete protocol, its inputs, and the surrounding application. Goldwasser, Micali, and Rackoff; Boneh and Shoup.

Post-quantum posture

Not applicable to a goal itself. A concrete realization can be vulnerable, plausible, or depend on its components. Changing a key-establishment algorithm does not also change signatures, stored evidence, or a proof system. NIST has finalized standards for module-lattice-based key encapsulation (ML-KEM), module-lattice-based signatures (ML-DSA), and stateless hash-based signatures (SLH-DSA); these are building blocks, not complete system guarantees. NIST's post-quantum project.

How to read the chapter

Begin with confidentiality, integrity, and authenticity. Read anonymity and unlinkability together, then forward secrecy for protection across time. The remaining pages explain evidence, disputes, and continued service. Each concept page states assumptions, missing guarantees, failure modes, and the maturity of example mechanisms.

Sources and changing specification statuses were checked on **27 October 2026**. Foundational papers remain useful for definitions; their age is not a claim about current deployment. This editorial source check is not an expert cryptographic audit. The next source review is due **27 April 2027**.

Further reading

- RFC 4949: Internet Security Glossary, Version 2 - definitions; an informational glossary, not an implementation standard.
- Boneh and Shoup, A Graduate Course in Applied Cryptography - deeper treatment of encryption, authentication, and security models.
- Pfitzmann and Hansen, privacy terminology, v0.34 - anonymity, pseudonymity, and unlinkability.
- Threat Model Checklist and Confidence Models - worksheets for evaluating a system.

Confidentiality

One-sentence intuition

Confidentiality keeps protected content from parties who are not allowed to read it. Boneh and Shoup, Part I.

Where it sits in the taxonomy

- Level: security goal (level 1).
- Parent category: Security Goals.
- Related concepts: Integrity, Anonymity, Authenticated Encryption.

Problem it solves

Alice wants to send Bob a private message across a network that other people can observe. Encryption can hide the message content from those observers; it does not decide whether Bob should be trusted with it. The security model must specify what attackers can observe and whether they can also alter traffic. Boneh and Shoup, Parts I-II.

Mental model

Think of an opaque envelope: it hides the letter but may leave its address, size, and delivery time visible. The analogy describes content secrecy, not anonymous delivery.

Minimal example

Alice sends a form to Bob's website using Transport Layer Security (TLS) 1.3. With authenticated setup and uncompromised endpoints, a network observer cannot read the protected application data. Bob's server can read it, and the observer can still see traffic patterns. TLS ends at its endpoints; an intermediary that terminates TLS is an endpoint too. RFC 8446, §1, §2, and Appendix E.

Security properties

For an encryption scheme, ask which messages an attacker can distinguish, with what capabilities and allowed leakage. Authenticated encryption with associated data (AEAD) adds tamper detection; the associated data is authenticated but **not encrypted**. Boneh and Shoup, Parts I-II; RFC 5116, §2.

What it does not provide

- Sender anonymity, unlinkability, or concealment of all sizes and timing.
- Integrity from encryption alone; it must be part of the scheme or protocol.
- Protection of plaintext copied by a recipient, malware, or a log.

These boundaries follow the separation between encryption security and application or traffic-analysis risks. Boneh and Shoup; RFC 8446, Appendix E.

Assumptions

Keep keys secret, generate them correctly, and follow the scheme's nonce and usage limits. A **nonce** is a per-encryption input; many schemes require it to be unique for a given key. The application must also limit who can request decryption or receive plaintext. RFC 5116, §§2-3 and §5.

Post-quantum posture

Not applicable to the goal. Concrete protection depends on encryption and key establishment. NIST identifies classical public-key cryptography as a quantum migration concern and has standardized ML-KEM for key establishment. Replacing that component does not address endpoint leaks. NIST post-quantum project; FIPS 203.

Confidence model

Mathematical-assumption + client-side-secret: the scheme must resist the modeled attacker, and every endpoint that sees plaintext or keys must protect them. An encrypted database is not protected from an application that is authorized to decrypt it. Boneh and Shoup.

Common constructions

Symmetric encryption and public-key encryption are primitives. Secure channels are protocols combining encryption, authentication, and key establishment. AEAD is a standard way to combine content secrecy and integrity. RFC 5116; RFC 8446.

Use cases

Private network traffic and encrypted records are common uses. In both cases, identify where decryption happens and which parties can obtain its output. Boneh and Shoup.

Composition patterns

Pair confidentiality with Authenticity to establish the intended peer, and with Forward Secrecy if later key theft should not expose past traffic. These are separate requirements in a channel design. RFC 8446, Appendix E.

Failure modes and anti-patterns

> **ENCRYPTION CAN FAIL OUTSIDE THE CIPHER**
>
> Reusing a nonce in a scheme such as AES-GCM can break confidentiality and authentication. Plaintext logging, incorrect peer authentication, and exposed decryption keys can bypass otherwise sound encryption. Use established protocols and audited implementations; this page is not an implementation recipe. RFC 5116, §5.1.1; RFC 8446, Appendix E.

Maturity and deployment

Widely deployed mechanisms: AEAD and TLS 1.3 are standardized. This describes the mechanisms, not a maturity rating for the abstract goal or an endorsement of every configuration. RFC 5116; RFC 8446.

Related concepts

- Authenticated Encryption
- Secure Channels
- Metadata Leakage

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography, Parts I-II - encryption security models.
- RFC 5116, §§2-3 and §5 - authenticated encryption and nonce requirements.
- RFC 8446, Appendix E - TLS guarantees and limits.
- FIPS 203 - ML-KEM, a key-establishment building block.

Integrity

One-sentence intuition

Integrity lets a verifier detect an unauthorized change to protected data. Boneh and Shoup, Part I.

Where it sits in the taxonomy

- Level: security goal (level 1).
- Parent category: Security Goals.
- Related concepts: Authenticity, Message Authentication Codes, Digital Signatures.

Problem it solves

If Alice sends “pay Bob 10,” an attacker should not be able to turn it into “pay Bob 100” and have the altered instruction accepted. Cryptographic integrity makes such forgery infeasible under a stated attack model; it does not physically prevent someone from changing bytes. Boneh and Shoup, message authentication.

Mental model

A tamper-evident seal tells Bob that the package no longer matches what was sealed. Bob still needs a trustworthy way to recognize the seal and must check it before acting.

Minimal example

A webhook sender attaches a message authentication code (MAC), a keyed check value, to a request. The receiver verifies it using their shared secret. An attacker without that secret should not be able to change the covered bytes and produce an accepted tag. Either secret holder can create tags, so this is not public proof of which holder sent the request. Boneh and Shoup, MACs.

Security properties

- Detect changes and new forgeries to the data covered by the authenticator.
- Bind context, such as a record identifier or protocol version, if it is included in the authenticated input.
- Allow an application to reject failed verification before using the protected result.

Authenticated encryption with associated data (AEAD) covers both encrypted content and designated unencrypted context. RFC 5116, §§2.1-2.2.

What it does not provide

- Secrecy of the data.
- Freshness: a valid old message can still be replayed.
- Truth, business validity, or human approval of the contents.

- Completeness: a missing record is not necessarily detected.

A bare hash is also insufficient if an attacker can replace both the file and its expected hash. These limits follow from what the authenticated input and trust anchor actually cover. Boneh and Shoup; RFC 5116, §1.

Assumptions

The scheme must resist forgery, keys or expected digests must come from a trustworthy source, and verification must cover every field the application relies on. For AEAD, respect nonce requirements and treat authentication failure as failure, not as usable plaintext. RFC 5116, §§2-3.

Post-quantum posture

Not applicable to the goal. A concrete MAC, hash, or signature has its own assumptions and parameter requirements. Post-quantum signature standards include ML-DSA and SLH-DSA; their existence does not upgrade an older signature automatically. NIST post-quantum project.

Confidence model

Client-side-secret for MACs; **public-verifiability** plus authentic verification keys for signatures. A Merkle proof instead needs a trusted or authenticated root. The application must interpret exactly the bytes that were verified. Boneh and Shoup; RFC 9162, §2.

Common constructions

MACs, digital signatures, and AEAD protect different settings. Merkle trees provide compact proofs that data matches a particular root; authenticating that root is a separate step. Boneh and Shoup; RFC 9162, §2.

Use cases

Webhook checks, software update verification, and authenticated records all need integrity. The first may use a shared key; the others may need public verification. Boneh and Shoup.

Composition patterns

Add Authenticity to establish the expected origin. Add authenticated counters, challenges, and receiver state when replay matters; an authenticated timestamp alone still needs an acceptance policy. RFC 5116, §1; Boneh and Shoup.

Failure modes and anti-patterns

> **⚠ CHECK THE EXACT DATA THAT DRIVES THE ACTION**
>
> Authenticating the amount while omitting the recipient leaves the recipient unprotected. Accepting a checksum from the same untrusted source as the file leaves both replaceable. Using an old valid tag as evidence of a fresh request leaves replay possible. Boneh and Shoup; RFC 5116, §1.

Maturity and deployment

Widely deployed mechanisms: MACs, signatures, and authenticated encryption are established families. Correct input coverage and verification behavior remain application responsibilities. Boneh and Shoup; RFC 5116.

Related concepts

- Authenticated Encryption
- Accumulators and Merkle Trees
- Transcript Binding

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography - MACs, signatures, and authenticated data.
- RFC 5116 - AEAD and the separate need for replay protection.
- RFC 9162, §2 - Merkle inclusion and consistency proofs.

Authenticity

One-sentence intuition

Authenticity gives you evidence that data or a peer corresponds to the origin you expected. RFC 4949, authentication entries.

Where it sits in the taxonomy

- Level: security goal (level 1).
- Parent category: Security Goals.
- Related concepts: Integrity, Digital Signatures, Anonymous Credentials.

Problem it solves

A message can be perfectly intact and still come from an impostor. Authentication establishes an origin or peer under a trust model. **Authorization** is the separate decision about what that party may do. Recognizing a key does not automatically grant it permission. RFC 4949, authentication and authorization.

Mental model

Think of checking a badge against a trusted directory. A badge with a convincing design is not enough; the issuer, holder, and purpose must match what you intended to check.

Minimal example

In a certificate-authenticated TLS 1.3 connection, the server proves possession of a certificate's private key. The client must validate the certificate and match the service identity to the site it meant to contact. A valid certificate for a different site is insufficient. TLS also supports pre-shared-key authentication, which has a different trust model. RFC 8446, §§2 and 4.4; RFC 9525.

Security properties

Distinguish **data-origin authentication** (“which key or party produced this data?”) from **peer authentication** (“who is participating in this session?”). Peer authentication needs freshness and session context; an old signed message alone does not establish a live peer. RFC 4949; RFC 8446, Appendix E.1.

What it does not provide

- Confidentiality or anonymity by default.
- Truth of a signed statement.
- Proof that a key holder is authorized for a particular operation.
- Proof of human intent, or that a device has not been compromised.

A signature verifies against a key and message; identity, authorization, and key custody require additional evidence and policy. FIPS 186-5; RFC 4949.

Assumptions

Obtain authentic verification keys or shared secrets, check the intended identity, and bind the protocol context to the authenticated data. Define how keys become trusted, how delegation works, and how compromise or revocation changes acceptance. RFC 8446, §4.4; RFC 9525.

Post-quantum posture

Not applicable to the goal. RSA and elliptic-curve signatures face quantum attacks. NIST's standardized post-quantum alternatives include ML-DSA and SLH-DSA. A deployment also needs compatible certificates, protocols, and trust anchors. NIST post-quantum project.

Confidence model

Public-verifiability + trusted-issuer for certificate-based signatures; **client-side-secret** for shared-key authentication. Key pinning uses a different initial trust path. In every case, ask who established the key-to-party binding. Boneh and Shoup; RFC 9525.

Common constructions

Digital signatures and MACs are primitives; authenticated key exchange and certificate validation belong to protocols. Anonymous credential protocols can authenticate an attribute or eligibility without revealing a civil identity. Boneh and Shoup; Camenisch and Lysyanskaya.

Use cases

Authenticating websites, verifying release artifacts, and accepting issuer-backed credentials all require the verifier to know which authority it intended to trust. RFC 8446; FIPS 186-5; Camenisch and Lysyanskaya.

Composition patterns

Pair authentication with Integrity, Transcript Binding, and application authorization. Include the operation and audience in the authenticated context so that evidence for one purpose is not accepted for another. RFC 8446, §4.4.3; Boneh and Shoup.

Failure modes and anti-patterns

> **⚠ A VALID SIGNATURE CAN ANSWER THE WRONG QUESTION**
>
> Accepting any valid key, skipping service-identity checks, or verifying bytes that omit the recipient can authenticate the wrong party or action. A stolen authorized key can also produce valid evidence. Authentication must include identity, context, and compromise handling. RFC 9525; FIPS 186-5.

Maturity and deployment

Widely deployed mechanisms: signatures, MACs, and authenticated TLS. The trust model and key lifecycle still determine what an application may infer. Boneh and Shoup; RFC 8446.

Related concepts

- Secure Channels
- Non-Repudiation
- Key Rotation and Migration

Further reading

- RFC 4949 - authentication versus authorization.
- RFC 8446, §4.4 - TLS authentication.
- RFC 9525: Service Identity in TLS - matching the intended service; supersedes RFC 6125.
- FIPS 186-5: Digital Signature Standard - signature capabilities and limits.

Anonymity

One-sentence intuition

Anonymity means an observer cannot sufficiently identify you among the people who could plausibly have performed an action. Pfitzmann and Hansen, §3.

Where it sits in the taxonomy

- Level: security goal (level 1).
- Parent category: Security Goals.
- Related concepts: Unlinkability, Mixnets, Anonymous Membership.

Problem it solves

A service may need to accept an eligible user's action without learning which eligible user acted. The question is always "anonymous **to whom**, among **which candidates**, given **which observations**?" A hidden name alone does not answer it. Pfitzmann and Hansen, §§2-3.

Mental model

Think of a crowd in which several people could have sent a message. If one person wears a uniquely identifiable coat, the size of the crowd overstates their protection. An anonymity set need not make every member equally likely. Pfitzmann and Hansen, §3.

Minimal example

Alice proves that she holds an eligible membership credential without revealing which member she is. That can hide her identity in the credential interaction. If she submits while logged into a named account, that surrounding application can still identify her. Anonymous credentials and application-level observations must be evaluated together. Camenisch and Lysyanskaya; RFC 9576, §6.

Security properties

Specify sender or recipient anonymity, the observer's capabilities, and how much identification advantage is allowed. Some protocols hide a participant's identity while still letting a verifier check eligibility. This is a property of a particular proof or credential protocol, not of every zero-knowledge proof (ZKP). Pfitzmann and Hansen, §§3 and 5; Camenisch and Lysyanskaya.

What it does not provide

- Confidentiality of the content.
- Unlinkability between repeated actions: a stable pseudonym may connect them.
- Concealment of network addresses or participation unless explicitly included.
- Equal protection for every member of a nominally large set.

Anonymity, pseudonymity, and unlinkability are related but distinct. Pfitzmann and Hansen, §§3-5 and 9.

Assumptions

Define the corruption and collusion model, how many plausible users remain, and what the observer sees. A mixnet may require an honest relay; a credential system may instead rely on a cryptographic privacy definition that tolerates issuer-verifier collusion. Do not assume that every system has the same non-collusion requirement. Chaum; Camenisch and Lysyanskaya.

Post-quantum posture

Not applicable to the goal. Analyze the credential, proof, encryption, and channel separately. Quantum resistance of one component does not establish anonymity for the whole interaction. NIST post-quantum project; RFC 9576, §6.

Confidence model

Possible models include **one-honest-party**, **non-collusion**, **mathematical-assumption**, and **client-side-secret**, depending on the protocol. State who must remain honest and what they must not learn; do not infer this from the word "anonymous." Chaum; RFC 9576, §§4 and 6.

Common constructions

Mixnets obscure correspondence between incoming and outgoing messages. Anonymous credential protocols can prove an entitlement without identifying its holder. These address different observation points and may need to be combined. Chaum; Camenisch and Lysyanskaya.

Use cases

Anonymous access and private credential presentations are examples. Voting and payments need additional rules for eligibility, duplicate use, and public evidence. Camenisch and Lysyanskaya; RFC 9576.

Composition patterns

Combine anonymous authentication with Unlinkability and an explicit Metadata Leakage model. If an abuse-control identifier deliberately links actions, specify the scope of that exception. Privacy Pass illustrates how deployment metadata can narrow otherwise cryptographic privacy. RFC 9576, §§3-6.

Failure modes and anti-patterns

> ⚠ **COUNT PLAUSIBLE USERS, NOT REGISTERED ACCOUNTS**
>
> Timing, rare attributes, user-specific issuer keys, or a named login can isolate someone within a much larger nominal population. A valid privacy proof does not neutralize these observations. RFC 9576, §§6.1-6.3.

Maturity and deployment

Mature but specialized constructions: mixnets and anonymous credentials have established research foundations. Operational anonymity remains dependent on the concrete deployment and observer model; no universal anonymity strength follows from a construction's name. Chaum; Camenisch and Lysyanskaya; RFC 9576.

Related concepts

- Anonymous Credentials
- Unlinkability
- Private Payments

Further reading

- Pfitzmann and Hansen, privacy terminology, v0.34, §§2-5 - precise definitions and observer models.
- Chaum, Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms (1981) - mix-based communication.
- Camenisch and Lysyanskaya, anonymous credentials (2001) - anonymous, unlinkable credential use.
- RFC 9576: The Privacy Pass Architecture, §6 - practical anonymity-set reduction.

Unlinkability

One-sentence intuition

Unlinkability limits what an observer can learn about whether two items, actions, or identities are related. Pfitzmann and Hansen, §4.

Where it sits in the taxonomy

- Level: security goal (level 1).
- Parent category: Security Goals.
- Related concepts: Anonymity, Anonymous Tokens, Nullifiers.

Problem it solves

An observer may not know your name but may still build a detailed profile by linking your visits. Unlinkability asks whether the observer can connect the visits, not merely whether they can identify the person. Specify the relationship being hidden: two presentations, an issuance and redemption, or a sender and recipient. Pfitzmann and Hansen, §§4-5 and 9.

Mental model

Imagine using separate tickets for separate visits. Changing the ticket number helps only if neither the tickets nor the surrounding observations reveal their shared origin. A stable account cookie can undo the separation.

Minimal example

Alice obtains a Privacy Pass token and later presents it to a website. The token protocol can prevent the website from matching that token to its issuance event. Whether the website can recognize Alice through an address, account, or timing is a separate deployment question. RFC 9576, §§3.3-3.6 and 6.

Security properties

A claim must name the observer and the relationship it hides. Privacy Pass distinguishes several unlinkability properties across its client, issuer, attester, and origin roles. A guarantee against one role does not automatically hold against all roles acting together. RFC 9576, §§3-4.

What it does not provide

- Content confidentiality or anonymous networking.
- Immunity to timing correlation and identifying attributes.
- Proof that a token cannot be transferred to someone else.
- Duplicate-use prevention: a service may still need to remember spent tokens.

In particular, unlinkability and non-transferability are different properties. Privacy Pass discusses token sharing and replay separately from privacy. RFC 9576, §§6 and 7.1.

Assumptions

Use the correct cryptographic issuance and presentation protocol, consistent public parameters, and the deployment's stated collusion model. Blind signatures can hide the issuance-redemption relation cryptographically even when roles collude; identifying metadata can still defeat system-level privacy. Non-collusion is therefore a specific architectural assumption, not a universal requirement of unlinkability. RFC 9576, §§3-4 and 6; Chaum.

Post-quantum posture

Not applicable to the goal. The blind-signature, credential, or token construction has its own assumptions. Classical RSA-based blind signatures do not become quantum resistant because the transport changes. Chaum; NIST post-quantum project.

Confidence model

Mathematical-assumption for the cryptographic unlinkability property; sometimes **non-collusion** for the overall deployment. Client behavior and public-parameter consistency also matter. Name the parties that could reconstruct the hidden relationship. RFC 9576, §§4 and 6.

Common constructions

Blind signatures and randomized anonymous credential presentations can hide links between interactions. A **nullifier** is different: it deliberately creates a repeatable identifier within a chosen scope to detect repeat use. It supports a limited exception to unlinkability rather than providing unlinkability by itself. Chaum; Camenisch and Lysyanskaya; Zerocash, §4.

Use cases

Anonymous token redemption, repeated credential presentations, and private payments each hide different relationships. Their claims should name those relationships instead of promising general “untraceability.” RFC 9576; Camenisch and Lysyanskaya; Zerocash.

Composition patterns

When using Anti-Double-Use Nullifiers, define the application or event that shares an identifier. Reusing the same link identifier across unrelated contexts can expose an unwanted relationship. This is an application of the distinction between scoped and cross-context pseudonyms. Pfitzmann and Hansen, §11.

Failure modes and anti-patterns

> ⚠ **DIFFERENT CRYPTOGRAPHIC VALUES CAN STILL BE CORRELATED**
>
> A user-specific issuer key, a unique issuance time, or a stable browser identifier can connect otherwise unlinkable interactions. Key rotation can also partition users into smaller groups; evaluate that privacy trade-off alongside compromise handling. RFC 9576, §§6.1-6.3.

Maturity and deployment

Mature but specialized mechanisms: blind signatures and anonymous credentials have long-standing constructions; Privacy Pass has standardized architecture and token protocols. This does not give every deployment the same metadata protection. RFC 9576.

Related concepts

- Blind Signatures
- Anonymous Credentials
- Privacy-Preserving Revocation

Further reading

- Pfitzmann and Hansen, privacy terminology, §§4-5 and 11 - relationships and scoped pseudonyms.
- RFC 9576, §§3-7 - token unlinkability and deployment risks.
- Chaum, Blind Signatures for Untraceable Payments (CRYPTO 1982; proceedings 1983) - foundational construction.
- Zerocash (2014), §4 - deliberate repeat-spend detection in a private payment protocol.

Forward Secrecy

One-sentence intuition

Forward secrecy limits the damage of a later key theft: past session secrets should remain protected when the specified long-term key is compromised. RFC 8446, Appendix E.1.

Where it sits in the taxonomy

- Level: security goal (level 1), usually evaluated for a protocol over time.
- Parent category: Security Goals.
- Related concepts: Confidentiality, Secure Channels, Secure Messaging.

Problem it solves

An attacker records encrypted conversations, then later steals a server's long-term authentication key. A forward-secret exchange is designed so that this theft alone cannot reconstruct the old traffic keys. It does not help if the attacker also saved the old traffic secrets or plaintext. RFC 8446, §2 and Appendix E.

Mental model

Each conversation uses a temporary key that is destroyed afterward. Stealing the identity key later should not recreate those temporary keys. Destruction is an essential part of the model, not an optional housekeeping step.

Minimal example

Alice connects to a server using certificate-authenticated TLS 1.3 with ephemeral Diffie-Hellman key exchange. “Ephemeral” means temporary key-exchange secrets. If they and the old traffic keys are erased, later theft of the server's certificate key alone should not decrypt recorded application traffic. This example does **not** cover every TLS 1.3 mode. RFC 8446, §§2.2-2.3 and Appendix E.

Security properties

Question	Property or limit
Does later long-term key theft reveal completed sessions?	Forward secrecy aims to prevent this.
Does theft of current endpoint state reveal earlier message keys?	A ratchet can provide this if old keys were erased.
Can communication become confidential again after a state compromise?	Post-compromise security; it needs additional protocol conditions and fresh secret input.

A one-way key update can protect earlier keys while still letting a thief derive future ones. The last two properties must therefore be assessed separately. RFC 8446, Appendix E.2; Double Ratchet, §§2 and 8.

What it does not provide

- Protection for a session whose relevant ephemeral or traffic secrets were stolen.
- Erasure of plaintext in backups, screenshots, or logs.
- Automatic recovery while an attacker still controls a device.
- Post-quantum security or metadata privacy by itself.

These are explicit limits of channel and ratchet security models. RFC 8446, Appendix E; Double Ratchet, §8.

Assumptions

Generate fresh secret contributions, authenticate the exchange, erase obsolete secrets, and define which compromise happens when. Retained debugging keys or cloned endpoint state can invalidate the intended protection. RFC 8446, Appendix E; Double Ratchet, §8.

Post-quantum posture

Not applicable to the goal. Recorded classical Diffie-Hellman exchanges remain vulnerable to a sufficiently capable quantum attacker, even if temporary keys were erased. Post-quantum key establishment addresses a different threat than later theft of an authentication key. NIST post-quantum project. The current Double Ratchet specification includes Sparse Post-Quantum Ratchet and Triple Ratchet designs; assess the exact variant, not the name alone. Double Ratchet, §§5-6.

Confidence model

Mathematical-assumption + client-side-secret: key establishment must resist the modeled attacker, and endpoints must erase the old secrets. Authentication, compromise timing, and retained skipped-message keys affect the actual boundary. RFC 8446, Appendix E; Double Ratchet, §8.

Common constructions

Ephemeral key exchange and ratcheting protocols offer different scopes of forward secrecy. A key encapsulation mechanism (KEM) alone does not guarantee it: the lifetime of its decapsulation key and the surrounding authenticated protocol matter. RFC 8446; FIPS 203; Double Ratchet.

Use cases

Protecting past web sessions and past messaging keys are representative uses. State the unit being protected: a completed connection, an earlier key generation, or an individual message. RFC 8446, Appendix E; Double Ratchet, §2.

Composition patterns

Combine with Authenticity and explicit compromise recovery. Ordinary key rotation does not retroactively protect sessions whose keys were derived from a now-exposed long-term secret. RFC 8446, Appendix E; Double Ratchet, §8.

Failure modes and anti-patterns

> ⚠ **TLS 1.3 HAS MODE-SPECIFIC EXCEPTIONS**
>
> Pre-shared-key (PSK) authentication **without** an ephemeral Diffie-Hellman contribution lacks forward secrecy against later PSK compromise. Zero round-trip time (0-RTT) early data is not forward secret and has separate replay risks. Adding ephemeral exchange for subsequent data does not upgrade that early data. RFC 8446, §§2.2-2.3 and 8.

Maturity and deployment

Widely deployed mechanisms: ephemeral exchanges in TLS and ratcheting messaging. Post-quantum variants need their own protocol and implementation assessment; classical deployment experience does not settle their properties. RFC 8446; Double Ratchet, §§5-6.

Related concepts

- Key Encapsulation and Exchange
- Key Rotation and Migration
- Post-Quantum Posture

Further reading

- RFC 8446, §§2.2-2.3 and Appendix E - forward secrecy, early data, and compromise limits.
- Signal, The Double Ratchet Algorithm, revision 4 (4 November 2025) - message-key evolution and post-quantum variants.
- FIPS 203 - ML-KEM's interface and security scope.

Verifiability

One-sentence intuition

Verifiability means you can check a stated claim using evidence, instead of accepting the claimant's word. Cryptographic proofs make this precise through properties such as completeness and soundness. Goldwasser, Micali, and Rackoff.

Where it sits in the taxonomy

- Level: security goal (level 1).
- Parent category: Security Goals.
- Related concepts: Auditability, Zero-Knowledge Proofs, Membership Proofs.

Problem it solves

An operator says that a record is included in a log or that a computation followed the rules. Verifiability gives a checker a way to assess that particular claim. It moves trust toward the verification rules, their inputs, and their assumptions; it does not remove all trust. RFC 9162, §2; Goldwasser, Micali, and Rackoff.

Mental model

Think of a worked answer that can be checked against a precise question. The check is useful only if it asks the question you actually care about.

Minimal example

A log operator gives Alice a Merkle inclusion proof for a certificate. Alice checks the proof against a particular authenticated tree root and tree size. Success establishes inclusion in that tree; it does not show that the certificate was legitimately issued or that the operator shows everyone the same tree. Those need separate monitoring and consistency checks. RFC 9162, §§2.1 and 11.

Security properties

- **Completeness:** honest proofs of valid statements should be accepted under the scheme's definition.
- **Soundness:** a false statement should not be accepted except with the permitted error probability or attack advantage.
- **Verification scope:** public verification is available to anyone with the public evidence; designated verification may require a particular party's secret.

These are different properties. Zero knowledge adds a disclosure limit; it is not a synonym for soundness or public verifiability. Goldwasser, Micali, and Rackoff; Boneh and Shoup, proof systems.

What it does not provide

- Truth of an external input: a proof about a supplied temperature does not measure the room.

- Correctness of rules that were encoded incorrectly.
- Availability of underlying data or continued service.
- Privacy unless the mechanism explicitly provides it.

An inclusion proof, for example, is about a tree entry, not the legitimacy of that entry or the completeness of the world's records. RFC 9162, §§2 and 11.

Assumptions

The intended statement must be encoded correctly. Obtain authentic roots or verification parameters, bind all relevant inputs, run the complete verifier, and state any trusted-setup assumptions. Soundness is always relative to a particular proof system and adversary model. Boneh and Shoup, proof systems.

Post-quantum posture

Not applicable to the goal. Proofs, hashes, commitments, and signatures must be assessed individually. In a signed Merkle log, quantum-resistant tree hashing alone does not upgrade a vulnerable log-signing key. RFC 9162, §§2 and 10; NIST post-quantum project.

Confidence model

Depends: public-verifiability for public proofs; a designated verifier or shared secret for other mechanisms. Mathematical assumptions, setup, and authentic public inputs remain separate dependencies. Do not infer public verifiability from the word “proof.” Boneh and Shoup.

Common constructions

Signatures verify a message under a key; Merkle proofs verify membership or consistency relative to roots; proof systems verify formally specified statements. They answer different questions. Boneh and Shoup; RFC 9162, §2.

Use cases

Log inclusion and consistency checks, credential verification, and computation proofs use this goal. Ask which claim each check actually establishes before extending it to an entire application. RFC 9162; Boneh and Shoup.

Composition patterns

Integrity protects the evidence, Authenticity identifies the expected key or authority, and Auditability preserves evidence for later review. Certificate Transparency shows why proofs still need monitoring and responses. RFC 9162, §§8 and 11.

Failure modes and anti-patterns

> ⚠ **CHECK THE STATEMENT AS WELL AS THE PROOF**
>
> A valid proof against the wrong root, application, or public inputs answers the wrong question. A log inclusion proof also cannot reveal every unlogged event. Verification is bounded by its inputs and rules. RFC 9162, §§2 and 11; Boneh and Shoup.

Maturity and deployment

Widely deployed mechanisms: signatures and authenticated data structures. Advanced proof systems have different maturity levels; their common ability to verify claims is not evidence of equal deployment readiness. Boneh and Shoup; RFC 9162.

Related concepts

- Accumulators and Merkle Trees
- Transcript Binding
- ZK Rollups

Further reading

- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems (1989) - proof properties and zero knowledge.
- Boneh and Shoup, A Graduate Course in Applied Cryptography - definitions and proof-system models.
- RFC 9162: Certificate Transparency Version 2.0, §§2 and 11 - what log proofs do and do not establish; an experimental protocol specification.

Auditability

One-sentence intuition

Auditability means a process leaves enough trustworthy, accessible evidence for a meaningful later review. This combines evidence protection with log-management and review procedures. NIST SP 800-92, §§2-3 and 5.

Where it sits in the taxonomy

- Level: security goal (level 1), usually applied to a system or process.
- Parent category: Security Goals.
- Related concepts: Verifiability, Accountability, Integrity.

Problem it solves

After an incident, "the operator says everything was fine" is insufficient. A reviewer needs records of the relevant actions, enough context to interpret them, and a way to detect tampering. Merely generating logs does not ensure they are retained, inspected, or useful. NIST SP 800-92, §§2-3 and 5.

Mental model

Think of a flight recorder with a defined scope. A protected recording can help reconstruct what happened, but it cannot answer questions about events that were never captured.

Minimal example

A certificate log publishes signed tree heads and provides inclusion and consistency proofs. A domain owner monitors entries for unexpected certificates and retains evidence for investigation. The proofs help review the published history; monitoring is what identifies suspect issuance. The log does not decide on its own whether issuance was legitimate. RFC 9162, §§8 and 11.

Security properties

A useful audit design specifies event coverage, integrity, ordering or time context, retention, and reviewer access. These are requirements to establish, not guarantees automatically supplied by a hash chain. NIST SP 800-92, §§3 and 5.

A consistency proof can check that one Merkle tree extends another. Detecting inconsistent views served to different users additionally requires those views to be compared or otherwise exposed to appropriate monitors. RFC 9162, §§2.1.4 and 11.3.

What it does not provide

- Proof that no unrecorded action occurred.
- Truth of every recorded claim.

- Automatic detection, remediation, or punishment.
- Privacy: logs may expose identities, secrets, and relationships.

Logging policy and review determine these boundaries. NIST SP 800-92, §§2-3; RFC 9162, §11.

Assumptions

Define what must be recorded and who can review it. Protect collection, transport, storage, and deletion; retain evidence for the relevant review period. If independent auditing is part of the claim, reviewers need sufficient access and independence from the operator being checked. NIST SP 800-92, §§3-5.

Post-quantum posture

Not applicable to the goal. Long-lived evidence inherits the assumptions of signatures, hashes, timestamps, and storage. Evidence signed today is not automatically reliable forever under future attacks. RFC 3161, §§1 and 4; NIST post-quantum project.

Confidence model

Public-verifiability for public log proofs, or **operational-audit** for restricted evidence. Both need available records and a review process; the latter also needs trusted, sufficiently independent reviewers. RFC 9162, §§8 and 11; NIST SP 800-92.

Common constructions

Append-only transparency logs, signed records, timestamps, and protected centralized log collection support different audit requirements. A timestamp attests to a digest's existence by a time under its authority's trust model; it does not prove that the underlying event was true. RFC 9162; RFC 3161; NIST SP 800-92.

Use cases

Certificate issuance monitoring and security incident investigation are representative uses. Choose evidence for the questions the reviewer must answer, rather than collecting everything. RFC 9162, §11; NIST SP 800-92, §3.

Composition patterns

Verifiability supplies specific checks; auditability preserves the evidence needed to repeat them. Accountability adds attribution under rules and a response process. Public evidence and restricted incident records may require different disclosure policies. RFC 9162, §11; NIST SP 800-92, §§3-5.

Failure modes and anti-patterns

> ⚠ **A TAMPER-EVIDENT LOG CAN STILL BE INCOMPLETE**
>
> An operator can omit an event before logging it. A log can also present conflicting histories unless the design exposes that behavior. Collecting plaintext or permanent identifiers "for audit" can create new privacy failures. Define coverage, cross-checks, access, and retention together. RFC 9162, §11.3; NIST SP 800-92, §§2-3.

Maturity and deployment

Widely deployed practices: security logging and operational review. Cryptographic transparency logs provide more specific checks; RFC 9162 describes an experimental version 2 protocol and should not be read as a claim that every deployed log uses that version. NIST SP 800-92; RFC 9162.

Related concepts

- Verifiability
- Accountability
- Metadata Leakage

Further reading

- NIST SP 800-92: Guide to Computer Security Log Management, §§2-5 - collection, review, access, and retention.
- RFC 9162, §§2, 8, and 11 - proofs, monitoring, and misbehaving logs.
- RFC 3161: Time-Stamp Protocol - timestamp authority assumptions and limitations.

Accountability

One-sentence intuition

Accountability lets a system attribute a defined violation to a participant using evidence that others can assess. Haeberlen, Kuznetsov, and Druschel.

Where it sits in the taxonomy

- Level: security goal (level 1), usually applied to a protocol or system.
- Parent category: Security Goals.
- Related concepts: Authenticity, Auditability, Non-Repudiation.

Problem it solves

Detecting an error is not always enough: a system may need to establish which participant violated which rule. A useful accountability design must also protect honest participants from false accusations. Neither attribution nor detection automatically supplies a punishment or a repair procedure. PeerReview.

Mental model

Think of evidence plus a rulebook. A signed statement records what a key endorsed. The rulebook explains whether that statement is prohibited and who can respond.

Minimal example

Suppose a log signs two different Merkle roots for the same tree size under the same log identity. Anyone who obtains both signed tree heads can check the inconsistency and attribute the signatures to that log key. The evidence does not reveal which employee acted or whether the key was stolen. Comparing conflicting views is essential. RFC 9162, §11.3.

Security properties

An accountability claim should specify:

- **Detection scope:** which observable violations yield evidence?
- **Attribution:** which key, role, or node does the evidence identify?
- **Protection against framing:** can an honest participant rebut a false accusation?

PeerReview supplies such guarantees for a defined distributed-system model, with deterministic behavior and correct witnesses; it is not a theorem about every logged system. PeerReview.

What it does not provide

- Detection of every private or unobservable violation.
- Proof that a particular human controlled the implicated key.

- Automatic sanctions, compensation, or recovery.
- Public identity disclosure as a necessary condition.

A node or pseudonym can be accountable within a system's rules. Extending that attribution to a person requires a separate identity and key-control model. PeerReview; FIPS 186-5.

Assumptions

Define violations precisely, provision authentic keys, protect evidence, and keep it available to checkers. Witnesses and communication must meet the chosen protocol's assumptions. If consequences depend on a governance or dispute process, that process must actually be able to act. PeerReview; RFC 9162, §11.

Post-quantum posture

Not applicable to the goal. Attribution based on signatures or commitments inherits their resistance to forgery. Archival evidence and current authentication may need different migration plans. FIPS 186-5; NIST post-quantum project.

Confidence model

Mixed: mathematical-assumption and authentic key bindings for evidence, public-verifiability or trusted reviewers for checking it, and operational procedures for responding. PeerReview additionally needs correct witnesses; a signature alone cannot replace them. PeerReview.

Common constructions

Signed messages, tamper-evident records, witness protocols, and independently checkable evidence can support accountability. A final threshold signature generally proves authorization by a group key; it does not by itself identify the individual signers or provide blame evidence. That needs a protocol with additional evidence. PeerReview; RFC 9591, §§5 and 7.

Use cases

Distributed-service fault investigation and transparency-log disputes are examples. They attribute specific protocol violations rather than making a general claim about a participant's honesty. PeerReview; RFC 9162, §11.

Composition patterns

Authenticity binds evidence to keys; Auditability keeps it available for review. Decide separately who may see identifying evidence and how compromised keys are handled in a dispute. PeerReview; FIPS 186-5.

Failure modes and anti-patterns

> **DETECTION IS NOT ATTRIBUTION OR ENFORCEMENT**
>
> A missing message may reflect delay, failure, or selective exclusion. Do not automatically label it proven malice. Conversely, two contradictory signed statements can supply direct evidence of a rule violation without proving the signer's motive. Define the evidence threshold and the response for each case. PeerReview; RFC 9162, §11.3.

Maturity and deployment

Mature but specialized mechanisms: authenticated records and evidence-based fault detection have established constructions. Each system still needs a defensible violation model and operational response; cryptographic evidence does not establish institutional reliability. PeerReview.

Related concepts

- Auditability
- Non-Repudiation
- Threshold Cryptography

Further reading

- Haeberlen, Kuznetsov, and Druschel, PeerReview: Practical Accountability for Distributed Systems (SOSP 2007) - observable faults, witnesses, and protection from framing.
- RFC 9162, §11.3 - evidence of log misbehavior.
- RFC 9591: FROST, §§5 and 7 - threshold signing and protocol-specific security considerations.
- FIPS 186-5 - what signatures establish under key-management assumptions.

Deniability

One-sentence intuition

Deniability aims to let participants authenticate a conversation without giving them a transcript that cryptographically proves the other's statements to outsiders. Borisov, Goldberg, and Brewer.

Where it sits in the taxonomy

- Level: security goal (level 1), usually evaluated for a protocol.
- Parent category: Security Goals.
- Related concepts: Authenticity, Non-Repudiation, Secure Messaging.

Problem it solves

Alice wants assurance that she is talking to Bob, but does not want their exchange to create portable cryptographic evidence of every statement. Local authentication and third-party proof are different requirements. Off-the-Record (OTR) communication was designed around this distinction. Borisov, Goldberg, and Brewer.

Mental model

Think of a conversation without a transferable signed receipt. The other participant may believe what happened, but the protocol transcript alone should not uniquely prove it to someone else.

Minimal example

Alice and Bob share a secret message authentication code (MAC) key established within a suitable authenticated protocol. Bob checks a tag during the conversation. Because Bob also knows the MAC key, the tag alone cannot prove to an outsider that only Alice could have created it. This illustrates the distinction; a shared MAC alone is not a complete deniable messaging protocol. Borisov, Goldberg, and Brewer.

Security properties

A deniability definition specifies who can simulate plausible evidence and which adversary should be unable to distinguish it from a real exchange. Hiding message authorship after the fact, hiding participation, and resisting an adversary who cooperates during the session are different goals. The informal claim “deniable chat” does not settle all of them. Borisov, Goldberg, and Brewer; Di Raimondo, Gennaro, and Krawczyk.

What it does not provide

- Secrecy from the recipient.
- Prevention of screenshots, recordings, or witness testimony.
- Sender anonymity or concealment of contact metadata.

- A guarantee about how people or a legal process will assess evidence.

The cryptographic goal concerns the transferability of protocol evidence, not the impossibility of disclosure or belief. Borisov, Goldberg, and Brewer.

Assumptions

The whole authentication protocol must support the intended deniability definition. Public signatures, server records, or device artifacts may add evidence beyond the modeled transcript. State whether the attacker acts during the exchange or only receives records afterward. Di Raimondo, Gennaro, and Krawczyk; Borisov, Goldberg, and Brewer.

Post-quantum posture

Not applicable to the goal. Check the exact authenticated key exchange and its deniability argument against the modeled quantum attacker. A post-quantum key exchange or a new ratchet does not automatically establish deniability for the complete protocol. Di Raimondo, Gennaro, and Krawczyk; NIST post-quantum project.

Confidence model

Mixed: the protocol's authentication and simulation properties, endpoint behavior, and the scope of available evidence all matter. Erasing old keys can help Forward Secrecy, but forward secrecy alone does not imply deniability. Borisov, Goldberg, and Brewer.

Common constructions

Deniable authenticated key exchange, symmetric message authentication, and protocol-specific disclosure of obsolete authentication keys appear in OTR-style designs. Avoiding public signatures on message content can help, but is not a complete proof of deniability. Borisov, Goldberg, and Brewer; Di Raimondo, Gennaro, and Krawczyk.

Use cases

Private conversations and off-the-record negotiations may want local authentication without permanent transferable receipts. A system whose purpose is public evidence has a different requirement. Borisov, Goldberg, and Brewer.

Composition patterns

Combine deniable authentication with Confidentiality and forward secrecy. Treat receipt-freeness as a separate voting goal: a deniable chat transcript alone does not stop vote buying or credential surrender. Borisov, Goldberg, and Brewer; Benaloh and Tuinstra.

Failure modes and anti-patterns

> ⚠ **A RATCHET IS NOT A DENIABILITY CERTIFICATE**
>
> Key evolution addresses compromise over time. Deniability also depends on authentication, transcripts, and the adversary model. Adding an externally verifiable signature to each message can create precisely the evidence the application wanted to avoid. Borisov, Goldberg, and Brewer; Di Raimondo, Gennaro, and Krawczyk.

Maturity and deployment

Mature but specialized: deniable messaging and key-exchange constructions have an established literature. Stronger adversary models and application-level evidence still need protocol-specific expert analysis; an educational sketch is not deployment guidance. Borisov, Goldberg, and Brewer; Di Raimondo, Gennaro, and Krawczyk.

Related concepts

- Non-Repudiation
- Forward Secrecy
- Make Receipts Useless

Further reading

- Borisov, Goldberg, and Brewer, Off-the-Record Communication, or, Why Not To Use PGP (WPES 2004) - intuition and design trade-offs.
- Di Raimondo, Gennaro, and Krawczyk, Deniable Authentication and Key Exchange (CCS 2006) - formal deniability models.
- Benaloh and Tuinstra, Receipt-Free Secret-Ballot Elections (STOC 1994) - the related voting requirement.

Non-Repudiation

One-sentence intuition

Non-repudiation aims to preserve evidence that a third party can use to resolve a later denial of an action. RFC 4949, non-repudiation entries.

Where it sits in the taxonomy

- Level: security goal (level 1), applied to a protocol or evidence process.
- Parent category: Security Goals.
- Related concepts: Authenticity, Accountability, Deniability.

Problem it solves

Bob receives an instruction and wants evidence for a later dispute in which Alice denies issuing it. A digital signature can support that evidence, provided the verifier can establish which key was trusted, what it signed, and the relevant key-control conditions. It does not make all denial impossible. FIPS 186-5; RFC 4949.

Mental model

Think of a signed receipt with a documented chain of custody. The mark matters, but so do the exact statement, the signer's authority, and the conditions under which the receipt was created and kept.

Minimal example

A release process signs an artifact digest and a release statement naming the product and version. A reviewer later verifies the signature with the trusted release key. If an independent timestamp is retained, it can add evidence that the signed material existed by a particular time. Neither check proves that the release was free of bugs or that a particular employee approved it. FIPS 186-5; RFC 3161, §§1-2.

Security properties

Distinguish evidence of **origin** from evidence of **receipt**. Alice's signature on an instruction can support origin evidence; it does not prove Bob received or acted on it. The system must specify the action being evidenced and the third party expected to assess it. RFC 4949, non-repudiation service.

What it does not provide

- Proof of a specific human's intent or physical presence.
- Protection after an attacker obtains the signing key.
- Proof of delivery from an origin signature alone.
- Automatic legal effect or a guaranteed outcome in a dispute.
- Deniability for the same statement and evidence audience.

Digital signatures support evidence under key-management and identity assumptions; the surrounding process supplies their interpretation. FIPS 186-5; RFC 4949.

Assumptions

Keep the private key under the intended control, provision authentic verification keys, and retain the exact signed bytes and relevant identity, validation, and timestamp evidence. Define how key compromise, revocation, and validation at a past time affect acceptance. FIPS 186-5; RFC 3161, §§2 and 4.

Post-quantum posture

Not applicable to the goal. Long-lived evidence depends on signatures, timestamp authorities, hashes, and archival policy. NIST's ML-DSA and SLH-DSA standards provide post-quantum signature building blocks; they do not retroactively strengthen existing classical signatures. NIST post-quantum project.

Confidence model

Mixed: mathematical-assumption for signature security, trusted-issuer or another authentic key directory for identity, key-custody controls, and trusted timing when required. The dispute process must understand these limitations. FIPS 186-5; RFC 3161.

Common constructions

Digital signatures, certificate-based identity binding, signed acknowledgments, and trusted timestamps support different parts of an evidence process. A MAC shared by Alice and Bob cannot, by itself, show a third party which of them generated a message. Boneh and Shoup, MACs and signatures; RFC 3161.

Use cases

Signed release statements, administrative approvals, and signed transaction instructions can preserve origin evidence. Receipt evidence needs an acknowledgment or a separate delivery protocol. FIPS 186-5; RFC 4949.

Composition patterns

Pair with Integrity, Auditability, explicit key validity rules, and an evidence-retention policy. Avoid adding public signatures to a protocol that requires deniability for those same messages. RFC 3161, §4; Borisov, Goldberg, and Brewer.

Failure modes and anti-patterns

> ⚠ **A SIGNATURE PROVES LESS THAN AN APPROVAL SCREEN SUGGESTS**
>
> A valid signature over ambiguous bytes, the wrong document version, or an incomplete instruction may not support the intended dispute. A timestamp establishes a time-related claim about a digest, not the truth of the content or the signer's intent. FIPS 186-5; RFC 3161.

Maturity and deployment

Widely deployed mechanisms: signatures and public-key infrastructure support evidence processes. Their technical maturity does not establish the adequacy of a particular legal or organizational process. FIPS 186-5; RFC 4949.

Related concepts

- Digital Signatures
- Accountability
- Deniability

Further reading

- RFC 4949 - origin and receipt evidence; technical versus broader meanings.
- FIPS 186-5: Digital Signature Standard - signature guarantees and key-management dependencies.
- RFC 3161: Time-Stamp Protocol, especially §4 - timestamp security considerations.
- Boneh and Shoup - why public signatures differ from shared-key MACs.

Availability

One-sentence intuition

Availability means authorized users can obtain the intended service when required, within the system's stated performance and failure limits. RFC 4949, availability.

Where it sits in the taxonomy

- Level: security goal (level 1), usually evaluated for a system.
- Parent category: Security Goals.
- Related concepts: Censorship Resistance, Threshold Cryptography, Key Rotation and Migration.

Problem it solves

A system can keep secrets and reject forged messages while still refusing every useful request. Availability addresses service failure, including lost keys, unreachable participants, stalled processing, and resource exhaustion. State what must complete and within what time; “online” is too vague. RFC 4949, availability and denial of service.

Mental model

Think of a building with several working entrances. Several doors do not help if they all depend on the same failed access controller. Likewise, counting servers does not establish independent failure tolerance. Castro and Liskov, §2.

Minimal example

A 3-of-5 signing service needs three cooperating signers for an operation. If two are offline and the other three follow the protocol, the threshold is still attainable. If three are offline, it is not. This arithmetic is a necessary condition, not a guarantee of completion: a chosen signer can withhold a response and abort a FROST signing attempt. Retrying and selecting participants are additional system responsibilities. RFC 9591, §§5 and 7.

Security properties

Define availability relative to a workload, timing requirement, and fault model. In distributed protocols, **liveness** means the required progress eventually occurs under stated conditions; it is distinct from a safety claim such as “no conflicting result is accepted.” Byzantine-fault-tolerant replication illustrates that these properties need separate network assumptions. RFC 4949; Castro and Liskov, §§2-3.

What it does not provide

- Confidentiality or integrity of responses.
- Progress during every network partition or unlimited denial-of-service attack.
- Selective-inclusion guarantees merely because a service is generally reachable.

- Recoverability of data merely because a commitment or proof exists.

A proof can check data that is supplied without ensuring it will be supplied. Separate availability requirements from verification requirements. RFC 4949; RFC 9162, §§2 and 11.

Assumptions

Specify reachable participants, tolerated faults, network conditions, capacity, and recovery dependencies. Independence matters: separate machines may share administrators, credentials, software bugs, or a provider outage. A threshold number does not capture all these dependencies. Castro and Liskov, §2; RFC 9591, §7.

Post-quantum posture

Not applicable to the goal. Quantum-vulnerable authentication can undermine a concrete service, but adopting post-quantum primitives does not solve overload, key loss, or correlated outages. Evaluate both cryptographic and operational dependencies. NIST post-quantum project; RFC 4949.

Confidence model

Mixed: a t-of-n-threshold may describe a signing requirement; a consensus protocol may have a different honest-party threshold and timing model. State the threshold for each operation and how many participants must be reachable and cooperative, not just uncompromised. RFC 9591; Castro and Liskov.

Common constructions

Replication, protocol-specific fault tolerance, recovery procedures, and redundant access paths can support availability. Threshold signing distributes key control, but can add a participation dependency; it does not automatically improve uptime. Castro and Liskov; RFC 9591, §7.

Use cases

Threshold custody and replicated services need explicit completion and recovery requirements. A website health check is insufficient if users cannot retrieve required data or obtain the necessary signatures. RFC 4949; RFC 9591.

Composition patterns

Evaluate Censorship Resistance for particular users, and Verifiability for returned results. Design recovery alongside confidentiality: making a secret easier to recover changes who may gain access to it. RFC 4949; RFC 9591, §7.

Failure modes and anti-patterns

> ⚠ **COUNT SHARED DEPENDENCIES AS WELL AS PARTICIPANTS**
>
> A committee can meet its nominal size and still fail together. A reachable signer can also refuse to cooperate. Test the actual completion and recovery paths against the fault model; do not treat replication or a signature threshold as a standalone availability guarantee. Castro and Liskov, §2; RFC 9591, §7.

Maturity and deployment

Widely deployed techniques: replication and operational recovery. Byzantine fault tolerance and threshold-signing services require more specialized assumptions. Their guarantees must be taken from the chosen protocol, not generalized from a replica count. Castro and Liskov; RFC 9591.

Related concepts

- Censorship Resistance
- Threshold Cryptography
- ZK Rollups

Further reading

- RFC 4949 - availability and denial-of-service terminology.
- Castro and Liskov, Practical Byzantine Fault Tolerance (OSDI 1999), §§2-3 - network and failure assumptions, safety, and liveness.
- RFC 9591: FROST, §7 - participation, aborts, and operational responsibilities.

Censorship Resistance

One-sentence intuition

Censorship resistance limits an adversary's ability to selectively prevent a valid submission from reaching its intended destination or being included, under stated network and operator assumptions. Inclusion-list designs illustrate this system-level goal. EIP-7805.

Where it sits in the taxonomy

- Level: security goal (level 1), applied to protocols and systems.
- Parent category: Security Goals.
- Related concepts: Availability, Encrypted Mempools, Delayed Reveal.

Problem it solves

A service can work for most users while one operator blocks Alice. Censorship resistance asks what Alice can do then: reach another operator, use an enforced inclusion path, or obtain evidence of exclusion. **Detecting censorship and overcoming it are separate guarantees.** EIP-7805, Motivation and Security Considerations.

Mental model

Think of a delivery service with an adversarial gatekeeper. Another entrance helps only if Alice can reach it and the operator controlling final delivery cannot simply ignore it.

Minimal example

In the **draft** Fork-choice enforced Inclusion Lists (FOCIL) proposal, a validator committee distributes transaction inclusion lists. Attesters check whether a proposed block satisfies their stored lists before voting for it. Conditions still include propagation deadlines, transaction validity, and available block capacity. This illustrates a proposed mechanism, not a claim about deployed Ethereum behavior. Status checked **27 October 2026**. EIP-7805, Specification and Security Considerations.

Security properties

A concrete claim should state the valid submissions covered, the maximum exclusion delay if any, tolerated censor control, network conditions, and the enforced consequence of exclusion. FOCIL's conditions demonstrate why “every valid transaction is immediately included” would be too strong. EIP-7805, Rationale and Security Considerations.

What it does not provide

- Payload confidentiality, sender anonymity, or fair transaction ordering.
- Inclusion of invalid submissions or submissions outside fee and capacity rules.
- Progress when an adversary can indefinitely block every usable network path.

- Inclusion merely because refusal can later be proved.

FOCIL deliberately leaves transaction placement within a block unconstrained and uses conditional inclusion rules. EIP-7805, Core Properties.

Assumptions

Users must reach the required participants before relevant deadlines. Enough participants must follow the inclusion rules, and the underlying consensus must remain live. An “honest or accountable” operator is not enough by itself: accountability needs an effective enforcement or alternative delivery path. EIP-7805, Consensus Liveness and Core Properties.

Post-quantum posture

Not applicable to the goal. A concrete design depends on authentication, consensus, and any encryption it uses. Quantum-resistant payload encryption alone does not protect a consensus system whose authentication can be forged. EIP-7805, validation rules; NIST post-quantum project.

Confidence model

Mixed: reachable honest participants, consensus fault thresholds, and external timing conditions may all matter. FOCIL describes a one-honest-member inclusion-list assumption alongside consensus and propagation requirements; that is not one honest participant for the entire system. EIP-7805, Core Properties and Consensus Liveness.

Common constructions

Alternative submission routes and enforced inclusion lists address access or inclusion. Encrypted submissions address what an operator can inspect before ordering; visible sender, fee, or timing can still enable selection. Encryption and fairness therefore need separate arguments. EIP-7805; Daian et al., Flash Boys 2.0.

Use cases

Blockchain transaction admission and sequencer bypass are representative requirements. For any fallback, specify the actual path, delay, fee, and party that must process it; the label “decentralized” is insufficient. EIP-7805, Motivation.

Composition patterns

Combine with Availability, a stated ordering policy, and Auditability where exclusion evidence is feasible. Content secrecy can reduce some selection opportunities while adding decryption availability requirements. Flash Boys 2.0; EIP-7805, Security Considerations.

Failure modes and anti-patterns

> ⚠ **A PROPOSAL IS NOT A DEPLOYED GUARANTEE**
>
> As checked on **27 October 2026**, EIP-7805 is marked **Draft** and the older EIP-7547 is marked **Stagnant**. Do not cite either as evidence that a live network enforces its rules. Also distinguish intentional exclusion from delay, invalidity, or capacity limits before claiming proof of censorship. EIP-7805; EIP-7547.

Maturity and deployment

System-specific; emerging mechanisms: the inclusion-list proposals cited here are not final specifications. Assess a particular network's deployed rules, assumptions, and reachable fallback paths separately. EIP-7805; EIP-7547.

Related concepts

- Availability
- Encrypted Mempools
- Private Payments

Further reading

- EIP-7805: Fork-choice enforced Inclusion Lists - draft mechanism; read its conditional inclusion and liveness assumptions.
- EIP-7547: Inclusion lists - stagnant earlier proposal, retained as historical context.
- Daian et al., Flash Boys 2.0 (IEEE S&P 2020; preprint 2019) - transaction ordering and extraction incentives, not a proof of inclusion guarantees.

Additional Security Goals

Some goals become clearer when you ask what happens beyond an ordinary honest-user interaction: a voter cooperates with a coercer, a protocol stops making progress, or an operator changes transaction order. These remain **security goals (taxonomy level 1)**, even when evaluated for an entire system. The examples below are conceptual, not deployable protocol sketches.

Receipt-freeness

Intuition: a voter should not be able to give a buyer or coercer convincing evidence of their vote, within the election's model. Ballot secrecy hides the choice from an observer; receipt-freeness also addresses a voter who tries to demonstrate that choice. Benaloh and Tuinstra.

Minimal example: encrypting a vote hides its contents. But if the voter can disclose the encryption randomness and prove which plaintext produced the posted ciphertext, that ciphertext may serve as a receipt. Publishing encrypted ballots is therefore not enough by itself. Benaloh and Tuinstra.

Assumptions and limits: the protocol must specify what private interactions and secrets a voter can expose. A receipt-free design does not necessarily stop forced abstention, credential surrender, or continuous observation. Those belong in a stronger coercion model. Benaloh and Tuinstra; Juels, Catalano, and Jakobsson.

Coercion resistance

Intuition: a voter should have a strategy to cast a vote reflecting their own choice while making compliance with a coercer's demands indistinguishable from resistance, under a specified adversary model. The model must define what the coercer can observe and demand. Juels, Catalano, and Jakobsson.

In the Juels-Catalano-Jakobsson model, demands can include voting a particular way, abstaining, or surrendering credentials. Its construction relies on an anonymous channel for voting and a protected registration interaction; these assumptions are part of the result, not incidental conveniences. This is a research construction and model, not a general guarantee for remote voting systems. Juels, Catalano, and Jakobsson.

Goal	Question it addresses	What remains outside the claim
Ballot secrecy	Can an observer learn the vote?	A willing voter may reveal evidence.
Receipt-freeness	Can the voter convincingly demonstrate the vote?	Other forms of coercion may remain.
Coercion resistance	Can the voter resist the modeled coercer's strategy?	Attacks excluded by that model remain.

The distinction follows Benaloh and Tuinstra and Juels, Catalano, and Jakobsson. Comparisons of strength require compatible definitions and assumptions; avoid treating these labels as universal guarantees.

> **REMOTE VOTING NEEDS SYSTEM-LEVEL SCRUTINY**
>
> Neither encryption nor a zero-knowledge proof (ZKP) alone supplies receipt-freeness or coercion resistance. Registration, devices, private opportunities to act, publication rules, and the coercer's observations can determine whether the design meets its model. Implementation requires specialist evaluation. Benaloh and Tuinstra; Juels, Catalano, and Jakobsson.

Read E-Voting, Coercion-Resistant Voting, and Make Receipts Useless for the surrounding protocols and patterns.

Liveness and fair ordering

Liveness asks whether a protocol eventually makes required progress under its network and fault assumptions. It does not automatically supply a useful deadline. Castro and Liskov, §§2-3. See Availability.

Fair ordering requires an explicit rule about relative order: for example, what observations count as evidence that one transaction arrived first. Hiding payloads before ordering can reduce some information advantages; it does not itself prove a fair order or guaranteed inclusion. Daian et al.; EIP-7805, Core Properties. See Encrypted Mempools and Censorship Resistance.

Post-quantum posture

Not applicable to the goals. Their implementations depend on encryption, credentials, proofs, authentication, and operational assumptions. NIST's post-quantum standards address particular primitives; they do not establish coercion resistance or fair ordering. NIST post-quantum project; Juels, Catalano, and Jakobsson.

Confidence model and maturity

Depends on the system: a voting construction may need trusted registration, a protected channel, and an honest-party or non-collusion assumption. A live distributed protocol needs its own fault and timing model. The cited coercion-resistant construction is research work; the cited FOCIL mechanism is a draft proposal as checked on **27 October 2026**. Juels, Catalano, and Jakobsson; Castro and Liskov; EIP-7805.

Related concepts

- Security Goals - the chapter's main comparison tables.
- Deniability - limits on transferable conversational evidence.
- Verifiability, Auditability, and Accountability - checking results, reviewing records, and attributing violations.

Further reading

- Benaloh and Tuinstra, Receipt-Free Secret-Ballot Elections (STOC 1994) - the foundational receipt-freeness problem.

- Juels, Catalano, and Jakobsson, Coercion-Resistant Electronic Elections (WPES 2005) - stronger coercion demands and explicit channel assumptions.
- Castro and Liskov, Practical Byzantine Fault Tolerance (OSDI 1999) - progress versus safety under a fault model.
- Daian et al., Flash Boys 2.0 - ordering incentives and attacks.

Discrete Logarithm

Discrete Logarithm

One-sentence intuition

The discrete logarithm assumption says that exponentiation in certain finite groups is easy, but reversing it is computationally hard.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: digital signatures, Pedersen commitments, pairings, key exchange

Problem it solves

Discrete-logarithm hardness gives cryptographic schemes a one-way structure: a secret exponent can create public values that are easy to check or combine but hard to invert.

Mental model

Think of repeatedly stepping around a huge clock. If someone tells you the start point, the step rule, and the final point, finding the exact number of steps should be infeasible in the chosen group.

Minimal example

In a group generated by g, a public value may be written as:

$$Y = g^x$$

The secret is x. The discrete logarithm problem is to recover x from g and Y.

Security properties

- Supports one-way public-key structure in suitable groups.
- Underlies Diffie-Hellman-style key agreement and Schnorr-style signatures.
- Enables binding in many discrete-logarithm-based commitment schemes.

What it does not provide

- Security in every group.
- Protection against quantum computers.
- Safety against bad parameters, small subgroups, or nonce reuse.

Assumptions

The group must be chosen so that known classical attacks are infeasible at the target security level. Implementations must also validate group elements and avoid leaking or reusing secret nonces.

Post-quantum posture

Vulnerable. Shor's algorithm breaks discrete logarithms on a sufficiently large cryptographically relevant quantum computer.

Confidence model

Confidence comes from mathematical-assumption hardness, public parameter review, implementation correctness, and side-channel resistance.

Common constructions

- Diffie-Hellman and elliptic-curve Diffie-Hellman.
- Schnorr, ECDSA, and EdDSA-style signatures.
- Pedersen commitments.
- Pairing-based constructions, with additional assumptions.

Concrete groups and schemes

Concrete family	Common examples	Where it appears	Key differences and cautions
Prime-order elliptic-curve groups	Curve25519, Curve448, P-256, P-384	ECDH, EdDSA, ECDSA, Schnorr-style signatures	Quantum-vulnerable; subgroup and encoding rules are curve-specific.
Blockchain curves	secp256k1	Blockchain signatures and key recovery conventions	Quantum-vulnerable; ecosystem conventions are not interchangeable with other ECDSA settings.
Finite-field groups	FFDHE safe-prime groups	Diffie-Hellman compatibility and standards profiles	Quantum-vulnerable; use reviewed groups and validate parameters.
Discrete-log commitments	Pedersen commitments, inner-product commitments	Confidential values, range proofs, vector commitments	Binding relies on discrete-logarithm hardness and generator setup.
Discrete-log proof systems	Schnorr identification, Bulletproofs	Authentication proofs and range proofs	Fiat-Shamir transcript binding and nonce handling are critical.

Use cases

- Key agreement.
- Digital signatures.
- Commitments and range proofs.
- Pairing-based proof systems.

Composition patterns

Discrete-logarithm assumptions are often paired with transcript hashing, domain separation, nonces, and zero-knowledge proofs. The surrounding protocol must bind public keys and contexts explicitly.

Failure modes and anti-patterns

- Using weak groups or parameters.
- Failing subgroup checks.
- Reusing signature nonces.
- Treating a discrete-logarithm-based system as post-quantum.

Maturity and deployment

Mature and widely deployed, but quantum-vulnerable.

Related concepts

- Digital signatures
- Pedersen commitments
- Key encapsulation and exchange

Further reading

- Diffie and Hellman, New Directions in Cryptography.
- Shor, Algorithms for Quantum Computation; Discrete Logarithms and Factoring.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Elliptic Curves

One-sentence intuition

Elliptic curves provide finite groups where discrete-logarithm problems can support compact signatures, key exchange, commitments, and pairings.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate.
- Parent category: Discrete Logarithm.
- Related concepts: Digital Signatures, Key Encapsulation and Exchange, Pairings.

Problem it solves

Elliptic-curve groups give efficient public-key cryptography with smaller keys and signatures than many finite-field alternatives, while still relying on a well-studied discrete-logarithm hardness assumption.

Mental model

The curve is the playing field; the base point is a public direction; multiplying by a secret scalar is easy, but recovering that scalar from the resulting point should be hard.

Minimal example

In X25519, Alice and Bob exchange curve points derived from secret scalars. Each combines the other party's public point with their own scalar to obtain a shared secret that a secure-channel protocol must authenticate and bind into a transcript.

Security properties

- Efficient discrete-logarithm-based public-key operations.
- Compact signatures and key agreement.
- Optional algebraic structure for commitments, accumulators, and pairings in selected curves.

What it does not provide

- Post-quantum security.
- Safe encodings by default.
- Endpoint authentication without a protocol.
- Protection from invalid-curve, subgroup, or side-channel implementation mistakes.

Assumptions

- The chosen curve has a large prime-order subgroup or safe cofactor handling.
- Scalar multiplication, point validation, and encodings follow the scheme specification.

- Randomness and nonce handling are correct for signature schemes that require it.
- Implementations resist timing, fault, and invalid-input attacks.

Post-quantum posture

Vulnerable. Shor's algorithm breaks elliptic-curve discrete logarithms on a sufficiently capable quantum computer.

Confidence model

Confidence comes from mathematical-assumption security, public parameter review, conservative curve selection, and implementation review.

Common constructions

- X25519 and X448 key agreement.
- Ed25519 and Ed448 signatures.
- ECDSA over NIST curves or secp256k1.
- Pairing-friendly curves such as BLS12-381 and BN254.

Use cases

- TLS and Noise handshakes.
- Wallet and software signatures.
- Commitments and zero-knowledge systems.
- Threshold signing and verifiable randomness.

Composition patterns

Elliptic-curve operations must be wrapped in protocols that bind identities, suites, public keys, and transcripts. The curve operation is a substrate, not a complete channel or signature policy.

Failure modes and anti-patterns

- Invalid point or subgroup attacks.
- ECDSA nonce reuse.
- Ambiguous encodings.
- Choosing obscure or custom curves.
- Treating curve choice as the only security decision.

Maturity and deployment

Widely deployed and mature, but quantum-vulnerable. New systems should plan migration paths for public-key uses that need long-term security.

Related concepts

- Discrete Logarithm
- Pairings
- Post-Quantum Posture

Further reading

- RFC 7748: Elliptic Curves for Security.
- RFC 8032: Edwards-Curve Digital Signature Algorithm.
- NIST FIPS 186-5: Digital Signature Standard.

Factoring and RSA

One-sentence intuition

RSA-style cryptography relies on arithmetic modulo a large composite number whose prime factors are secret.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: public-key encryption, digital signatures, accumulators, VDFs

Problem it solves

RSA gives a public-key trapdoor structure: public operations can be easy while the corresponding private operation depends on knowing the factorization of the modulus.

Mental model

Multiplying two large primes is easy. Recovering those primes from only their product is believed to be hard for classical computers when the primes are generated correctly and are large enough.

Minimal example

An RSA modulus is:

$$N = pq$$

The factors p and q are secret. Many RSA-based systems rely on the difficulty of recovering them from N.

Security properties

- Supports trapdoor public-key encryption and signatures when used in safe schemes.
- Supports RSA accumulators and some time-delay constructions under additional assumptions.

What it does not provide

- Security for textbook RSA.
- Protection against quantum computers.
- Safety if primes, padding, or key sizes are wrong.

Assumptions

RSA systems assume correct prime generation, adequate modulus sizes, safe padding or signature schemes, and protection of the private exponent or factorization.

Post-quantum posture

Vulnerable. Shor's algorithm breaks integer factoring on a sufficiently large cryptographically relevant quantum computer.

Confidence model

Confidence comes from mathematical-assumption hardness, parameter generation, padding or scheme design, and private-key protection.

Common constructions

- RSA encryption schemes.
- RSA signature schemes.
- RSA accumulators.
- Groups of unknown order used in some VDF or timelock designs.

Concrete schemes and parameter families

Scheme or substrate	Common role	Key differences and cautions
RSA-OAEP	Public-key encryption and hybrid encryption	Safe padding is part of the scheme; textbook RSA is not safe encryption.
RSA-PSS	Digital signatures	Preferable to legacy deterministic RSA signatures when RSA signatures are required.
RSA PKCS #1 v1.5 encryption/signatures	Legacy compatibility	Historically fragile; should be treated as legacy context, not a modern design target.
RSA accumulators	Compact membership witnesses	Require an RSA modulus with a clear trust story; setup and update rules dominate confidence.
Unknown-order groups for delay	Repeated-squaring timelocks and VDFs	May use RSA groups or class groups; factorization or setup assumptions must be explicit.
RSA modulus sizes	2048-bit, 3072-bit, 4096-bit profiles	Classical security margin changes with size; all remain quantum-vulnerable.

Use cases

- Legacy public-key encryption and signatures.
- Accumulators.
- Some delay and timelock constructions.

Composition patterns

RSA is commonly used inside hybrid encryption, certificate systems, signatures, and accumulator protocols. Each use needs its own security notion and padding or proof layer.

Failure modes and anti-patterns

- Using textbook RSA directly.
- Reusing primes or generating weak primes.
- Confusing factoring hardness with security of every RSA-based scheme.
- Treating RSA-based systems as post-quantum.

Maturity and deployment

Mature and widely deployed historically, but quantum-vulnerable and being migrated away from in long-term security designs.

Related concepts

- Public-key encryption
- Digital signatures
- Accumulators and Merkle trees

Further reading

- Rivest, Shamir, and Adleman, A Method for Obtaining Digital Signatures and Public-Key Cryptosystems.
- Shor, Algorithms for Quantum Computation; Discrete Logarithms and Factoring.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Code-Based Assumptions

One-sentence intuition

Code-based cryptography relies on the hardness of decoding noisy error-correcting-code data without secret structure.

Where it sits in the taxonomy

- Level: mathematical assumption.
- Parent category: post-quantum public-key assumptions.
- Related concepts: Lattices, Key Encapsulation and Exchange, Post-Quantum Posture.

Problem it solves

Code-based assumptions provide a long-studied post-quantum direction for public-key encryption and key encapsulation, complementing lattice-based schemes.

Mental model

A public key describes a scrambled code. Anyone can add noise, but only the holder of the secret structure can efficiently remove enough noise to decode.

Minimal example

In a McEliece-style KEM, a sender encodes a secret with a public code and adds errors. The recipient uses a hidden decoding structure to recover the secret.

Security properties

- Public-key encryption or KEM security under decoding assumptions.
- Plausible resistance to known quantum attacks when parameters are conservative.
- Long history of cryptanalysis for some families.

What it does not provide

- Small public keys in many classic systems.
- Signatures by default.
- Security for arbitrary codes or ad hoc parameters.
- Protection from implementation side channels.

Assumptions

- Decoding random-looking linear codes remains hard at selected parameters.
- The public key does not leak exploitable structure.
- Parameter sets are chosen from reviewed schemes rather than custom code families.

Post-quantum posture

Plausible. Code-based KEMs are considered post-quantum candidates, but exact confidence depends on scheme family, parameters, and cryptanalysis.

Confidence model

Confidence comes from mathematical-assumption security, conservative parameter selection, public cryptanalysis, and implementation review.

Common constructions

- Classic McEliece-style encryption and KEMs.
- HQC-style code-based KEMs.
- Niederreiter-style variants.

Use cases

- Post-quantum key establishment.
- Diversifying away from lattice-only migration.
- Long-term confidentiality planning.

Composition patterns

Code-based KEMs are composed into secure-channel handshakes, usually with KDFs, transcript binding, authentication, and sometimes hybrid classical-plus-post-quantum key exchange.

Failure modes and anti-patterns

- Underestimating large public-key sizes.
- Using unreviewed code families.
- Ignoring side-channel and decoding-failure behavior.
- Treating "post-quantum" as deployment maturity.

Maturity and deployment

Mature but specialized. McEliece-style assumptions are old, while standardization and deployment profiles for code-based KEMs are still evolving.

Related concepts

- Key Encapsulation and Exchange
- Post-Quantum Posture
- Concrete Algorithms and Schemes

Further reading

- McEliece, "A Public-Key Cryptosystem Based on Algebraic Coding Theory".
- NIST IR 8545.
- Classic McEliece project.

Pairings

One-sentence intuition

A pairing is a special map between algebraic groups that lets hidden exponent relationships become publicly checkable.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: SNARKs, anonymous credentials, signatures, trusted setup

Problem it solves

Pairings enable compact checks of multiplicative relationships in exponents. This is useful for short signatures, identity-based encryption, polynomial commitments, and many succinct proof systems.

Mental model

A pairing acts like a structured calculator for exponent relationships: it can check that two independently formed group elements contain matching hidden exponent products without revealing those exponents.

Minimal example

At a high level, a bilinear pairing has a relation like:

$$e(g^a, h^b) = e(g, h)^{ab}$$

This bilinearity is powerful, but it comes with specialized assumptions and parameter choices.

Security properties

- Supports compact verification of algebraic relations.
- Enables succinct proof systems and short signature schemes under pairing-specific assumptions.

What it does not provide

- Post-quantum security.
- General-purpose privacy.
- Safety without careful curve and subgroup choices.

Assumptions

Pairing-based systems rely on elliptic-curve group assumptions, pairing-specific hardness assumptions, subgroup checks, and sometimes trusted setup or structured reference strings.

Post-quantum posture

Vulnerable. Common pairings are built from elliptic-curve groups whose discrete logarithm problems are broken by Shor's algorithm.

Confidence model

Confidence comes from mathematical-assumption hardness, curve selection, subgroup validation, setup correctness where required, and careful implementation.

Common constructions

- Pairing-based SNARKs.
- BLS-style signatures.
- Identity-based encryption.
- Polynomial commitments.

Concrete curves and schemes

Family or scheme	Common examples	Where it appears	Key differences and cautions
Pairing-friendly curves	BLS12-381, BN254	SNARKs, BLS signatures, KZG commitments	Quantum-vulnerable; curve security level and subgroup checks are not interchangeable.
BLS signatures	BLS signature variants over pairing-friendly curves	Aggregatable signatures, threshold signatures	Compact aggregation, but domain separation and rogue-key defenses matter.
KZG commitments	KZG polynomial commitments	Rollups, data availability, polynomial openings	Very compact openings; relies on pairings and usually a structured reference string.
Groth16-style SNARKs	Pairing-based succinct proofs	ZK circuits with very small proofs	Often circuit-specific trusted setup; posture inherits pairing vulnerability.
Identity-based encryption	Boneh-Franklin-style systems	Specialized key-management models	Key generator trust is central, not an implementation detail.

Use cases

- Succinct proof verification.
- Aggregatable signatures.
- Anonymous credential schemes.
- Verifiable shuffles and commitments in specialized protocols.

Composition patterns

Pairings are often composed with trusted setup, commitments, zero-knowledge proof statements, and public verification. The setup and subgroup model must be part of the protocol description.

Failure modes and anti-patterns

- Missing subgroup checks.
- Using weak or obsolete curves.
- Hiding trusted setup assumptions behind a "succinct proof" label.
- Treating pairing-based succinctness as post-quantum.

Maturity and deployment

Mature but specialized. Pairing-based systems are deployed, but they require expert parameter and implementation review.

Related concepts

- SNARKs, STARKs, and Bulletproofs
- Anonymous credentials
- Trusted setup

Further reading

- Boneh and Franklin, Identity-Based Encryption from the Weil Pairing.
- Groth, On the Size of Pairing-Based Non-interactive Arguments.
- Shor, Algorithms for Quantum Computation; Discrete Logarithms and Factoring.

Lattices

One-sentence intuition

Lattice-based cryptography relies on the apparent hardness of finding or distinguishing structured noisy points in high-dimensional grids.

Where it sits in the taxonomy

- Level: mathematical assumption or substrate
- Parent category: assumptions
- Related concepts: post-quantum cryptography, homomorphic encryption, key encapsulation, signatures

Problem it solves

Lattice assumptions provide a foundation for many post-quantum candidates and for advanced tools such as fully homomorphic encryption.

Mental model

Imagine a high-dimensional grid where noise slightly moves points away from clean grid locations. Some problems ask an adversary to recover hidden structure from those noisy samples.

Minimal example

Learning with errors (LWE)-style problems involve noisy linear equations. Informally:

$$b = As + e$$

The secret is s, and e is small noise. Recovering s or distinguishing these samples from random should be hard for suitable parameters.

Security properties

- Supports post-quantum key encapsulation and signatures in standardized families.
- Supports homomorphic encryption in many modern schemes.
- Can provide worst-case to average-case style security reductions in some settings.

What it does not provide

- Automatic security for every parameter set.
- Simple implementations.
- Metadata privacy or access control by itself.

Assumptions

Security depends on the concrete lattice problem, dimension, modulus, noise distribution, reduction quality, and implementation side-channel posture.

Post-quantum posture

Plausible. Lattice-based schemes are central post-quantum candidates, but each scheme and parameter set still needs concrete analysis.

Confidence model

Confidence comes from mathematical-assumption hardness, parameter selection, standardization review, and implementation discipline.

Common constructions

- Learning with errors and module-LWE schemes.
- Lattice-based key encapsulation.
- Lattice-based signatures.
- Fully homomorphic encryption schemes.

Concrete assumptions and schemes

Family or scheme	Common role	Key differences and cautions
LWE and module-LWE	Foundation for KEMs and encryption	Parameter choices control concrete security and failure behavior.
SIS and module-SIS	Foundation for signatures and commitments	Often appears in lattice signatures and proof systems.
NTRU-style lattices	Key encapsulation and encryption families	Different structure and parameter trade-offs from module-LWE families.
ML-KEM	Standardized post-quantum key encapsulation	Plausibly post-quantum; larger public keys and ciphertexts affect protocols.
ML-DSA	Standardized post-quantum signatures	Plausibly post-quantum; signature size and deterministic/randomized signing choices matter.
BFV, BGV, CKKS, TFHE/FHEW	Homomorphic encryption families	Workload fit, noise growth, bootstrapping, and approximation behavior differ sharply.

Use cases

- Post-quantum key establishment.
- Post-quantum signatures.
- Homomorphic encryption.
- Some functional-encryption research.

Composition patterns

Lattice-based components are often combined with symmetric encryption, KDFs, signatures, and protocol transcript binding. Migration designs may combine classical and post-quantum mechanisms during transition.

Failure modes and anti-patterns

- Choosing ad hoc parameters.
- Ignoring decryption-failure behavior.
- Treating all lattice assumptions as interchangeable.
- Assuming post-quantum posture removes implementation risk.

Maturity and deployment

Emerging to deployed depending on the scheme. ML-KEM and ML-DSA are standardized, while many advanced lattice tools remain specialized or research-stage.

Related concepts

- Post-quantum posture
- Homomorphic encryption
- Key encapsulation and exchange

Further reading

- Regev, On Lattices, Learning with Errors, Random Linear Codes, and Cryptography.
- NIST FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.
- Gentry, Fully Homomorphic Encryption Using Ideal Lattices.

Random Oracle Model

One-sentence intuition

The random oracle model treats a hash function as an ideal public random function inside a security proof.

Where it sits in the taxonomy

- Level: mathematical assumption or model
- Parent category: assumptions
- Related concepts: hash functions, Fiat-Shamir transforms, zero-knowledge proofs

Problem it solves

The model lets designers prove security for efficient protocols that use hash functions as if every new query returned an independent random value.

Mental model

Imagine a public notebook that answers every new input with a fresh random-looking output and always repeats the same answer for the same input.

Minimal example

A protocol proof may analyze calls to an ideal oracle:

$$y \leftarrow \mathcal{H}(x)$$

In implementation, the oracle is replaced with a concrete hash function such as SHA-2, SHA-3, or a proof-system-specific hash.

Security properties

- Supports security arguments for many practical hash-based transforms.
- Helps model Fiat-Shamir-style challenge generation.

What it does not provide

- A guarantee that any concrete hash exactly behaves like a random oracle.
- Protection against bad domain separation.
- A substitute for checking the implemented statement or protocol context.

Assumptions

The proof assumes ideal random-oracle behavior. The implementation assumes the chosen hash function and domain separation are good enough for the intended security target.

Post-quantum posture

Not applicable to the model itself. Concrete hash functions can be plausibly post-quantum with adequate output lengths, while the protocol's surrounding assumptions may not be.

Confidence model

Confidence comes from the proof model, conservative hash selection, domain separation, and awareness that the model is idealized.

Common constructions

- Fiat-Shamir transforms.
- Hash-and-sign style designs.
- Non-interactive proof systems.
- Hash-based commitments and nullifiers.

Concrete instantiations

Concrete choice	Typical role	Key differences and cautions
SHA-256 / SHA-512	General random-oracle-like hashing in protocols	Conservative deployed choices, but domain labels and transcript encodings are still required.
SHA3 and SHAKE	Random-oracle-like hashing and extendable output	Variable output from SHAKE must be treated as a parameter, not an afterthought.
Hash-to-curve suites	Mapping arbitrary strings into elliptic-curve groups	Needs standardized encodings; ad hoc mappings can bias outputs or break proofs.
Poseidon / Rescue / MiMC	Proof-system-specific hashes in circuits	Chosen for circuit efficiency; the security argument is tied to the proof-system context.
Fiat-Shamir transcripts	Challenge derivation for non-interactive proofs	Transcript encoding must bind statements, public inputs, protocol version, and domain.

Use cases

- Security proofs for practical protocols.
- Challenge derivation.
- Transcript binding.

Composition patterns

Random-oracle-model arguments often appear with zero-knowledge proofs, signatures, commitments, and protocol transcripts. The same hash function should be domain-separated across roles.

Failure modes and anti-patterns

- Treating a random-oracle proof as an implementation proof for every hash.
- Reusing transcript encodings ambiguously.
- Omitting domain labels.
- Confusing the model with a concrete primitive.

Maturity and deployment

Mature as a proof model, but idealized. It should be named explicitly when a construction relies on it.

Related concepts

- Hash functions
- Zero-knowledge proofs
- Commitments

Further reading

- Bellare and Rogaway, Random Oracles are Practical; A Paradigm for Designing Efficient Protocols.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Trusted Setup

Trusted Setup

One-sentence intuition

Trusted setup is a parameter-generation process whose hidden secrets must be destroyed or never known for the resulting system to be sound or private.

Where it sits in the taxonomy

- Level: setup or trust assumption
- Parent category: assumptions
- Related concepts: SNARKs, pairings, common reference strings, public verifiability

Problem it solves

Some efficient proof systems and commitments need structured public parameters. A setup process creates those parameters, but the security model must say what hidden setup material could break the system if retained.

Mental model

Think of forging a lock with a temporary master key. The lock can be public, but the temporary master key must be destroyed; otherwise whoever keeps it may forge openings or proofs.

Minimal example

A setup ceremony may output public parameters:

$$\mathrm{pp} \leftarrow \mathrm{Setup}(\lambda)$$

The dangerous part is any hidden trapdoor or toxic waste used to create pp.

Security properties

- Enables compact or efficient proof systems in some families.
- Can support public verification if setup assumptions hold.

What it does not provide

- Trustlessness.
- Protection if all ceremony participants collude or the trapdoor remains.
- A guarantee that the statement being proven is correct.

Assumptions

The setup ceremony, transcript, randomness contributions, and software environment must match the construction's assumptions. Multi-party ceremonies often rely on at least one honest participant destroying their secret contribution.

Post-quantum posture

Depends on the construction. Many trusted-setup systems are pairing-based and quantum-vulnerable; the setup model itself is a trust assumption rather than a post-quantum primitive.

Confidence model

Confidence comes from trusted-setup or one-honest-party ceremony assumptions, transcript auditability, implementation review, and public verification of parameters.

Common constructions

- Structured reference strings.
- Powers-of-tau style ceremonies.
- Circuit-specific or universal SNARK setup.
- Multi-party parameter-generation ceremonies.

Concrete setup patterns

Setup pattern	Common examples	Key differences and cautions
Circuit-specific setup	Groth16-style circuits	Small proofs, but every circuit may need its own setup assumptions.
Universal structured setup	Powers-of-tau, PLONK-style SRS reuse	Can support many circuits up to parameter limits; toxic waste and transcript audit remain central.
Transparent setup	STARK-style systems, many Bulletproof-style systems	Avoids trusted toxic waste, but still has parameters and implementation assumptions.
Updatable ceremonies	Multi-party SRS updates	Often rely on at least one honest contribution; update verification must be public.
Application-specific parameters	KZG commitments, accumulator parameters	Reusing parameters outside their intended scope can invalidate the confidence model.

Use cases

- Pairing-based SNARKs.
- Polynomial commitments.
- Some anonymous credential or accumulator systems.

Composition patterns

Trusted setup is commonly combined with public verifiability: anyone can verify proofs after setup, but only if the setup assumptions are accepted.

Failure modes and anti-patterns

- Hiding the setup assumption from users.
- Losing ceremony transcripts.
- Treating "multi-party setup" as equivalent to no setup.
- Reusing parameters outside their intended scope.

Maturity and deployment

Mature but specialized. Setup ceremonies are deployed in some proof-system ecosystems, but they remain a major confidence-model component.

Related concepts

- SNARKs, STARKs, and Bulletproofs
- Pairings
- Confidence models

Further reading

- Groth, On the Size of Pairing-Based Non-interactive Arguments.
- Bowe, Gabizon, and Miers, Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model.

Common Reference Strings

One-sentence intuition

A common reference string is public setup data that parties use to run or verify a cryptographic protocol.

Where it sits in the taxonomy

- Level: assumption or setup substrate.
- Parent category: Trusted Setup.
- Related concepts: Polynomial Commitments, SNARKs, STARKs, and Bulletproofs.

Problem it solves

Some proof systems and commitments need shared public parameters before proving begins. A CRS gives provers and verifiers common data, but it can also introduce setup trust.

Mental model

The CRS is the public measuring device. Everyone can use it, but if someone secretly calibrated it with a hidden trapdoor, they may be able to fake measurements.

Minimal example

A Groth16 circuit has a proving key and verification key derived from setup. If toxic waste from setup survives, an attacker may create false proofs for that circuit.

Security properties

- Shared public parameters for proving and verification.
- Sometimes universal or updatable setup across circuits.
- Sometimes transparent setup with no toxic waste, depending on the system.

What it does not provide

- Correct setup by itself.
- Toxic-waste destruction unless the ceremony guarantees it.
- Post-quantum security by default.
- Correct circuit or statement design.

Assumptions

- Setup participants followed the ceremony rules.
- At least one honest participant destroyed secret contribution in multi-party ceremonies when required.
- Verifiers use the correct CRS or verification key for the statement.

- Public parameters are authenticated and versioned.

Post-quantum posture

Depends on the proof system and commitment scheme. A transparent CRS can avoid toxic waste but may still rely on assumptions whose post-quantum posture varies.

Confidence model

Confidence may come from trusted-setup, one-honest-party setup ceremonies, public-verifiability of ceremony transcripts, or transparent parameter generation.

Common constructions

- Circuit-specific CRS.
- Universal structured reference strings.
- Updatable setup ceremonies.
- Transparent public randomness and hash-based parameters.

Use cases

- Pairing-based SNARKs.
- KZG commitments.
- Verifiable computation.
- ZK-rollup verifier keys.

Composition patterns

CRS data must be bound into proofs, verification keys, application versions, and migration procedures. A verifier should never silently accept proofs under an unexpected CRS.

Failure modes and anti-patterns

- Toxic waste retained by setup participants.
- Verifying proofs under the wrong verification key.
- Losing ceremony transcripts.
- Treating universal setup as universally safe.
- Omitting CRS version from protocol metadata.

Maturity and deployment

Mature but sensitive. Setup-dependent systems are deployed, but setup transparency and ceremony auditability remain central to confidence.

Related concepts

- Trusted Setup
- Polynomial Commitments
- Transcript Binding

Further reading

- Groth, "On the Size of Pairing-Based Non-interactive Arguments".
- Bowe, Gabizon, and Miers, "Scalable Multi-party Computation for zk-SNARK Parameters".
- Kate, Zaverucha, and Goldberg, "Constant-Size Commitments to Polynomials and Their Applications".

Additional Assumptions and Substrates

This page is now a routing overview for substrate and model concepts that appear across concrete schemes.

Classification matrix

Concept	Level	Typical role	Post-quantum posture
Elliptic curves	mathematical assumption / substrate	Groups for signatures, key agreement, commitments, pairings	vulnerable for discrete-log-based uses
Finite-field groups	mathematical assumption / substrate	Diffie-Hellman, Schnorr-style protocols, classically deployed groups	vulnerable
Code-based assumptions	mathematical assumption	Post-quantum encryption and key encapsulation candidates	plausible, scheme-dependent
Hash-to-curve	substrate / encoding method	Maps arbitrary strings into curve/group elements	depends on target group and encoding
Common reference strings	setup assumption	Public parameters for proofs, commitments, and protocols	depends on generation model
Standard model vs random oracle	proof model distinction	Explains whether a proof idealizes a hash function	not-applicable to the distinction itself

Elliptic curves as a substrate

Elliptic curves provide finite groups with compact representations and efficient operations. They instantiate ECDH, ECDSA, EdDSA, Schnorr signatures, Pedersen commitments, BLS signatures, pairings, accumulators, and many proof-system commitments.

What they provide:

- Efficient group operations.
- Compact keys and signatures compared with many finite-field alternatives.
- Mature implementation ecosystems for selected curves.

What they do not provide:

- Post-quantum security for discrete-logarithm-based schemes.
- Safe parameter generation if a curve or subgroup is ad hoc.
- Complete protocol security without validation, domain separation, and context binding.

Assumptions and failure modes:

- The relevant elliptic-curve discrete-logarithm problem must be hard.
- Implementations must validate points, handle cofactors correctly, and use constant-time scalar operations.
- Failures include invalid-curve attacks, subgroup confusion, nonce reuse in signatures, and using the wrong curve for a protocol's security target.

Finite-field groups

Finite-field groups are classic substrates for Diffie-Hellman, discrete-log assumptions, and Schnorr-style protocols.

What they provide:

- A well-studied discrete-logarithm setting.
- Standards-based safe-prime groups for interoperability.
- Conceptual simplicity for some proofs.

What they do not provide:

- Post-quantum security.
- Safety for small, custom, or trapdoored groups.
- Automatic validation of peer public values.

Assumptions and failure modes:

- The finite-field discrete-logarithm problem must be hard at the chosen size.
- Parameters must be reviewed, public values must be validated, and small-subgroup attacks must be prevented.

Code-based assumptions

Code-based cryptography relies on the hardness of decoding or related problems for error-correcting codes. It is a major post-quantum family alongside lattices, hashes, multivariate assumptions, and isogenies.

What it provides:

- A different post-quantum assumption family from lattices.
- Mature history through McEliece-style encryption.
- Diversity for migration planning.

What it does not provide:

- Small public keys by default.
- Automatic protection from implementation side channels.
- A universal replacement for signatures, proofs, or advanced primitives.

Assumptions and failure modes:

- The decoding problem must remain hard for the selected code family and parameters.
- Deployments must handle key sizes, decapsulation behavior, and constant-time implementation.
- The posture is plausible for reviewed schemes, but scheme-specific analysis matters.

Hash-to-curve

Hash-to-curve maps arbitrary strings into elliptic-curve points or group elements in a way that is suitable for cryptographic protocols.

What it provides:

- A standardized way to turn identifiers, messages, or transcript data into group elements.
- Domain separation through suite IDs and context labels.
- Avoidance of ad hoc encodings that bias outputs or fail on edge cases.

What it does not provide:

- Security for the protocol using the point.
- Protection if the wrong suite, domain tag, or curve is selected.
- Post-quantum security when the target group is quantum-vulnerable.

Assumptions and failure modes:

- The hash function and suite mapping must match the protocol's target group.
- Failures include missing domain separation, non-uniform encodings, and accepting invalid points.

Common reference strings

A common reference string (CRS) is public setup material used by a proof system, commitment scheme, or protocol.

What it provides:

- Shared parameters needed for proving, verifying, committing, or encrypting.
- Sometimes succinctness or efficiency unavailable in transparent settings.
- A fixed public context for all participants.

What it does not provide:

- Honest generation by itself.
- Toxic-waste destruction.
- Public verifiability unless the setup process or update ceremony is designed for it.

Assumptions and failure modes:

- The CRS generation model must be explicit: trusted, updatable, transparent, or one-honest-party.
- Failures include hidden trapdoors, biased parameters, mismatched CRS files, and using a CRS for the wrong circuit or domain.

Standard model vs random oracle distinctions

The standard model analyzes protocols without idealizing hash functions as perfect random oracles. The random oracle model treats a hash function as an ideal public random function for proof purposes.

What it provides:

- A way to compare proof assumptions.
- A warning that "proved secure" depends on the model.
- Vocabulary for Fiat-Shamir-style transformations and hash-based protocol proofs.

What it does not provide:

- A guarantee that a real hash function exactly behaves like the ideal model.
- A simple ranking where every standard-model construction is always better in practice.
- Protection from implementation or composition mistakes.

Assumptions and failure modes:

- Random-oracle proofs rely on the heuristic that the concrete hash behaves well enough for the use.
- Standard-model constructions may require stronger setup, larger parameters, or more complex assumptions.

Confidence model

These concepts have different confidence models: mathematical-assumption for curve, finite-field, and code-based hardness; trusted-setup or public-verifiability for common reference strings; and model-assumption for random-oracle distinctions.

Related concepts

- Discrete Logarithm
- Elliptic Curves
- Code-Based Assumptions
- Pairings
- Lattices
- Random Oracle Model
- Trusted Setup
- Common Reference Strings

Further reading

- RFC 7748, "Elliptic Curves for Security".
- RFC 7919, "Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for TLS".
- RFC 9380, "Hashing to Elliptic Curves".
- Bellare and Rogaway, "Random Oracles are Practical".

- NIST, "Post-Quantum Cryptography Project".

Basic Primitives

Basic Primitives Overview

Basic primitives provide narrow cryptographic guarantees. They are not complete systems.

Examples

- Hash functions compress data into fixed-length digests.
- Symmetric encryption protects bulk data under a shared secret key.
- Authenticated encryption combines confidentiality with ciphertext integrity and authenticated public context.
- Message authentication codes authenticate messages with a shared secret key.
- Key derivation functions separate keys by context.
- Password hashing slows offline guessing of low-entropy secrets.
- Randomness and nonces provide freshness or uniqueness where schemes require it.
- Commitments bind a party to a hidden value.
- Digital signatures authenticate messages.
- Public-key encryption lets a sender encrypt to a recipient's public key.
- Key encapsulation and exchange establish shared secret material.
- Key-committing encryption binds ciphertext validity to the intended key where ambiguity matters.
- Secret sharing splits a secret across multiple shares.

Reading rule

For each primitive, ask what it guarantees, what it assumes, what it does not provide, and what breaks when it is composed incorrectly.

Coverage note

This section covers the main symmetric-key and public-key primitives that most applied systems rely on. See Password Hashing, Key-Committing Encryption, and Primitive Engineering Concepts for PRFs, PRPs, and nonce-misuse-resistant encryption.

Primitive Engineering Concepts

This page is now a routing overview for recurring engineering risks around keys, passwords, nonces, and symmetric-key abstractions.

Classification matrix

Concept	Level	Typical role	Maturity
Pseudorandom functions (PRFs)	basic primitive	Keyed deterministic function that looks random without the key	mature
Pseudorandom permutations (PRPs)	basic primitive	Keyed reversible permutation, often block-cipher-like	mature
Password hashing	basic primitive / storage pattern	Slows offline guessing of low-entropy secrets	widely deployed
Nonce-misuse-resistant encryption	basic primitive / scheme family	Reduces damage from accidental nonce reuse	mature but specialized
Key-committing encryption	basic primitive / scheme property	Binds ciphertext validity to one key or key commitment	emerging

Pseudorandom functions

A pseudorandom function is a keyed deterministic function whose outputs should be indistinguishable from random to parties that do not know the key.

Security properties:

- Pseudorandom output under a secret key.
- Deterministic repeatability for the same key and input.
- Keyed domain separation when used through labeled inputs or KDF contexts.

What it does not provide:

- Public verifiability.
- Confidentiality of inputs.
- Safe password storage by itself.
- Protection if one key is reused across unrelated domains.

Assumptions and failure modes:

- The PRF construction must remain secure and the key must remain secret.
- Failures include missing domain separation, using raw hashes as PRFs, and exposing outputs for low-entropy inputs without rate limits.

Pseudorandom permutations

A pseudorandom permutation is a keyed, invertible permutation that should look random to parties without the key. Block ciphers such as AES are commonly modeled as PRPs or strong PRPs in many designs.

Security properties:

- Reversible keyed transformation on fixed-size blocks.
- Pseudorandom behavior under chosen-input models, depending on the construction.
- Useful substrate for modes of operation.

What it does not provide:

- Variable-length encryption by itself.
- Authentication.
- Safe handling of repeated blocks without a mode.

Assumptions and failure modes:

- The block cipher or permutation must be used through a reviewed mode.
- Failures include using ECB mode, designing custom modes, and ignoring block-size limits.

Password hashing

Password hashing stores a verifier for a low-entropy secret in a way that makes offline guessing more expensive.

Security properties:

- Per-password salt prevents shared precomputation.
- Work factors slow guessing.
- Memory-hard designs raise the cost of parallel hardware attacks.

What it does not provide:

- High entropy if the password is weak.
- Protection after a successful online guess.
- Password-authenticated key exchange by itself.
- Recovery or revocation policy.

Assumptions and failure modes:

- Parameters must match current hardware and risk tolerance.
- Failures include unsalted hashes, fast hashes such as raw SHA-256 for password storage, weak reset flows, and logging passwords before hashing.

Nonce-misuse-resistant encryption

Nonce-misuse-resistant encryption reduces the damage caused by accidental nonce reuse. Synthetic-IV constructions such as AES-SIV and AES-GCM-SIV are common examples.

Security properties:

- Confidentiality and integrity with less catastrophic behavior under repeated nonces.
- Deterministic or synthetic-IV operation depending on the scheme.
- Associated-data binding when the scheme supports AEAD.

What it does not provide:

- A license to ignore nonce design.
- Protection from key compromise.
- Hiding of equality when deterministic encryption is used on repeated plaintext/context pairs.

Assumptions and failure modes:

- The construction must be a reviewed misuse-resistant scheme, not an ad hoc patch around a nonce-based mode.
- Failures include confusing misuse resistance with full misuse immunity and leaking repeated plaintext equality.

Key-committing encryption

Key-committing encryption makes it hard for a ciphertext to decrypt or authenticate successfully under more than one key. This matters in group messaging, abuse reporting, and systems where ciphertexts may be presented under disputed keys.

Security properties:

- Ciphertext validity is bound to a key or key commitment.
- Reduces ambiguity about which key produced or decrypts a ciphertext.
- Can support accountability or message-franking designs.

What it does not provide:

- Public attribution by itself.
- Sender identity without authentication.
- Deniability unless the surrounding protocol is designed for it.
- Automatic safety for all AEAD schemes.

Assumptions and failure modes:

- The committing property must be proved for the concrete encryption construction.
- Failures include using a non-committing AEAD where key ambiguity matters, omitting associated-data context, and treating key commitment as a signature.

Post-quantum posture

PRFs, PRPs, password hashing, and misuse-resistant symmetric encryption are generally plausible with conservative parameters. Key-committing encryption inherits posture from the underlying symmetric scheme and any public-key or signature layer around it.

Confidence model

Confidence comes from mathematical-assumption or symmetric-key security, parameter selection, implementation review, and correct operational handling of keys, salts, nonces, and associated data.

Related concepts

- Hash Functions
- Authenticated Encryption
- Key Derivation Functions
- Password Hashing
- Key-Committing Encryption
- Randomness and Nonces
- Secure Channels

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography".
- RFC 9106, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications".
- RFC 8452, "AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption".
- Rogaway and Shrimpton, "Deterministic Authenticated-Encryption".
- Grubbs et al., "Message Franking via Committing Authenticated Encryption".

Hash Functions

One-sentence intuition

A cryptographic hash function maps data to a fixed-length digest in a way that should resist finding collisions or reversing the digest.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: primitive
- Related concepts: Commitments, Accumulators and Merkle Trees, Nullifiers

Problem it solves

Systems often need a compact, deterministic representation of data for identifiers, commitments, integrity checks, and transcript binding. Cryptographic hash functions are useful when that representation must resist preimage, second-preimage, or collision attacks.

Mental model

Think of a stable fingerprint for data: easy to compute, but hard to reverse or intentionally duplicate.

Minimal example

A file download page publishes SHA-256(file). A user hashes the downloaded file and checks that the digest matches.

Security properties

- Preimage resistance.
- Second-preimage resistance.
- Collision resistance.

What it does not provide

- Encryption.
- Authentication unless used in a construction such as a MAC.
- Hiding for low-entropy secrets without careful salting or keying.

Assumptions

The chosen hash function must be within its intended security lifetime, outputs must be long enough for the security target, and protocols must use clear domain separation when the same function is reused in different roles.

Post-quantum posture

Plausible with appropriate output lengths and parameters. Hash functions are often used as post-quantum building blocks, but the security target must account for quantum search speedups.

Confidence model

Confidence comes from public algorithm scrutiny, parameter choice, domain separation, and correct use. Hashing a low-entropy secret is not enough to make it hidden.

Common constructions

Concrete algorithms and schemes

Family	Examples	Where readers usually see it	Key differences and cautions
SHA-2	SHA-256, SHA-384, SHA-512	General-purpose hashing, signatures, Merkle trees, protocol transcripts	Conservative deployed default; choose output length for the security target and domain-separate protocol roles.
SHA-3 and SHAKE	SHA3-256, SHA3-512, SHAKE128, SHAKE256	Hashing, extendable-output functions, post-quantum schemes	Sponge-based design; SHAKE outputs variable length, so the output length is a security parameter.
BLAKE family	BLAKE2, BLAKE3	File integrity, application protocols, high-throughput hashing	Fast and widely used in software; check whether the exact variant is standardized or ecosystem-specific.
ZK-friendly hashes	Poseidon, Rescue, MiMC, Griffin	Zero-knowledge circuits and proof systems	Optimized for arithmetic circuits, not a drop-in replacement for general-purpose hashing unless the full protocol expects that choice.
Legacy or broken hashes	SHA-1, MD5	Old protocols, compatibility checks, forensic context	Collision resistance is broken or deprecated; include only to explain legacy risk, not for new designs.

Use cases

- Commitments.
- Merkle trees.
- Content addressing.
- Transcript binding in protocols.

Composition patterns

- Hashes of small secrets can be brute-forced.
- Reusing the same hash input format across contexts can create ambiguity.

Common adjacent concepts: Commitments, Accumulators and Merkle Trees, Nullifiers.

Failure modes and anti-patterns

- Hashing passwords without a password-hashing scheme.
- Treating a hash of a small secret as hidden.
- Using obsolete or broken hash functions.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: medium. Parameter sensitivity: medium.

Related concepts

- Commitments
- Accumulators and Merkle Trees
- Nullifiers

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Grover, A Fast Quantum Mechanical Algorithm for Database Search.
- NIST FIPS 180-4, Secure Hash Standard.
- NIST FIPS 202, SHA-3 Standard.

Symmetric Encryption

Symmetric Encryption

One-sentence intuition

Symmetric encryption uses the same secret key to encrypt and decrypt data.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: encryption
- Related concepts: Message Authentication Codes, Key Derivation Functions, Public-Key Encryption

Problem it solves

Many systems need to store or transmit data so only parties with the same secret can read it. Symmetric encryption turns confidentiality into a key-management problem: protect the key, choose safe modes, and avoid reusing dangerous state.

Mental model

Think of a locked box where the same secret key locks and unlocks the contents.

Minimal example

A backup tool encrypts a local archive with a secret key before uploading it to storage.

Security properties

- Confidentiality under the chosen attack model.
- Efficient protection for bulk data when used through a safe mode or authenticated-encryption construction.

For most new protocol designs, the safer interface is authenticated encryption, not bare encryption; the AEAD interface is standardized in RFC 5116.

What it does not provide

- Key agreement or key distribution.
- Authentication unless combined with a message authentication code or authenticated encryption.
- Metadata privacy for sizes, timing, or access patterns.

Assumptions

The key must remain secret, nonces or initialization vectors must follow the scheme rules, and the encryption mode must match the threat model. Raw block ciphers should not be used as complete encryption schemes.

Post-quantum posture

Plausible with appropriate key sizes and conservative parameters. Quantum search affects security margins, so symmetric-key migration often increases key sizes rather than replacing the primitive family (Grover 1996).

Confidence model

Confidence comes from secret-key control, public scrutiny of the algorithm, correct nonce handling, and authenticated use when active attackers can modify ciphertexts.

Common constructions

Concrete algorithms and schemes

Scheme or mode	Primitive family	Typical role	Key differences and cautions
AES-GCM	AES block cipher plus Galois/Counter Mode	Authenticated encryption for network protocols and storage	Very common and hardware-accelerated; nonce reuse is catastrophic under the GCM security requirements in NIST SP 800-38D.
ChaCha20-Poly1305	Stream cipher plus MAC	Authenticated encryption in software and mobile environments	Often faster without AES hardware; nonces must still be unique per key as specified in RFC 8439.
XChaCha20-Poly1305	Extended-nonce ChaCha20-Poly1305 variant	Applications that want random nonces with a larger nonce space	Useful engineering shape, but check ecosystem support and protocol compatibility.
AES-GCM-SIV / AES-SIV	Misuse-resistant authenticated encryption	Systems where accidental nonce reuse is a realistic risk	More forgiving of nonce mistakes, but not a license to ignore nonce design.
AES-CBC plus MAC	Legacy composition	Older protocols and compatibility layers	Only safe with correct encrypt-then-MAC composition and padding handling; avoid for new designs when AEAD is available.
AES-ECB	Raw block-cipher mode	Legacy anti-pattern	Reveals repeated plaintext blocks and should not be used for protecting structured data.

Use cases

- Bulk data encryption.
- Encrypted backups.
- The data-encryption part of hybrid encryption.

Composition patterns

- Symmetric encryption often needs authenticated encryption.
- Key establishment must be handled separately.

Common adjacent concepts: Message Authentication Codes, Key Derivation Functions, Public-Key Encryption.

Failure modes and anti-patterns

- Reusing nonces in modes that require uniqueness.
- Using encryption without authentication, despite the AEAD interface in RFC 5116 being the safer default for active-attack settings.
- Designing a custom mode around a block cipher or stream cipher.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: medium.

Related concepts

- Message Authentication Codes
- Key Derivation Functions
- Public-Key Encryption

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.
- NIST FIPS 197, Advanced Encryption Standard.
- RFC 5116, An Interface and Algorithms for Authenticated Encryption.
- RFC 8439, ChaCha20 and Poly1305 for IETF Protocols.

Authenticated Encryption

Authenticated Encryption

One-sentence intuition

Authenticated encryption protects a message's confidentiality and lets the receiver reject modified ciphertexts.

Where it sits in the taxonomy

- Level: basic primitive.
- Parent category: symmetric cryptography.
- Related concepts: Symmetric encryption, Message authentication codes, Randomness and nonces.

Problem it solves

Many systems need both secrecy and tamper detection. Encrypting without authentication can allow an attacker to modify ciphertexts and learn from error behavior, while authenticating the plaintext separately can fail if the composition order, keys, or associated context are wrong; AEAD was standardized as a single interface for this combined goal in RFC 5116.

Authenticated encryption gives applications a single interface for "encrypt this plaintext, authenticate this public context, and reject invalid ciphertexts."

Mental model

Think of a sealed envelope with a tamper-evident seal. The inside of the envelope is hidden. The label on the outside may remain visible, but the seal binds that label to the hidden contents so an attacker cannot swap either one without detection.

Minimal example

A protocol sends:

- plaintext: `transfer 10 tokens`;
- associated data: `chain_id=1, protocol=v2, sender=alice`;
- nonce: a value that must be unique for the key.

The receiver decrypts only if the ciphertext, tag, associated data, key, and nonce all verify together. If the associated data changes to `chain_id=2`, verification fails even though the associated data was never encrypted.

Security properties

- Confidentiality of the plaintext under the scheme's chosen security notion.

- Integrity of the ciphertext and plaintext.
- Authenticity under the symmetric key: only someone with the key should produce ciphertexts that verify.
- Associated-data integrity: public context is authenticated even though it is not encrypted.

What it does not provide

- Non-repudiation or public verifiability.
- Sender identity when several parties share the same key.
- Protection from nonce reuse unless the specific scheme is misuse-resistant.
- Hiding of associated data, message length, timing, or traffic patterns.
- Post-compromise protection after the symmetric key is exposed.

Assumptions

The key must remain secret, nonces must satisfy the scheme's exact uniqueness or randomness rule, associated data must include the full protocol context that needs binding, and implementations must reject invalid tags before releasing plaintext.

Post-quantum posture

Plausible when instantiated with symmetric-key algorithms using appropriate key and tag lengths. Quantum search reduces effective brute-force margins, so parameter choices still matter (Grover 1996). The surrounding key-establishment or signature layer may be quantum-vulnerable even if the authenticated-encryption algorithm is not.

Confidence model

Confidence comes from the symmetric-key assumption, correct nonce management, unambiguous associated-data encoding, and implementations that do not leak plaintext or tag-check behavior through side channels.

Common constructions

Construction	Typical use	Caution
AES-GCM	Network protocols and hardware-accelerated platforms	Catastrophic nonce reuse under one key; NIST specifies GCM requirements and nonce handling in SP 800-38D.
ChaCha20-Poly1305	Software-oriented secure channels	Nonce uniqueness is still required by the IETF construction in RFC 8439.
XChaCha20-Poly1305	Systems that want larger random nonces	Widely used, but not every protocol registry includes it.
AES-GCM-SIV / AES-SIV	Misuse-resistant encryption	More forgiving of nonce mistakes, but still has limits and different performance.
AES-CCM	Constrained and wireless protocols	Nonce formatting and length choices are easy to get wrong.

Use cases

- TLS record protection.
- Encrypted application messages.
- File and backup encryption.
- Token sealing.
- Encrypted database fields where context must be bound as associated data.

Composition patterns

Authenticated encryption is commonly fed by a key derivation function, uses randomness and nonces, and appears inside secure-channel protocols. It often binds transcript hashes, version numbers, identities, or domain separators as associated data.

Failure modes and anti-patterns

- Reusing a nonce with AES-GCM or ChaCha20-Poly1305 under the same key.
- Decrypting or parsing plaintext before tag verification succeeds.
- Leaving protocol version, sender, recipient, or purpose outside associated data.
- Mixing encrypted values from different protocols because encodings are not domain-separated.
- Treating authenticated encryption as a signature.
- Using AES-CBC or AES-CTR without a correct authentication layer.

Maturity and deployment

Widely deployed. Authenticated encryption with associated data (AEAD) is the standard interface for modern combined confidentiality and integrity designs (RFC 5116), but incorrect nonce handling remains a common implementation risk.

Related concepts

- Symmetric encryption
- Message authentication codes
- Randomness and nonces
- Key derivation functions
- Metadata leakage

Further reading

- RFC 5116, An Interface and Algorithms for Authenticated Encryption.
- RFC 8439, ChaCha20 and Poly1305 for IETF Protocols.
- Rogaway, Authenticated-Encryption with Associated-Data.
- NIST SP 800-38D, Recommendation for Block Cipher Modes of Operation: GCM and GMAC.

Key-Committing Encryption

One-sentence intuition

Key-committing encryption makes it hard for one ciphertext to validate under multiple different keys.

Where it sits in the taxonomy

- Level: basic primitive or scheme property.
- Parent category: Authenticated Encryption.
- Related concepts: Secure Messaging, Transcript Binding.

Problem it solves

Some systems need to know which key a ciphertext is valid under. Ordinary authenticated encryption may allow ambiguous ciphertexts in unusual multi-key settings, which can matter for abuse reporting, message franking, or disputed-key workflows.

Mental model

The ciphertext carries a cryptographic fingerprint of the key relationship. A valid opening should commit to one key context rather than being plausibly valid under several.

Minimal example

A messaging system encrypts a reportable message. Later, a moderation flow needs evidence that the ciphertext corresponds to the reported key context, not a different key chosen after the fact.

Security properties

- Ciphertext validity is bound to a key or key commitment.
- Reduces key ambiguity in multi-key systems.
- Can support accountability or message-franking workflows when composed carefully.

What it does not provide

- Public attribution by itself.
- Sender identity without authentication.
- Deniability unless the full protocol is designed for it.
- Safety for arbitrary AEAD schemes.

Assumptions

- The concrete encryption construction has a proved committing property.
- Associated data binds the recipient, protocol, version, and message context.
- Key generation and key identifiers are not attacker-confusable.

Post-quantum posture

Depends on the underlying encryption, authentication, and any public-key layer around it. Symmetric-only committing designs can be plausible with conservative parameters.

Confidence model

Confidence comes from mathematical-assumption or symmetric-key security, proof of the committing property, and careful context binding.

Common constructions

- Committing authenticated encryption.
- Message franking constructions.
- Deterministic authenticated-encryption components in limited contexts.

Use cases

- Secure messaging abuse reporting.
- Multi-recipient encryption with disputed keys.
- Systems where ciphertext/key ambiguity is a security problem.

Composition patterns

Key-committing encryption is often combined with transcript binding, sender authentication, audit logs, or moderation workflows. It should not be swapped into a protocol without checking deniability and privacy goals.

Failure modes and anti-patterns

- Treating ordinary AEAD as key-committing.
- Omitting associated-data context.
- Confusing key commitment with a digital signature.
- Breaking deniability expectations in messaging systems.

Maturity and deployment

Emerging. The motivation is deployed in secure-messaging designs, but exact scheme properties are specialized and require expert review.

Related concepts

- Authenticated Encryption
- Secure Messaging
- Transcript Binding

Further reading

- Grubbs et al., "Message Franking via Committing Authenticated Encryption".
- Rogaway and Shrimpton, "Deterministic Authenticated-Encryption".
- RFC 8452: AES-GCM-SIV.

Message Authentication Codes

One-sentence intuition

A message authentication code (MAC) lets parties sharing a secret key detect forged or modified messages.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: authentication
- Related concepts: Symmetric Encryption, Key Derivation Functions, Digital Signature

Problem it solves

When two parties share a secret, they need a cheap way to reject messages that were modified or forged in transit. A message authentication code (MAC) attaches an authenticity tag that only someone with the key should be able to produce.

Mental model

Think of a tamper-evident tag that only people with the shared secret can create or verify.

Minimal example

An API request carries HMAC(key, method || path || body); the server recomputes it before accepting the request.

Security properties

- Message integrity.
- Symmetric-key authenticity between parties that share the key.

What it does not provide

- Public verifiability.
- Non-repudiation.
- Confidentiality.
- Protection if all verifiers share the same signing key and one is compromised.

Assumptions

The MAC key must remain secret, message encoding must be unambiguous, and verifiers must bind the tag to the right protocol context and replay rules.

Post-quantum posture

Plausible for standard MAC constructions with appropriate key and tag sizes. The surrounding key-exchange or provisioning layer may still be quantum-vulnerable.

Confidence model

Confidence comes from symmetric key secrecy, unforgeability of the MAC construction, and replay protection in the surrounding protocol.

Common constructions

Concrete algorithms and schemes

Scheme	Built from	Typical role	Key differences and cautions
HMAC-SHA-256 / HMAC-SHA-512	Hash function	General-purpose MAC and KDF component	Mature and conservative; key separation and unambiguous message encoding still matter.
CMAC-AES	AES block cipher	MACs in AES-centered systems	Avoids naive CBC-MAC pitfalls when used as specified.
GMAC	GCM authentication component	Authentication when AES-GCM infrastructure is already present	Requires nonce discipline; misuse can break authenticity.
Poly1305	One-time MAC, commonly paired with ChaCha20	AEAD internals such as ChaCha20-Poly1305	Requires one-time keys derived correctly; do not reuse Poly1305 keys directly.
KMAC	SHA-3/cSHAKE family	SHA-3-based keyed hashing	Useful where SHA-3 primitives are already part of the design.
CBC-MAC	Block cipher	Narrow legacy setting	Unsafe when copied outside its fixed-length, single-key assumptions.

Use cases

- Authenticated APIs.
- Protocol transcript authentication.
- Authenticated encryption internals.

When confidentiality and integrity are both required for ciphertexts, prefer a reviewed authenticated-encryption scheme over designing a custom encryption-plus-MAC composition.

Composition patterns

- MAC keys should be separated by role and protocol.
- A MAC is not a digital signature.

Common adjacent concepts: Symmetric Encryption, Key Derivation Functions, Digital Signature.

Failure modes and anti-patterns

- Reusing one MAC key across unrelated protocols without domain separation.
- Comparing tags with timing leaks.
- Treating a MAC as a digital signature that third parties can verify.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: medium. Parameter sensitivity: medium.

Related concepts

- Symmetric Encryption
- Key Derivation Functions
- Digital Signature

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.
- NIST SP 800-38B, Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication.

Key Derivation Functions

Key Derivation Functions

One-sentence intuition

A key derivation function (KDF) turns shared secret material into context-specific cryptographic keys.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: key-management
- Related concepts: Symmetric Encryption, Key Encapsulation and Exchange, Message Authentication Codes

Problem it solves

Raw shared secrets are rarely safe to use directly as application keys. Key derivation functions (KDFs) extract and expand secret material into separate context-bound keys so different purposes do not accidentally share the same cryptographic state.

Mental model

Think of a key schedule that turns one master secret into separate labeled keys for separate jobs.

Minimal example

TLS derives separate client-write and server-write traffic keys from one handshake secret and transcript labels.

Security properties

- Key separation between contexts.
- Pseudorandom derived keys when the input secret has enough entropy.
- Binding to protocol labels, salts, and transcript context when used correctly.

What it does not provide

- Entropy that was not present in the input.
- Password hardening unless the KDF is specifically designed for passwords.
- Agreement that the input secret is authentic.

Assumptions

The input secret must have adequate entropy for the use case, salts and labels must be chosen correctly, and each derived key must have a clearly separated purpose.

Post-quantum posture

Plausible for hash-based KDFs with appropriate parameters. The posture of the key-establishment step that produced the input secret must be classified separately.

Confidence model

Confidence comes from entropy in the input material, domain separation in the KDF inputs, and disciplined key lifecycle management.

Common constructions

Concrete algorithms and schemes

Scheme	Main input shape	Typical role	Key differences and cautions
HKDF	High-entropy shared secret plus salt and info labels	Session key schedules and protocol key separation	Fast KDF; not a password-hashing scheme.
PBKDF2	Password plus salt and iteration count	Legacy password-based key derivation	Widely deployed, but easier to accelerate than modern memory-hard schemes.
scrypt	Password plus salt and memory/cost parameters	Password hashing and key derivation	Adds memory cost; parameters must match attacker hardware assumptions.
Argon2id	Password plus salt and memory/time/parallelism parameters	Password storage and password-derived keys	Modern memory-hard default when available; parameters need operational tuning.
NIST counter-mode KDFs	Shared secret plus labels and context	Key management in NIST-profiled systems	Good for structured key derivation when labels and context are explicit.

Use cases

- Deriving encryption and MAC keys from a shared secret.
- Session-key schedules.
- Context-separated keys for protocols.

Composition patterns

- KDF context must bind protocol transcripts and algorithm choices.
- Passwords need password-hashing schemes, not only fast KDFs.

Common adjacent concepts: Symmetric Encryption, Key Encapsulation and Exchange, Message Authentication Codes.

Failure modes and anti-patterns

- Reusing one derived key for multiple purposes.

- Feeding low-entropy passwords into a fast KDF.
- Omitting transcript or domain labels, which can create cross-protocol key reuse.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: medium. Parameter sensitivity: medium.

Related concepts

- Symmetric Encryption
- Key Encapsulation and Exchange
- Message Authentication Codes

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.
- RFC 5869, HMAC-based Extract-and-Expand Key Derivation Function.

Password Hashing

One-sentence intuition

Password hashing stores a salted, deliberately expensive verifier so stolen password databases are harder to brute-force offline.

Where it sits in the taxonomy

- Level: basic primitive and storage pattern.
- Parent category: Key Derivation Functions.
- Related concepts: Password-Authenticated Key Exchange, Randomness and Nonces.

Problem it solves

Passwords are usually low entropy. If a database stores raw passwords or fast hashes, an attacker who steals it can test guesses cheaply. Password hashing raises the cost of each guess.

Mental model

The salt prevents attackers from reusing one giant lookup table. The work factor makes every guess expensive. Memory hardness makes specialized parallel hardware less attractive.

Minimal example

A server stores `Argon2id(password, salt, parameters)` plus the salt and parameters. During login, it recomputes the verifier and compares it in constant time.

Security properties

- Per-password salts defeat shared precomputation.
- Work and memory factors slow offline guessing.
- Parameter metadata supports future upgrades.

What it does not provide

- High entropy for weak passwords.
- Protection against online guessing without rate limits.
- Password-authenticated key exchange.
- Safe reset or recovery flows.
- Protection if plaintext passwords are logged before hashing.

Assumptions

- Salt generation is random and unique enough.
- Parameters match current hardware and risk tolerance.

- The chosen function is designed for password storage.
- Online guessing and reset flows are separately controlled.

Post-quantum posture

Plausible with conservative parameters. Quantum search can reduce brute-force margins, but password strength, rate limits, and work factors dominate practical security.

Confidence model

Confidence depends on client-side-secret quality, server-side verifier handling, parameter selection, and operational controls around login and recovery.

Common constructions

- Argon2id.
- scrypt.
- PBKDF2, mainly for legacy or compatibility contexts.
- bcrypt, mainly for legacy deployments with parameter caveats.

Use cases

- Password verifier storage.
- Password-derived local encryption keys with careful UX.
- Migration from legacy password databases.

Composition patterns

Password hashing is paired with rate limits, breach monitoring, multi-factor authentication, and parameter migration. It is distinct from PAKE, which changes the login protocol rather than only the stored verifier.

Failure modes and anti-patterns

- Raw SHA-256 or unsalted fast hashes.
- Global salts reused for every user.
- Parameters never upgraded.
- Passwords logged before hashing.
- Reset flows weaker than the password verifier.

Maturity and deployment

Widely deployed, but often misconfigured. Mature deployments periodically review parameters and recovery policy.

Related concepts

- Key Derivation Functions
- Password-Authenticated Key Exchange
- Secure Channels

Further reading

- RFC 9106: Argon2.
- OWASP Password Storage Cheat Sheet.
- Boneh and Shoup, "A Graduate Course in Applied Cryptography".

Randomness and Nonces

One-sentence intuition

Randomness and nonces provide the fresh or unique values that many cryptographic schemes need to stay secure.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: implementation-substrate
- Related concepts: Digital Signature, Commitments, Symmetric Encryption

Problem it solves

Many cryptographic schemes break when they reuse supposedly fresh values or rely on predictable randomness. Randomness and nonces provide uniqueness or unpredictability so keys, signatures, encryption modes, and protocols do not repeat unsafe internal state.

Mental model

Think of freshness labels and unpredictable draws that keep repeated cryptographic operations from colliding.

Minimal example

AES-GCM encrypts several records under one key, and each record uses a distinct nonce so authentication and confidentiality do not collapse.

Security properties

- Unpredictability for secrets, keys, salts, and some protocol challenges.
- Uniqueness for nonces in schemes that require no reuse.

What it does not provide

- Security if the surrounding scheme is misused.
- Confidentiality or authentication by itself.
- A substitute for domain separation.

Assumptions

Random values must come from an appropriate random-number generator, nonces must satisfy the exact uniqueness or unpredictability rule of the scheme, and failures must be detectable where possible.

Post-quantum posture

Not applicable as a standalone category. Randomness quality matters equally in classical and post-quantum systems.

Confidence model

Confidence depends on local entropy sources, deterministic derivation where appropriate, implementation checks, and operational monitoring for reuse or generator failure.

Common constructions

Concrete generators and nonce patterns

Mechanism or pattern	Typical role	Key differences and cautions
Operating-system CSPRNGs	General key, salt, nonce, and challenge generation	Usually the right source for application randomness; failures often come from bypassing it.
Hash_DRBG / HMAC_DRBG / CTR_DRBG	Deterministic random bit generators	Common in standards-oriented libraries and hardware modules; require correct seeding and reseeding.
ChaCha20-based CSPRNGs	Fast software randomness expansion	Common in operating systems and libraries; security depends on seed handling and state protection.
Counter nonces	Unique nonces for one key and one stream of messages	Good when state is reliable; dangerous if state rolls back or keys are reused.
Random nonces	Large nonce spaces where collision probability is negligible	Requires enough nonce bits; small random nonces can collide under load.
Synthetic IV / deterministic nonce designs	Misuse-resistant encryption modes and deterministic signatures	Reduces reliance on external randomness, but only within schemes designed for that model.

Use cases

- Encryption nonces.
- Signature nonces.
- Commitment randomness.
- Protocol challenges.

Composition patterns

- Nonce rules differ by scheme and cannot be guessed.
- Randomness failures can break otherwise sound primitives.

Common adjacent concepts: Digital Signature, Commitments, Symmetric Encryption.

Failure modes and anti-patterns

- Reusing a signature nonce in schemes where it exposes the private key.

- Reusing encryption nonces in modes that require uniqueness.
- Treating predictable identifiers as random values.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: high.

Related concepts

- Digital Signature
- Commitments
- Symmetric Encryption

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Katz and Lindell, Introduction to Modern Cryptography.

Commitments

Commitments

One-sentence intuition

A commitment lets someone lock in a value now and reveal it later.

Where it sits in the taxonomy

- Level: basic primitive
- Parent category: primitives
- Related concepts: hash functions, Pedersen commitments, zero-knowledge proofs

Problem it solves

Commitments solve the problem of delayed disclosure. A party can choose a value, publish evidence that the value has been fixed, and later open the commitment so others can check that the value did not change.

Mental model

Think of placing a message in a locked box and publishing the box. Later, the sender opens the box. A good commitment hides the message while the box is locked and prevents the sender from opening the same box to a different message.

Minimal example

A simple hash commitment can look like:

$$c = H(m \,\|\, r)$$

The opening is the pair (m, r).

The verifier recomputes the hash during opening and checks that it matches the published commitment.

More abstractly, a commitment scheme has two operations:

$$c \leftarrow \mathrm{Commit}(m; r)$$

$$\mathrm{Open}(c, m, r) \in \{\mathrm{accept}, \mathrm{reject}\}$$

Security properties

- Hiding: the commitment should not reveal the committed value before opening.
- Binding: the committer should not be able to open the same commitment to two different values.

Some schemes are computationally hiding or binding; others are perfectly hiding or binding under their model.

What it does not provide

- Authentication of the committer.
- Proof that the committed value is valid.
- Confidentiality after the opening is revealed.
- Protection against weak randomness.
- Agreement about when openings are allowed.

Assumptions

Hash commitments usually rely on properties of the hash function and sufficient randomness. Pedersen commitments rely on group assumptions and generator choices.

Post-quantum posture

Depends on the construction. Hash-based commitments can be plausibly post-quantum with appropriate hash choices and parameters. Pedersen commitments and other discrete-logarithm-based commitments are quantum-vulnerable.

Confidence model

Confidence comes from the hiding and binding assumptions of the concrete commitment scheme, plus correct randomness and unambiguous opening rules. A commitment does not identify the committer unless authentication is added.

Common constructions

- Hash-based commitments.
- Pedersen commitments.
- Merkle tree commitments to many values.

Concrete algorithms and schemes

Scheme	Typical role	Hiding and binding shape	Key differences and cautions
Hash commitment	Commit to one value with `H(message		randomness)`
Pedersen commitment	Commit to numeric values with additive homomorphism	Perfectly hiding, computationally binding under discrete-logarithm assumptions	Useful for range proofs and confidential values; quantum-vulnerable.

Scheme	Typical role	Hiding and binding shape	Key differences and cautions
Merkle commitment	Commit to a list or set of values	Binding from collision resistance	Efficient membership proofs; privacy depends on leaf encoding and whether paths reveal structure.
KZG commitment	Polynomial or vector commitment	Binding under pairing assumptions and setup model	Very compact openings; quantum-vulnerable and often setup-dependent.
Inner-product-argument commitments	Vector/polynomial commitments in discrete-logarithm groups	Binding under discrete-logarithm assumptions	Avoids trusted setup in common forms, but proofs can be larger and quantum-vulnerable.

Use cases

- Commit-reveal protocols.
- Private voting and delayed ballot reveal.
- Range proofs and confidential transactions.
- Randomness beacons and fair ordering protocols.

Composition patterns

Commitments are often combined with zero-knowledge proofs to prove facts about a hidden value without opening it. They are also used with signatures when a system must bind a commitment to an authenticated party.

Failure modes and anti-patterns

- Reusing or choosing predictable randomness can expose the committed value.
- Forgetting authentication allows anyone to submit a commitment.
- Treating a commitment as encryption confuses delayed disclosure with recipient-controlled confidentiality.
- Opening rules can be abused if the protocol does not define deadlines and abort handling.

Maturity and deployment

Commitments are mature and widely used, but concrete schemes still depend on careful parameter choices and protocol context.

Related concepts

- Pedersen commitments
- Zero-knowledge proofs
- Private aggregation

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- Pedersen, Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing.

Pedersen Commitments

One-sentence intuition

A Pedersen commitment hides a value while preserving additive structure.

Where it sits in the taxonomy

- Level: structured primitive
- Parent category: commitments
- Related concepts: commitments, homomorphic commitments, range proofs

Problem it solves

Pedersen commitments let a system commit to numeric values and later use algebraic relationships between commitments. This is useful when a protocol needs to keep values hidden while proving or aggregating relationships about them.

Mental model

The commitment mixes the message with a private random mask. Observers see a group element that looks randomized, while the committer can later reveal both the value and the mask.

Minimal example

A common notation is:

$$C = g^m h^r$$

Here `m` is the message, `r` is randomness, and `g` and `h` are group generators. The opening is `m, r`.

The additive structure appears when two commitments are combined:

$$C_1 \cdot C_2 = g^{m_1+m_2} h^{r_1+r_2}$$

Inline notation also works: $C = g^m h^r$.

Security properties

- Hiding: if `r` is random and secret, the commitment hides `m`.
- Binding: under discrete logarithm assumptions, a committer should not be able to open one commitment to two different messages.
- Additive homomorphism: multiplying commitments corresponds to adding their committed values.

What it does not provide

- Authentication.
- Proof that `m` is in an allowed range.
- Protection if randomness is reused, predictable, or revealed.
- Binding if the discrete logarithm relationship between `g` and `h` is known to the committer.

Assumptions

Pedersen commitments depend on a suitable group where discrete logarithms are hard and on safe generation of independent generators. The randomness must be sampled correctly and kept secret until opening.

Post-quantum posture

Vulnerable. Standard Pedersen commitments rely on discrete-logarithm hardness, so they should not be treated as post-quantum commitments.

Confidence model

Confidence comes from the group assumption, independent public generator setup, and correct randomness. Binding can fail if a party knows the discrete-logarithm relation between the generators. Hiding can fail if randomness is reused, predictable, or revealed.

Common constructions

Pedersen commitments are usually instantiated in elliptic curve or finite-field groups. Concrete deployments must choose parameters and libraries carefully.

Concrete instantiations

Instantiation	Common setting	Key differences and cautions
Elliptic-curve Pedersen commitments	Confidential transactions, range proofs, ZK protocols	Efficient and common; curve choice, generator derivation, and subgroup handling matter.
Finite-field Pedersen commitments	Older protocols and threshold/verifiable secret sharing	Same discrete-logarithm posture, but parameter sizes and subgroup checks differ.
Pedersen vector commitments	Commit to several values with independent generators	Useful for proving linear relations; binding depends on no party knowing generator relations.
Bulletproof-style commitments	Range proofs and confidential amounts	Often built over Pedersen commitments; range proofs are needed to prevent overflow or negative-value attacks.

Use cases

- Confidential transactions.
- Range proofs.
- Private tallying.

- Commitments inside zero-knowledge proof systems.

Composition patterns

Pedersen commitments are often paired with range proofs to show that hidden values are non-negative or within a bound. They can also support private tallying because commitments to individual votes can be combined into a commitment to the total.

Failure modes and anti-patterns

- Bad randomness can reveal values or allow linkage.
- Reusing randomness across commitments can leak relationships between messages.
- Unknown generator setup can break binding if a party knows the relation between generators.
- Homomorphism can be dangerous if a protocol forgets to constrain values with range proofs.

Maturity and deployment

Pedersen commitments are mature but should be used through well-reviewed libraries and protocol designs.

Related concepts

- Commitments
- Range proofs
- Private aggregation

Further reading

- Pedersen, Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing.
- Bünz et al., Bulletproofs: Short Proofs for Confidential Transactions and More.

Digital Signatures

One-sentence intuition

A digital signature lets a private key holder authorize a message so anyone with the public key can verify it.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: authentication
- Related concepts: Anonymous Credentials, Threshold Cryptography, Public-Key Encryption

Problem it solves

Open systems need a way for anyone to verify that a message was authorized by a private-key holder. Digital signatures provide public verifiability for origin and integrity, but only if keys, contexts, and verification rules are handled correctly.

Mental model

Think of a public wax seal: anyone can check the seal, but only the private key holder should be able to create it.

Minimal example

A package maintainer signs a release digest; users verify the signature against the maintainer public key before installing.

Security properties

- Message authenticity.
- Integrity.
- Non-repudiation in some legal or operational contexts, depending on key control.

What it does not provide

- Confidentiality.
- Anonymity.
- Proof that the signer understood the message.
- Protection if the private key is stolen.

Assumptions

The signature scheme must resist forgery, the private key must remain secret, and verifiers must bind the public key to the right signer, protocol, message format, and domain; modern signature standards make

the approved scheme and parameter choices explicit (NIST FIPS 186-5).

Post-quantum posture

Depends on the signature scheme. RSA, ECDSA, EdDSA, and Schnorr-style signatures are quantum-vulnerable under Shor's algorithm for factoring and discrete logarithms (Shor 1994), while finalized post-quantum signature standards such as ML-DSA and SLH-DSA are designed for post-quantum migration (NIST FIPS 204, NIST FIPS 205). Falcon/FN-DSA is selected for ongoing standardization, so its deployment status should be checked separately (NIST IR 8581).

Confidence model

Confidence comes from the signer controlling the private key, verifiers binding the public key to the right identity and context, and the signature scheme resisting forgery.

Common constructions

Concrete algorithms and schemes

Scheme family	Examples	Common role	Post-quantum posture and cautions
EdDSA	Ed25519, Ed448	General-purpose signatures where ecosystem support exists	Quantum-vulnerable; deterministic signing helps avoid random nonce failures, but context binding still matters (RFC 8032).
ECDSA	ECDSA P-256, ECDSA secp256k1	TLS, certificates, blockchain transactions	Quantum-vulnerable; nonce reuse or biased nonces can expose the private key (NIST FIPS 186-5).
Schnorr-style signatures	BIP-340 Schnorr, protocol-specific Schnorr variants	Blockchains, multisignatures, zero-knowledge protocols	Quantum-vulnerable; batch verification and multisignature variants need careful domain separation.
RSA-PSS	RSA Probabilistic Signature Scheme	Legacy public-key infrastructure and compatibility	Quantum-vulnerable; prefer PSS over older RSA PKCS #1 v1.5 signatures in new RSA designs.
BLS signatures	BLS12-381 or BN254 deployments	Aggregatable signatures and threshold signing	Quantum-vulnerable and pairing-based; subgroup checks and domain separation are critical.
ML-DSA	Module-lattice signature standard	Post-quantum migration	Plausibly post-quantum; larger keys and signatures affect protocol design (NIST FIPS 204).
SLH-DSA	Stateless hash-based signature standard	Conservative post-quantum signatures	Plausibly post-quantum; signatures are large and performance differs sharply from elliptic-curve schemes (NIST FIPS 205).
Falcon / FN-DSA	Compact lattice signature selected for ongoing NIST standardization	Future post-quantum option where smaller signatures matter	Not one of the three finalized 2024 FIPS standards; track FIPS 206 status before treating as finalized (NIST IR 8581).
Legacy signatures	DSA, RSA PKCS #1 v1.5 signatures	Compatibility and verification of old artifacts	Keep as legacy context; do not present as a modern default.

Use cases

- Software updates.
- Blockchain transactions.
- Credential issuance.
- Protocol transcript authentication.

Composition patterns

- Signatures must bind protocol, domain, and message encoding.
- Legal non-repudiation depends on operational key control.

Common adjacent concepts: Anonymous Credentials, Threshold Cryptography, Public-Key Encryption.

Failure modes and anti-patterns

- Signing ambiguous encodings.
- Reusing nonces in schemes where nonce uniqueness is required (NIST FIPS 186-5).
- Failing to bind signatures to domain, chain, or protocol context.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: high.

Related concepts

- Anonymous Credentials
- Threshold Cryptography
- Public-Key Encryption

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- NIST FIPS 186-5, Digital Signature Standard.
- RFC 8032, Edwards-Curve Digital Signature Algorithm.
- NIST FIPS 204, Module-Lattice-Based Digital Signature Standard.
- NIST FIPS 205, Stateless Hash-Based Digital Signature Standard.
- NIST CSRC, Post-Quantum Cryptography Project.

Public-Key Encryption

One-sentence intuition

Public-key encryption lets anyone encrypt to a public key while only the private key holder can decrypt.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: encryption
- Related concepts: Homomorphic Encryption, Threshold Cryptography, Electronic Voting

Problem it solves

A sender often needs to protect data for a recipient before the two parties share a secret. Public-key encryption solves that bootstrapping problem by letting anyone encrypt to a public key while only the private-key holder can decrypt.

Mental model

Think of a mailbox with a public slot and a private key: anyone can put a message in, but only the owner can open it.

Minimal example

A sender uses a recipient public key to encapsulate a fresh data-encryption key, then encrypts the message with authenticated encryption.

Security properties

- Confidentiality of plaintexts under the chosen security model.
- Sometimes sender anonymity or ciphertext unlinkability, depending on the construction and context.

What it does not provide

- Sender authentication.
- Message validity beyond successful decryption.
- Protection against metadata leaks.
- Safety if keys are misbound to identities.

Assumptions

The scheme must meet the intended security notion, public keys must be authenticated, private keys must remain secret, and encryption must be used through a safe scheme or hybrid construction rather than raw textbook operations; HPKE is one standardized framework for composing KEM, KDF, and AEAD choices (RFC 9180).

Post-quantum posture

Depends on the scheme. RSA and elliptic-curve public-key encryption or key agreement are quantum-vulnerable under Shor's algorithm for factoring and discrete logarithms (Shor 1994). Post-quantum key encapsulation mechanisms such as ML-KEM are designed for migration, but protocol integration still matters (NIST FIPS 203).

Confidence model

Confidence comes from key authenticity, correct encryption or encapsulation, secure private-key handling, and the recipient's ability to decrypt. If the public key is bound to the wrong party, confidentiality can fail even when the encryption algorithm is sound.

Common constructions

Concrete algorithms and schemes

Scheme or suite	Typical role	Key differences and cautions
RSA-OAEP	Legacy public-key encryption and hybrid encryption	Quantum-vulnerable; safe padding is mandatory and raw RSA is not encryption.
HPKE	Standard hybrid public-key encryption framework	Composes KEM, KDF, and AEAD choices; security depends on authenticated public keys and mode selection (RFC 9180).
ECIES-style schemes	Hybrid encryption over elliptic-curve key agreement	Quantum-vulnerable and variant-heavy; interoperability and authentication details vary.
Integrated encryption in protocols	TLS 1.3, Signal-style sessions, Noise patterns	Public-key operations establish keys; application data is protected with symmetric encryption.
Post-quantum KEM-based hybrids	ML-KEM plus AEAD through a key schedule	Plausibly post-quantum at the KEM layer; ciphertext size, failure behavior, and authentication still matter (NIST FIPS 203).
Legacy RSA encryption	RSAES-PKCS1-v1_5	Compatibility only; historically fragile against padding-oracle mistakes.

Use cases

- Secure messaging.
- Hybrid encryption.
- Encrypted ballots.
- Key exchange support, depending on the protocol.

Composition patterns

- Encryption must be combined with authentication or proofs when validity matters.
- Hybrid constructions need correct key encapsulation and data encapsulation.

Common adjacent concepts: Homomorphic Encryption, Threshold Cryptography, Electronic Voting.

Failure modes and anti-patterns

- Using raw textbook encryption instead of authenticated, padded, or hybrid constructions.
- Forgetting key authentication.
- Treating encrypted data as valid without additional checks or proofs.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: high.

Related concepts

- Homomorphic Encryption
- Threshold Cryptography
- Electronic Voting

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- NIST FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.
- RFC 9180, Hybrid Public Key Encryption.

Key Encapsulation and Exchange

One-sentence intuition

Key encapsulation and key exchange let parties establish shared secret material over an insecure channel.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: key-management
- Related concepts: Public-Key Encryption, Key Derivation Functions, Lattices

Problem it solves

Parties that start with no shared secret need a way to agree on fresh key material across an observable network. Key exchange and key encapsulation mechanisms (KEMs) solve that setup step so later symmetric encryption and authentication can protect session traffic.

Mental model

Think of two parties creating the same fresh secret in public while outsiders cannot compute it.

Minimal example

A sender encapsulates to a recipient public key, both derive the same shared secret, and a KDF turns it into an encryption key.

Security properties

- Shared secret establishment.
- Forward secrecy in protocols designed for ephemeral secrets.
- Authentication when combined with signatures, certificates, pre-shared keys, or authenticated transcripts.

What it does not provide

- Entity authentication by itself.
- Application-data encryption without a subsequent key schedule and encryption layer.
- Metadata privacy for who talked to whom and when.

Assumptions

The scheme-specific hardness assumption must hold, public keys or identities must be authenticated when authentication is required, and derived keys must be bound to the full transcript; TLS 1.3 is a standard example of transcript-bound key schedule design (RFC 8446).

Post-quantum posture

Depends on the concrete mechanism. Diffie-Hellman and elliptic-curve Diffie-Hellman are quantum-vulnerable under Shor's algorithm for discrete logarithms (Shor 1994), while standardized post-quantum KEMs such as ML-KEM are designed for migration (NIST FIPS 203).

Confidence model

Confidence comes from the key-establishment assumption, authentication binding, fresh ephemeral secret handling, and correct derivation of application keys from the shared secret.

Common constructions

Concrete algorithms and schemes

Mechanism	Family	Typical role	Key differences and cautions
X25519	Elliptic-curve Diffie-Hellman	Modern key agreement in protocols and libraries	Quantum-vulnerable; usually simple and robust when used through established libraries (RFC 7748).
X448	Elliptic-curve Diffie-Hellman	Higher-security-margin key agreement	Quantum-vulnerable; less widely deployed than X25519 (RFC 7748).
P-256 ECDH	Elliptic-curve Diffie-Hellman	TLS and standards-oriented environments	Quantum-vulnerable; point validation and library correctness matter.
FFDHE	Finite-field Diffie-Hellman groups	Compatibility and standards profiles	Quantum-vulnerable; use reviewed safe-prime groups, not ad hoc parameters.
ML-KEM	Module-lattice KEM	Post-quantum key encapsulation	Plausibly post-quantum; protocol designers must handle larger keys and ciphertexts (NIST FIPS 203).
HQC	Code-based KEM selected for ongoing NIST standardization	Backup or alternative post-quantum KEM family	Selected by NIST in 2025 for future standardization; not a finalized FIPS standard as of this review (NIST HQC selection).
HPKE KEM suites	KEM plus KDF plus AEAD framework	Hybrid encryption and application protocols	HPKE is a composition framework; the selected KEM determines posture (RFC 9180).
Hybrid classical/PQ exchange	Classical ECDH plus ML-KEM or similar	Migration period key establishment	Reduces single-assumption risk, but transcript binding and failure handling must be explicit.

Use cases

- Hybrid public-key encryption.
- Transport security handshakes.
- Secure messaging session setup.

Composition patterns

- Key exchange must feed a KDF and authenticated transcript.

- Hybrid post-quantum migration must define combiner behavior and transcript binding, rather than just concatenating outputs informally.

Common adjacent concepts: Public-Key Encryption, Key Derivation Functions, Lattices.

Failure modes and anti-patterns

- Establishing a key with an unauthenticated attacker.
- Failing to bind the transcript, identities, and algorithm choices into derived keys.
- Reusing ephemeral secrets where freshness is required.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk is expert-only because authentication, transcript binding, hybrid combiners, key derivation, and failure handling are protocol-level decisions (RFC 8446, RFC 9180). Parameter sensitivity: high.

Related concepts

- Public-Key Encryption
- Key Derivation Functions
- Lattices

Further reading

- Boneh and Shoup, A Graduate Course in Applied Cryptography.
- NIST FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.
- RFC 9180, Hybrid Public Key Encryption.
- NIST, NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption.

Secret Sharing

One-sentence intuition

Secret sharing splits a secret into shares so that only an authorized subset can reconstruct it.

Where it sits in the taxonomy

- Level: basic-primitive
- Parent category: threshold
- Related concepts: Threshold Cryptography, Secure Aggregation, Electronic Voting

Problem it solves

Some systems need to protect a secret from single-point compromise while still allowing recovery or action by an authorized group. Secret sharing distributes risk across shares and defines which subsets can reconstruct the value.

Mental model

Think of splitting a vault code across people so no small group can open the vault, but a quorum can.

Minimal example

A backup key is split into five shares, and any three can reconstruct it if two shares are lost.

In a threshold scheme, a secret can be split so that any `t` shares reconstruct it, while fewer than `t` shares should reveal nothing useful:

$$\text{threshold} = t \quad \text{out of} \quad n$$

For Shamir-style secret sharing, the dealer can choose a random polynomial whose constant term is the secret:

$$f(z) = s + a_1 z + \cdots + a_{t-1} z^{t-1}$$

Each participant receives one point on that polynomial:

$$\text{share}_i = (i, f(i))$$

Security properties

- Confidentiality against parties below the reconstruction threshold.
- Availability when enough shares survive.

What it does not provide

- Authentication of shares unless added.
- Protection against maliciously corrupted shares unless verification is added.
- Automatic key rotation or operational security.

Assumptions

Shares must be generated with correct randomness, distributed over authenticated channels, stored independently, and reconstructed only under the intended threshold and governance rules.

Post-quantum posture

Plausible for the information-theoretic core of schemes such as Shamir secret sharing. The full system may still depend on quantum-vulnerable authentication, transport encryption, signatures, or storage controls.

Confidence model

Confidence is threshold-based. Fewer than `t` shares should not reveal the secret; at least `t` valid shares can reconstruct it. Availability fails if too many shares are lost, and confidentiality fails if enough shareholders collude.

Common constructions

Concrete algorithms and schemes

Scheme	Typical role	Key differences and cautions
Shamir secret sharing	Threshold sharing over a finite field	Information-theoretic privacy below threshold; requires authenticated distribution and careful reconstruction.
Additive secret sharing	MPC and simple split-control workflows	Simple and efficient, but usually needs all shares or protocol-specific reconstruction.
Feldman verifiable secret sharing	Publicly checkable share consistency	Adds public verification, but commitments can reveal structure and rely on discrete-logarithm assumptions.
Pedersen verifiable secret sharing	Verifiable sharing with hiding commitments	Hides coefficients better than Feldman-style commitments, but inherits generator and group assumptions.
Distributed key generation	Threshold key creation without one dealer knowing the whole secret	Protocol, not just a sharing algorithm; participant authentication and abort handling dominate risk.

Use cases

- Threshold key custody.
- Backup and recovery.
- Distributed decryption.
- MPC building blocks.

Composition patterns

- Share custody is an operational security problem.
- Threshold choices encode governance assumptions.

Common adjacent concepts: Threshold Cryptography, Secure Aggregation, Electronic Voting.

Failure modes and anti-patterns

- Losing too many shares.
- Letting one organization control enough shares.
- No process for detecting invalid shares.

Maturity and deployment

Classified as mature. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: medium. Parameter sensitivity: medium.

Related concepts

- Threshold Cryptography
- Secure Aggregation
- Electronic Voting

Further reading

- Shamir, How to Share a Secret.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Structured Primitives

Structured Primitives Overview

Structured primitives add algebraic, access-control, timing, or delegation structure to basic cryptographic building blocks.

Examples

- Homomorphic commitments preserve arithmetic relationships.
- Homomorphic encryption computes on encrypted values.
- Threshold cryptography distributes authority across parties.
- Functional encryption reveals only an approved function of encrypted data.
- Verifiable delay functions make computation take sequential time.
- Polynomial commitments and vector commitments bind larger algebraic or indexed objects.
- Verifiable random functions produce publicly verifiable pseudorandom outputs.
- Blind signatures authorize hidden messages.
- Verifiable encryption encrypts while proving a statement about the plaintext.
- E-cash primitives support private issuance, spending, and double-spend handling.

Safety note

The extra structure is useful because it exposes controlled relationships. The same structure can create surprising failure modes if the protocol forgets to constrain inputs or metadata.

See Advanced Structured Primitives, Polynomial Commitments, Vector Commitments, Blind Signatures, Verifiable Random Functions, Verifiable Encryption, and E-cash Primitives.

Advanced Structured Primitives

Advanced Structured Primitives

This page is now a routing overview for primitives that add structure beyond basic encryption, signatures, commitments, or hashes.

This is a grouped overview. Polynomial Commitments and Vector Commitments now have standalone pages because their assumptions and failure modes are important enough to track separately.

Classification matrix

Concept	Level	Typical role	Main caution
Verifiable random functions (VRFs)	structured primitive	Publicly verifiable pseudorandom output from a secret key	Usually signature-like and scheme-specific.
Blind signatures	structured primitive	Signer authorizes a hidden message	Blindness does not imply unlinkable spending by itself.
Polynomial commitments	structured primitive / proof-system building block	Commit to a polynomial and open evaluations	Setup, pairing, or transcript assumptions vary.
Vector commitments	structured primitive	Commit to indexed values with compact openings	Update, non-membership, and setup models differ.
Verifiable encryption	structured primitive	Encrypt while proving something about the plaintext	Easy to prove the wrong statement or leak metadata.
E-cash primitives	structured primitive family	Issue, transfer, and redeem digital coins with privacy controls	Double-spend handling and issuer trust dominate.

Verifiable random functions

A verifiable random function (VRF) lets a secret-key holder produce pseudorandom output plus a public proof that the output was derived from a specific input and public key.

Security properties:

- Pseudorandom output to parties without the secret key.
- Public verifiability of correct evaluation.
- Uniqueness: one valid output for a given key and input, depending on the scheme.

What it does not provide:

- Unbiased public randomness if the key holder can choose whether to publish.
- Anonymity of the key holder.
- Protection if the secret key is compromised.

Assumptions and failure modes:

- Many deployed VRFs rely on elliptic-curve assumptions and are quantum-vulnerable.
- Failures include grinding inputs, withholding unfavorable outputs, and missing domain separation.

Blind signatures

A blind signature lets a user obtain a signature on a message without the signer seeing the message.

Security properties:

- Blindness: the signer should not link issuance to the final signed message.
- Unforgeability: users should not obtain more valid signatures than allowed.
- Public or issuer-side verification, depending on the scheme.

What it does not provide:

- Revocation.
- Double-spend prevention.
- Protection from network or timing linkage.
- Attribute disclosure control unless combined with a credential protocol.

Assumptions and failure modes:

- The signature assumption must hold and the blinding protocol must be implemented correctly.
- Failures include issuing without rate limits, using linkable metadata, or assuming blindness survives payment, transport, or redemption logs.

Polynomial commitments

A polynomial commitment binds a committer to a polynomial while allowing compact proofs of evaluations.

Security properties:

- Binding to a polynomial.
- Compact opening proofs for claimed evaluations.
- Often batching or aggregation support.

What it does not provide:

- Hiding unless the scheme includes it.
- Transparent setup in all constructions.
- Post-quantum posture by default.

Assumptions and failure modes:

- KZG commitments rely on pairings and structured setup; inner-product commitments rely on discrete-log assumptions; FRI-style commitments rely more heavily on hashes and coding theory.

- Failures include toxic-waste setup, domain mismatch, and verifying an opening for the wrong polynomial or evaluation point.

Vector commitments

A vector commitment binds to an indexed list of values and supports compact proofs for individual positions.

Security properties:

- Binding to a vector.
- Opening proofs for selected indices.
- Sometimes efficient updates.

What it does not provide:

- Privacy of the vector unless hiding is added.
- Freshness without authenticated roots or update rules.
- Efficient non-membership in every construction.

Assumptions and failure modes:

- The posture depends on whether the scheme is Merkle/hash-based, RSA-based, pairing-based, or polynomial-commitment-based.
- Failures include stale roots, ambiguous indexing, and update-witness inconsistency.

Verifiable encryption

Verifiable encryption lets a party encrypt a value while proving that the ciphertext encrypts a plaintext satisfying a public statement.

Security properties:

- Plaintext confidentiality under the encryption scheme.
- Public or verifier-specific proof that the encrypted plaintext has a required property.
- Binding between proof, ciphertext, public key, and context.

What it does not provide:

- Decryption fairness.
- Correct behavior by the decryptor after decryption.
- Metadata privacy for ciphertext size, recipient, or timing.

Assumptions and failure modes:

- The encryption and proof-system assumptions both matter.
- Failures include proving a weak statement, omitting recipient/context binding, and relying on a decryptor who can refuse service.

E-cash primitives

E-cash primitives support issuance, transfer, redemption, and double-spend handling for digital value with privacy goals.

Security properties:

- Issuer authenticity for minted value.
- Spending privacy under the scheme's model.
- Double-spend detection or prevention.
- Sometimes offline detection of double-spenders.

What it does not provide:

- Monetary policy or solvency guarantees.
- Network anonymity.
- Protection from ledger, timing, or amount metadata.
- Regulation or dispute resolution.

Assumptions and failure modes:

- Confidence may come from blind signatures, commitments, zero-knowledge proofs, accumulators, nullifiers, or issuer ledgers.
- Failures include linkable issuance/redemption, broken double-spend rules, and confusing privacy of coins with privacy of users.

Post-quantum posture

Depends on the construction. Hash-based vector commitments may be plausible; pairing, RSA, and elliptic-curve-based commitments, VRFs, blind signatures, and many e-cash systems are quantum-vulnerable unless replaced by post-quantum schemes.

Confidence model

Confidence varies: mathematical-assumption for core schemes, public-verifiability for openings and proofs, trusted-issuer for e-cash issuance, and trusted-setup for some commitment and proof-system deployments.

Related concepts

- Commitments
- Accumulators and Merkle Trees
- Blind Signatures
- Verifiable Random Functions
- Verifiable Encryption
- E-cash Primitives

- Zero-Knowledge Proofs
- Private Payments
- Anonymous Tokens

Further reading

- Micali, Rabin, and Vadhan, "Verifiable Random Functions".
- Chaum, "Blind Signatures for Untraceable Payments".
- Kate, Zaverucha, and Goldberg, "Constant-Size Commitments to Polynomials and Their Applications".
- Catalano and Fiore, "Vector Commitments and Their Applications".
- Camenisch and Shoup, "Practical Verifiable Encryption and Decryption of Discrete Logarithms".
- Chaum, Fiat, and Naor, "Untraceable Electronic Cash".

Polynomial Commitments

One-sentence intuition

A polynomial commitment binds a prover to a polynomial while allowing short proofs about selected evaluations of that polynomial.

Where it sits in the taxonomy

- Level: structured primitive.
- Parent category: commitments and proof-system components.
- Related concepts: Commitments, Vector Commitments, Proof-System Components, ZK Rollups.

Problem it solves

Many proof systems need a prover to commit to a large witness polynomial and later prove that the committed polynomial evaluates to a claimed value at verifier-chosen points. Without a commitment scheme, the verifier would need too much witness data or would have no binding handle on the prover's claims.

Mental model

Think of the commitment as a sealed description of a curve. Later, the prover can reveal and prove the height of the curve at selected x-coordinates without publishing the entire curve.

Minimal example

A prover commits to a polynomial `f`. The verifier asks for `f(z)`. The prover returns `y` plus an opening proof. The verifier checks that the same committed polynomial really satisfies `f(z) = y`.

Security properties

- Binding to one polynomial or low-degree object.
- Compact evaluation openings.
- Batch verification in many schemes.
- Sometimes hiding, if the scheme includes blinding.

What it does not provide

- A complete proof system by itself.
- Zero knowledge unless hiding and protocol-level protections are added.
- Transparent setup in all constructions.
- Correct arithmetization of the program being proven.

Assumptions

- KZG commitments rely on pairings and structured reference strings.
- Inner-product commitments rely on discrete-logarithm assumptions.
- FRI-style polynomial commitment layers rely on hash functions, coding theory, and soundness parameters.
- Fiat-Shamir transformations require transcript binding when made non-interactive.

Post-quantum posture

Depends on the construction. KZG and inner-product-argument commitments are quantum-vulnerable because they rely on pairings or discrete logarithms. FRI-style hash-based constructions are often treated as plausibly post-quantum when parameters and hashes are conservative.

Confidence model

Confidence may come from mathematical-assumption, trusted-setup, public-verifiability, or transparent hash-based soundness. The verifier must check the commitment, evaluation point, claimed value, proof, and transcript context together.

Common constructions

- KZG commitments.
- Inner-product-argument commitments.
- FRI-style commitments.
- Pedersen-style vector or polynomial encodings in smaller protocols.

Use cases

- SNARKs and STARKs.
- Data availability commitments.
- ZK rollups.
- Verifiable computation.
- Batched opening proofs.

Composition patterns

Polynomial commitments are composed with arithmetization, Fiat-Shamir transcripts, lookup arguments, sumcheck, and recursive proof systems. The commitment binds algebraic objects; the proof system defines what those objects mean.

Failure modes and anti-patterns

- Verifying an opening for the wrong polynomial, domain, or evaluation point.
- Reusing structured setup with unclear toxic-waste assumptions.
- Omitting public inputs from the transcript.

- Treating a hiding commitment as hiding all metadata.
- Confusing a polynomial commitment with an ordinary hash commitment.

Maturity and deployment

Mature but specialized. KZG is deployed in proof systems and data-availability contexts. FRI-style commitments are deployed in STARK-style systems. Recursive and folding-heavy uses are still fast-moving.

Related concepts

- Commitments
- Vector Commitments
- SNARKs, STARKs, and Bulletproofs
- ZK Rollups

Further reading

- Kate, Zaverucha, and Goldberg, "Constant-Size Commitments to Polynomials and Their Applications".
- Ben-Sasson et al., "Fast Reed-Solomon Interactive Oracle Proofs of Proximity".
- Bowe, Grigg, and Hopwood, "Halo".

Vector Commitments

One-sentence intuition

A vector commitment binds a party to an indexed list of values while allowing compact proofs about individual positions.

Where it sits in the taxonomy

- Level: structured primitive.
- Parent category: commitments and authenticated data structures.
- Related concepts: Commitments, Accumulators and Merkle Trees, Polynomial Commitments.

Problem it solves

Systems often need a short public commitment to a large indexed state, plus proofs that a particular index contains a particular value. Downloading or verifying the whole vector is too expensive.

Mental model

The commitment is a signed table of contents for a large table. An opening proof shows that one row has a claimed value without publishing every row.

Minimal example

A server commits to balances `[10, 0, 7, 3]`. Later it proves that index `2` contains `7` against the same commitment. A verifier checks the proof without receiving the full vector.

Security properties

- Binding to indexed values.
- Compact openings for selected indices.
- Sometimes efficient updates and update proofs.
- Sometimes non-membership or zero-value proofs, depending on construction.

What it does not provide

- Privacy of vector values unless hiding is added.
- Freshness without an authenticated root or version.
- Efficient updates in every construction.
- Correct index semantics unless encodings are unambiguous.

Assumptions

- Merkle-style vector commitments rely on hash collision resistance.
- RSA or pairing-based accumulators rely on their respective algebraic assumptions.

- KZG or polynomial-based vector commitments inherit setup and pairing assumptions.
- Dynamic update systems require consistent witness-update rules.

Post-quantum posture

Depends on the construction. Hash-based Merkle commitments are plausible with conservative hashes. RSA, pairing, and elliptic-curve-based vector commitments are quantum-vulnerable unless replaced by post-quantum alternatives.

Confidence model

Confidence comes from mathematical-assumption or hash-function security, public-verifiability of openings, and operational control over roots, versions, and update rules.

Common constructions

- Merkle trees and sparse Merkle trees.
- RSA accumulators.
- Pairing-based accumulators.
- KZG or polynomial-commitment-backed vector commitments.

Use cases

- Transparency logs.
- Membership and non-membership proofs.
- Stateless client designs.
- Rollup and blockchain state commitments.
- Revocation lists and credential status sets.

Composition patterns

Vector commitments are often composed with signatures over roots, zero-knowledge proofs over openings, accumulators for membership, and update logs for freshness.

Failure modes and anti-patterns

- Accepting stale roots.
- Ambiguous leaf encoding or index encoding.
- Forgetting to bind tree depth, domain, or version.
- Mishandling update witnesses.
- Assuming inclusion proof privacy when paths or indices are public.

Maturity and deployment

Mature but construction-specific. Merkle commitments are widely deployed. More compact algebraic vector commitments are specialized and carry stronger setup or assumption caveats.

Related concepts

- Accumulators and Merkle Trees
- Polynomial Commitments
- Membership Proofs
- Privacy-Preserving Revocation

Further reading

- Catalano and Fiore, "Vector Commitments and Their Applications".
- Merkle, "A Digital Signature Based on a Conventional Encryption Function".
- Kate, Zaverucha, and Goldberg, "Constant-Size Commitments to Polynomials and Their Applications".

Blind Signatures

One-sentence intuition

A blind signature lets a signer authorize a message without seeing the exact message being signed.

Where it sits in the taxonomy

- Level: structured primitive.
- Parent category: Digital Signatures.
- Related concepts: Anonymous Tokens, Blind-Signature Credentials, Private Payments.

Problem it solves

Issuers often need to authorize tokens, credentials, or coins without being able to link issuance to later presentation or redemption.

Mental model

The user puts a message in a cryptographic envelope. The signer stamps the envelope. The user removes the envelope and keeps a valid signature on the hidden message.

Minimal example

A rate limiter issues one blind-signed token to a client. Later the client redeems the signed token, and the issuer can verify authenticity without directly linking redemption to issuance.

Security properties

- Blindness: the signer should not learn the signed message.
- Unforgeability: users should not obtain more valid signatures than authorized.
- Public or issuer-side verification, depending on the scheme.

What it does not provide

- Revocation by itself.
- Double-spend prevention.
- Network anonymity.
- Attribute privacy unless composed with a credential presentation protocol.

Assumptions

- The underlying signature assumption holds.
- The blinding protocol is implemented correctly.
- Issuance policy limits over-issuance.

- Presentation and transport metadata do not re-link users.

Post-quantum posture

Depends on the signature scheme. RSA and elliptic-curve blind signatures are quantum-vulnerable; post-quantum blind signatures are less mature.

Confidence model

Confidence comes from mathematical-assumption security, signer key protection, issuance policy, and metadata controls around issuance and redemption.

Common constructions

- Chaumian RSA blind signatures.
- Partially blind signatures.
- Blind signatures inside Privacy Pass-style token systems.

Use cases

- Anonymous tokens.
- E-cash and private payments.
- Credential issuance.
- Privacy-preserving rate limits.

Composition patterns

Blind signatures are often composed with issuance policy, token redemption logs, revocation mechanisms, and transport privacy. Blindness at the primitive layer does not automatically survive system metadata.

Failure modes and anti-patterns

- Unique issuance metadata.
- Redemption timing linkage.
- No rate limits.
- Treating blind signatures as complete anonymous credentials.

Maturity and deployment

Mature but specialized. Blind signatures are standardized in some forms and deployed in token systems, but privacy depends heavily on system integration.

Related concepts

- Anonymous Tokens

- Blind-Signature Credentials
- Privacy-Preserving Revocation

Further reading

- Chaum, "Blind Signatures for Untraceable Payments".
- RFC 9474: RSA Blind Signatures.
- RFC 9576: The Privacy Pass Architecture.

Verifiable Random Functions

One-sentence intuition

A verifiable random function (VRF) produces pseudorandom output plus a public proof that the output came from a specific key and input.

Where it sits in the taxonomy

- Level: structured primitive.
- Parent category: pseudorandom functions and digital signatures.
- Related concepts: Digital Signatures, Randomness and Nonces, Domain Separation.

Problem it solves

Some systems need randomness that is unpredictable before evaluation but publicly checkable afterward, such as leader election, lotteries, and transparency mechanisms.

Mental model

A VRF is like a private dice roll with a receipt. The key holder rolls, and anyone can verify that this was the unique roll for that input and key.

Minimal example

A validator evaluates a VRF on an epoch number. If the output falls below a threshold, the validator is eligible to propose a block and publishes the proof.

Security properties

- Pseudorandom output to parties without the secret key.
- Public verifiability of correct evaluation.
- Uniqueness under the scheme's model.

What it does not provide

- Unbiased public randomness if the key holder can withhold outputs.
- Anonymity of the key holder.
- Protection after secret-key compromise.
- Domain separation unless the input is labeled.

Assumptions

- The VRF construction and group assumptions hold.
- Inputs include protocol, version, and context labels.
- Key holders cannot gain advantage by grinding inputs or withholding outputs.

Post-quantum posture

Common elliptic-curve VRFs are quantum-vulnerable. Post-quantum VRFs are less deployed and should be treated as specialized.

Confidence model

Confidence comes from mathematical-assumption security, public-verifiability, key protection, and anti-grinding protocol rules.

Common constructions

- ECVRF-style constructions.
- Signature-like VRFs.
- VRFs embedded in consensus or lottery protocols.

Use cases

- Blockchain leader election.
- Randomized committee selection.
- Transparency logs.
- Privacy-preserving rate limits.

Composition patterns

VRFs are composed with eligibility rules, threshold checks, public transcripts, and sometimes slashing or accountability. Protocols must address withholding and grinding.

Failure modes and anti-patterns

- Missing domain separation.
- Treating VRFs as unbiased randomness beacons.
- Allowing input grinding.
- Ignoring key compromise or rotation.

Maturity and deployment

Deployed, but mostly in specialized protocols. Most common deployments inherit elliptic-curve quantum vulnerability.

Related concepts

- Digital Signatures
- Domain Separation
- Secure Channels

Further reading

- Micali, Rabin, and Vadhan, "Verifiable Random Functions".
- RFC 9381: Verifiable Random Functions.

Verifiable Encryption

One-sentence intuition

Verifiable encryption encrypts a value while proving that the hidden plaintext satisfies a public statement.

Where it sits in the taxonomy

- Level: structured primitive.
- Parent category: encryption plus proof systems.
- Related concepts: Encrypt-Then-Prove, Zero-Knowledge Proofs, Commitments.

Problem it solves

Systems sometimes need public confidence about encrypted data without revealing the data, such as encrypted ballots, escrowed secrets, or dispute workflows.

Mental model

The ciphertext is sealed, but the prover attaches a certificate saying what kind of object is inside without opening it.

Minimal example

A voter encrypts a ballot and proves that the ciphertext encrypts one valid candidate choice.

Security properties

- Plaintext confidentiality under the encryption scheme.
- Proof that the plaintext satisfies a statement.
- Binding between ciphertext, recipient key, statement, and context.

What it does not provide

- Correct decryption behavior.
- Fairness.
- Metadata privacy for sender, recipient, size, or timing.
- A guarantee that the proved statement is the statement the system actually needs.

Assumptions

- Encryption assumptions and proof-system assumptions both hold.
- The proof binds the ciphertext, recipient key, public statement, and context.
- Decryptors are available and authorized when decryption is required.

Post-quantum posture

Depends on the encryption and proof system. Classical public-key encryption, pairings, and elliptic-curve commitments are quantum-vulnerable; lattice and hash-based components may be plausible.

Confidence model

Confidence is mixed: mathematical-assumption for encryption and proofs, public-verifiability for proof checks, and operational trust in decryption or recovery authorities.

Common constructions

- Encryption with a zero-knowledge proof of plaintext knowledge.
- Verifiable encryption of discrete logarithms.
- Encrypted ballots with validity proofs.

Use cases

- Electronic voting.
- Fair exchange and dispute workflows.
- Credential recovery or escrow.
- Private payments and compliance hooks.

Composition patterns

Verifiable encryption is commonly expressed as the Encrypt-Then-Prove pattern. The proof must bind to the exact ciphertext and recipient key.

Failure modes and anti-patterns

- Proving a statement about a commitment but not the ciphertext.
- Omitting recipient or context binding.
- Proving a statement too weak for the system goal.
- Relying on a decryptor who can refuse service.

Maturity and deployment

Mature but specialized. Correct statement design and composition require expert review.

Related concepts

- Encrypt-Then-Prove
- Zero-Knowledge Proofs
- Electronic Voting

Further reading

- Camenisch and Shoup, "Practical Verifiable Encryption and Decryption of Discrete Logarithms".
- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems".

E-Cash Primitives

One-sentence intuition

E-cash primitives support issuing, transferring, and redeeming digital value while limiting linkability under a stated model.

Where it sits in the taxonomy

- Level: structured primitive family.
- Parent category: Private Payments.
- Related concepts: Blind Signatures, Nullifiers, Commitments.

Problem it solves

Digital value systems need authenticity and double-spend control. Privacy-preserving systems also need to avoid linking issuance, spending, and redemption more than necessary.

Mental model

A digital coin is a signed or committed object. Spending reveals enough to prove validity and prevent reuse, but not necessarily the owner's identity.

Minimal example

A bank blindly signs a coin serial number. Later, the user spends the coin. The merchant verifies the issuer signature and the issuer rejects the same serial number if it appears again.

Security properties

- Issuer authenticity for minted value.
- Spending privacy under the scheme model.
- Double-spend detection or prevention.
- Sometimes offline identification of double spenders.

What it does not provide

- Solvency or monetary policy.
- Network anonymity.
- Regulation or dispute resolution.
- Privacy from all amount, timing, or redemption metadata.

Assumptions

- Issuer keys and issuance policy are sound.
- Double-spend database or ledger is available.

- Cryptographic privacy is not defeated by transport, wallet, or exchange metadata.

Post-quantum posture

Depends on blind signatures, commitments, proofs, accumulators, and ledger authentication. Many classical e-cash designs are quantum-vulnerable.

Confidence model

Confidence is mixed: trusted-issuer for minting, public-verifiability or issuer verification for spends, client-side-secret for coin ownership, and operational audit for issuance and redemption.

Common constructions

- Chaumian blind-signature e-cash.
- Serial-number or nullifier-based coins.
- ZK note systems.
- Accumulator-backed redemption or revocation.

Use cases

- Private payments.
- Anonymous tokens with value.
- Offline or semi-offline payment schemes.
- Rate-limited authorization tokens.

Composition patterns

E-cash primitives compose with wallets, ledgers, nullifiers, revocation, secure channels, and compliance or audit workflows. The primitive does not define the complete payment system.

Failure modes and anti-patterns

- Linkable issuance and redemption logs.
- Weak double-spend rules.
- Small anonymity sets.
- Confusing privacy of coins with privacy of users.

Maturity and deployment

Emerging as a system family. Classical ideas are mature, but modern deployments vary widely in trust and metadata posture.

Related concepts

- Private Payments

- Blind Signatures
- Anti-Double-Use Nullifiers

Further reading

- Chaum, Fiat, and Naor, "Untraceable Electronic Cash".
- Chaum, "Blind Signatures for Untraceable Payments".
- Zerocash: Decentralized Anonymous Payments from Bitcoin.

Homomorphic Commitments

One-sentence intuition

A homomorphic commitment lets commitments be combined in ways that correspond to operations on the hidden values.

Where it sits in the taxonomy

- Level: structured-primitive
- Parent category: commitment
- Related concepts: Pedersen Commitments, Commitments, Private Aggregation

Problem it solves

Protocols often need to keep values hidden while proving that sums, differences, or other relations over those values are consistent. Homomorphic commitments let verifiers check algebraic relationships without opening every committed value.

Mental model

Think of sealed numbers that can be added while still sealed, with the result opening consistently.

Minimal example

Two committed balances can be added to produce a commitment to the total, then a proof checks the total without opening each balance.

Security properties

- hiding
- binding
- additive structure

What it does not provide

- Proof that hidden values are in a valid range.
- Authentication.
- Protection against maliciously crafted commitments without additional checks.

Assumptions

The commitment scheme must retain its hiding and binding properties under the allowed homomorphic operation, and the surrounding protocol must define the arithmetic domain being committed to.

Post-quantum posture

Depends on the construction. Pedersen-style homomorphic commitments are quantum-vulnerable because they rely on discrete logarithms. Other commitment families need separate classification.

Confidence model

Confidence comes from the commitment binding and hiding assumptions plus explicit constraints on the hidden arithmetic domain. Homomorphic structure is useful only when invalid values are ruled out elsewhere.

Common constructions

Concrete schemes and families

Scheme	Homomorphic shape	Typical role	Key differences and cautions
Pedersen commitments	Additive over committed values	Confidential amounts, private tallying, range proofs	Mature and efficient, but quantum-vulnerable and generator setup-sensitive.
Pedersen vector commitments	Linear relations over committed vectors	Inner-product proofs and multi-value commitments	Requires independent generators or a sound generator-derivation process.
KZG commitments	Polynomial openings and linear combinations	Rollups, data availability, succinct polynomial proofs	Compact but pairing-based and usually setup-dependent.
Inner-product-argument commitments	Vector and polynomial commitments	Bulletproof-style systems and transparent-ish vector commitments	Avoids pairing setup in common forms, but proof sizes and verification costs differ.
Merkle commitments	Set/list commitment via hashes	Membership proofs and sparse state commitments	Not algebraically homomorphic, but often used as the hash-based alternative when homomorphism is not required.

Use cases

- Confidential sums.
- Private tallying.
- Range proof systems.

Composition patterns

- Homomorphic arithmetic can wrap or cancel invalid values.
- Range proofs or validity proofs are usually needed.

Common adjacent concepts: Pedersen Commitments, Commitments, Private Aggregation.

Failure modes and anti-patterns

Homomorphism can let invalid values cancel or wrap unless the protocol adds range proofs and clear arithmetic domains.

Maturity and deployment

Classified as mature. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: high.

Related concepts

- Pedersen Commitments
- Commitments
- Private Aggregation

Further reading

- Pedersen, Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing.
- Pedersen commitments

Homomorphic Encryption

One-sentence intuition

Homomorphic encryption allows computation on ciphertexts so that decrypting the result reveals the result of a computation on the plaintexts.

Where it sits in the taxonomy

- Level: structured-primitive
- Parent category: encryption
- Related concepts: Private Aggregation, Reveal Only a Function, Secure Aggregation

Problem it solves

Some applications need computation on sensitive data by parties who should not see the plaintext. Homomorphic encryption lets selected operations happen on ciphertexts, but shifts difficulty to parameter choices, leakage boundaries, and performance.

Mental model

Think of doing arithmetic on locked boxes so the final unlocked box contains the computed answer.

Minimal example

Clients encrypt measurements, a server adds ciphertexts, and trustees decrypt only the aggregate sum.

At a high level, evaluation over ciphertexts should agree with evaluating the function over plaintexts:

$$\mathrm{Dec}_{sk}(\mathrm{Eval}(f, c_1, \ldots, c_n)) = f(m_1, \ldots, m_n)$$

where each ciphertext hides a message:

$$c_i = \mathrm{Enc}_{pk}(m_i)$$

Security properties

- confidentiality
- encrypted computation

What it does not provide

- Automatic input validity.
- Protection against malicious outputs without verification.
- Metadata privacy.
- Practical efficiency for every workload.

Assumptions

The scheme-specific hardness assumption, parameter set, noise budget, key-management model, and implementation side-channel posture must all match the workload and threat model.

Post-quantum posture

Plausible for many lattice-based homomorphic encryption families, assuming appropriate parameters and implementations. The posture still depends on the concrete scheme, security level, and whether surrounding signatures, proofs, or key-management layers are post-quantum.

Confidence model

Confidence usually comes from a scheme-specific hardness assumption, correct parameter selection, secure key generation, and protection of decryption keys. In threshold or multi-key settings, the confidence model also includes the trustee or participant threshold.

Common constructions

Concrete schemes and families

Family	Computation style	Typical role	Key differences and cautions
BFV	Exact arithmetic over integers modulo a plaintext modulus	Private tallying and exact arithmetic workloads	Good for exact values; parameter selection and batching shape performance.
BGV	Exact arithmetic with leveled homomorphic evaluation	Exact encrypted computation	Similar use space to BFV, with different noise-management techniques.
CKKS	Approximate arithmetic over real or complex values	Machine learning and statistical analytics	Approximate results are part of the design; precision and scale management are security-relevant engineering choices.
TFHE / FHEW-style schemes	Boolean or small-gate bootstrapped computation	Bit-level computation and programmable bootstrapping	Can support frequent bootstrapping; performance profile differs from arithmetic-circuit schemes.
Threshold HE variants	Shared decryption key across trustees	Private tallying and multi-party analytics	Adds a `t-of-n` confidence model on top of the encryption scheme.

Use cases

- Private tallying.
- Confidential analytics.
- Encrypted computation services.

Composition patterns

- Outputs can reveal sensitive input information.
- Malicious inputs may require zero-knowledge validity proofs.

- Incorrect parameters can break correctness or security.

Common adjacent concepts: Private Aggregation, Reveal Only a Function, Secure Aggregation.

Failure modes and anti-patterns

- Choosing parameters that do not meet the security or correctness target.
- Ignoring noise growth or implementation limits.
- Revealing too much through outputs or repeated queries.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: expert-only. Parameter sensitivity: high.

Related concepts

- Private Aggregation
- Reveal Only a Function
- Secure Aggregation

Further reading

- Gentry, Fully Homomorphic Encryption Using Ideal Lattices.
- Microsoft Research, Microsoft SEAL.
- OpenFHE contributors, OpenFHE documentation.
- Zama, Concrete ML documentation.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Threshold Cryptography

One-sentence intuition

Threshold cryptography distributes a cryptographic power across several parties so that a quorum is required to act.

Where it sits in the taxonomy

- Level: structured-primitive
- Parent category: threshold
- Related concepts: Secret Sharing, Electronic Voting, Public-Key Encryption

Problem it solves

Critical keys and signing powers are dangerous when controlled by one machine or one person. Threshold cryptography splits authority across participants so action requires a quorum, while making availability and collusion assumptions explicit.

Mental model

Think of a cryptographic power split across operators so no single operator can act alone.

Minimal example

Five trustees hold decryption shares, and any three can jointly decrypt an election tally while fewer than three learn nothing.

Security properties

- distributed control
- compromise resilience
- availability at threshold

What it does not provide

- Trustlessness.
- Protection if the threshold colludes.
- Simple operations or recovery.
- Metadata privacy by itself.

Assumptions

The protocol must generate and protect shares correctly, define the threshold and recovery process, authenticate participants, and handle share refresh, replacement, and audit procedures.

Post-quantum posture

Depends on the underlying primitive. Threshold ECDSA or threshold Schnorr is quantum-vulnerable. Threshold versions of post-quantum signatures, encryption, or key encapsulation need separate analysis and are not automatically available just because a single-party primitive exists.

Confidence model

Confidence is `t-of-n`: the system assumes fewer than `t` parties collude for privacy or key misuse resistance, and at least `t` parties are available for liveness. Distributed key generation, share custody, and recovery policy are part of the model.

Common constructions

Common constructions vary by concrete scheme and deployment context. Use the concrete scheme or composition tables on this page to check the exact assumptions, setup model, and implementation risk.

Use cases

- threshold signatures
- threshold decryption
- trustee based voting

Composition patterns

- Threshold cryptography shifts trust into custody and governance.
- Liveness and confidentiality thresholds may differ.

Common adjacent concepts: Secret Sharing, Electronic Voting, Public-Key Encryption.

Failure modes and anti-patterns

- Bad distributed key generation.
- Poor share custody.
- Unclear quorum governance.
- No plan for rotation, slashing, replacement, or disaster recovery.

Maturity and deployment

Classified as mature. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: expert-only. Parameter sensitivity: high.

Source-depth notes

Threshold signing should cite both the signing protocol and the setup protocol. For Schnorr-style deployments, FROST is a protocol standard, while distributed key generation remains a separate trust and liveness concern. For BLS and ECDSA ecosystems, aggregation, pairing, nonce, and share-

generation assumptions differ enough that they should not be collapsed into one generic "threshold" claim.

Related concepts

- Secret Sharing
- Electronic Voting
- Public-Key Encryption

Further reading

- Shamir, How to Share a Secret.
- Gennaro, Jarecki, Krawczyk, and Rabin, Secure Distributed Key Generation for Discrete-Log Based Cryptosystems.
- RFC 9591, The FROST Protocol.
- Zcash Foundation, FROST implementation.
- drand, Protocol Specification.
- Boneh, Lynn, and Shacham, Short Signatures from the Weil Pairing.
- Boldyreva, Threshold Signatures, Multisignatures and Blind Signatures.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Functional Encryption

One-sentence intuition

Functional encryption lets a key reveal only a specific function of encrypted data, rather than the full plaintext.

Where it sits in the taxonomy

- Level: structured-primitive
- Parent category: encryption
- Related concepts: Homomorphic Encryption, Reveal Only a Function, Private Aggregation

Problem it solves

Sometimes a data holder wants another party to learn only a permitted computation over encrypted data, not the underlying plaintext. Functional encryption captures that goal with restricted decryption keys, at the cost of strong assumptions and careful function design.

Mental model

Think of giving someone a key that opens only one computed view of encrypted data, not the raw data.

Minimal example

An analyst receives a function key that reveals a risk score over encrypted records, not the underlying records.

Security properties

- controlled disclosure
- confidentiality
- least privilege decryption

What it does not provide

- General practicality for all functions.
- Simple deployment.
- Protection against outputs that are themselves revealing.
- A substitute for access-control design.

Assumptions

Assumptions are scheme-specific and often include a setup or key authority that issues function keys correctly. The system also assumes that the allowed function and repeated query pattern do not leak more than intended.

Post-quantum posture

Depends on the concrete construction. Functional encryption is a broad research area; posture should be classified per scheme and parameter set, not for the category as a whole.

Confidence model

Confidence often depends on a key authority or setup process that issues function keys. Even if the cryptography works, the allowed function can leak sensitive information, and repeated function outputs can become an inference channel.

Common constructions

Common constructions vary by concrete scheme and deployment context. Use the concrete scheme or composition tables on this page to check the exact assumptions, setup model, and implementation risk.

Use cases

- restricted analytics
- private classification
- reveal only a function

Composition patterns

- The allowed function output may reveal more than intended.
- Repeated function queries can leak inputs.
- Key authority trust can dominate the privacy model.

Common adjacent concepts: Homomorphic Encryption, Reveal Only a Function, Private Aggregation.

Failure modes and anti-patterns

- Issuing function keys that reveal too much.
- Ignoring leakage from repeated function outputs.
- Treating research-stage general functional encryption as a deployable access-control layer.

Maturity and deployment

Functional encryption remains largely research-stage for many general forms. Specialized forms may be more practical.

Related concepts

- Homomorphic Encryption
- Reveal Only a Function
- Private Aggregation

Further reading

- Boneh, Sahai, and Waters, Functional Encryption: Definitions and Challenges.

Timelock and VDFs

One-sentence intuition

Timelock tools and verifiable delay functions (VDFs) make information or outputs depend on the passage of sequential computation time.

Where it sits in the taxonomy

- Level: structured-primitive
- Parent category: delay
- Related concepts: Delayed Reveal, Commitments

Problem it solves

Some protocols need a public delay that cannot be skipped by adding parallel hardware, but can be checked quickly afterward. Timelocks and verifiable delay functions (VDFs) model time as a cryptographic resource, with security tied to sequential-work assumptions.

Mental model

Think of a puzzle that forces time to pass before a value can be opened, while the answer is easy to check.

Minimal example

A lottery seed is committed now and revealed only after a verifiable delay output is available.

A verifiable delay function can be read as a function that takes an input `x`, requires about `T` sequential steps to compute, and returns a result plus a proof:

$$(y, \pi) \leftarrow \mathrm{Eval}(x, T)$$

Verification should be much faster than evaluation:

$$\mathrm{Verify}(x, y, \pi) = 1$$

Security properties

- sequential delay
- public verifiability

What it does not provide

- Wall-clock fairness in every network setting.
- Protection against specialized hardware unless modeled.

- Confidentiality by themselves.

Assumptions

The delay parameter must reflect realistic sequential computation, the setup model must be sound for the chosen construction, and the protocol must account for network delay, denial of service, and hardware advantage.

Post-quantum posture

Depends on the construction. Some VDF and timelock designs rely on assumptions whose post-quantum status is not the same as hash-based or lattice-based primitives. Treat each construction separately.

Confidence model

Confidence comes from the sequential-delay assumption, the verification equation, parameter selection, and any setup process. The model is not "trusted time"; it is an assumption about how much sequential work an adversary can complete.

Common constructions

Concrete schemes and families

Scheme or family	Assumption shape	Typical role	Key differences and cautions
Repeated-squaring timelocks	Sequential squaring in an unknown-order group	Delayed disclosure and timelock puzzles	Delay depends on sequential work; setup and modulus/class-group choice matter.
Wesolowski VDF	Unknown-order group with compact proof	Publicly verifiable delay outputs	Compact verification, but assumption and setup choices must be explicit.
Pietrzak VDF	Unknown-order group with interactive/recursive proof structure	Verifiable delay outputs	Different proof and verification trade-offs from Wesolowski-style VDFs.
Class-group VDFs	Unknown-order class groups	Avoiding trusted RSA modulus generation	Parameter generation differs from RSA groups and still needs expert review.
Trusted delay services	External time authority or server	Engineering substitute for cryptographic delay	Not a VDF; confidence shifts to service trust and availability.

Use cases

- Randomness beacons.
- Delayed reveal.
- Leader election protocols.

Composition patterns

- Specialized hardware can change delay assumptions.
- Delay does not prevent denial of service.
- Verification only checks the computation, not the surrounding protocol goal.

Common adjacent concepts: Delayed Reveal, Commitments.

Failure modes and anti-patterns

- Underestimating hardware advantage.
- Confusing sequential work with trusted time.
- Ignoring denial-of-service and availability.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: expert-only. Parameter sensitivity: high.

Related concepts

- Delayed Reveal
- Commitments

Further reading

- Boneh et al., Verifiable Delay Functions.
- Wesolowski, Efficient Verifiable Delay Functions.

Accumulators and Merkle Trees

One-sentence intuition

Accumulators and Merkle trees commit to a collection while supporting compact membership, and sometimes non-membership, proofs.

Where it sits in the taxonomy

- Level: structured-primitive
- Parent category: authenticated-data-structure
- Related concepts: Membership Proofs, Hash Function, Anonymous Membership

Problem it solves

Systems often need to publish a compact digest of a large set while letting users prove membership, and sometimes non-membership, without sending the whole set. Accumulators and Merkle trees trade storage, update cost, proof size, and trust assumptions in different ways.

Mental model

Think of a compact root that stands for a whole collection and can be checked with a short path or witness.

Minimal example

A transparency log publishes a Merkle root, and a user verifies that a certificate appears under that root with an inclusion path.

Security properties

- set commitment
- membership verification
- compact proofs

What it does not provide

- Privacy of the set unless the design hides it.
- Freshness unless updates are authenticated.
- Protection against metadata leaks.

Assumptions

The set encoding must be canonical, roots must be authenticated, update rules must be clear, and verifiers must know which root or accumulator state is current.

Post-quantum posture

Depends on the accumulator. Merkle trees built from appropriate hash functions are plausibly post-quantum. RSA accumulators and elliptic-curve accumulators are quantum-vulnerable.

Confidence model

Confidence comes from the authenticated set root, update rules, and membership proof verification. Dynamic accumulators also need a freshness model so verifiers know which root is current.

Common constructions

Common constructions vary by concrete scheme and deployment context. Use the concrete scheme or composition tables on this page to check the exact assumptions, setup model, and implementation risk.

Use cases

- Certificate transparency.
- Blockchains and authenticated data structures.
- Anonymous membership sets.
- Airdrop eligibility lists.

Composition patterns

- Set membership is not the same as eligibility policy.
- Dynamic updates need a freshness story.

Common adjacent concepts: Membership Proofs, Hash Function, Anonymous Membership.

Failure modes and anti-patterns

- Ambiguous tree encoding.
- No domain separation between leaves and internal nodes.
- Treating membership as authorization without checking context.

Maturity and deployment

Classified as deployed. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: scheme-dependent.

Related concepts

- Membership Proofs
- Hash Function
- Anonymous Membership

Further reading

- Merkle, A Digital Signature Based on a Conventional Encryption Function.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Proof Systems

Proof Systems Overview

Proof systems let a prover convince a verifier that a statement is true. Some proof systems also hide the witness.

Examples

- Zero-knowledge proofs hide witnesses.
- Range proofs show a hidden value is within bounds.
- Membership proofs show inclusion in a set.
- SNARKs, STARKs, and Bulletproofs are proof-system families with different trade-offs.
- Proof-system components include FRI, folding schemes, recursive proofs, lookup arguments, sumcheck, and arithmetization.

Reading rule

Always identify the statement, the witness, public inputs, setup assumptions, verifier cost, prover cost, and metadata leaks.

See Proof-System Components, FRI, Folding Schemes, Arithmetization, Sumcheck, Recursive Proofs, and Lookup Arguments.

Proof-System Components

Proof-System Components

Modern proof systems are assembled from lower-level components. This page is now a routing overview for components that often appear inside SNARKs, STARKs, rollups, and recursive proving systems.

Component matrix

Component	Level	Typical role	Main caution
Polynomial commitments	structured primitive / proof-system component	Commit to polynomial witnesses and prove evaluations	Setup and assumption model vary sharply.
FRI	proof-system component	Prove low-degree structure in transparent proof systems	Parameter and hash choices drive soundness and size.
Folding schemes	proof-system component	Combine repeated computations or instances incrementally	Still fast-moving and construction-specific.
Recursive proofs	proof-system technique	Verify proofs inside other proofs	Security depends on cycles, transcript binding, and soundness composition.
Lookup arguments	proof-system component	Prove values belong to a table	Table commitments and multiplicity rules are subtle.
Sumcheck	proof-system component	Reduce claims about large sums to smaller checks	Soundness depends on field, degree, and verifier challenges.
Arithmetization	proof-system representation	Encode computation as constraints, circuits, traces, or polynomials	Bugs here prove the wrong program.

Polynomial commitments

In proof systems, polynomial commitments let a prover commit to witness polynomials and later prove evaluations at verifier-chosen points.

Security properties:

- Binding to committed polynomials.
- Compact evaluation openings.
- Batchable verification in many systems.

What it does not provide:

- A complete proof system by itself.
- Correct arithmetization.
- Transparent setup in all constructions.

Assumptions and failure modes:

- KZG is pairing-based and setup-dependent; IPA-style commitments are discrete-log-based; FRI-style commitments are usually hash-based.
- Failures include wrong evaluation domains, toxic waste, and transcript ambiguity.

FRI

FRI is an interactive oracle proof technique for showing that a function is close to a low-degree polynomial. It is central to many STARK-style proof systems.

Security properties:

- Low-degree testing with transparent setup.
- Hash-based commitment to evaluation layers.
- Scalable proof systems when combined with suitable arithmetization.

What it does not provide:

- Zero knowledge by itself.
- Correctness of the traced computation.
- Small proofs in every parameter regime.

Assumptions and failure modes:

- Confidence comes from soundness analysis, field choice, query counts, hash functions, and transcript binding.
- Failures include weak parameters, bad randomness derivation, and confusing low-degree proximity with program correctness.

Folding schemes

Folding schemes combine multiple proof instances into a smaller accumulated instance, often for incremental verifiable computation.

Security properties:

- Incremental compression of repeated computation claims.
- A path toward efficient recursion without proving a full verifier each step.
- Support for long-running or streaming computations in some constructions.

What it does not provide:

- A universal replacement for SNARKs or STARKs.
- Mature, interchangeable security assumptions across all schemes.
- Application correctness if the folded relation is wrong.

Assumptions and failure modes:

- Confidence is construction-specific and often depends on commitment schemes, transcript design, and soundness of the folded relation.
- Failures include accumulator misuse, finalization mistakes, and treating research-stage performance claims as deployed maturity.

Recursive proofs

Recursive proofs verify one proof inside another proof. They are used for proof aggregation, rollups, incremental computation, and succinct chains of verification.

Security properties:

- Proof composition.
- Compression of many checks into one proof.
- Public verifiability of an accumulated computation or history.

What it does not provide:

- Automatic soundness under arbitrary composition.
- Metadata privacy.
- Cheap proving if verifier circuits are large.

Assumptions and failure modes:

- Recursion depends on field compatibility, curve cycles or non-native arithmetic, transcript binding, and proof-system assumptions.
- Failures include verifying the wrong verification key, omitting public inputs, and accumulating invalid state.

Lookup arguments

Lookup arguments prove that committed or witnessed values appear in an approved table.

Security properties:

- Efficient range checks, opcode checks, or membership-in-table checks.
- Reduced constraint count for repeated fixed tables.
- Compatibility with many arithmetizations.

What it does not provide:

- Correct table construction.
- Privacy for table access patterns unless the system hides them.
- Soundness if multiplicities or table commitments are mishandled.

Assumptions and failure modes:

- Confidence depends on commitment binding, permutation or grand-product checks, table encoding, and transcript challenge generation.
- Failures include duplicate-handling bugs, wrong table versions, and public lookup values leaking sensitive state.

Sumcheck

The sumcheck protocol lets a prover convince a verifier that a large sum over a polynomial has a claimed value using a sequence of smaller checks.

Security properties:

- Reduces an exponential-size sum claim to interactive polynomial checks.
- Useful inside interactive proofs, GKR-style protocols, and many modern proof systems.
- Verifier work can be much smaller than direct recomputation.

What it does not provide:

- Zero knowledge by itself.
- Commitment to witness values unless combined with other tools.
- Soundness if degrees or fields are misstated.

Assumptions and failure modes:

- Confidence comes from polynomial degree bounds, verifier randomness, and finite-field soundness.
- Failures include wrong degree accounting and transcript ambiguity after Fiat-Shamir transformation.

Arithmetization

Arithmetization is the process of representing a computation as algebraic constraints, circuits, traces, or polynomial identities that a proof system can check.

Security properties:

- Defines the exact statement being proven.
- Bridges source-level computation and proof-system constraints.
- Enables efficient proving for the chosen proof family.

What it does not provide:

- Assurance that the encoded statement matches developer intent.
- Protection from bugs in constraint generation.
- Privacy for public inputs.

Assumptions and failure modes:

- The relation must be complete, sound, and bound to the right public inputs.

- Failures include underconstrained circuits, missing range checks, inconsistent encodings, and proving a property that is too weak for the system goal.

Post-quantum posture

Depends on the component and instantiation. FRI/STARK-style components are often treated as plausibly post-quantum when instantiated with appropriate hashes and parameters. Pairing-based polynomial commitments and many IPA-style commitments are quantum-vulnerable.

Confidence model

Confidence comes from public-verifiability, mathematical assumptions, transcript binding, setup model, and correctness of the arithmetized statement.

Related concepts

- Zero-Knowledge Proofs
- SNARKs, STARKs, and Bulletproofs
- FRI
- Folding Schemes
- Arithmetization
- Sumcheck
- Recursive Proofs
- Lookup Arguments
- Advanced Structured Primitives
- ZK Rollups

Further reading

- Lund, Fortnow, Karloff, and Nisan, "Algebraic Methods for Interactive Proof Systems".
- Ben-Sasson et al., "Fast Reed-Solomon Interactive Oracle Proofs of Proximity".
- Bünz et al., "Bulletproofs: Short Proofs for Confidential Transactions and More".
- Kate, Zaverucha, and Goldberg, "Constant-Size Commitments to Polynomials and Their Applications".
- Gabizon and Williamson, "Plookup: A Simplified Polynomial Protocol for Lookup Tables".
- Bowe, Grigg, and Hopwood, "Halo".
- Kothapalli, Setty, and Tzialla, "Nova".

FRI

One-sentence intuition

FRI proves that a committed function is close to a low-degree polynomial using repeated folding and random queries.

Where it sits in the taxonomy

- Level: proof-system component.
- Parent category: Proof-System Components.
- Related concepts: STARKs, ZK Rollups, Polynomial Commitments.

Problem it solves

Transparent proof systems need a way to convince verifiers that a large evaluated trace has low-degree structure without checking every point.

Mental model

FRI repeatedly compresses a long table into smaller tables. Random checks make it hard for a high-degree table to keep pretending to be low-degree.

Minimal example

A prover commits to evaluations of a polynomial-like trace. The verifier samples positions across folded layers and checks consistency until the final object is small enough to inspect.

Security properties

- Low-degree proximity testing.
- Transparent setup in common constructions.
- Scalable verification when paired with suitable commitments and parameters.

What it does not provide

- Zero knowledge by itself.
- Correct arithmetization.
- Small proofs in every parameter regime.
- Security if hash, field, or query parameters are weak.

Assumptions

- Field size, blowup factor, query count, and hash security match the target soundness.
- Fiat-Shamir transcript binding is correct for non-interactive proofs.
- Commitment layers bind each evaluation layer.

Post-quantum posture

Plausible when instantiated with conservative hashes and parameters. The posture comes from hash security, coding-theoretic soundness, and implementation choices.

Confidence model

Confidence comes from public-verifiability, transparent parameters, soundness analysis, and transcript binding.

Common constructions

- STARK-style low-degree testing.
- DEEP-FRI-style refinements.
- FRI-backed polynomial commitment layers.

Use cases

- STARK proof systems.
- Validity proofs.
- ZK rollups with transparent proving.

Composition patterns

FRI is composed with arithmetization, Merkle commitments, Fiat-Shamir transcripts, and sometimes recursion or proof aggregation.

Failure modes and anti-patterns

- Weak query counts.
- Bad domain or field choices.
- Treating low-degree proximity as program correctness.
- Missing transcript binding.

Maturity and deployment

Deployed in STARK-style systems, but parameters and performance engineering are specialized.

Related concepts

- Proof-System Components
- Arithmetization
- ZK Rollups

Further reading

- Ben-Sasson et al., "Fast Reed-Solomon Interactive Oracle Proofs of Proximity".
- Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity".

Folding Schemes

One-sentence intuition

Folding schemes combine multiple proof instances into a smaller accumulated instance that can be checked later.

Where it sits in the taxonomy

- Level: proof-system component.
- Parent category: recursion and incremental verifiable computation.
- Related concepts: Recursive Proofs, Sumcheck, Polynomial Commitments.

Problem it solves

Recursive proofs can be expensive if every step verifies a full proof inside another proof. Folding accumulates claims more directly for long-running computations.

Mental model

Instead of carrying a stack of receipts, each new receipt is folded into a running balance that is finalized later.

Minimal example

A prover repeatedly folds computation steps into an accumulator. At the end, the prover produces a final proof that the accumulated relation is valid.

Security properties

- Incremental compression of repeated claims.
- Support for streaming or long-running computations.
- Potentially cheaper recursion than proving a full verifier each step.

What it does not provide

- A universal replacement for SNARKs or STARKs.
- Mature, interchangeable assumptions across schemes.
- Correctness if the folded relation is wrong.

Assumptions

- The commitment scheme and folding relation are sound.
- Accumulators are finalized correctly.
- Fiat-Shamir transcripts bind each step and public input.

Post-quantum posture

Depends on commitments and proof-system choices. Many known efficient constructions use elliptic-curve assumptions and are quantum-vulnerable.

Confidence model

Confidence is construction-specific and comes from mathematical assumptions, transcript binding, and public-verifiability of the final proof.

Common constructions

- Nova-style folding.
- Halo-style accumulation.
- Incrementally verifiable computation frameworks.

Use cases

- Recursive proving.
- Long-running computation proofs.
- Rollup aggregation.

Composition patterns

Folding schemes compose with arithmetization, polynomial commitments, sumcheck-style protocols, and final proof systems.

Failure modes and anti-patterns

- Treating research performance claims as deployed maturity.
- Finalizing the wrong accumulator.
- Omitting public inputs from the folded relation.
- Reusing challenges across contexts.

Maturity and deployment

Research-stage to emerging. The area is fast-moving and should be reviewed frequently.

Related concepts

- Recursive Proofs
- ZK Rollups
- Transcript Binding

Further reading

- Bowe, Grigg, and Hopwood, "Halo".
- Kothapalli, Setty, and Tzialla, "Nova".

Arithmetization

Arithmetization

One-sentence intuition

Arithmetization turns a computation into algebraic constraints that a proof system can check.

Where it sits in the taxonomy

- Level: proof-system representation.
- Parent category: Proof-System Components.
- Related concepts: Lookup Arguments, Sumcheck, ZK Rollups.

Problem it solves

Proof systems do not verify source code directly. They verify constraints, traces, circuits, or polynomial identities. Arithmetization defines what statement is actually proven; PLONK-style systems are one example of making this algebraic constraint layer explicit (PLONK).

Mental model

Arithmetization is the translation layer between program behavior and proof-system mathematics.

Minimal example

To prove `x * y = z`, the circuit introduces variables for `x`, `y`, and `z` and a multiplication constraint. Larger programs are decomposed into many such constraints.

Security properties

- Defines the exact relation being proven.
- Enables efficient proof-system checks.
- Binds public inputs to witness constraints when designed correctly.

What it does not provide

- Assurance that constraints match developer intent.
- Privacy for public inputs.
- Protection from underconstrained circuits; circuit tooling still requires explicit constraint review (Circom documentation).
- Soundness if encodings or range checks are missing.

Assumptions

- Constraint generation is correct and complete.
- Public inputs, encodings, ranges, and state roots are included.
- The chosen field and representation support the computation safely.

Post-quantum posture

Not applicable to arithmetization itself. The posture comes from the proof system and commitments used to prove the arithmetized statement.

Confidence model

Confidence comes from public-verifiability, independent circuit review, test vectors, and constraints that exactly match the intended semantics; implementation documentation for circuit languages treats constraint generation and review as separate from writing source-level expressions (Circom documentation).

Common constructions

- R1CS.
- Plonkish constraint systems.
- AIR traces.
- Circuit and VM arithmetizations.

Use cases

- ZK circuits.
- Rollup state-transition proofs.
- Validity proofs for virtual machines.
- Private computation proofs.

Composition patterns

Arithmetization composes with polynomial commitments, lookup arguments, range checks, and transcript binding. A proof is only as meaningful as the relation it proves.

Failure modes and anti-patterns

- Underconstrained circuits (Circom documentation).
- Missing range checks.
- Incorrect public-input binding.
- Proving a property that is too weak for the application.

Maturity and deployment

Mature as a concept, but implementation risk is high and domain-specific because the proof only enforces the constraints actually generated, not the developer's informal intent (Circom documentation).

Related concepts

- Lookup Arguments
- ZK Rollups
- Transcript Binding

Further reading

- PLONK.
- Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity".
- Goldwasser, Kalai, and Rothblum, "Delegating Computation".
- Circom documentation.

Lookup Arguments

One-sentence intuition

Lookup arguments prove that witnessed values appear in an approved table.

Where it sits in the taxonomy

- Level: proof-system component.
- Parent category: Arithmetization.
- Related concepts: Range Proofs, Polynomial Commitments, ZK Rollups.

Problem it solves

Some checks are cheaper as table membership than as arithmetic constraints. Lookups help prove range checks, opcodes, byte decompositions, or fixed function tables efficiently.

Mental model

Instead of recomputing a rule, the prover shows that each value came from an approved table.

Minimal example

A circuit proves that a byte value is in the table `{0, ..., 255}` rather than enforcing a long bit-decomposition manually.

Security properties

- Efficient table membership checks.
- Reduced constraint counts for repeated fixed tables.
- Compatibility with many Plonkish arithmetizations.

What it does not provide

- Correct table construction.
- Privacy for public lookup values.
- Soundness if multiplicities or table commitments are wrong.
- Range semantics unless the table encodes the range correctly.

Assumptions

- Table commitment and encoding are correct.
- Multiplicity and permutation checks are sound.
- Transcript challenges bind table version and lookup relation.

Post-quantum posture

Depends on the surrounding proof system and commitment scheme. The lookup idea itself is not the source of post-quantum posture.

Confidence model

Confidence comes from public-verifiability, commitment binding, correct table construction, and transcript binding.

Common constructions

- Plookup-style arguments.
- Grand-product lookup checks.
- Fixed and dynamic lookup tables.

Use cases

- Range checks.
- Opcode and VM instruction checks.
- Byte decomposition.
- Hash and signature gadget optimization.

Composition patterns

Lookup arguments compose with arithmetization, polynomial commitments, public table roots, and sometimes vector commitments.

Failure modes and anti-patterns

- Duplicate-handling bugs.
- Wrong table version.
- Public lookup values leaking private state.
- Using a table that encodes the wrong semantics.

Maturity and deployment

Emerging to mature in modern proof systems. Details remain scheme-specific.

Related concepts

- Arithmetization
- Range Proofs
- Polynomial Commitments

Further reading

- Gabizon and Williamson, "Plookup".
- PLONK.

Sumcheck

One-sentence intuition

The sumcheck protocol convinces a verifier that a large polynomial sum has a claimed value using a sequence of smaller checks.

Where it sits in the taxonomy

- Level: proof-system component.
- Parent category: interactive proofs.
- Related concepts: Arithmetization, Folding Schemes, Zero-Knowledge Proofs.

Problem it solves

Many computations can be expressed as sums over large domains. Sumcheck lets a verifier avoid evaluating every term directly.

Mental model

The prover claims a huge spreadsheet total. The verifier asks randomized questions that reduce the total to smaller totals until only simple checks remain.

Minimal example

A prover claims that the sum of a multilinear polynomial over all Boolean inputs is `S`. The protocol reduces that claim one variable at a time using verifier challenges.

Security properties

- Reduces large sum claims to smaller polynomial checks.
- Verifier work can be much smaller than direct recomputation.
- Useful inside many modern proof systems.

What it does not provide

- Zero knowledge by itself.
- Commitment to witness values.
- Soundness if degree bounds or fields are misstated.
- Non-interactivity without Fiat-Shamir-style transformation.

Assumptions

- Polynomial degree bounds are correct.
- Verifier challenges are unpredictable.
- The field is large enough for the soundness target.

- Transcript binding is correct after Fiat-Shamir.

Post-quantum posture

Plausible for the information-theoretic protocol core. Concrete non-interactive systems inherit posture from commitments, hashes, and Fiat-Shamir assumptions.

Confidence model

Confidence comes from public-verifiability, finite-field soundness, verifier randomness, and correct degree accounting.

Common constructions

- Classical sumcheck.
- GKR-style protocols.
- Sumcheck inside polynomial IOPs and folding schemes.

Use cases

- Interactive proofs.
- Verifiable computation.
- Recursive and folding proof systems.
- Proof systems for arithmetic circuits.

Composition patterns

Sumcheck is composed with commitments to witness data, arithmetization, Fiat-Shamir transcripts, and final polynomial openings.

Failure modes and anti-patterns

- Wrong degree bounds.
- Reusing verifier challenges.
- Transcript ambiguity.
- Forgetting that sumcheck alone does not bind a hidden witness.

Maturity and deployment

Mature as a protocol component. Implementations require careful field and transcript design.

Related concepts

- Arithmetization
- Folding Schemes
- Transcript Binding

Further reading

- Lund, Fortnow, Karloff, and Nisan, "Algebraic Methods for Interactive Proof Systems".
- Goldwasser, Kalai, and Rothblum, "Delegating Computation".

Recursive Proofs

One-sentence intuition

Recursive proofs verify one proof inside another proof so many checks can be compressed into one accumulated proof.

Where it sits in the taxonomy

- Level: proof-system technique.
- Parent category: Proof-System Components.
- Related concepts: Folding Schemes, ZK Rollups, Transcript Binding.

Problem it solves

Systems may need to prove long histories, batches, or repeated computations. Recursion lets a proof attest to previous proofs rather than exposing every step.

Mental model

Each proof becomes an entry in a chain. A later proof verifies the previous link and adds new work.

Minimal example

A rollup proves a batch, then later proves that the previous batch proof verified and the next transition is valid.

Security properties

- Proof composition.
- Aggregation of many checks.
- Public verification of accumulated history.

What it does not provide

- Automatic soundness under arbitrary composition.
- Cheap proving if the verifier circuit is large.
- Metadata privacy.
- Correctness if public inputs are omitted.

Assumptions

- The verifier circuit or relation is correct.
- Verification keys, public inputs, and prior proofs are transcript-bound.
- Field, curve, or hash choices support efficient recursion safely.

Post-quantum posture

Depends on the proof system. Pairing and elliptic-curve recursion is quantum-vulnerable; hash-based recursion may be plausible with conservative parameters.

Confidence model

Confidence comes from public-verifiability, mathematical assumptions, transcript binding, and correct verifier arithmetization.

Common constructions

- Proof-carrying data.
- Halo-style accumulation.
- Nova-style folding and finalization.
- Recursive SNARKs and STARKs.

Use cases

- Rollup proof aggregation.
- Incremental verifiable computation.
- Succinct blockchain history.
- Batch verification.

Composition patterns

Recursive proofs compose with arithmetized verifiers, folding schemes, polynomial commitments, and key/version management.

Failure modes and anti-patterns

- Verifying the wrong verification key.
- Omitting public inputs.
- Accumulating invalid state.
- Treating recursion as a substitute for data availability.

Maturity and deployment

Emerging. Recursion is deployed in some systems, but designs are specialized and fast-moving.

Related concepts

- Folding Schemes
- ZK Rollups
- Arithmetization

Further reading

- Bowe, Grigg, and Hopwood, "Halo".
- Kothapalli, Setty, and Tzialla, "Nova".

Zero-Knowledge Proofs

One-sentence intuition

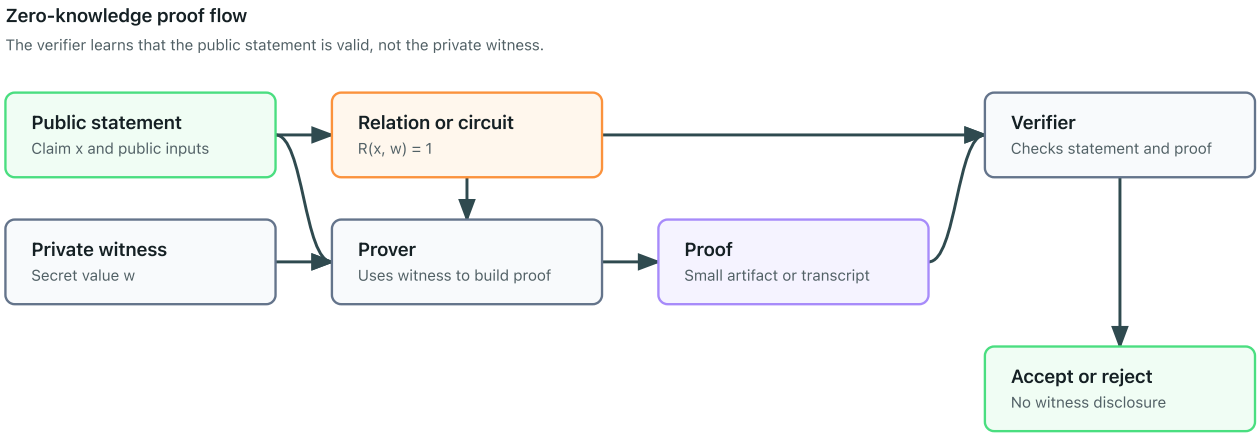
A zero-knowledge proof lets one party prove that a statement is true without revealing the secret information that makes it true.

Where it sits in the taxonomy

- Level: proof-system
- Parent category: proof-system
- Related concepts: SNARKs, STARKs, and Bulletproofs, Range Proofs, Membership Proofs

Problem it solves

Zero-knowledge proofs (ZKPs) are useful when a verifier needs confidence in a claim but should not learn the private witness behind the claim; the classical formulation comes from Goldwasser, Micali, and Rackoff's knowledge-complexity work (GMR 1985).



Statement vs witness

The statement is the public claim being proven. The witness is the private information that makes the statement true.

Many proof systems can be read as proving that there exists a private witness `w` such that a public relation accepts the public statement `x`:

$$\exists w : R(x, w) = 1$$

The verifier should learn that this relation is satisfied, not the witness itself.

Examples:

Statement	Witness
This commitment contains a number between 0 and 100	The committed number and randomness
I know a credential signed by an issuer	The credential and secret key
This encrypted vote is one of the allowed choices	The plaintext vote and encryption randomness

Mental model

Think of proving you know a valid solution without handing over the solution.

Minimal example

A prover shows they know a credential signed by an issuer and that it contains an age-over-18 attribute, without revealing the credential.

- Membership: prove a secret appears in a committed list without revealing which entry.
- Valid vote: prove an encrypted ballot encodes one allowed choice.
- Range proof: prove a hidden amount is non-negative and below a limit.

Security properties

- Completeness: honest proofs for true statements verify.
- Soundness: false statements should not verify except with negligible probability.
- Zero-knowledge: the proof should not reveal the witness beyond the truth of the statement.

What it does not provide

- Encryption of arbitrary data.
- Network anonymity.
- Protection against metadata leaks.
- A complete protocol by themselves.
- Safety if the statement being proven is the wrong statement.
- Protection against a compromised witness.

Assumptions

Assumptions vary by proof system. Some systems require trusted setup, some rely on hash functions, some rely on elliptic curve assumptions, and some are designed around transparent setup; these choices are visible in systems such as Groth16, STARKs, and Bulletproofs (Groth16, STARKs, Bulletproofs).

Post-quantum posture

Depends on the proof system. Hash-based transparent systems such as many STARK-style systems are commonly treated as plausibly post-quantum (Ben-Sasson et al. 2018), while many pairing-based SNARKs and discrete-logarithm-based range proofs are quantum-vulnerable under Shor's algorithm (Shor 1994).

Confidence model

Confidence comes from three layers: the proof-system assumptions, the correctness of the statement being proven, and the setup model. A proof can verify correctly while still proving the wrong statement for the application; circuit tooling documentation repeatedly treats constraint completeness as a separate engineering responsibility (Circom documentation).

Common constructions

Concrete proof-system families

Family or scheme	Setup model	Typical role	Key differences and cautions
Sigma protocols	Often interactive or Fiat-Shamir transformed	Knowledge proofs and identification-style protocols	Simple building blocks; transcript binding controls non-interactive security.
Groth16	Circuit-specific trusted setup	Very small proofs and fast verification	Pairing-based and quantum-vulnerable; setup is tied to the circuit (Groth16).
PLONK-style systems	Often universal/updatable setup	General-purpose SNARK proving stacks	More flexible setup than Groth16-style systems, but assumptions and arithmetization choices vary (PLONK).
STARKs	Transparent setup	Scalable transparent proofs	Usually hash-based and plausibly post-quantum; proofs are larger than many SNARKs (STARKs).
Bulletproofs	No trusted setup in common forms	Range proofs and inner-product statements	Discrete-logarithm based and quantum-vulnerable; verification cost grows with statement size (Bulletproofs).
Folding and accumulation systems	Varies by construction	Recursive proofs and incremental verifiable computation	Maturity and assumptions are construction-specific; do not treat all folding schemes as interchangeable.

Family vs concrete proof systems

"Zero-knowledge proof" is a broad family. SNARKs, STARKs, and Bulletproofs are concrete proof-system families with different trade-offs in proof size, verifier cost, prover cost, setup assumptions, post-quantum posture, and implementation maturity.

Use cases

- Prove membership in a set without revealing which member, depending on the construction.
- Prove that a hidden value is in a valid range.
- Prove that a vote, transaction, or credential use follows specified rules.
- Reduce trust in validators who would otherwise need to inspect private data.

Composition patterns

- A proof can be valid for a statement that is too weak for the protocol.
- Public inputs can reveal identity or linkage.

- Setup assumptions vary significantly across proof systems.

Common adjacent concepts: SNARKs, STARKs, and Bulletproofs, Range Proofs, Membership Proofs.

Failure modes and anti-patterns

- Proving a weak statement that does not match the system's real security goal.
- Ignoring public inputs that leak identity or linkage.
- Reusing witnesses, nullifiers, or randomness in linkable ways.
- Treating a proof system as a complete privacy protocol.
- Deploying research-stage proving systems without expert review.

Maturity and deployment

ZKPs are a mature field, but concrete systems vary from widely deployed to research-stage. Maturity must be evaluated per construction and implementation, not inferred from the umbrella term "zero knowledge."

Related concepts

- Range proofs
- Membership proofs
- Nullifiers

Further reading

- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.
- Ben-Sasson et al., Scalable, transparent, and post-quantum secure computational integrity.
- Bünz et al., Bulletproofs: Short Proofs for Confidential Transactions and More.
- Groth, On the Size of Pairing-Based Non-interactive Arguments.
- Gabizon, Williamson, and Ciobotaru, PLONK.
- iden3, Circom documentation.

SNARKs, STARKs, and Bulletproofs

One-sentence intuition

SNARKs, STARKs, and Bulletproofs are families of proof systems that package zero-knowledge or succinct verification with different costs and assumptions.

Where it sits in the taxonomy

- Level: proof-system
- Parent category: proof-system-family
- Related concepts: Zero-Knowledge Proofs, Range Proofs, Trusted Setup

Problem it solves

Applications that use succinct or zero-knowledge proofs must choose among families with very different setup, proof-size, verifier-cost, prover-cost, and assumption profiles. Comparing SNARKs, STARKs, and Bulletproofs helps match a proof system to a threat model and deployment setting.

Mental model

Think of different proof-system toolchains that compress confidence in a computation with different setup, size, and cost trade-offs.

Minimal example

A rollup may use a SNARK for small verifier cost, while a transparency-focused system may use a STARK to avoid trusted setup.

Security properties

- succinct verification
- soundness
- zero knowledge depending on mode

What it does not provide

- A correct statement automatically.
- Metadata privacy.
- Protection against implementation bugs.
- A complete protocol.

Assumptions

Assumptions are family-specific: pairing-based SNARKs often depend on elliptic-curve and setup assumptions, STARK-style systems typically depend on hash choices and transparent protocols, and

Bulletproof-style systems usually depend on discrete-logarithm assumptions (Groth16, STARKs, Bulletproofs).

Post-quantum posture

Depends on the family and construction. Many deployed pairing-based SNARKs are quantum-vulnerable because pairing groups inherit discrete-logarithm vulnerability under Shor's algorithm (Shor 1994). STARK-style systems are often treated as plausibly post-quantum when instantiated with appropriate hash functions (Ben-Sasson et al. 2018). Bulletproof-style systems are usually discrete-logarithm based and therefore quantum-vulnerable (Bulletproofs).

Confidence model

Confidence comes from the proof-system assumptions, setup model, and statement design. Pairing-based SNARKs may require trusted or universal setup, including ceremony assumptions for setup parameters (Bowe, Gabizon, and Miers 2017). STARK-style systems are usually transparent. Bulletproof-style systems avoid trusted setup but still rely on discrete-logarithm assumptions.

Common constructions

Concrete systems and families

System or family	Category	Setup model	Common use	Main caution
Groth16	Pairing-based SNARK	Circuit-specific trusted setup	Very small proofs in deployed ZK systems	Setup and circuit specificity dominate confidence (Groth16).
PLONK-style systems	Polynomial-commitment SNARK	Often universal or updatable setup	General-purpose circuits and rollups	Exact assumptions depend on the commitment scheme and transcript design (PLONK).
Marlin / Sonic-style systems	Universal-setup SNARK family	Universal structured setup	General circuits with reusable setup	Setup is reusable but still a setup assumption.
STARKs	Transparent proof system	Transparent	Scalable computation proofs	Larger proofs; hash and FRI parameters are security-critical (STARKs).
FRI / DEEP-FRI	Low-degree testing component	Transparent	STARK-style proof systems	It is a component, not the full application statement.
Bulletproofs	Inner-product proof system	No trusted setup in common forms	Range proofs and confidential transactions	Discrete-logarithm based and quantum-vulnerable (Bulletproofs).
Halo / Nova-style systems	Accumulation or folding families	Varies	Recursive and incremental proofs	Rapidly evolving; maturity is implementation-specific.

Comparison snapshot

Family	Common strengths	Common trade-offs
SNARKs	Small proofs and fast verification	Some constructions need trusted setup or pairing assumptions; Groth16 is the canonical small-proof example (Groth16).
STARKs	Transparent setup and hash-based assumptions	Larger proofs and heavier verification than many SNARKs (STARKs).
Bulletproofs	No trusted setup and useful range proofs	Verification can be heavier for large statements (Bulletproofs).

Use cases

- zk rollups
- private transactions
- verifiable computation

Composition patterns

- Proof-family choice changes the whole confidence model.
- Public inputs can reveal identity or linkage.

Common adjacent concepts: Zero-Knowledge Proofs, Range Proofs, Trusted Setup.

Failure modes and anti-patterns

- Choosing a proof family based only on proof size.
- Treating trusted setup, transparent setup, and universal setup as interchangeable.
- Proving a statement that omits important public inputs.
- Ignoring prover cost, verification cost, and implementation maturity.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk is expert-only because production safety depends on concrete circuits, transcript design, parameters, serialization, and proving-stack behavior (halo2 Book, gnark documentation, Circom documentation). Parameter sensitivity: high.

Source-depth notes

The families in this page should be read through their concrete construction papers, not only through umbrella terms. Groth16, PLONK, STARK/FRI systems, Bulletproofs, Halo, Nova, lookup arguments, and arithmetization each move assumptions into different places: setup, pairings, hashes, transcript binding, field choice, or circuit correctness.

Related concepts

- Zero-Knowledge Proofs
- Range Proofs
- Trusted Setup

Further reading

- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.
- Groth, On the Size of Pairing-Based Non-interactive Arguments.
- Ben-Sasson et al., Scalable, transparent, and post-quantum secure computational integrity.
- Ben-Sasson et al., Fast Reed-Solomon Interactive Oracle Proofs of Proximity.
- Bünz et al., Bulletproofs: Short Proofs for Confidential Transactions and More.
- Gabizon, Williamson, and Ciobotaru, PLONK.
- Bowe, Grigg, and Hopwood, Halo.
- Kothapalli, Setty, and Tzialla, Nova.
- Gabizon and Williamson, Plookup.
- Electric Coin Company, The halo2 Book.
- Consensys, gnark documentation.
- arkworks contributors, arkworks ecosystem.
- iden3, Circom documentation.

Range Proofs

One-sentence intuition

A range proof shows that a hidden value lies within an allowed interval.

Where it sits in the taxonomy

- Level: proof-system
- Parent category: proof-system
- Related concepts: Pedersen Commitments, Zero-Knowledge Proofs, SNARKs, STARKs, and Bulletproofs

Problem it solves

Hidden values can be invalid. In private payments, a value may need to be non-negative. In voting, a ballot may need to be one of a small set of choices.

The typical statement is that a hidden value lies in a public interval:

$$m \in [0, 2^k - 1]$$

When the value is inside a commitment, the proof should be bound to that exact commitment:

$$C = \mathrm{Commit}(m; r)$$

Mental model

Think of proving a sealed number is between two bounds without revealing the number.

Minimal example

A private payment proves the committed amount is non-negative and below 2^64 without revealing the amount.

Security properties

- value validity
- soundness
- zero knowledge depending on construction

What it does not provide

- Authentication.
- Correct tallying by itself.
- Protection against metadata leaks.

- A guarantee that the range was the right policy choice.

Assumptions

The proof system must be sound, the proof must be bound to the exact commitment or ciphertext, and the range must be encoded without field wraparound or overflow ambiguity.

Post-quantum posture

Depends on the proof system and commitment scheme. Bulletproof-style range proofs are usually discrete-logarithm based and quantum-vulnerable; hash-based or STARK-style approaches may be plausibly post-quantum if the full construction supports the required statement.

Confidence model

Confidence comes from public verification of the range statement, the soundness of the proof system, and correct binding to the commitment or ciphertext being constrained.

Common constructions

Concrete schemes and families

Scheme or family	Commitment/proof base	Typical role	Key differences and cautions
Bulletproof range proofs	Pedersen commitments plus inner-product arguments	Confidential transactions and hidden balances	No trusted setup in common forms, but discrete-logarithm based and quantum-vulnerable.
Boudot-style interval proofs	Integer commitments	Earlier range-proof constructions	Useful historically; parameter and efficiency trade-offs differ from Bulletproofs.
SNARK-based range proofs	Groth16, PLONK-style systems	Validity proofs inside larger circuits	Compact verification, but setup and statement correctness matter.
STARK-based range checks	STARK constraints and lookups	Transparent proof systems	Plausibly post-quantum when the full stack is; proof size and constraint design matter.
Lookup-based range checks	Plookup-style tables or custom lookup arguments	Modern circuit systems	Efficient for bounded values, but table binding and field encoding must be explicit.

Use cases

- Confidential transactions.
- Private voting.
- Rate limits.
- Private statistics.

Composition patterns

- The proved interval must match the application policy.
- Arithmetic domains and overflows must be explicit.

Common adjacent concepts: Pedersen Commitments, Zero-Knowledge Proofs, SNARKs, STARKs, and Bulletproofs.

Failure modes and anti-patterns

- Proving the wrong bound.
- Ignoring overflow or field wraparound.
- Failing to bind the proof to the correct commitment or context.

Maturity and deployment

Classified as mature. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: expert-only. Parameter sensitivity: high.

Related concepts

- Pedersen Commitments
- Zero-Knowledge Proofs
- SNARKs, STARKs, and Bulletproofs

Further reading

- Bünz et al., Bulletproofs: Short Proofs for Confidential Transactions and More.
- Boudot, Efficient Proofs that a Committed Number Lies in an Interval.

Membership Proofs

One-sentence intuition

A membership proof shows that an item belongs to a committed set.

Where it sits in the taxonomy

- Level: proof-system
- Parent category: proof-system
- Related concepts: Accumulators and Merkle Trees, Anonymous Membership, Zero-Knowledge Proofs

Problem it solves

A verifier may need evidence that an item is in a committed set without receiving the full set. Membership proofs provide that evidence, and their privacy and update properties depend heavily on the underlying commitment or accumulator.

Mental model

Think of proving your name appears on a committed list without asking the verifier to trust a separate database lookup.

Minimal example

A client proves an account key is included under a published Merkle root by sending the leaf and authentication path.

Security properties

- set membership
- public verifiability
- privacy depending on construction

What it does not provide

- Authorization policy by itself.
- Privacy unless the proof hides which member is used.
- Freshness unless the set commitment is current.

Assumptions

The set commitment must be authentic, leaf encoding must be unambiguous, and the verifier must know which root or accumulator state is current for the application.

Post-quantum posture

Depends on the construction. Hash-based Merkle inclusion proofs can be plausibly post-quantum with appropriate hashes, while many algebraic accumulators rely on assumptions that may be quantum-vulnerable.

Confidence model

Confidence comes from public verification of the set commitment, collision resistance or accumulator soundness, and correct binding between the proof and the policy context.

Common constructions

Concrete schemes and families

Scheme	Set commitment	Typical role	Key differences and cautions
Merkle inclusion proof	Merkle root	Allowlists, transparency logs, blockchain state	Hash-based and compact logarithmic proofs; reveals path information unless wrapped in zero knowledge.
Sparse Merkle proof	Sparse Merkle root	Large key spaces, nullifier sets, account state	Supports non-membership patterns; encoding and default nodes must be canonical.
RSA accumulator witness	RSA accumulator value	Compact set membership and revocation	Quantum-vulnerable and setup-sensitive; witness updates are operationally important.
Bilinear accumulator witness	Pairing-based accumulator	Anonymous credentials and specialized protocols	Quantum-vulnerable; setup and subgroup checks matter.
KZG opening proof	Polynomial/vector commitment	Verkle-style state and data availability	Very compact openings; pairing and setup assumptions are central.
ZK membership proof	Merkle or accumulator proof inside a ZKP	Anonymous membership	Hides which member is used, but inherits proof-system and set-root assumptions.

Use cases

- Proving eligibility.
- Showing inclusion in a Merkle tree.
- Anonymous membership when combined with zero knowledge.

Composition patterns

- Membership must be bound to the correct policy context.
- Proof paths or public inputs can reveal the member.

Common adjacent concepts: Accumulators and Merkle Trees, Anonymous Membership, Zero-Knowledge Proofs.

Failure modes and anti-patterns

- Using stale set roots.
- Ambiguous leaf encoding.
- Revealing the member through the proof path or metadata.

Maturity and deployment

Classified as mature. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: scheme-dependent.

Related concepts

- Accumulators and Merkle Trees
- Anonymous Membership
- Zero-Knowledge Proofs

Further reading

- Merkle, A Digital Signature Based on a Conventional Encryption Function.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Protocols

Protocols Overview

Protocols specify how parties interact. They combine primitives, messages, state transitions, checks, and failure handling.

Protocol questions

- Who participates?
- What are their inputs and outputs?
- What can each adversary observe or corrupt?
- What metadata is public?
- What happens if a party aborts?
- What assumptions are required?

Covered protocol families

- Additional protocol families
- Secure channels
- Password-authenticated key exchange
- Oblivious transfer
- Private information retrieval
- Anonymous credentials
- Anonymous tokens
- Blind-signature credentials
- Nullifiers
- Oblivious pseudorandom functions
- Private set intersection
- Secure aggregation
- Multi-party computation
- Mixnets
- Electronic voting

Safety note

A protocol can fail even if every primitive inside it is sound.

Protocol coverage is still intentionally grouped in some areas. The grouped page remains a map for adjacent families and comparison context.

Additional Protocol Families

Additional Protocol Families

This page is now a routing overview for protocol families that appear repeatedly in privacy-preserving systems, secure messaging, identity, and private payments.

This is a grouped overview. Secure Channels, Oblivious Transfer, Private Information Retrieval, and Password-Authenticated Key Exchange now have standalone pages.

Protocol matrix

Protocol family	Goal	Typical building blocks	Main caution
Oblivious transfer	Receiver obtains one of several sender messages without revealing which one	public-key crypto, OPRFs, OT extension	Core MPC building block; adversary model matters.
Private information retrieval	Client retrieves a database item without revealing which item	coding, homomorphic encryption, PIR-specific protocols	Server/database size and access-pattern assumptions matter.
Password-authenticated key exchange	Parties derive a strong session key from a password without exposing it to offline guessing	PAKE protocol, KDF, authenticated transcript	Not the same as password hashing.
Secure channels	Establish authenticated, encrypted sessions	key exchange, signatures/PSKs, KDFs, AEAD	Metadata and endpoint compromise remain.
Anonymous tokens	Issue or redeem tokens without stable identity linkage	blind signatures, OPRFs, accumulators, rate limits	Token privacy depends on issuance, redemption, and transport metadata.
Blind-signature credentials	Issue credentials using blind signatures	blind signatures, selective disclosure, issuer policy	Blind issuance does not solve revocation or misuse.
Private-payment protocols	Transfer value while hiding payer, payee, amount, or linkage under a model	commitments, nullifiers, ZKPs, e-cash, ledgers	Ledger metadata and double-spend rules dominate.

Oblivious transfer

Oblivious transfer (OT) lets a receiver learn one selected message from a sender while the sender does not learn the selection and the receiver does not learn the other messages.

Security goals:

- Receiver choice privacy.
- Sender message privacy for unchosen messages.
- Correctness under the stated adversary model.

Non-goals:

- Hiding that an interaction occurred.
- Fairness or guaranteed delivery by itself.
- General MPC without additional protocol structure.

Assumptions and failure modes:

- OT protocols vary from public-key-based base OT to efficient OT extension.
- Failures include using semi-honest OT in malicious settings, weak correlation checks, and failing to authenticate channels.

Private information retrieval

Private information retrieval (PIR) lets a client retrieve an item from a database without revealing which item was requested.

Security goals:

- Query privacy from one or more servers.
- Correctness of returned data under the protocol model.
- Sometimes database privacy, depending on symmetric PIR variants.

Non-goals:

- Hiding client identity or timing.
- Protecting writes or updates by default.
- Avoiding all leakage from repeated queries.

Assumptions and failure modes:

- Single-server computational PIR relies on cryptographic assumptions; multi-server PIR often relies on non-collusion.
- Failures include server collusion, repeated-query linkage, and database-version ambiguity.

Password-authenticated key exchange

Password-authenticated key exchange (PAKE) lets parties derive a strong shared key using a password while resisting offline dictionary attacks from passive transcripts.

Security goals:

- Session key establishment.
- Resistance to offline password guessing from recorded handshakes.
- Mutual or asymmetric authentication, depending on the PAKE.

Non-goals:

- Protection against online guessing without rate limits.
- Password storage policy.

- Metadata privacy.

Assumptions and failure modes:

- PAKEs must bind identities, transcript, and protocol parameters into the derived key.
- Failures include downgrade, missing identity binding, weak password reset flows, and treating PAKE as a password manager.

Secure channels

Secure-channel protocols protect streams or sessions between endpoints. TLS 1.3, HPKE-based application channels, Noise patterns, Signal X3DH, and Double Ratchet are common examples or components.

Security goals:

- Confidentiality and integrity for session data.
- Endpoint authentication under a certificate, key, pre-shared key, or identity model.
- Forward secrecy when ephemeral secrets and key evolution are used correctly.
- Replay and downgrade protection, depending on protocol.

Non-goals:

- Hiding endpoint IP addresses or timing.
- Protecting compromised endpoints.
- Making all application messages deniable.

Assumptions and failure modes:

- Confidence comes from authenticated key exchange, transcript binding, KDFs, AEAD, and implementation review.
- Failures include certificate/key misbinding, transcript truncation, downgrade negotiation, nonce misuse, and storing session keys too long.

Anonymous tokens

Anonymous-token protocols let a user obtain or redeem authorization tokens while limiting linkability between issuance and redemption.

Security goals:

- Unlinkability between issuance and redemption under the protocol model.
- Issuer authenticity.
- Rate limiting or one-token-per-event controls.

Non-goals:

- Network anonymity.

- Sybil resistance beyond the issuance policy.
- Revocation without extra machinery.

Assumptions and failure modes:

- Tokens may use blind signatures, OPRFs, accumulators, or public redemption logs.
- Failures include unique token metadata, timing linkage, shared browser/device identifiers, and issuer/verifier collusion.

Blind-signature credentials

Blind-signature credentials use blind signatures as an issuance mechanism so the issuer can authorize a holder without seeing the exact token or credential presented later.

Security goals:

- Blind issuance.
- Credential authenticity.
- Limited disclosure or unlinkability when combined with a suitable presentation protocol.

Non-goals:

- Attribute privacy by default.
- Revocation.
- Non-transferability without a holder secret or device binding.

Assumptions and failure modes:

- Confidence comes from blind-signature unforgeability, issuer policy, holder secret protection, and presentation design.
- Failures include over-issuing credentials, linking by metadata, and treating blind signatures as complete anonymous credentials.

Private-payment protocols

Private-payment protocols transfer value while hiding selected transaction details such as payer, payee, amount, or linkage.

Security goals:

- Value conservation.
- Double-spend prevention or detection.
- Transaction privacy under a stated ledger or issuer model.
- Public auditability where needed.

Non-goals:

- Network anonymity by default.

- Compliance, recovery, or governance.
- Protection from amount, timing, or exchange metadata unless designed.

Assumptions and failure modes:

- Designs may use e-cash, blind signatures, commitments, nullifiers, zero-knowledge proofs, accumulators, or threshold decryption.
- Failures include metadata linkage, trusted-issuer abuse, weak note/nullifier design, and confusing local privacy with ledger-wide anonymity.

Post-quantum posture

Depends on the concrete protocol. Many deployed secure-channel, blind-signature, OT, OPRF, and private-payment systems rely on RSA, elliptic curves, pairings, or discrete-log assumptions. Symmetric components may be plausible, but authentication and proof layers must be reviewed separately.

Confidence model

Confidence may come from mathematical-assumption security, public-verifiability, trusted issuers, non-colluding servers, one-honest-party, or client-side secrets. Every protocol page using these families should state the exact model.

Related concepts

- MPC
- Oblivious Pseudorandom Functions
- Anonymous Tokens
- Blind-Signature Credentials
- Private Set Intersection
- Authenticated Encryption
- Advanced Structured Primitives

Further reading

- Rabin, "How to Exchange Secrets by Oblivious Transfer".
- Chor et al., "Private Information Retrieval".
- RFC 9383, "OPAQUE: An Asymmetric PAKE Protocol".
- RFC 8446, "The Transport Layer Security (TLS) Protocol Version 1.3".
- RFC 9180, "Hybrid Public Key Encryption".
- Signal, "The X3DH Key Agreement Protocol" and "The Double Ratchet Algorithm".
- RFC 9576, "The Privacy Pass Architecture".
- Chaum, "Blind Signatures for Untraceable Payments".
- Ben-Sasson et al., "Zerocash: Decentralized Anonymous Payments from Bitcoin".

Secure Channels

Goal

Establish an authenticated, encrypted session between endpoints so application data has confidentiality, integrity, replay protection, and usually forward secrecy under a stated endpoint-authentication model.

Participants

- Client or initiator.
- Server or responder.
- Optional certificate authority, identity provider, pre-shared-key issuer, or key-transparency service.
- Network adversary that can observe, delay, replay, drop, and inject messages.

Inputs and outputs

Inputs:

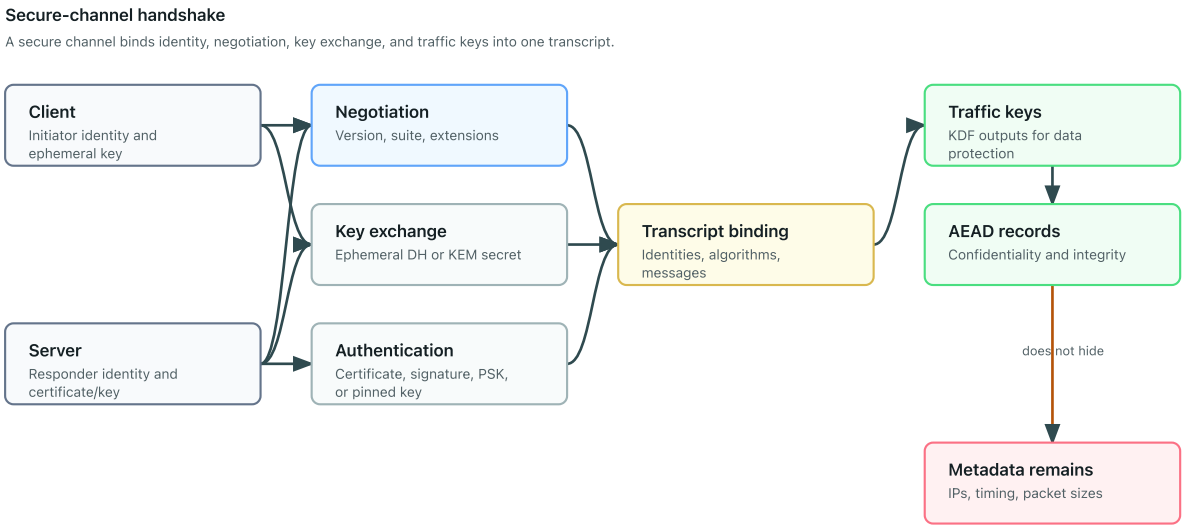
- Endpoint identities, public keys, certificates, pre-shared keys, or trust anchors.
- Supported algorithm suites and protocol versions.
- Fresh randomness and ephemeral key shares.
- Application context such as hostnames, service names, or channel bindings.

Outputs:

- Session traffic keys.
- Authenticated handshake transcript.
- Exported channel bindings or application keys, when supported.
- Failure if authentication, negotiation, or transcript checks fail.

Building blocks

- Key exchange or key encapsulation.
- Digital signatures, certificates, pre-shared keys, or authenticated public keys.
- Key derivation functions.
- Authenticated encryption with associated data.
- Transcript binding and domain separation.



Security goals

- Confidentiality and integrity for session data.
- Endpoint authentication according to the configured identity model.
- Forward secrecy when fresh ephemeral secrets are used and erased.
- Replay and downgrade resistance when the protocol binds transcript, version, and suite choices.
- Key separation between handshake, application data, and exported keys.

Non-goals

- Hiding endpoint IP addresses, packet timing, or traffic volume.
- Protecting compromised endpoints.
- Solving application authorization.
- Making every transcript deniable.
- Preventing all denial of service.

Threat model

The usual model assumes an active network attacker. The attacker can observe and modify traffic but cannot break the selected cryptographic assumptions, compromise endpoint secrets during the protected session, or subvert the trust anchor. Stronger models add post-compromise recovery, key transparency, delegated credentials, anonymity networks, or post-quantum hybrid key establishment.

Protocol sketch

1. Endpoints negotiate a version, cipher suite, and authentication mode.
2. They exchange ephemeral key material or encapsulated secrets.
3. Each side derives handshake secrets from the key exchange and transcript.
4. The authenticated party proves control of a private key, certificate chain, or pre-shared key.

5. Both sides derive traffic keys and bind them to the transcript.
6. Application data is encrypted with AEAD under monotonically managed nonces or sequence numbers (RFC 5116).

Trust assumptions

- Trust anchors, public keys, or pre-shared keys are authentic.
- Ephemeral secrets are generated with strong randomness and erased when required.
- Transcript hashes include identities, algorithms, public keys, and negotiation choices.
- Implementations reject downgrade, replay, certificate, and hostname failures.

Post-quantum posture

Depends on the concrete suite. Classical TLS 1.3, Noise, X25519, ECDSA, EdDSA, and many Signal-style deployments rely on discrete-logarithm assumptions and are quantum-vulnerable under Shor's algorithm (Shor 1994). Symmetric encryption and KDF layers can be plausible with conservative parameters, but authentication and key establishment need post-quantum or hybrid migration; ML-KEM and ML-DSA are the relevant finalized NIST standards for those migration paths (NIST FIPS 203, NIST FIPS 204).

Confidence model

Confidence is mixed: mathematical-assumption for key exchange and authentication, public-key infrastructure or pinned-key trust for endpoint identity, client-side-secret for endpoint private keys, and implementation review for parsing, transcript binding, and state-machine correctness.

Metadata leaks

- Endpoint addresses and routing metadata.
- Server name or certificate metadata unless hidden by additional mechanisms.
- Timing, packet sizes, connection counts, and session duration.
- Authentication method and sometimes client identity.

Failure modes

- Accepting a certificate or public key for the wrong identity.
- Omitting algorithm choices or identities from the transcript.
- Reusing nonces or sequence numbers under an AEAD key (RFC 5116).
- Supporting downgrade to legacy versions or weak suites.
- Storing session secrets too long.
- Treating HPKE or raw Diffie-Hellman as a complete secure channel without authentication and replay handling; HPKE is an encryption framework, not a full transport protocol (RFC 9180).

Variants

- TLS 1.3 for web and service transport (RFC 8446).

- HPKE-based application encryption, often as a component rather than a full channel (RFC 9180).
- Noise handshakes for explicitly selected peer-to-peer patterns.
- Signal X3DH plus Double Ratchet for asynchronous secure messaging (X3DH, Double Ratchet).
- Post-quantum or hybrid handshakes that combine classical and post-quantum key establishment.

Source-depth notes

Secure-channel source coverage should distinguish transport standards, application encryption components, and messaging protocols. TLS 1.3 is a complete channel protocol (RFC 8446); HPKE is a building block for application encryption (RFC 9180); MLS standardizes group messaging key management (RFC 9420); Signal X3DH and Double Ratchet cover asynchronous messaging patterns (X3DH, Double Ratchet). Post-quantum migration requires reviewing key establishment and authentication separately.

Where it is used

- Web transport security.
- API and service-to-service calls.
- Secure messaging.
- Blockchain peer-to-peer networking.
- Application-layer encrypted objects and session establishment.

Further reading

- RFC 8446: The Transport Layer Security Protocol Version 1.3.
- RFC 5116: An Interface and Algorithms for Authenticated Encryption.
- RFC 9180: Hybrid Public Key Encryption.
- RFC 9420: The Messaging Layer Security Protocol.
- RFC 9750: The Messaging Layer Security Architecture.
- The Noise Protocol Framework.
- Signal X3DH and Double Ratchet.
- RFC 5869: HKDF.

Password-Authenticated Key Exchange

Goal

Let parties establish a strong session key from a low-entropy password while preventing passive transcript capture from enabling offline password guessing.

Participants

- Client or user with a password.
- Server or peer with either the same password, a transformed password record, or an augmented PAKE envelope.
- Active network attacker.

Inputs and outputs

Inputs:

- Password or password-derived registration record.
- Party identities and protocol parameters.
- Fresh randomness.
- Optional server public key, OPRF key, or credential envelope.

Outputs:

- Shared session key if authentication succeeds.
- Failure if the password, record, or transcript check fails.

Building blocks

- PAKE-specific algebra or OPRF construction.
- Key derivation functions.
- Authenticated transcript binding.
- Sometimes a secure channel, envelope encryption, or server-side rate limiting.

Security goals

- Session key establishment from a password.
- Resistance to offline dictionary attacks from recorded handshakes.
- Mutual or asymmetric authentication, depending on the protocol.
- Protection against server-record compromise in augmented PAKEs, depending on the design.

Non-goals

- Protection against unlimited online guessing.

- Password reset, account recovery, or credential lifecycle policy.
- Metadata privacy.
- Security if the password is phished and used in a real session.

Threat model

The network attacker can record, replay, and modify messages. PAKE should prevent the attacker from testing password guesses offline from transcripts. The server still needs rate limits and account protections against online guessing. Augmented PAKEs reduce damage from server-record compromise but do not make weak passwords strong.

Protocol sketch

1. The client and server exchange PAKE messages derived from the password or password record.
2. The protocol hides enough password-dependent material to prevent offline testing.
3. Both parties bind identities, parameters, and messages into a transcript.
4. A key confirmation step verifies that both parties derived the same session key.
5. The session key is used directly or fed into a secure-channel key schedule.

Trust assumptions

- The PAKE protocol is implemented exactly as specified.
- Password registration records are generated and stored correctly.
- Identities, protocol versions, and suite parameters are transcript-bound.
- Online guessing is rate limited and monitored.

Post-quantum posture

Depends on the PAKE. OPAQUE as standardized in RFC 9383 uses prime-order-group OPRF suites and is quantum-vulnerable at that layer. Some PAKE research explores post-quantum assumptions, but deployment maturity and standardization vary.

Confidence model

Confidence is mixed: mathematical-assumption for the PAKE and OPRF layers, client-side-secret for the password, operational controls for online guessing, and implementation review for transcript and envelope handling.

Metadata leaks

- Account identifier, timing, and login frequency.
- Failure behavior, lockout state, and recovery flows.
- Server identity and supported suite metadata.

Failure modes

- Treating password hashing as a PAKE.
- Missing identity or parameter binding.
- Allowing downgrade to a weaker password protocol.
- Leaking password-derived material through logs, telemetry, or reset flows.
- Omitting online rate limits.

Variants

- Balanced PAKE, where both parties share a password.
- Augmented PAKE, where the server stores a password-derived record.
- OPAQUE-style OPRF-based PAKE.
- Password-authenticated secure-channel handshakes.

Where it is used

- Password login protocols.
- Device pairing.
- Password-derived secure channels.
- Systems that want less server-side verifier exposure than traditional password hashes.

Further reading

- RFC 9383: OPAQUE.
- RFC 9497: OPRFs Using Prime-Order Groups.
- RFC 9106: Argon2.

Oblivious Transfer

Goal

Let a receiver obtain one selected message from a sender while the sender does not learn which message was selected and the receiver learns nothing about the unselected messages.

Participants

- Sender with two or more candidate messages.
- Receiver with a private selection.
- Optional setup, correlation generator, or base-OT functionality in efficient extensions.

Inputs and outputs

Inputs:

- Sender messages.
- Receiver choice index or selection bit.
- Security parameters and protocol mode.
- Optional authenticated channel and base OT material.

Outputs:

- Receiver obtains only the selected message.
- Sender learns no selection information beyond allowed leakage.
- Failure if checks, authentication, or consistency tests fail.

Building blocks

- Public-key encryption or key exchange for base OT.
- OT extension for many cheap OTs from fewer expensive base OTs.
- Hash functions, commitments, correlation checks, and authenticated channels.
- Sometimes OPRFs, MPC preprocessing, or correlated randomness.

Security goals

- Receiver choice privacy.
- Sender message privacy for unchosen messages.
- Correctness under the specified adversary model.
- Consistency against malicious parties when the protocol includes malicious-security checks.

Non-goals

- Hiding that the parties interacted.

- Fairness or guaranteed delivery.
- General multi-party computation by itself.
- Output privacy after the receiver uses the selected message elsewhere.

Threat model

OT can be defined against semi-honest or malicious adversaries. Semi-honest security assumes parties follow the protocol but try to infer extra information. Malicious security allows arbitrary deviations and requires extra consistency checks. Using a semi-honest OT where malicious behavior is possible is a common system-level error.

Protocol sketch

1. The receiver encodes a private choice into public-key or correlation material.
2. The sender uses that material to mask each candidate message.
3. Only the receiver's selected branch can be unmasked.
4. In extension protocols, a small number of base OTs bootstrap many derived OTs.
5. Malicious-secure variants add checks that parties used consistent choices and correlations.

Trust assumptions

- The selected OT construction matches the adversary model.
- Channels authenticate participants.
- Base OTs and extension seeds are generated correctly.
- Correlation checks are not skipped when malicious security is required.

Post-quantum posture

Depends on the construction. Many deployed base OTs use classical public-key assumptions such as elliptic-curve or finite-field discrete logarithms and are quantum-vulnerable. OT extension mostly uses symmetric primitives after setup, but the base OT and authentication layers determine the overall posture.

Confidence model

Confidence comes from mathematical assumptions, authenticated channels, and adversary-model alignment. Some systems also rely on one-honest-party or honest-majority assumptions when OT is embedded in larger MPC protocols.

Metadata leaks

- Participation, timing, and message sizes.
- Number of OTs and batch structure.
- Abort behavior that may reveal malformed inputs or failed checks.

Failure modes

- Using semi-honest OT in malicious environments.
- Skipping correlation checks in OT extension.
- Failing to authenticate channels.
- Reusing seeds, choices, or setup material across domains.
- Treating OT as a complete privacy-preserving computation protocol.

Variants

- 1-out-of-2 OT and 1-out-of-n OT.
- Random OT and correlated OT.
- Base OT and OT extension.
- Semi-honest, covert, and malicious-secure OT.
- Post-quantum candidate OTs from lattice or code assumptions.

Where it is used

- MPC.
- Private set intersection.
- Garbled circuits.
- Secure function evaluation.
- Preprocessing for private computation systems.

Further reading

- Rabin, "How to Exchange Secrets with Oblivious Transfer".
- Canetti, "Universally Composable Security".
- Boneh and Shoup, "A Graduate Course in Applied Cryptography".

Private Information Retrieval

Goal

Let a client retrieve an item from a database without revealing to the server which item was requested.

Participants

- Client with a private query index.
- One or more database servers.
- Optional non-colluding replicas, preprocessing service, or audit layer.

Inputs and outputs

Inputs:

- Client query index.
- Server database or encoded database.
- Public parameters, keys, or preprocessing material.

Outputs:

- Client obtains the requested record.
- Server learns limited information about the query according to the PIR model.
- Server may or may not learn that a client queried at a given time.

Building blocks

- Coding-theoretic PIR.
- Homomorphic encryption or other computational PIR tools.
- Secret sharing or replicated databases for multi-server PIR.
- Authenticated data structures when database integrity matters.

Security goals

- Query privacy from one or more servers.
- Correctness of the returned record.
- Sometimes database privacy in symmetric PIR, where the client should learn only the requested item.
- Optional robustness against faulty or malicious servers.

Non-goals

- Hiding the client's network identity.
- Hiding query timing or frequency.
- Protecting writes, updates, or database freshness by default.

- Preventing leakage from repeated queries or returned content.

Threat model

Single-server computational PIR relies on cryptographic assumptions. Multi-server information-theoretic PIR often relies on non-collusion between servers. If servers collude beyond the threshold, query privacy can fail. Malicious-server models require additional integrity and consistency checks.

Protocol sketch

1. The client encodes the desired index into a private query.
2. The server or servers evaluate the query over the database.
3. The client decodes the response to recover the requested item.
4. Optional integrity checks prove that the response corresponds to the committed database version.

Trust assumptions

- Non-collusion holds in replicated-server designs.
- Computational assumptions hold in single-server designs.
- Database version and encoding are bound to the query.
- Integrity checks are used when the server may return malformed data.

Post-quantum posture

Depends on the construction. Coding-theoretic and information-theoretic multi-server PIR can avoid classical public-key assumptions but depend on non-collusion and data replication. Homomorphic-encryption-based PIR may be plausibly post-quantum when based on well-parameterized lattice schemes.

Confidence model

Confidence may come from non-collusion, mathematical assumptions, public-verifiability for database commitments, or operational audit of server independence.

Metadata leaks

- Query timing, client identity, and request frequency.
- Database partition or shard being queried.
- Response size and error behavior.
- Repeated-query linkage.

Failure modes

- Assuming one-server PIR is private without cryptographic assumptions.
- Treating non-colluding servers as independent when they share operators or logs.
- Returning stale or malicious database versions.

- Ignoring access-pattern leakage outside the PIR query itself.
- Using PIR where the returned item reveals the query anyway.

Variants

- Single-server computational PIR.
- Multi-server information-theoretic PIR.
- Symmetric PIR.
- Keyword PIR and private lookup systems.
- Batched and preprocessing-heavy PIR.

Where it is used

- Private contact discovery.
- Certificate transparency or key transparency lookup designs.
- Private blocklist and breach lookup.
- Privacy-preserving database queries.

Further reading

- Chor et al., "Private Information Retrieval".
- Boneh and Shoup, "A Graduate Course in Applied Cryptography".
- Metadata Leakage.

Anonymous Tokens

Goal

Issue and redeem authorization tokens while limiting linkability between issuance and redemption.

Participants

- Client or holder.
- Issuer that authorizes token issuance.
- Redeemer or verifier.
- Optional attester, rate limiter, or public redemption log.

Inputs and outputs

Inputs: issuance policy, holder request, token key material, redemption context, and transport metadata.

Outputs: a token or proof accepted by the verifier, or failure if the token is invalid, duplicated, expired, or out of context.

Building blocks

- Blind signatures or OPRFs.
- Token binding and domain separation.
- Rate limits or issuance policy.
- Optional accumulators, public keys, or redemption logs.

Security goals

- Unlinkability between issuance and redemption under the protocol model.
- Issuer authenticity.
- One-token-per-event or rate-limited authorization.
- Context binding for token use.

Non-goals

- Network anonymity.
- Sybil resistance beyond the issuance policy.
- Revocation without extra machinery.
- Protection from device or browser identifiers.

Threat model

Issuers and verifiers may try to link token issuance and redemption. The model must state whether issuer-verifier collusion is allowed and what metadata they observe.

Protocol sketch

1. The client obtains authorization under an issuance policy.
2. The issuer issues a blinded or oblivious token.
3. The client unblinds or derives a redeemable token.
4. The verifier checks authenticity, context, and duplicate-use rules.

Trust assumptions

- Issuer keys and issuance policy are sound.
- The anonymity set is large enough.
- Transport metadata and device identifiers do not re-link redemption.

Post-quantum posture

Depends on blind-signature, OPRF, and authentication suites. Many standardized suites are currently classical.

Confidence model

Confidence comes from trusted-issuer policy, mathematical assumptions, client-side token secrecy, and operational separation between issuance and redemption.

Metadata leaks

- Timing and batch size.
- Client network identifiers.
- Issuer/verifier collusion data.
- Redemption context and device state.

Failure modes

- Unique token metadata.
- Small issuance batches.
- Shared browser identifiers.
- Issuer and verifier logs joined by timing.

Variants

- Blind-signature tokens.
- OPRF-based tokens.
- Publicly verifiable tokens.
- Rate-limited anonymous credentials.

Where it is used

- Privacy Pass-style systems.
- Abuse prevention.
- Private rate limits.
- Anonymous authorization.

Further reading

- RFC 9576: Privacy Pass Architecture.
- RFC 9497: OPRFs Using Prime-Order Groups.
- RFC 9474: RSA Blind Signatures.

Blind-Signature Credentials

Goal

Issue credentials through blind signatures so an issuer can authorize a holder without seeing the exact token or presentation later.

Participants

- Issuer.
- Holder.
- Verifier.
- Optional revocation or status service.

Inputs and outputs

Inputs: issuer policy, holder attributes or eligibility, blinded message, presentation context, and verifier policy.

Outputs: a signed credential or token, and later a verifier decision.

Building blocks

- Blind signatures.
- Holder secrets.
- Selective disclosure or presentation protocol.
- Revocation and status mechanisms.

Security goals

- Blind issuance.
- Credential authenticity.
- Limited disclosure under the presentation protocol.
- Unlinkability when metadata and revocation support it.

Non-goals

- Attribute privacy by default.
- Revocation privacy by default.
- Non-transferability without holder binding.
- Protection from rare-attribute re-identification.

Threat model

Issuers and verifiers may collude to link issuance and presentation. Holders may try to forge or share credentials. Verifiers may over-collect attributes.

Protocol sketch

1. Holder blinds a credential message or token.
2. Issuer checks eligibility and signs the blinded value.
3. Holder unblinds and stores the credential.
4. Holder presents a credential or derived proof to a verifier.
5. Verifier checks issuer authenticity, context, and status policy.

Trust assumptions

- Issuer is trusted for claims.
- Blind signature unforgeability and blindness hold.
- Holder secrets remain protected when non-transferability matters.
- Revocation does not become a tracking beacon.

Post-quantum posture

Depends on the signature and proof system. Common RSA or elliptic-curve designs are quantum-vulnerable.

Confidence model

Confidence comes from trusted-issuer claims, mathematical assumptions, holder secrets, and operational privacy controls around status checks.

Metadata leaks

- Issuance timing.
- Verifier identity.
- Rare disclosed attributes.
- Revocation or status queries.

Failure modes

- Treating blind signatures as complete anonymous credentials.
- Over-issuing credentials.
- Linking by metadata or revocation checks.
- No holder binding when transfer is a threat.

Variants

- Blind-signature bearer credentials.
- Partially blind credentials.
- Selective-disclosure credentials with blind issuance.

Where it is used

- Anonymous authorization.
- Private rate limits.
- Credential wallets.
- Token systems.

Further reading

- Chaum, "Blind Signatures for Untraceable Payments".
- RFC 9474: RSA Blind Signatures.
- W3C Verifiable Credentials Data Model v2.0.

Anonymous Credentials

Goal

Anonymous credentials let a user prove authorization or attributes without revealing a stable identity.

Participants

- Issuer.
- Holder.
- Verifier.

Inputs and outputs

The issuer gives a credential to a holder. Later, the holder proves selected claims to a verifier.

Building blocks

- Digital signatures or credential schemes.
- Zero-knowledge proofs.
- Revocation or status mechanisms.

Security goals

- Selective disclosure.
- Unlinkability between presentations, depending on the scheme.
- Issuer authenticity.

Non-goals

- Network anonymity.
- Coercion resistance.
- Protection if the holder exposes secrets.

Threat model

Designs must consider issuer-verifier collusion, verifier tracking, credential sharing, and revocation leakage; credential-status mechanisms are part of the W3C credential model and can expose privacy-sensitive presentation behavior if composed poorly (W3C VC Data Model 2.0, W3C Bitstring Status List v1.0).

Protocol sketch

1. The issuer verifies eligibility and issues a credential.
2. The holder stores the credential and secret.

3. The holder proves a claim to a verifier.
4. The verifier checks the proof, issuer, context, and revocation status.

Trust assumptions

The issuer may be trusted to issue correctly. Some systems require non-collusion or privacy-preserving revocation infrastructure.

Post-quantum posture

Depends on the credential signature scheme, presentation proof, accumulator, and revocation mechanism. The anonymous-credential pattern itself is not enough to determine post-quantum posture; pairing-based credential signatures inherit quantum-vulnerable discrete-logarithm assumptions under Shor's algorithm (Shor 1994).

Confidence model

Confidence usually depends on a trusted issuer, holder-controlled secrets, verifier-side checks, and revocation infrastructure. Some designs also require non-collusion between issuer and verifier to preserve privacy.

Metadata leaks

Timing, verifier identity, IP addresses, rare attributes, and revocation checks can identify the holder (Metadata Leakage, W3C Bitstring Status List v1.0).

Failure modes

- Attributes are too identifying.
- Revocation checks create tracking.
- Presentations are not bound to the right context.
- Credentials are transferable when the system assumes they are not.

Variants

- Selective disclosure credentials.
- Anonymous credentials with nullifiers.
- Unlinkable presentations.

Concrete schemes and systems

Scheme or family	Typical role	Key differences and cautions
CL signatures / Idemix-style credentials	Anonymous credentials with selective disclosure	Mature academic lineage; issuer trust and revocation design are central (Camenisch and Lysyanskaya).

Scheme or family	Typical role	Key differences and cautions
BBS+ signatures	Selective-disclosure credentials and unlinkable presentations	Pairing-based and quantum-vulnerable; useful for compact multi-message disclosure (IETF BBS draft, Shor 1994).
SD-JWT / selective-disclosure verifiable credentials	Practical web credential ecosystems	Easier web integration, but not automatically unlinkable against issuer/verifier correlation (RFC 9901).
ZK credential systems	Credentials proven inside a zero-knowledge proof	Can hide more metadata, but inherits proof-system assumptions and circuit correctness risk.
Accumulator-based revocation	Private or semi-private status checks	Revocation can reintroduce linkability if freshness checks are not designed carefully.

Source-depth notes

Credential pages should distinguish mature anonymous-credential schemes from web credential profiles that primarily provide issuer authenticity and selective disclosure. BBS-based W3C and IETF work should be checked against its current draft or recommendation status before making deployment claims (W3C Data Integrity BBS Cryptosuites, IETF BBS draft), while SD-JWT is standardized for selective disclosure and SD-JWT VC status should be checked separately (RFC 9901, SD-JWT VC draft).

Where it is used

- Private access control.
- Eligibility proofs.
- Age or membership claims.

Further reading

- Camenisch and Lysyanskaya, An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation.
- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.
- IETF CFRG, The BBS Signature Scheme.
- W3C, Data Integrity BBS Cryptosuites v1.0.
- RFC 9901, Selective Disclosure for JSON Web Tokens.
- IETF OAuth, SD-JWT-based Verifiable Digital Credentials.
- OpenID Foundation, OpenID for Verifiable Credential Issuance.
- OpenID Foundation, OpenID for Verifiable Presentations.
- Hyperledger, AnonCreds Specification.
- W3C, Bitstring Status List v1.0.

Nullifiers

Goal

A nullifier helps enforce one-person-one-action, or one-secret-one-action, without directly revealing the identity behind the action.

Participants

- User: holds a secret or credential.
- Verifier or system: checks whether the action is valid.
- Public registry or ledger: records used nullifiers to prevent reuse.

Inputs and outputs

- Input: a user secret and a context such as an election, airdrop, or application identifier.
- Output: a public nullifier value.

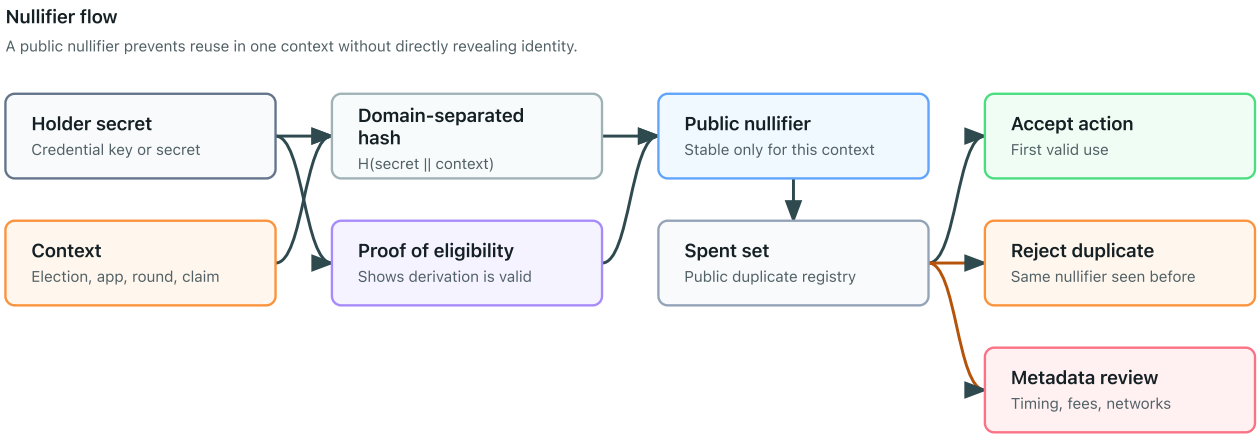
A common mental model is:

$$N = H(\text{secret} \,\|\, \text{context})$$

For the same secret, changing the context should change the nullifier:

$$H(s \,\|\, c_1) \neq H(s \,\|\, c_2)$$

Real systems normally derive nullifiers inside a larger protocol and may prove correctness with a zero-knowledge proof.



Building blocks

- Hash functions.

- Anonymous credentials or membership proofs.
- Zero-knowledge proofs.
- A public spent-nullifier set.

Security goals

- Prevent double use in the same context.
- Avoid revealing the user's identity directly.
- Avoid linking actions across different contexts when domain separation is correct.

Non-goals

- A nullifier does not by itself prove eligibility.
- A nullifier does not hide network metadata.
- A nullifier does not prevent coercion.
- A nullifier does not protect a user whose secret is compromised.

Threat model

The system assumes adversaries may observe public nullifiers and try to link them to users or reuse credentials. The design must prevent cross-context linkage and reject duplicate nullifiers.

Protocol sketch

1. A user obtains or holds an eligibility secret.
2. For a specific context, the user derives a nullifier.
3. The user proves, often in zero knowledge, that the nullifier was derived from an eligible secret.
4. The system checks that the nullifier has not appeared before.
5. The system records the nullifier after accepting the action.

Trust assumptions

Trust assumptions depend on the issuer, eligibility registry, proof system, and public registry. If the issuer can deanonymize credentials or the registry censors submissions, privacy and availability may fail.

Post-quantum posture

Depends on the surrounding stack. A hash-derived nullifier can be plausibly post-quantum if the hash function and parameters are appropriate, but eligibility proofs, credential signatures, and membership proofs may rely on quantum-vulnerable assumptions.

Confidence model

Confidence comes from the holder secret, domain-separated context, public spent-nullifier registry, and a proof that the nullifier was derived from an eligible secret. The nullifier alone only supports non-reuse; it does not establish eligibility.

Metadata leaks

Nullifiers can still be linked through timing, network address, account funding, transaction fees, wallet behavior, or reused contexts.

Failure modes

- Reusing a context across applications links actions.
- Omitting eligibility proofs allows arbitrary nullifiers.
- Weak secrets allow guessing attacks.
- Poor domain separation creates accidental cross-context linkage.
- Public ordering can expose behavioral patterns.

Variants

- Per-application nullifiers.
- Per-election nullifiers.
- Rate-limited nullifiers.
- Nullifiers derived from anonymous credentials.

Concrete constructions and patterns

Construction	Typical role	Key differences and cautions
Hash nullifier	One-use marker derived from a secret and context	Simple shape, but weak secrets or poor domain separation can make it guessable or linkable.
ZK-derived nullifier	Public nullifier plus proof of correct derivation	Common in anonymous membership systems; statement must bind eligibility, context, and nullifier.
Serial-number e-cash pattern	Preventing double-spend of anonymous tokens	Mature idea, but issuance, spend, and revocation details vary by system.
Semaphore-style nullifier	Anonymous signal or one-action-per-group pattern	Tied to Merkle membership and zero-knowledge proof assumptions.
Rate-limit nullifier	Allows limited actions per epoch or context	Context design controls linkability and replay boundaries.

Where it is used

- Private voting.
- Anonymous airdrops.
- Anonymous signaling.
- Private access systems with rate limits.

Further reading

- Camenisch and Lysyanskaya, An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation.

- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.

Oblivious Pseudorandom Functions

Goal

An oblivious pseudorandom function (OPRF) lets a client learn `F(k, x)` for its private input `x` and a server-held key `k`, without the server learning `x` or the output and without the client learning `k`.

Participants

- Client: holds the private input.
- Server: holds the PRF key.
- Optional verifier role: in verifiable variants, the client checks that the server used the expected key.

Inputs and outputs

- Client input: private value `x`.
- Server input: secret PRF key `k`.
- Output: the client learns `F(k, x)`.
- Public parameters: group, hash-to-group/hash-to-scalar rules, domain separation tags, and protocol mode.

Building blocks

- A pseudorandom function abstraction.
- A group or other algebraic structure suitable for the construction.
- Blinding and unblinding operations.
- Hash-to-group or encoding rules.
- Optional zero-knowledge proof of correct evaluation for verifiable OPRFs.

Security goals

- Client input privacy from the server.
- Server key privacy from the client.
- Pseudorandomness of the output to parties without the key.
- Verifiability in VOPRF variants, where the client can check the server used the advertised key.

Non-goals

- Hiding that a client contacted the server.
- Preventing online rate limits or denial of service.
- Protecting low-entropy inputs from guessing once outputs are exposed.
- Providing anonymity by itself.
- Making the server stateless or non-censoring.

Threat model

The usual model considers a malicious or curious server that should not learn the client's input and a malicious client that should not learn the server key or evaluate the PRF on arbitrary values without interaction. Application-level privacy also depends on transport privacy, rate limiting, replay rules, and whether inputs have enough entropy.

Protocol sketch

1. The client maps `x` into the protocol domain and blinds it.
2. The client sends the blinded value to the server.
3. The server evaluates using key `k` on the blinded value.
4. The server returns the evaluated value, and in verifiable modes also returns a proof.
5. The client verifies any proof, unblinds the response, and derives `F(k, x)`.

Trust assumptions

The algebraic assumptions for the chosen construction must hold, encodings and domain separation must be unambiguous, the server key must remain secret, and implementations must validate group elements and reject malformed inputs.

Post-quantum posture

Depends on the construction. Standard prime-order-group OPRFs inherit discrete-logarithm vulnerability to large quantum computers. Post-quantum OPRF designs exist as research and engineering work, but posture must be assessed per construction and parameter set.

Confidence model

Confidence comes from mathematical-assumption hardness, server key secrecy, public parameter correctness, and client-side verification in VOPRF or POPRF modes. Operational confidence also depends on server availability and abuse controls.

Metadata leaks

- Client-server contact timing.
- Server identity.
- Request volume and retry patterns.
- Public input in partially oblivious variants.
- Account, network, or payment metadata outside the OPRF transcript.

Failure modes

- Reusing an OPRF output as if it were an authentication token without binding it to context.
- Forgetting domain separation between applications.
- Accepting invalid group elements or non-canonical encodings.

- Using low-entropy inputs without a rate-limited or password-hardened design.
- Assuming OPRFs provide anonymity or hide network metadata.
- Treating a non-verifiable OPRF as if the server key were publicly committed.

Variants

- OPRF: client learns the PRF output; server learns neither input nor output.
- VOPRF: client can verify the server used the expected key.
- POPRF: public input is included in the PRF computation, useful for context binding.
- Application-specific designs: password-hardening services, privacy-preserving tokens, and private set intersection variants.

Where it is used

- Password-hardened authentication and secret recovery.
- Privacy-preserving rate limits or token issuance.
- Private set intersection.
- Credential and anonymous-token systems.
- Some privacy-preserving contact discovery designs.

Further reading

- RFC 9497, Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups.
- Jarecki and Liu, Efficient Oblivious Pseudorandom Function with Applications to Adaptive OT and Secure Computation of Set Intersection.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Private Set Intersection

Goal

Private set intersection (PSI) lets parties learn the overlap between their private sets while revealing as little as possible about non-overlapping elements.

Participants

- Two or more set holders.
- Optional helper or server roles in delegated, outsourced, or multi-party variants.
- Optional auditor or verifier in variants that prove correct computation.

Inputs and outputs

- Inputs: each participant's private set.
- Outputs: the intersection, the size of the intersection, or a function of matching records, depending on the protocol.
- Public parameters: hash functions, encodings, protocol suite, security level, and sometimes public keys or setup material.

Building blocks

- Oblivious pseudorandom functions (OPRFs).
- Diffie-Hellman-style blinding or public-key encryption.
- Oblivious transfer.
- Garbled circuits or multi-party computation.
- Homomorphic encryption in some constructions.
- Hashing, canonical encoding, and domain separation.

Security goals

- Hide non-matching elements under the stated threat model.
- Reveal only the intended output: intersection, cardinality, or approved function.
- Bind comparisons to canonical encodings so equal items match and distinct items do not accidentally collide.
- Support semi-honest or malicious security depending on the construction.

Non-goals

- Hiding the fact that a PSI run happened.
- Hiding set sizes unless the protocol pads or otherwise protects them.
- Preventing inference from the intersection itself.
- Handling fuzzy, approximate, or dirty data safely by default.

- Providing fairness if one party aborts after learning output.

Threat model

PSI protocols vary sharply. A semi-honest protocol assumes participants follow the protocol but try to learn extra information from transcripts. A malicious-secure protocol must handle malformed inputs, selective aborts, and attempts to bias or expand the result. Multi-party, delegated, and asymmetric PSI variants add different collusion and availability assumptions.

Protocol sketch

One common OPRF-style two-party shape is:

1. The client encodes and blinds each element in its set.
2. The server evaluates an OPRF on blinded elements without seeing them.
3. The client unblinds outputs and compares them with server-provided encodings of the server set.
4. The client learns matching elements or intersection size, depending on the design.
5. Malicious-secure variants add proofs, consistency checks, or cut-and-choose-style defenses.

Trust assumptions

Inputs must be encoded canonically, the chosen construction's assumptions must hold, each party must use the agreed security parameters, and the output policy must match the privacy claim. If helper servers are used, the protocol must state whether privacy depends on non-collusion.

Post-quantum posture

Depends on the construction. Diffie-Hellman and many OPRF-based PSI protocols are quantum-vulnerable. PSI from symmetric-key multi-party computation or post-quantum assumptions may be plausible, but concrete posture depends on the protocol, parameters, and authentication layer.

Confidence model

Confidence may come from mathematical assumptions, a semi-honest or malicious-secure proof, public verification of commitments or proofs, or non-collusion among helper services. These models are not interchangeable.

Metadata leaks

- Set sizes unless padded.
- Timing, retry, and abort behavior.
- Which party learns the output.
- Intersection size if cardinality is revealed.
- Network identifiers and operational logs.
- Distributional clues from rare or predictable elements.

Failure modes

- Treating PSI as "privacy preserving" without stating exactly what output is revealed.
- Comparing low-entropy identifiers that are easy to guess offline.
- Ignoring set-size leakage.
- Using inconsistent normalization, causing false matches or missed matches.
- Running repeated PSI queries that allow differencing attacks.
- Using a semi-honest protocol against malicious parties.
- Revealing too much through post-processing of matched records.

Variants

- PSI with intersection output.
- PSI-cardinality, where only the intersection size is revealed.
- PSI with associated payloads, where matching records unlock extra values.
- Multi-party PSI.
- Delegated or outsourced PSI.
- Fuzzy or approximate PSI, which usually has stronger leakage concerns.

Where it is used

- Private contact discovery.
- Fraud and abuse signal sharing.
- Privacy-preserving record linkage.
- Audience overlap and measurement.
- Federated learning cohort construction.
- Credential or allowlist checks when full lists should not be shared.

Further reading

- Freedman, Nissim, and Pinkas, Efficient Private Matching and Set Intersection.
- RFC 9497, Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups.
- Bonawitz et al., Practical Secure Aggregation for Privacy-Preserving Machine Learning.
- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.

Secure Aggregation

Goal

Secure aggregation lets a server learn an aggregate of client values without learning each individual value.

Participants

- Clients that hold private values.
- Aggregator or server.
- Sometimes helper servers.

Inputs and outputs

Clients input values. The server learns an aggregate such as a sum or average.

Building blocks

- Secret sharing.
- Pairwise masks.
- Authentication.
- Dropout handling.

Security goals

- Hide individual values from the aggregator under the chosen collusion model.
- Recover the aggregate if enough clients complete the protocol.

Non-goals

- Privacy for tiny groups.
- Protection against malicious values unless validation is added.
- Differential privacy unless added separately.

Threat model

The model must specify how many clients or servers may collude, whether clients can drop out, and whether malicious clients can submit malformed updates.

Protocol sketch

1. Clients authenticate and agree on masks.
2. Each client sends a masked value.
3. The server combines masked values.

4. Masks cancel or are reconstructed according to the protocol.
5. The server learns only the aggregate.

Trust assumptions

Assumptions depend on the number of clients, dropout thresholds, authentication, and helper-server model.

Post-quantum posture

Depends on the transport, authentication, key agreement, and masking primitives. The aggregation pattern can be made from post-quantum components, but many deployed channels and signatures may not be post-quantum today.

Confidence model

Confidence comes from the collusion threshold, dropout model, authentication, and whether helper servers are assumed not to collude. Small cohorts can defeat privacy even if the protocol messages are cryptographically protected.

Metadata leaks

The server may learn which clients participated, when they connected, and the aggregate for small cohorts.

Failure modes

- Small cohorts reveal individuals.
- Malicious clients poison the aggregate.
- Dropout handling leaks masks or values.
- Repeated aggregates enable differencing attacks.

Variants

- Single-server secure aggregation.
- Multi-server aggregation.
- Aggregation with differential privacy.

Concrete protocol families

Family or system	Typical role	Key differences and cautions
Bonawitz-style secure aggregation	Federated learning with client dropout	Pairwise masks cancel in aggregate; dropout handling and cohort size dominate privacy.
Prio / Prio+ style systems	Private telemetry with validity checks	Adds client-side proof or verification machinery so malformed values are harder to inject.

Family or system	Typical role	Key differences and cautions
Secret-shared aggregation	Multi-server private aggregation	Privacy depends on non-collusion or corruption threshold between helper servers.
Homomorphic-encryption aggregation	Encrypted sums with trustee or server decryption	Useful when client coordination is limited; key management and output leakage remain.
Differentially private aggregation	Aggregate release with noise	Not just cryptography; privacy budget and repeated releases are central.

Where it is used

- Federated analytics.
- Federated learning.
- Private telemetry.

Further reading

- Bonawitz et al., Practical Secure Aggregation for Privacy-Preserving Machine Learning.
- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.

Multi-Party Computation

Goal

Multi-party computation (MPC) lets parties jointly compute a function over their inputs while keeping those inputs private under a stated adversary model.

Participants

Multiple parties with private inputs. Some protocols also use dealers or preprocessing parties.

Inputs and outputs

Each party supplies an input. The protocol reveals an output to designated parties.

Building blocks

- Secret sharing.
- Oblivious transfer.
- Oblivious pseudorandom functions in some specialized protocols.
- Commitments.
- Zero-knowledge proofs, in malicious-secure settings.

Security goals

- Input privacy.
- Correctness.
- Sometimes fairness or guaranteed output delivery.

Non-goals

- Hiding the output.
- Preventing inference from the output.
- Simplicity of deployment.

Threat model

MPC protocols differ for semi-honest, malicious, honest-majority, dishonest-majority, synchronous, and asynchronous settings.

Protocol sketch

Parties encode or share inputs, evaluate a circuit or arithmetic computation collaboratively, then reconstruct the allowed output.

Trust assumptions

Assumptions include corruption thresholds, network model, setup, and whether preprocessing is trusted or verifiable.

Post-quantum posture

Depends on the protocol and its building blocks. Information-theoretic MPC variants can avoid public-key assumptions in some settings, while many practical protocols rely on signatures, oblivious transfer, commitments, or channels whose posture must be checked separately.

Confidence model

Confidence is model-specific: honest-majority, dishonest-majority, threshold, semi-honest, or malicious. The page for a concrete MPC protocol should state the maximum corrupt set it tolerates and what happens on abort.

Metadata leaks

Participation, timing, circuit shape, aborts, and output values can leak information.

Failure modes

- Selecting a protocol for the wrong adversary model.
- Revealing too much through outputs.
- Ignoring aborts or fairness.
- Weak authentication between parties.

Variants

- Secret-sharing-based MPC.
- Garbled circuits.
- MPC with preprocessing.
- Threshold signing as a specialized use case.

Concrete protocol families

Family	Common setting	Typical role	Key differences and cautions
Yao garbled circuits	Two-party computation	Boolean-circuit MPC	Often efficient for two parties; oblivious transfer and malicious-security upgrades matter.
GMW	Multi-party Boolean circuits	General MPC with interactive rounds	Round complexity and network latency can dominate.
BGW	Honest-majority arithmetic MPC	Information-theoretic MPC in suitable settings	Requires honest majority and synchronous-style assumptions.

Family	Common setting	Typical role	Key differences and cautions
SPDZ-style protocols	Preprocessed arithmetic MPC	Dishonest-majority computation with offline preprocessing	Preprocessing generation and MAC checks are part of the confidence model.
MASCOT / OT-extension preprocessing	Practical malicious-secure preprocessing	Generating correlated randomness for SPDZ-like protocols	Relies on oblivious transfer and implementation-specific security choices.
Threshold signing protocols	FROST, threshold ECDSA, threshold BLS	Specialized MPC for signing	The signing protocol inherits both MPC threshold assumptions and signature-scheme assumptions.

Where it is used

- Threshold custody.
- Private analytics.
- Auctions.
- Collaborative risk scoring.
- Private set intersection variants.

See also Private Set Intersection, which can be built from MPC techniques or from more specialized protocol families.

Further reading

- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.
- Keller, MP-SPDZ: A Versatile Framework for Multi-Party Computation.
- EMP toolkit contributors, EMP toolkit.
- FRESCO contributors, FRESCO documentation.
- Boneh and Shoup, A Graduate Course in Applied Cryptography.

Mixnets

Goal

A mixnet breaks the link between message senders and outputs by batching, re-encrypting or transforming, and shuffling messages through one or more mix servers.

Participants

- Senders.
- Mix servers.
- Recipients or bulletin board.

Inputs and outputs

Senders submit messages. The mixnet outputs the same logical messages in a shuffled form.

Building blocks

- Public-key encryption.
- Verifiable shuffles.
- Commitments and proofs.

Security goals

- Sender-message unlinkability within an anonymity set.
- Verifiable correct shuffling in some designs.

Non-goals

- Perfect anonymity against global traffic analysis without batching assumptions.
- Protection if all mix servers collude.
- Coercion resistance by itself.

Threat model

The model must state how many mix servers may be corrupt and what the adversary observes about timing and network traffic.

Protocol sketch

Messages are collected into a batch, passed through mixes that transform and shuffle them, and then published or delivered.

Trust assumptions

Privacy often relies on at least one honest mix server and adequate batching.

Post-quantum posture

Depends on the encryption, signatures, and shuffle proofs used by the mixnet. A mixnet can have a one-honest-server privacy model while still relying on quantum-vulnerable public-key primitives.

Confidence model

The common confidence model is one-honest-party plus batching: privacy can hold if at least one mix server honestly shuffles and enough messages are mixed together. Verifiable mixnets add public verification for correct shuffling.

Metadata leaks

Batch size, timing, message size, and participation patterns can reveal users.

Failure modes

- Small batches.
- All mixes collude.
- Malformed shuffles.
- Side-channel leakage through message formats.

Variants

- Chaumian mixnets.
- Verifiable mixnets.
- Decryption mixnets.

Concrete protocols and packet formats

Family or system	Typical role	Key differences and cautions
Chaumian mixnets	Batched anonymous message delivery	Privacy depends on batching and at least one honest mix.
Verifiable shuffle mixnets	Elections and public bulletin boards	Adds proofs that shuffles are correct; proof system and public auditability matter.
Decryption mixnets	Encrypted ballots or messages decrypted through mixes	Mix servers transform and decrypt layers; trustee collusion breaks privacy.
Sphinx packet format	Anonymous messaging packets	Hides routing metadata inside fixed-format packets; timing analysis still matters.
Loopix-style mixnets	Continuous-time anonymous messaging	Adds delays and cover traffic; latency and traffic assumptions are part of the model.

Where it is used

- Electronic voting.
- Anonymous messaging.
- Privacy-preserving publication systems.

Further reading

- Chaum, Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms.
- Benaloh, Verifiable Secret-Ballot Elections.

Electronic Voting

Goal

Electronic voting systems aim to collect, protect, tally, and verify votes under demanding privacy, integrity, and coercion constraints.

Participants

- Voters.
- Election authority.
- Trustees or tally authorities.
- Observers and auditors.

Inputs and outputs

Voters input ballots. The system outputs a tally and audit evidence.

Building blocks

- Authentication or eligibility checks.
- Ballot encryption.
- Zero-knowledge proofs.
- Mixnets or homomorphic tallying.
- Threshold decryption.

Security goals

- Eligibility.
- Ballot secrecy.
- Verifiability.
- Integrity.
- Coercion resistance, in stronger systems.

Non-goals

No single cryptographic primitive makes a voting system safe. Operational procedures, usability, legal rules, and public auditability matter.

Threat model

Voting systems must consider malicious voters, corrupt authorities, compromised devices, coercers, network observers, and public trust.

Protocol sketch

Eligible voters cast protected ballots. Ballots are checked for validity, anonymized or aggregated, tallied, and audited.

Trust assumptions

Assumptions may include trustee honesty thresholds, device integrity, public bulletin boards, and audit processes.

Post-quantum posture

Depends on the election cryptography: voter authentication, ballot encryption, proofs, signatures, mixnets, and trustee key management may each have different posture. A voting system should not receive a single post-quantum label unless every relevant layer is classified.

Confidence model

Confidence usually comes from threshold trustees, public verifiability, eligibility authorities, client-device assumptions, and audit procedures. Ballot secrecy may be `t-of-n`, while integrity may come from public verification and challenge processes.

Metadata leaks

Timing, voter check-in, device identifiers, and small precinct totals can leak information.

Failure modes

- Validity checks leak votes.
- Receipts enable coercion.
- Malware changes ballots before encryption.
- Trustees collude.
- The audit trail is too complex for public confidence.

Variants

- Mixnet-based voting.
- Homomorphic tallying.
- End-to-end verifiable voting.
- Coercion-resistant voting protocols.

Concrete systems and protocol families

Family or system	Tally/privacy shape	Key differences and cautions
Helios-style voting	Public bulletin board, encrypted ballots, public verification	Useful for low-coercion settings; not coercion-resistant by default.

Family or system	Tally/privacy shape	Key differences and cautions
Mixnet tallying	Ballots are shuffled before decryption or publication	Stronger unlinkability when at least one mix is honest; shuffle proofs and batch size matter.
Homomorphic tallying	Encrypted ballots combine into an encrypted tally	Efficient for simple ballot formats; validity proofs must prevent malformed ballots.
Threshold decryption elections	Trustees jointly decrypt tally or ballots	Ballot secrecy depends on trustee threshold and key ceremony.
Coercion-resistant protocols	Receipt-free or fake-credential-resistant designs	Much harder system problem; usability and registration assumptions are critical.
DAO voting experiments	On-chain eligibility, nullifiers, ZK proofs, public tally	Ledger metadata, wallet funding, and coercion risks often dominate cryptography.

Where it is used

- Government elections in limited settings.
- Organization voting.
- DAO governance experiments.

Further reading

- Benaloh, Verifiable Secret-Ballot Elections.
- Chaum, Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms.
- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.

Systems and Applications

Systems and Applications Overview

Systems and applications combine assumptions, primitives, proof systems, protocols, and operational controls into user-facing behavior. These pages use a case-study format, but their taxonomy level is `system`.

Current systems and applications

System or application	Main design question
Private payments	How can value move while limiting transaction linkage and preserving monetary rules?
ZK rollups	How do validity proofs compose with data availability, sequencing, and governance?
Secure messaging	How do channels, ratchets, devices, backups, and metadata interact?
Identity wallets	How do credentials, selective disclosure, issuers, and revocation compose?
Encrypted mempools	Which ordering and committee assumptions remain after transaction contents are encrypted?
Private machine-learning analytics	What do secure aggregation, MPC, FHE, and output controls each protect?
Private DAO voting	How do eligibility, ballot secrecy, duplicate-vote prevention, and tallying compose?
Coercion-resistant voting	What additional system properties are needed beyond ballot secrecy?
Anonymous airdrops	How can eligibility and one-time claims be enforced without public identity linkage?

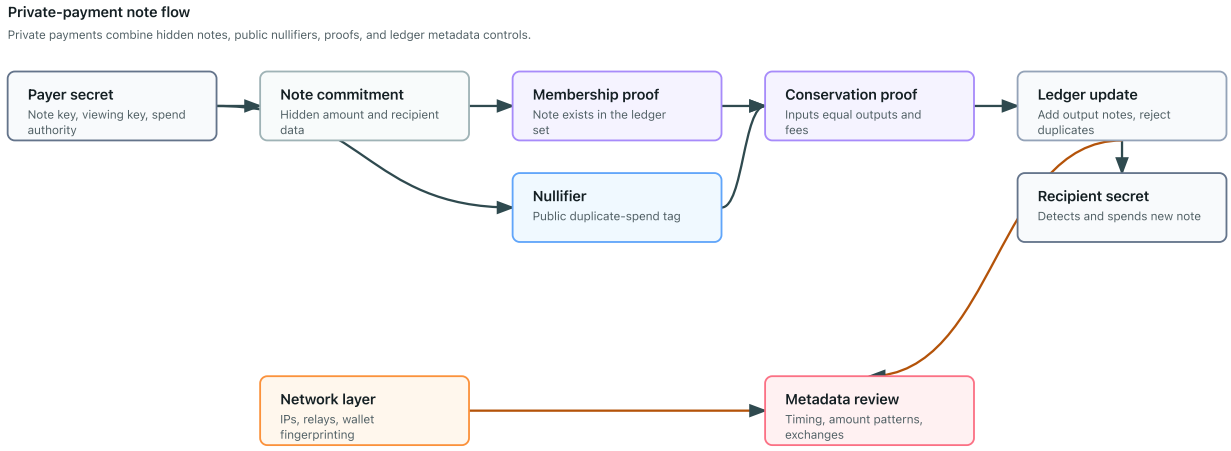
Reading rule

Track goals, non-goals, building blocks, leaks, and maturity warnings separately.

Private Payments

Overview

Private-payment systems transfer value while hiding selected details such as payer, payee, amount, balance, or transaction linkage. No design hides everything: privacy depends on the ledger model, issuer model, wallet behavior, network layer, and metadata available to observers; this difference is visible across e-cash and shielded-ledger designs (Chaum, Fiat, and Naor, Zerocash).



Goals

- Preserve value conservation.
- Prevent or detect double spending.
- Hide transaction linkage, participants, and amounts under a stated model.
- Allow public or issuer-side audit of monetary rules.
- Support recovery, viewing, or compliance workflows only when explicitly designed.

Non-goals

- Network anonymity by default.
- Solvency, governance, or monetary policy guarantees.
- Protection from exchange, wallet, timing, or amount metadata.
- Perfect privacy with small anonymity sets.
- Safe custody or key recovery by itself.

Building blocks

- Commitments to notes, balances, or amounts.
- Nullifiers or serial numbers for duplicate-spend detection.
- Zero-knowledge proofs for conservation and authorization.
- Accumulators or Merkle trees for note membership.

- Blind signatures or issuer ledgers in e-cash systems.
- Secure channels and wallet key management.

Metadata leaks

- Deposit and withdrawal timing.
- Amounts, denominations, fees, or change patterns.
- Network addresses and wallet fingerprinting.
- Exchange, bridge, or merchant records.
- Viewing-key, compliance, or recovery workflows.

Post-quantum posture

Depends on the full stack. Many deployed private-payment systems use elliptic-curve commitments, pairings, or SNARKs that are quantum-vulnerable under Shor's algorithm (Shor 1994). Hash-based proof systems and symmetric components may be plausible, but signatures, commitments, note encryption, and ledger authentication must all be reviewed (STARKs).

Confidence model

Confidence is mixed: public-verifiability for ledger rules and proofs, client-side-secret for notes and nullifiers, trusted-issuer for account or e-cash systems, and operational trust for wallets, exchanges, bridges, and network routing.

Failure modes

- Linking deposits, withdrawals, and payments by timing or amount.
- Small anonymity sets.
- Wallet compromise or viewing-key leakage.
- Nullifier or serial-number reuse.
- Broken note commitment or memo encryption.
- Trusted issuer abuse or insolvency in issuer-backed systems.
- Treating ledger privacy as user anonymity.

Source-depth notes

Private-payment source coverage should separate e-cash lineage, shielded-ledger protocol specifications, and metadata analysis. A deployed protocol specification can describe note and nullifier mechanics, but it does not by itself prove wallet, exchange, bridge, or network privacy (Zcash Protocol Specification, Metadata Leakage).

Related concepts

- Nullifiers
- Anti-Double-Use Nullifiers

- Commitments
- Zero-Knowledge Proofs
- Private Information Retrieval

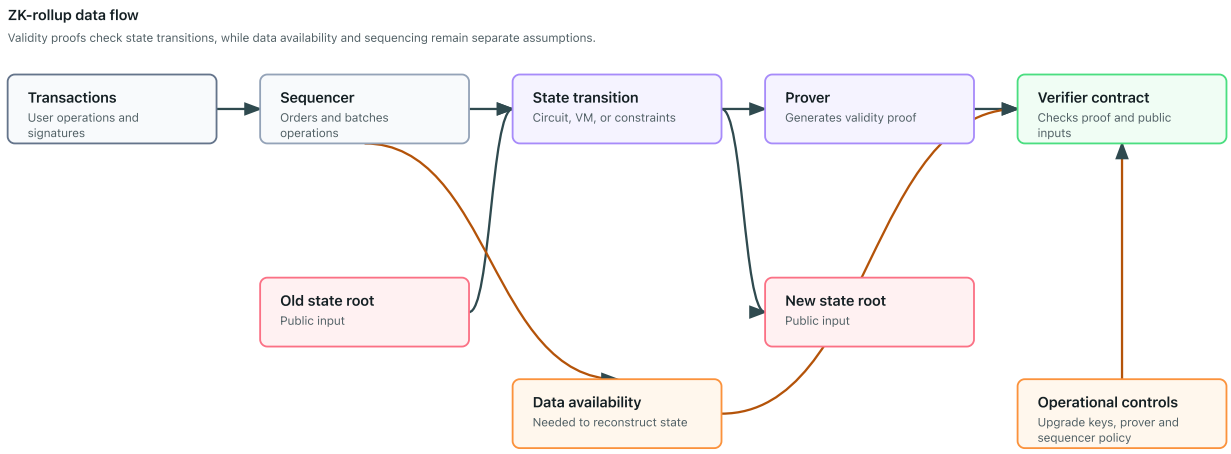
Further reading

- Chaum, Fiat, and Naor, "Untraceable Electronic Cash".
- Zerocash: Decentralized Anonymous Payments from Bitcoin.
- Chaum, "Blind Signatures for Untraceable Payments".
- Zcash Protocol Specification.
- The Orchard Book.
- Metadata Leakage.

ZK Rollups

Overview

ZK rollups use succinct validity proofs to convince verifiers that a batch of state transitions was applied correctly. In many deployed systems, "ZK" means validity proof, not necessarily transaction privacy; the privacy question depends on what is public input, witness data, and on-chain calldata rather than on the proof-family name alone (Groth16, STARKs).



Goals

- Verify many state transitions with low verifier cost.
- Preserve the base chain's ability to reject invalid state updates.
- Reduce on-chain execution work.
- Sometimes provide privacy when transaction data or witnesses are hidden.

Non-goals

- Data availability by itself.
- Decentralized sequencing or proving by default.
- Privacy unless the rollup explicitly hides inputs, outputs, and metadata.
- Correctness of application logic outside the proven transition.
- Protection from governance or upgrade-key misuse.

Building blocks

- Arithmetization of the state-transition function.
- Polynomial commitments or FRI-style commitment layers.
- SNARKs, STARKs, or recursive proof systems.
- Public inputs for old state root, new state root, batch data, and verifier keys.
- Data availability mechanism.

- Sequencer, prover, and verifier contracts or services.

Metadata leaks

- Batch timing and ordering.
- Public transaction data unless privacy is part of the design.
- Sequencer behavior and censorship patterns.
- Prover timing, fees, and failed-batch behavior.
- Bridge deposits and withdrawals.

Post-quantum posture

Depends on the proof system and surrounding stack. Pairing-based SNARK rollups are quantum-vulnerable because pairing groups inherit discrete-logarithm vulnerability under Shor's algorithm (Shor 1994). STARK-style systems based on hashes and FRI are often treated as plausibly post-quantum with conservative parameters (Ben-Sasson et al. 2018), but signatures, bridges, upgrade keys, and data-availability commitments may remain vulnerable.

Confidence model

Confidence is mixed: public-verifiability for validity proofs, mathematical-assumption or transparent soundness for the proof system, operational trust in sequencers and provers, and governance trust in upgrade mechanisms.

Failure modes

- Proving a transition relation that does not match intended application logic; circuit constraints still require separate review from application source code (Circom documentation).
- Verifying a proof against the wrong verifier key or public inputs.
- Data unavailable even though the validity proof verifies.
- Centralized sequencer censorship.
- Trusted setup compromise for setup-dependent proof systems.
- Bridges or upgrade keys bypassing the proof system.

Related concepts

- Proof-System Components
- Polynomial Commitments
- SNARKs, STARKs, and Bulletproofs
- Transcript Binding
- Private Payments

Further reading

- Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity".

- Kate, Zaverucha, and Goldberg, "Constant-Size Commitments to Polynomials and Their Applications".
- Groth, "On the Size of Pairing-Based Non-interactive Arguments".
- Ben-Sasson et al., "Fast Reed-Solomon Interactive Oracle Proofs of Proximity".
- iden3, Circom documentation.

Secure Messaging

Overview

Secure messaging systems protect message contents across asynchronous delivery, device changes, and compromise scenarios. They are systems, not just encrypted channels; X3DH and Double Ratchet cover one asynchronous messaging design family, while MLS standardizes group messaging key management (Signal X3DH, Signal Double Ratchet, RFC 9420).

Goals

- Message confidentiality and integrity.
- Participant authentication.
- Forward secrecy and post-compromise recovery where supported.
- Deniability in some designs.

Non-goals

- Complete metadata privacy.
- Protection after endpoint compromise.
- Guaranteed deletion from recipients or backups.
- Social-graph privacy by default.

Building blocks

- Secure channels.
- X3DH-style initial key agreement.
- Double Ratchet or group messaging key schedules.
- Signatures, KDFs, AEAD, and key transparency.

Metadata leaks

- Contact graph and delivery timing.
- Device count and key changes.
- Message size and frequency.
- Backup and notification behavior.

Post-quantum posture

Depends on identity keys, key agreement, signatures, and ratchet design. Many deployed systems still use classical elliptic-curve assumptions that are quantum-vulnerable under Shor's algorithm, though hybrid migration is possible (Shor 1994, NIST FIPS 203).

Confidence model

Confidence is mixed: client-side-secret for device keys, mathematical assumptions for key agreement and signatures, and operational trust in key directories, delivery servers, and backup policy.

Failure modes

- Endpoint compromise.
- Cloud backups exposing plaintext or keys.
- Users ignoring key-change warnings.
- Metadata retained by servers.
- Multi-device synchronization weakening assumptions.

Related concepts

- Secure Channels
- Key-Committing Encryption
- Deniability

Further reading

- Signal X3DH.
- Signal Double Ratchet.
- RFC 9420: Messaging Layer Security.
- NIST FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.

Identity Wallets

Overview

Identity wallets store credentials and present claims to verifiers, sometimes with selective disclosure or privacy-preserving proofs; the W3C Verifiable Credentials data model standardizes the credential data model, not a complete privacy-preserving wallet system (W3C VC Data Model 2.0).

Goals

- Holder-controlled credential presentation.
- Issuer authenticity.
- Selective disclosure.
- Optional unlinkability across presentations.

Non-goals

- Trustlessness; issuers still matter.
- Protection from rare-attribute re-identification.
- Revocation privacy by default.
- Device compromise protection.

Building blocks

- Digital signatures or anonymous credentials.
- Holder binding.
- Selective disclosure proofs.
- Status lists, accumulators, or revocation registries.
- Secure storage and recovery.

Metadata leaks

- Issuer/verifier collusion.
- Presentation timing.
- Rare attributes.
- Status-check queries.
- Wallet software fingerprinting.

Post-quantum posture

Depends on credential signatures, holder binding, revocation commitments, and wallet secure storage. Many deployed credential stacks rely on classical signatures that are quantum-vulnerable under Shor's algorithm (Shor 1994).

Confidence model

Confidence comes from trusted issuers, holder-side secrets, verifier policy, wallet implementation, and status infrastructure.

Failure modes

- Over-disclosure by wallet UX.
- Revocation checks tracking holders (W3C Bitstring Status List v1.0).
- Device compromise or credential export.
- Issuer keys not rotated or revoked cleanly.

Source-depth notes

Identity-wallet source coverage should distinguish the credential data model, proof cryptosuites, and presentation profiles. W3C Data Integrity ECDSA and EdDSA are credential signature profiles (ECDSA Cryptosuites, EdDSA Cryptosuites); BBS, SD-JWT, and SD-JWT VC deployment claims should be checked against their current standards status (W3C BBS Cryptosuites, RFC 9901, SD-JWT VC draft).

Related concepts

- Anonymous Credentials
- Blind-Signature Credentials
- Privacy-Preserving Revocation

Further reading

- W3C Verifiable Credentials Data Model v2.0.
- W3C Bitstring Status List v1.0.
- W3C Data Integrity ECDSA Cryptosuites v1.0.
- W3C Data Integrity EdDSA Cryptosuites v1.0.
- W3C Data Integrity BBS Cryptosuites v1.0.
- RFC 9901: Selective Disclosure for JSON Web Tokens.
- IETF SD-JWT-based Verifiable Digital Credentials draft.
- OpenID for Verifiable Credential Issuance.
- OpenID for Verifiable Presentations.
- European Digital Identity Wallet Architecture and Reference Framework.
- ISO/IEC 18013-5 mobile driving licence application.
- Hyperledger AnonCreds Specification.
- Camenisch and Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials".

Encrypted Mempools

Overview

Encrypted mempools hide transaction contents before ordering, inclusion, or a reveal phase, usually to reduce pre-trade leakage and some forms of front-running.

Goals

- Hide transaction contents before ordering.
- Bind encrypted submissions to reveal or decryption rules.
- Reduce content-based front-running.
- Support fairer sequencing under a stated model.

Non-goals

- Guaranteed inclusion.
- Censorship resistance by itself.
- Hiding sender network metadata.
- Solving all miner or maximal extractable value risks.

Building blocks

- Public-key encryption or threshold encryption.
- Commitments to encrypted transactions.
- Sequencing and reveal rules.
- Slashing, availability, or committee accountability.

Metadata leaks

- Submission timing.
- Sender network path.
- Gas or fee patterns.
- Transaction size.
- Censorship and reveal behavior.

Post-quantum posture

Depends on the encryption, threshold scheme, signatures, and ledger authentication. Many current designs rely on classical public-key assumptions.

Confidence model

Confidence is mixed: threshold or trusted decryption committee, sequencer assumptions, public-verifiability for commitments and inclusion, and operational audit for key release.

Failure modes

- Decryption keys released before ordering.
- Committee collusion or unavailability.
- Censorship of encrypted transactions.
- Metadata still revealing strategy.
- Reveal failures or griefing.

Source-depth notes

Encrypted-mempool coverage should treat current Ethereum designs as draft-stage unless the page is explicitly about a deployed system. Source depth should include MEV motivation, threshold-decryption designs, batching proposals, and the limitations of encrypted ordering under censorship, size, timing, and reveal failures.

Related concepts

- Delayed Reveal
- Threshold Issuance
- Secure Channels

Further reading

- Daian et al., "Flash Boys 2.0".
- Canetti, "Universally Composable Security".
- EIP-8184: LUCID encrypted mempool.
- Goes, Ferveo: Threshold Decryption for Mempool Privacy in BFT Networks.
- Condorelli et al., Mempool Privacy via Batched Threshold Encryption.
- Shutter Network, Shutter documentation.
- Shutter Network, Shutter API documentation.

Private Machine-Learning Analytics

Overview

Private machine-learning analytics tries to learn aggregate model updates, statistics, or predictions without exposing raw participant data.

Goals

- Input privacy under a stated aggregation or computation model.
- Aggregate correctness or robustness.
- Output privacy when differential privacy or release controls are added.

Non-goals

- Preventing all inference from model outputs.
- Protecting tiny cohorts.
- Validating malicious client updates by default.
- Hiding participation in every deployment.

Building blocks

- Secure aggregation.
- MPC or homomorphic encryption.
- Differential privacy.
- Client update validation.
- Cohort selection and release policy.

Metadata leaks

- Participation and dropout.
- Cohort size.
- Model version and timing.
- Aggregate outputs across repeated releases.

Post-quantum posture

Depends on the cryptographic stack. Secure aggregation can rely heavily on symmetric primitives after setup, while public-key, HE, or MPC components may vary.

Confidence model

Confidence is mixed: honest-majority or non-collusion for aggregation, mathematical assumptions for cryptographic computation, and operational trust in release policy.

Failure modes

- Differencing attacks across releases.
- Model inversion or membership inference.
- Poisoned updates.
- Dropout or small-cohort leakage.

Related concepts

- Secure Aggregation
- Private Aggregation
- Homomorphic Encryption

Further reading

- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning".
- Dwork et al., "Calibrating Noise to Sensitivity in Private Data Analysis".

Private DAO Voting

Overview

This case study sketches a private DAO voting system as a composition exercise. It is not a deployable protocol.

Goals

- Eligible members can vote.
- Ballots remain private until, or unless, tally disclosure is intended.
- A member cannot vote twice in the same election.
- Invalid ballots are rejected.
- The final tally is verifiable.

Non-goals

- Full coercion resistance.
- Protection against all traffic analysis.
- Production deployment guidance.
- Governance security outside the voting protocol.
- Legal or regulatory compliance.

Possible building blocks

Requirement	Possible building block
Anonymous eligibility	anonymous credentials or membership proofs
Anti-double-vote protection	context-specific nullifiers
Ballot secrecy	public-key encryption or threshold encryption
Valid ballot proofs	zero-knowledge proofs
Private tallying	homomorphic encryption, MPC, or threshold decryption
Delayed reveal	timelock pattern or governance-defined opening phase

Concrete composition options

Requirement	Concrete options	Main caution
Eligibility set	Merkle tree, sparse Merkle tree, accumulator, anonymous credential issuer	Set construction affects anonymity, updates, and revocation.
Nullifier	Hash nullifier, Semaphore-style nullifier, credential serial number	Context binding controls cross-election linkability.
Ballot privacy	ElGamal-style encryption, threshold encryption, homomorphic encryption	Many classical options are quantum-vulnerable; trustees and keys dominate confidence.

Requirement	Concrete options	Main caution
Validity proof	Groth16, PLONK-style proof, STARK, Bulletproof-style range proof	Proof must bind ballot, election context, nullifier, and eligibility.
Tally	Homomorphic tally, mixnet tally, MPC tally	The tally method determines trustee, batching, and audit assumptions.

Simple architecture

1. The DAO publishes an election context, eligible voter set, and ballot rules.
2. Each eligible voter derives a context-specific nullifier.
3. The voter encrypts or commits to a ballot.
4. The voter submits a zero-knowledge proof that the ballot is valid and the nullifier is derived from an eligible secret.
5. The system rejects duplicate nullifiers.
6. The tally is computed privately and revealed with verification data.

Privacy leaks

- Submission timing can correlate voters with ballots.
- Wallet funding and transaction fees can identify voters.
- Small voter groups can reveal preferences from the final tally.
- Reused contexts or poor domain separation can link votes across elections.
- Public discussion, delegation, or governance forums can leak intent.

Post-quantum posture

Depends on every selected building block: credential signatures, membership proofs, nullifiers, ballot encryption, validity proofs, and tallying. A DAO voting design should be treated as post-quantum only if each layer has been classified.

Confidence model

A typical design combines several confidence models: trusted or semi-trusted eligibility source, holder secrets for anonymous voting, public nullifier registry for non-reuse, proof-system soundness for ballot validity, and threshold trustees or MPC participants for tally privacy.

Coercion limitations

Ballot secrecy is not the same as coercion resistance. A voter may be pressured to reveal credentials, prove how they voted, vote under observation, or sell access to a voting key. Coercion-resistant voting needs additional protocol and operational design.

Maturity warning

> **Warning.**
> Private DAO voting combines fast-moving privacy tooling with governance incentives and public ledgers. Treat this as a design map, not an implementation recommendation.

Failure modes

- Eligibility source is centralized or manipulable.
- Nullifier construction links voters across elections.
- Invalid encrypted ballots pass because validity proofs are incomplete.
- Tally authorities collude or lose decryption shares.
- User interfaces accidentally reveal votes or create receipts.
- Governance rules do not define aborts, challenges, or disputed tallies.

Related concepts

- Nullifiers
- Zero-knowledge proofs
- Private aggregation
- Coercion-resistant voting

Further reading

- Benaloh, Verifiable Secret-Ballot Elections.
- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.
- Bonawitz et al., Practical Secure Aggregation for Privacy-Preserving Machine Learning.

Coercion-Resistant Voting

Overview

Coercion-resistant voting aims to prevent a voter from proving how they voted, even if a coercer pressures them.

Goals

- Ballot secrecy.
- Receipt-freeness.
- Resistance to forced abstention or forced choice, depending on the model.
- Verifiable tallying.

Non-goals

- Solving all physical coercion.
- Protecting compromised voter devices by cryptography alone.
- Simple deployment.

Building blocks

- Anonymous credentials.
- Re-voting or credential recovery.
- Mixnets or homomorphic tallying.
- Zero-knowledge proofs.
- Careful user experience and operational procedures.

Concrete composition options

Requirement	Concrete options	Main caution
Eligibility	Anonymous credentials, recovery credentials, private registration lists	Registration can itself create coercion and tracking channels.
Ballot privacy	Mixnet tallying, homomorphic tallying, threshold decryption	The tally style changes trustee and metadata assumptions.
Receipt resistance	Re-voting, fakeable transcripts, credential invalidation	Must match the coercion timing model.
Validity and audit	ZK ballot-validity proofs, public bulletin board, verifiable shuffles	Public audit data must not become a voter-held receipt.
Trustee model	Threshold trustees, distributed key generation, public ceremony	Trustee collusion and key loss are system-level failures.

Privacy leaks

Timing, small groups, device compromise, and social pressure can defeat formal privacy claims.

Post-quantum posture

Depends on voter authentication, credentials, ballot encryption, proof systems, mixnets or tallying, signatures, and trustee key management. Coercion resistance is a system property and does not receive a single post-quantum label unless every cryptographic layer is classified.

Confidence model

Confidence usually combines public verifiability, threshold trustees, trusted eligibility processes, client-side secrecy, and operational controls that make receipts unavailable, fakeable, or superseded by later actions.

Maturity warning

> **Warning.**
> Coercion resistance is a demanding system property. Treat any simple claim of coercion resistance with skepticism unless the threat model is precise.

Failure modes

- The voter interface creates a receipt.
- Device compromise observes or changes the vote.
- Coercers demand credentials before voting.
- Small groups or public tallies make votes inferable.
- Re-voting or recovery rules are too hard for real voters to use.

Related concepts

- Electronic voting
- Make receipts useless
- Mixnets

Further reading

- Benaloh, Verifiable Secret-Ballot Elections.
- Electronic voting

Anonymous Airdrop

Overview

An anonymous airdrop lets eligible users claim once without publicly linking the claim to their original identity.

Goals

- Eligibility.
- One claim per eligible user.
- Claim privacy.
- Public auditability of the spent claim set.

Non-goals

- Network anonymity.
- Protection from identity leaks during funding or withdrawal.
- Sybil resistance beyond the eligibility source.

Possible building blocks

- Eligibility set commitment.
- Membership proof.
- Nullifier.
- Zero-knowledge proof.
- Private withdrawal or shielding mechanism.

Concrete composition options

Requirement	Concrete options	Main caution
Eligibility commitment	Merkle tree, sparse Merkle tree, accumulator	The set root must be authenticated and deduplicated.
Membership proof	Merkle proof inside ZK, accumulator witness, credential presentation	Proof choice determines setup, posture, and witness update risk.
One-claim enforcement	Hash nullifier, ZK-derived nullifier, credential serial number	Nullifier context must be campaign-specific.
Claim privacy	Shielded pool, delayed withdrawal, mixnet-style relay	Public ledger metadata can still identify claimants.
Validity proof	Groth16, PLONK-style, STARK-style proof	The proof must include eligibility, nullifier correctness, and claim rules.

Privacy leaks

Claim timing, gas funding, wallet reuse, and exchange withdrawals can identify claimants.

Post-quantum posture

Depends on the selected membership proof, nullifier construction, signatures, shielding mechanism, and transaction layer. A hash-based nullifier does not make the whole airdrop post-quantum if the credential or proof system is quantum-vulnerable.

Confidence model

Confidence combines a trusted or publicly auditable eligibility source, holder-controlled secrets, public spent-nullifier checks, proof-system soundness, and operational controls around funding and withdrawal privacy.

Failure modes

- Eligibility set contains duplicates.
- Nullifiers are linkable across campaigns.
- Claims reveal wallet funding patterns.
- The proof omits a required context.

Related concepts

- Nullifiers
- Anonymous membership
- Anti-double-use nullifiers

Further reading

- Nullifiers
- Membership proofs

Design Patterns

Design Patterns Overview

Design patterns are recurring ways to compose cryptographic tools into system behavior.

Examples

- Anonymous membership proves eligibility without naming the user.
- Anti-double-use nullifiers prevent repeated anonymous actions.
- Private aggregation reveals totals without exposing individual values.
- Delayed reveal separates commitment time from opening time.
- Reveal only a function limits disclosure to a computed output.
- Make receipts useless reduces coercion and vote-buying risk.
- Domain separation and transcript binding prevent cross-protocol and cross-statement confusion.
- Privacy-preserving revocation manages status checks without turning every presentation into a tracking event.
- Encrypt-then-prove proves validity of encrypted values.
- Threshold issuance splits authorization across issuers.
- Key rotation and migration manages cryptographic agility over time.

Reading rule

A pattern is not a complete protocol. Treat it as a design shape that still needs assumptions, threat models, and implementation review.

See Operational Design Patterns, Domain Separation, Transcript Binding, Privacy-Preserving Revocation, Encrypt-then-prove, Threshold Issuance, and Key Rotation and Migration.

Operational Design Patterns

Operational Design Patterns

These design patterns are recurring ways to compose primitives, bind context, and manage cryptographic state over time. This page is now a routing overview.

This is a grouped overview. Domain Separation, Transcript Binding, and Privacy-Preserving Revocation now have standalone pages.

Pattern matrix

Pattern	Main purpose	Common building blocks	Main failure
Encrypt-then-prove	Prove a statement about encrypted data	encryption, commitments, ZKPs	proof not bound to ciphertext or recipient
Threshold issuance	Avoid one issuer/key having unilateral power	threshold signatures, MPC, DKG	quorum collusion or availability failure
Privacy-preserving revocation	Revoke access or credentials without tracking everyone	accumulators, status lists, ZKPs	revocation checks become tracking beacons
Domain separation	Prevent cross-protocol confusion	labels, transcript hashes, KDF contexts	reusing keys or hashes across domains
Transcript binding	Bind outputs to the full protocol transcript	KDFs, signatures, Fiat-Shamir, AEAD associated data	omitting identities, algorithms, or public inputs
Key rotation and migration	Replace keys or algorithms over time	versioning, re-encryption, signatures, KDFs	stale keys, downgrade, or split trust roots

Encrypt-then-prove

Encrypt-then-prove combines encryption with a proof that the encrypted plaintext satisfies a public statement.

Typical use:

- Encrypted ballots with validity proofs.
- Encrypted bids or orders with range constraints.
- Verifiable encryption for dispute or recovery workflows.

Security properties:

- Plaintext confidentiality.
- Public or verifier-local evidence about a property of the plaintext.
- Binding between ciphertext, recipient key, statement, and context.

What it does not provide:

- Correct decryption behavior.

- Fairness after the proof verifies.
- Metadata privacy for sender, recipient, size, or timing.

Failure modes:

- Proving a statement about a commitment but not the actual ciphertext.
- Forgetting to bind recipient public key or election/order ID.
- Revealing too much through public inputs.

Threshold issuance

Threshold issuance splits credential, token, or signature issuance across multiple parties so no single issuer can act alone.

Typical use:

- Anonymous credentials.
- Threshold signatures for custody.
- E-cash or token issuance.
- DAO or governance authorization.

Security properties:

- Issuance requires a threshold of participants.
- Some designs provide public verification of issuer participation.
- Reduces single-key compromise risk.

What it does not provide:

- Trustlessness.
- Availability if too many issuers are offline.
- Privacy if issuers collude or log metadata.

Failure modes:

- Treating threshold as decentralization without governance.
- No key-refresh or recovery process.
- Issuers share infrastructure or operational control.

Revocation with privacy

Revocation with privacy lets a verifier reject revoked credentials, keys, or notes without turning every check into a tracking event.

Typical use:

- Anonymous credentials.
- Identity wallets.

- E-cash and private payments.
- Access tokens.

Security properties:

- Revoked items should fail.
- Non-revoked holders should avoid unnecessary linkage.
- Revocation evidence should be fresh enough for the threat model.

What it does not provide:

- Perfect privacy if revocation sets are small or checks are online.
- Recovery from issuer abuse.
- Non-transferability.

Failure modes:

- Online status checks that reveal every presentation.
- Tiny revocation anonymity sets.
- Stale accumulators or status lists.
- Revocation identifiers that become stable trackers.

Domain separation

Domain separation labels cryptographic operations so values from one protocol, purpose, chain, or version cannot be replayed as another.

Typical use:

- Hash inputs.
- KDF context strings.
- Signature messages.
- Fiat-Shamir transcripts.
- AEAD associated data.

Security properties:

- Prevents cross-protocol replay or confusion.
- Makes encodings unambiguous.
- Allows safe key and primitive reuse within designed limits.

What it does not provide:

- Security if the underlying primitive is broken.
- Protection from omitted fields.
- A substitute for key separation where separate keys are required.

Failure modes:

- Signing a message without chain, protocol, or version.
- Hashing concatenated fields without length encoding.
- Reusing the same label across incompatible contexts.

Transcript binding

Transcript binding feeds the full protocol context into signatures, KDFs, AEAD associated data, or Fiat-Shamir challenges.

Typical use:

- Secure-channel handshakes.
- Key exchange.
- Zero-knowledge proofs.
- Threshold signing.
- Rollup and bridge proofs.

Security properties:

- Outputs are tied to the identities, algorithms, messages, and public inputs that produced them.
- Downgrade and unknown-key-share attacks are harder.
- Verifiers check the intended statement.

What it does not provide:

- Correctness if the transcript omits critical data.
- Privacy for public transcript fields.
- Protection from endpoint compromise.

Failure modes:

- Omitting algorithm choices.
- Not binding peer identities.
- Proving or signing a transcript hash computed by an untrusted party without reconstruction.

Key rotation and migration

Key rotation and migration replaces keys, parameters, or algorithms while preserving continuity and limiting damage from compromise.

Typical use:

- Certificate and signing-key rotation.
- Post-quantum hybrid migration.
- Re-encryption of stored data.
- Credential issuer key updates.

Security properties:

- Limits exposure from old keys.
- Creates a controlled path to stronger algorithms.
- Allows revocation or deprecation of unsafe material.

What it does not provide:

- Automatic security for data already exposed.
- Trust continuity without authentic migration records.
- User recovery by itself.

Failure modes:

- Old and new keys both accepted indefinitely.
- Downgrade to legacy algorithms.
- Re-encryption without authenticating metadata.
- Lost ability to verify historical artifacts.

Post-quantum posture

Not applicable to the patterns themselves. Rotation and migration are especially important for post-quantum transition because a system may need hybrid operation, algorithm agility, and a plan for deprecating quantum-vulnerable keys.

Confidence model

Confidence depends on the pattern: public-verifiability for transcripts and proofs, t-of-n-threshold for threshold issuance, client-side-secret for rotation state, and operational audit for revocation and migration.

Related concepts

- Composability
- Authenticated Encryption
- Threshold Cryptography
- Encrypt-then-prove
- Threshold Issuance
- Key Rotation and Migration
- Additional Security Goals
- Post-Quantum Posture

Further reading

- Boneh and Shoup, "A Graduate Course in Applied Cryptography".
- Canetti, "Universally Composable Security; A New Paradigm for Cryptographic Protocols".

- RFC 8446, "The Transport Layer Security (TLS) Protocol Version 1.3".
- RFC 9380, "Hashing to Elliptic Curves".
- NIST SP 800-57, "Recommendation for Key Management".

Domain Separation

One-sentence intuition

Domain separation labels cryptographic operations so values valid in one context cannot be replayed or reinterpreted in another.

Where it sits in the taxonomy

- Level: design pattern.
- Parent category: composition and implementation hygiene.
- Related concepts: Transcript Binding, Hash Functions, Key Derivation Functions.

Problem it solves

Cryptographic values are often just bytes. Without explicit labels and encodings, the same hash, signature, proof challenge, or KDF output may be interpreted across protocols, chains, versions, or purposes.

Mental model

Domain separation is a namespace for cryptographic bytes. It tells the verifier, "this value belongs to exactly this protocol, version, role, and purpose."

Minimal example

Instead of signing `amount || recipient`, a wallet signs an encoded message that includes `protocol = ExamplePay`, `version = 2`, `chain_id`, `amount`, `recipient`, and a length-delimited structure.

Security properties

- Reduces cross-protocol replay.
- Makes encodings and protocol roles explicit.
- Helps separate keys, hash inputs, KDF outputs, and Fiat-Shamir challenges.
- Supports versioning and migration.

What it does not provide

- Security if the underlying primitive is broken.
- A substitute for separate keys where key separation is required.
- Protection if critical fields are omitted.
- Privacy for public labels.

Assumptions

- Labels and encodings are specified before deployment.

- Implementations use the same canonical encoding.
- Version, suite, role, and application context are included when relevant.
- Reviewers can tell which domain a value belongs to.

Post-quantum posture

Not applicable to the pattern itself. The posture comes from the underlying primitive, but domain separation remains necessary in post-quantum and hybrid protocols.

Confidence model

Confidence depends on unambiguous specifications, independent verifier reconstruction, implementation review, and test vectors that cover labels and encoding boundaries.

Common constructions

- Labeled hash prefixes.
- KDF info/context fields.
- AEAD associated data.
- Signature domain tags.
- Fiat-Shamir transcript labels.
- Hash-to-curve domain separation tags.

Use cases

- Wallet signatures.
- Secure-channel key schedules.
- Multi-chain protocols.
- ZK proof transcripts.
- Credential presentations.
- Versioned APIs.

Composition patterns

Domain separation is usually paired with transcript binding. Domain separation names the context; transcript binding commits to the actual messages and public inputs inside that context.

Failure modes and anti-patterns

- Hashing concatenated fields without lengths.
- Reusing one label across incompatible protocols.
- Omitting chain ID, protocol version, role, or verification key.
- Relying on UI text instead of signed structured data.
- Adding labels after deployment without migration rules.

Maturity and deployment

Widely deployed as an engineering pattern, but often under-specified. Mature protocols usually include explicit labels, transcript hashes, and key-schedule contexts.

Related concepts

- Transcript Binding
- Secure Channels
- Random Oracle Model
- Concrete Algorithms and Schemes

Further reading

- RFC 9380: Hashing to Elliptic Curves.
- RFC 8446: TLS 1.3.
- Boneh and Shoup, "A Graduate Course in Applied Cryptography".

Transcript Binding

One-sentence intuition

Transcript binding feeds the full protocol context into keys, signatures, proofs, or challenges so outputs cannot be detached from the messages that produced them.

Where it sits in the taxonomy

- Level: design pattern.
- Parent category: composition and protocol state.
- Related concepts: Domain Separation, Secure Channels, Zero-Knowledge Proofs.

Problem it solves

Many attacks replace, omit, reorder, or reinterpret protocol messages. Binding the transcript forces participants and verifiers to derive keys or challenges from the same complete context.

Mental model

The transcript is the protocol's receipt. A derived key, signature, or proof challenge should be valid only for that exact receipt.

Minimal example

In a handshake, both sides hash the version, cipher suite, identities, ephemeral keys, and prior messages. Finished messages authenticate that transcript before application keys are trusted.

Security properties

- Binds outputs to identities, algorithms, public keys, public inputs, and messages.
- Reduces downgrade and unknown-key-share attacks.
- Helps Fiat-Shamir challenges refer to the intended proof statement.
- Supports channel binding to higher-level protocols.

What it does not provide

- Protection if the transcript omits a critical field.
- Privacy for public transcript fields.
- Endpoint security after compromise.
- Correctness of application semantics outside the bound transcript.

Assumptions

- Each party reconstructs the transcript independently.
- All security-critical fields are included in canonical order and encoding.

- Domain separation distinguishes protocol phases and roles.
- Verification rejects missing, duplicated, or out-of-order messages.

Post-quantum posture

Not applicable to the pattern itself. Transcript binding is still required for post-quantum, hybrid, and classical protocols.

Confidence model

Confidence comes from public-verifiability or local-verifiability of the bound transcript, implementation review, canonical encodings, and test vectors that cover downgrade and substitution cases.

Common constructions

- Handshake transcript hashes.
- KDF context strings.
- AEAD associated data.
- Fiat-Shamir challenge transcripts.
- Signature prehashes over structured messages.

Use cases

- TLS 1.3 handshakes.
- Noise protocol patterns.
- ZK proof systems.
- Threshold signing.
- Rollup state-transition proofs.
- Wallet and bridge authorization.

Composition patterns

Transcript binding composes with domain separation, key schedules, proof-system public inputs, and authenticated encryption. The common rule is: if a verifier would care about a field, bind it before deriving the output.

Failure modes and anti-patterns

- Not binding peer identities.
- Omitting algorithm choices or protocol versions.
- Letting an untrusted party supply a transcript hash without reconstruction.
- Not binding public inputs or verification keys in proof systems.
- Binding a UI string but not the canonical machine-readable message.

Maturity and deployment

Widely deployed in mature protocols, but subtle implementation bugs remain common because transcript boundaries are protocol-specific.

Related concepts

- Domain Separation
- Secure Channels
- Polynomial Commitments
- ZK Rollups

Further reading

- RFC 8446: TLS 1.3.
- Bellare and Rogaway, "Random Oracles are Practical".
- The Noise Protocol Framework.

Privacy-Preserving Revocation

One-sentence intuition

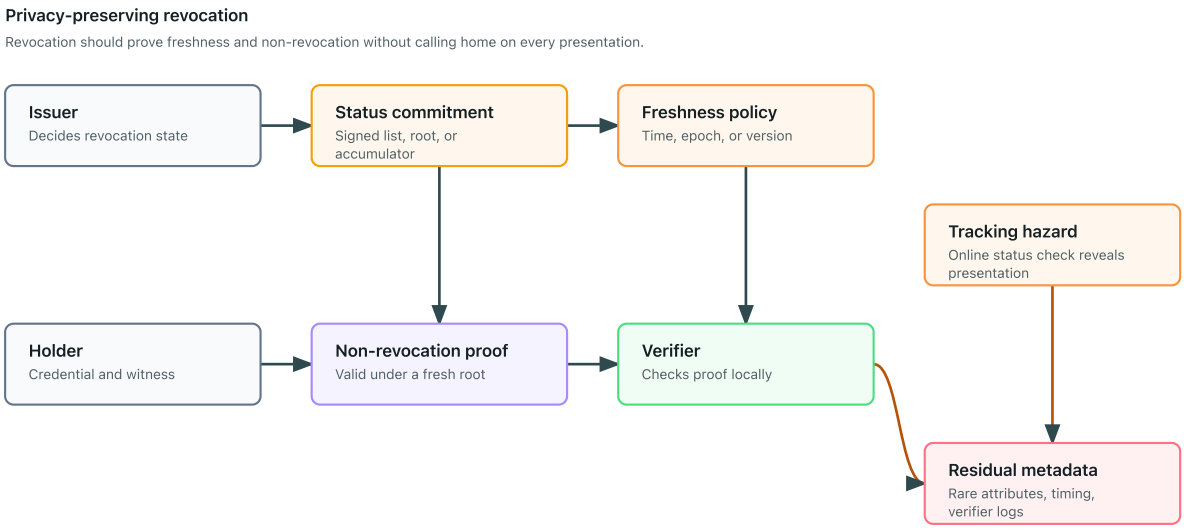
Privacy-preserving revocation lets verifiers reject revoked credentials, keys, or tokens without making every status check a tracking event.

Where it sits in the taxonomy

- Level: design pattern.
- Parent category: operational cryptography and credential systems.
- Related concepts: Anonymous Credentials, Vector Commitments, Accumulators and Merkle Trees.

Problem it solves

Systems need a way to stop accepting compromised, expired, or abusive credentials. Naive online status checks can reveal every presentation to the issuer or status service; credential status mechanisms are explicitly part of verifiable-credential ecosystems (W3C VC Data Model 2.0, W3C Bitstring Status List v1.0).



Mental model

The holder proves "my credential is not on the revoked list" against a public status commitment, without asking the issuer about this exact presentation.

Minimal example

An issuer publishes a signed accumulator or status-list root. A holder presents a credential plus a proof that its revocation handle is not revoked under the current root. The verifier checks the root freshness and proof locally.

Security properties

- Revoked items fail verification.
- Non-revoked holders avoid unnecessary issuer callbacks.
- Status evidence is bound to a version or time.
- Verifiers can check revocation under a stated freshness policy.

What it does not provide

- Perfect privacy when revocation sets are tiny.
- Recovery from issuer abuse.
- Non-transferability by itself.
- Protection from verifier collusion, timing linkage, or rare attributes.

Assumptions

- The revocation root, list, accumulator, or status commitment is authentic and fresh enough.
- Holders can obtain and update witnesses without revealing presentations.
- Verifiers enforce freshness and reject stale status evidence.
- Revocation identifiers are not stable cross-context trackers unless intentionally disclosed.

Post-quantum posture

Depends on the construction. Hash-based status lists and Merkle proofs can be plausible with conservative hashes. Pairing, RSA, or elliptic-curve accumulators are quantum-vulnerable under Shor's algorithm unless replaced by post-quantum alternatives (Shor 1994).

Confidence model

Confidence is mixed: trusted-issuer for revocation decisions, public-verifiability for signed status commitments, client-side-secret for holder binding, and operational audit for publication cadence and witness updates.

Common constructions

- Signed status lists.
- Bitstring status lists (W3C Bitstring Status List v1.0).
- Dynamic accumulators.
- Merkle or sparse-Merkle revocation sets.
- Zero-knowledge non-revocation proofs.

Use cases

- Anonymous credentials.
- Verifiable credentials and identity wallets.

- Access tokens.
- Private payments and e-cash.
- Device or key compromise handling.

Composition patterns

Privacy-preserving revocation is composed with issuer signatures, holder binding, vector commitments or accumulators, freshness policies, and sometimes ZK proofs that hide the revocation handle.

Failure modes and anti-patterns

- Online status checks that reveal every presentation.
- Tiny revocation sets that identify holders.
- Stale accumulators or status lists.
- Stable revocation identifiers reused across contexts.
- Witness update services that log holder activity.

Maturity and deployment

Emerging. Status lists are deployed in credential ecosystems (W3C VC Data Model 2.0, W3C Bitstring Status List v1.0), while stronger accumulator and zero-knowledge revocation designs are specialized and harder to operate.

Related concepts

- Anonymous Credentials
- Vector Commitments
- Metadata Leakage
- Identity Wallets

Further reading

- W3C Verifiable Credentials Data Model v2.0.
- W3C Bitstring Status List v1.0.
- Camenisch and Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials".
- Catalano and Fiore, "Vector Commitments and Their Applications".
- RFC 9576: The Privacy Pass Architecture.

Encrypt-Then-Prove

One-sentence intuition

Encrypt-then-prove encrypts data and proves a statement about the hidden plaintext.

Where it sits in the taxonomy

- Level: design pattern.
- Parent category: encryption plus proof composition.
- Related concepts: Verifiable Encryption, Zero-Knowledge Proofs, Commitments.

Problem it solves

Verifiers may need confidence that encrypted data is well formed without learning the data itself.

Mental model

The ciphertext hides the object; the proof verifies a public rule about what is inside.

Minimal example

An encrypted ballot includes a proof that the hidden vote is one of the allowed choices.

Security properties

- Plaintext confidentiality.
- Proof of a public property.
- Binding between proof, ciphertext, recipient key, and context.

What it does not provide

- Correct decryption behavior.
- Fairness.
- Metadata privacy.
- Correctness if the statement is too weak.

Assumptions

- Encryption and proof-system assumptions both hold.
- The proof references the exact ciphertext and recipient key.
- Public inputs include the right context.

Post-quantum posture

Depends on the encryption and proof system. A post-quantum layer can be weakened by classical signatures, commitments, or setup.

Confidence model

Confidence is mixed: mathematical assumptions for encryption and proofs, public-verifiability for proof checks, and operational trust in decryptors.

Common constructions

- Encrypted ballot validity proofs.
- Verifiable encryption.
- Encrypted order or bid constraints.

Use cases

- Voting.
- Private payments.
- Auctions.
- Recovery and dispute workflows.

Composition patterns

Encrypt-then-prove composes with transcript binding, domain separation, key management, and metadata minimization.

Failure modes and anti-patterns

- Proof not bound to ciphertext.
- Recipient key omitted.
- Public inputs leak sensitive values.
- Decryptor can refuse service.

Maturity and deployment

Mature as a pattern, but system-specific statement design needs expert review.

Related concepts

- Verifiable Encryption
- Electronic Voting
- Transcript Binding

Further reading

- Camenisch and Shoup, "Practical Verifiable Encryption and Decryption of Discrete Logarithms".
- Goldwasser, Micali, and Rackoff, "The Knowledge Complexity of Interactive Proof Systems".

Threshold Issuance

One-sentence intuition

Threshold issuance requires several authorities to cooperate before a credential, token, or signature is issued.

Where it sits in the taxonomy

- Level: design pattern.
- Parent category: threshold cryptography and operational governance.
- Related concepts: Threshold Cryptography, Blind-Signature Credentials, Anonymous Tokens.

Problem it solves

A single issuer key can become a single point of compromise, abuse, or censorship. Threshold issuance splits that authority.

Mental model

No one clerk can stamp a credential alone. A quorum must cooperate, and the verifier sees one valid issued object or proof of quorum.

Minimal example

Three of five issuer servers jointly produce a blind signature for a credential request after policy checks pass.

Security properties

- Issuance requires a threshold.
- Reduces single-key compromise risk.
- Can improve auditability of issuer participation.

What it does not provide

- Trustlessness.
- Availability if too many issuers are offline.
- Privacy if issuers collude or log metadata.
- Good governance by itself.

Assumptions

- At most the allowed number of issuers collude or are compromised.
- Distributed key generation or key shares are set up correctly.
- Issuers are operationally independent enough for the model.

Post-quantum posture

Depends on the threshold signature or issuance scheme. Many threshold signature deployments are classical and quantum-vulnerable.

Confidence model

Confidence comes from t-of-n-threshold assumptions, operational independence, key-share protection, and public or operational audit.

Common constructions

- Threshold signatures.
- Distributed key generation.
- Threshold blind signatures.
- Multi-issuer credential issuance.

Use cases

- Credential issuance.
- Token minting.
- Custody and governance authorization.
- E-cash and anonymous token systems.

Composition patterns

Threshold issuance composes with issuer policy, revocation, audit logs, blind issuance, and rate limits. It changes who can issue, not what the issued object means.

Failure modes and anti-patterns

- Issuers share infrastructure or operators.
- No key refresh or recovery.
- Quorum collusion.
- Treating threshold as decentralization without governance.

Maturity and deployment

Emerging to mature depending on scheme. Operational assumptions are often more fragile than the cryptographic threshold.

Related concepts

- Threshold Cryptography
- Blind Signatures
- Privacy-Preserving Revocation

Further reading

- RFC 9591: FROST.
- Shamir, "How to Share a Secret".
- Canetti, "Universally Composable Security".

Key Rotation and Migration

One-sentence intuition

Key rotation and migration replace keys, parameters, or algorithms while preserving continuity and limiting compromise impact.

Where it sits in the taxonomy

- Level: design pattern.
- Parent category: operational cryptography.
- Related concepts: Post-Quantum Posture, Domain Separation, Secure Channels.

Problem it solves

Keys expire, leak, age, or become tied to algorithms that need replacement. Systems need a controlled path to move forward without silently breaking trust.

Mental model

Migration is a signed bridge from old trust to new trust. Rotation defines when and how traffic moves over that bridge.

Minimal example

A service signs a new public key with an old trusted key, publishes an activation time, and rejects old signatures after a deprecation window.

Security properties

- Limits exposure from old keys.
- Enables algorithm migration.
- Supports revocation and deprecation.
- Preserves trust continuity when authenticated correctly.

What it does not provide

- Recovery of already exposed data.
- User recovery by itself.
- Trust continuity without authentic migration records.
- Protection if old and new keys are both compromised.

Assumptions

- Old trust roots are still trustworthy during migration.
- Versioning and domain separation are explicit.

- Rollback and downgrade are rejected.
- Operators can inventory cryptographic dependencies.

Post-quantum posture

Not applicable to the pattern itself. It is central to post-quantum transition because systems need crypto agility, hybrid deployment, and deprecation plans.

Confidence model

Confidence is mixed: operational audit for key lifecycle, public-verifiability for signed migration records, and client-side-secret for private-key custody.

Common constructions

- Key epochs.
- Signed key-transition records.
- Hybrid key exchange.
- Certificate rotation and revocation.
- Re-encryption with authenticated metadata.

Use cases

- Certificate rotation.
- Post-quantum migration.
- Credential issuer key updates.
- Storage re-encryption.

Composition patterns

Rotation composes with transcript binding, domain separation, audit logs, and client update policy. Every consumer must know which keys are valid for which purpose and time.

Failure modes and anti-patterns

- Accepting old and new keys indefinitely.
- Downgrade to legacy algorithms.
- Re-encryption without authenticating metadata.
- Losing historical verification.

Maturity and deployment

Mature operational pattern, but failures are common because migration spans code, policy, clients, and archives.

Related concepts

- Post-Quantum Posture
- Secure Channels
- Domain Separation

Further reading

- NIST SP 800-57 Part 1 Revision 5.
- NIST NCCoE Migration to Post-Quantum Cryptography.

Anonymous Membership

One-sentence intuition

Anonymous membership lets someone prove they belong to an eligible group without revealing which member they are.

Where it sits in the taxonomy

- Level: design-pattern
- Parent category: design-pattern
- Related concepts: Anonymous Credentials, Membership Proofs, Nullifiers

Problem it solves

Many systems need to check that a participant belongs to an approved set without learning which approved member is acting. Anonymous membership patterns preserve eligibility while reducing identity linkage across actions.

Mental model

Think of a guarded doorway that checks a valid badge from a crowd without learning which badge holder is at the door.

Minimal example

A voter proves they are in an eligibility Merkle tree and submits a ballot without revealing which leaf belongs to them.

Security properties

- anonymity
- eligibility
- unlinkability depending on context

What it does not provide

- Anti-double-use unless nullifiers or rate limits are added.
- Network anonymity.
- Coercion resistance.
- Privacy for very small groups.

Assumptions

The eligibility source, membership commitment, proof system, and verifier context binding must all match the intended group. Privacy also assumes the anonymity set is large enough and that presentations are

not linkable through metadata.

Post-quantum posture

Not applicable to the pattern by itself. A concrete anonymous-membership system inherits posture from its credential scheme, membership proof, hash function, signatures, and transport layer.

Confidence model

Confidence usually depends on a trusted issuer or public membership set, holder-controlled secrets, public verification of membership proofs, and non-linking contexts.

Common constructions

Common building blocks

- Anonymous credentials.
- Merkle or accumulator membership proofs.
- Zero-knowledge proofs.
- Context binding.

Concrete compositions

Composition	Typical role	Main caution
Merkle membership plus zero-knowledge proof	Anonymous allowlist or group membership	The anonymity set is only the committed set, and stale roots can break eligibility.
Accumulator membership plus zero-knowledge proof	Compact anonymous membership with dynamic sets	Witness updates and accumulator setup must be part of the protocol.
BBS+ or CL anonymous credential	Attribute-based anonymous authorization	Issuer trust, revocation, and rare attributes can re-identify users.
Semaphore-style group membership	Anonymous signaling and one-action-per-group designs	Nullifiers context design controls linkability and rate limits.

Use cases

- eligibility proofs
- private access control
- private voting

Composition patterns

- Nullifiers or rate limits are needed to prevent repeated use.
- Rare attributes or small groups can deanonymize members.

Common adjacent concepts: Anonymous Credentials, Membership Proofs, Nullifiers.

Failure modes and anti-patterns

- The eligible set is too small.
- Membership data is stale or manipulable.
- Proofs are linkable across contexts.
- Metadata reveals the member.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: scheme-dependent.

Related concepts

- Anonymous Credentials
- Membership Proofs
- Nullifiers

Further reading

- Camenisch and Lysyanskaya, An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation.
- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.

Anti-Double-Use Nullifiers

One-sentence intuition

Anti-double-use nullifiers let a system reject repeated anonymous actions in the same context.

Where it sits in the taxonomy

- Level: design-pattern
- Parent category: design-pattern
- Related concepts: Nullifiers, Anonymous Membership, Zero-Knowledge Proofs

Problem it solves

Anonymous systems often need to stop a credential, coin, or eligibility proof from being used twice without learning who used it. Anti-double-use nullifiers give verifiers a public, context-bound value they can compare for repeats while keeping the underlying identity or secret hidden.

Mental model

Think of a private ticket that stamps the same public mark whenever it is used for one event, while hiding who held the ticket.

Minimal example

An airdrop claim includes a nullifier derived from the claimant secret and the airdrop ID; the contract rejects a second claim with the same nullifier.

Security properties

- double use prevention
- limited linkability
- anonymous rate limiting

What it does not provide

- Eligibility by itself.
- Protection against stolen secrets.
- Privacy if contexts are reused badly.
- Coercion resistance.

Assumptions

The private secret must have enough entropy, the context must be domain-separated, and the system must check a public spent-nullifier set before accepting an action.

Post-quantum posture

Not applicable to the pattern by itself. A concrete nullifier design inherits posture from the hash function, proof system, credential scheme, and public registry mechanism.

Confidence model

Confidence comes from client-side secret control, deterministic context binding, public duplicate detection, and a proof that the nullifier is derived from an eligible secret.

Common constructions

Pattern

1. Bind the action to a context.
2. Derive a public nullifier from a private secret and that context.
3. Prove the nullifier is well formed and eligible.
4. Reject the action if the nullifier was already used.

Concrete compositions

Composition	Typical role	Main caution
Hash secret plus context	Simple one-use marker	Only safe when the secret has high entropy and the context is domain-separated.
ZK proof plus nullifier	Anonymous voting, airdrops, signaling	The proof must bind membership, context, and nullifier into one statement.
Credential serial number	Anonymous credential spending or presentation limits	Revocation and issuer linkability need separate treatment.
Epoch-scoped nullifier	Rate limits per time window or application	Epoch design controls whether users are linkable across periods.

Use cases

- private voting
- anonymous airdrops
- anonymous signaling

Composition patterns

- The nullifier must be bound to an eligibility proof.
- Context reuse can link actions across systems.

Common adjacent concepts: Nullifiers, Anonymous Membership, Zero-Knowledge Proofs.

Failure modes and anti-patterns

- Reusing the same context links actions.

- Omitting domain separation.
- Storing side metadata that identifies the user.
- Accepting nullifiers without proving eligibility.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: high. Parameter sensitivity: medium.

Related concepts

- Nullifiers

Further reading

- Nullifiers
- Anonymous membership

Private Aggregation

One-sentence intuition

Private aggregation reveals a combined result without revealing each participant's individual value.

Where it sits in the taxonomy

- Level: design-pattern
- Parent category: design-pattern
- Related concepts: Secure Aggregation, Homomorphic Encryption, MPC

Problem it solves

Applications often need totals or statistics without collecting each participant's raw value. Private aggregation focuses on revealing only the combined result and managing what the aggregate, failures, and side channels still leak.

Mental model

Think of private aggregation as a narrow building block whose guarantee must be composed with the rest of the system.

Minimal example

A system uses private aggregation for one explicit role and handles authentication, metadata, and operational policy separately.

In a private poll, each voter submits an encrypted vote. The system combines encrypted votes and decrypts only the final tally.

If participant `i` holds a private value x_i, the system may reveal only the aggregate:

$$T = \sum_{i=1}^{n} x_i$$

The privacy goal is to reveal T without revealing the individual values $x_1, \ldots, x_n$.

Security properties

- Individual input privacy, under the chosen model.
- Aggregate correctness, if inputs are valid and the protocol completes.
- Limited disclosure of the final aggregate.

What it does not provide

- Privacy for very small groups.
- Protection against differencing attacks across repeated queries.
- Proof that inputs are valid unless validity checks are added.
- Protection against metadata leaks.
- Coercion resistance in voting contexts.

Assumptions

Assumptions vary by construction: encryption keys, threshold decryption, honest-majority assumptions, dropout handling, authenticated participants, and zero-knowledge proofs for input validity may all be required.

Post-quantum posture

Not applicable to the pattern by itself. The posture is inherited from the aggregation mechanism, authentication, proof system, transport, and key-management layers.

Confidence model

Confidence may come from threshold decryption, non-colluding helper servers, honest-majority MPC, or public verification of encrypted inputs. The page for a concrete design should state which model is being used.

Common constructions

Approaches

Approach	Useful when	Main risk
Homomorphic encryption	A public aggregator should combine ciphertexts	Key management and invalid inputs
Homomorphic commitments	Values need binding plus additive structure	Requires proofs that values are valid
Multi-party computation (MPC)	No single party should see inputs	Assumptions about parties and availability
Secure aggregation	Many clients report statistics	Dropout, malicious clients, and small groups

Concrete compositions

Composition	Typical role	Main caution
Paillier-style or additive homomorphic encryption tally	Simple encrypted sums in legacy or specialized systems	Quantum-vulnerable and key-management-heavy; validity proofs are still required.
Lattice HE tally	Post-quantum-oriented encrypted aggregation	Parameter choice and output leakage dominate practical risk.

Composition	Typical role	Main caution
Bonawitz-style secure aggregation	Federated learning and telemetry	Dropout handling, cohort size, and malicious updates are the main failure points.
Prio-style private telemetry	Aggregate statistics with validity checks	Requires a validation mechanism so clients cannot poison aggregates.
MPC-based aggregation	Multi-server or multi-party analytics	Collusion threshold and abort behavior must be explicit.

Use cases

- Voting and polling.
- Private telemetry.
- Private statistics.
- Federated analytics.
- Confidential financial sums.

Composition patterns

Private aggregation is often combined with range proofs, membership proofs, rate limits, threshold decryption, or differential privacy.

Failure modes and anti-patterns

- Aggregates over tiny groups reveal individuals.
- Repeated aggregates allow differencing attacks.
- Malicious users submit invalid or extreme values.
- A decryptor quorum can collude or lose keys.
- Metadata reveals who participated and when.

A differencing attack appears when two aggregates differ by one participant:

$$x_j = T(S \cup \{j\}) - T(S)$$

This is why query design, cohort size, and repeated releases matter.

Maturity and deployment

The pattern is mature, but concrete systems range from well deployed to experimental depending on scale, threat model, and implementation.

Related concepts

- Homomorphic encryption
- Secure aggregation
- Private set intersection

- Private DAO voting

Further reading

- Bonawitz et al., Practical Secure Aggregation for Privacy-Preserving Machine Learning.
- Gentry, Fully Homomorphic Encryption Using Ideal Lattices.
- Benaloh, Verifiable Secret-Ballot Elections.

Delayed Reveal

One-sentence intuition

Delayed reveal fixes information at one time and discloses it later.

Where it sits in the taxonomy

- Level: design-pattern
- Parent category: design-pattern
- Related concepts: Commitments, Timelock and VDFs

Problem it solves

Protocols sometimes need participants to commit to information before it is public, but reveal it only after a deadline, randomness event, or contest closes. Delayed reveal separates the time of choosing a value from the time of disclosing it.

Mental model

Think of sealing an answer in a tamper-evident envelope now and opening it after everyone has committed.

Minimal example

Bidders first publish commitments to bids. After the deadline, they open the commitments so late bidders cannot adapt to visible bids.

Security properties

- temporal binding
- delayed disclosure

What it does not provide

- Confidentiality after opening.
- Fairness if parties can abort without penalty.
- Authentication unless submissions are bound to identities or credentials.

Assumptions

The commitment or delay mechanism must bind the initial value, opening rules must be unambiguous, and the system must define what happens when a party refuses or misses the reveal phase.

Post-quantum posture

Not applicable to the pattern by itself. A concrete design inherits posture from its commitments, timelocks, signatures, and public bulletin board.

Confidence model

Confidence may come from mathematical binding, public-verifiability of openings, or external-timing assumptions when VDFs or timelocks are used.

Common constructions

Common building blocks

- Commitments.
- Timelocks or VDFs.
- Public bulletin boards.
- Opening deadlines and dispute rules.

Concrete compositions

Composition	Typical role	Main caution
Hash commit-reveal	Lotteries, auctions, simple delayed disclosure	Weak randomness reveals low-entropy committed values.
Pedersen commit-reveal	Numeric hidden values with later opening	Quantum-vulnerable; randomness and generator setup matter.
Timelock puzzle reveal	Delay without relying only on a human opener	Delay assumptions and hardware advantage must be modeled.
VDF-assisted reveal	Publicly verifiable delayed output	VDF setup and denial-of-service handling are part of the design.

Use cases

- commit reveal
- randomness beacons
- sealed bid processes

Composition patterns

- Delayed reveal needs deadlines and penalties.
- VDFs and timelocks add timing assumptions, not trusted wall-clock time.

Common adjacent concepts: Commitments, Timelock and VDFs.

Failure modes and anti-patterns

- Selective aborts.

- Weak randomness in commitments.
- Ambiguous opening formats.
- No rule for missed deadlines.

Maturity and deployment

Classified as mature. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: medium. Parameter sensitivity: scheme-dependent.

Related concepts

- Commitments
- Timelock and VDFs

Further reading

- Commitments
- Timelocks and VDFs

Reveal Only a Function

One-sentence intuition

Reveal only a function means exposing a computed result while keeping the underlying inputs hidden.

Where it sits in the taxonomy

- Level: design-pattern
- Parent category: design-pattern
- Related concepts: Functional Encryption, Homomorphic Encryption, MPC

Problem it solves

A party may need to disclose an approved result, such as a score, eligibility bit, or aggregate, without exposing the raw input that produced it. This pattern frames the boundary between useful disclosure and unnecessary data release.

Mental model

Think of a calculator that accepts hidden inputs and shows only the approved output, not the inputs themselves.

Minimal example

A service learns that a user score is above a threshold without learning the exact score.

Security properties

- controlled disclosure
- input privacy
- least information release

What it does not provide

- Protection if the function output is too revealing.
- Privacy across repeated queries without leakage analysis.
- Correctness unless inputs and computation are verified.

Assumptions

The allowed function must be chosen carefully, repeated queries must be controlled, and the underlying primitive or protocol must enforce that only the intended function result is revealed.

Post-quantum posture

Not applicable to the pattern by itself. A concrete system inherits posture from functional encryption, homomorphic encryption, multi-party computation, proof systems, and authentication.

Confidence model

Confidence depends on the mechanism: a key authority for functional encryption, threshold or honest-party assumptions for multi-party computation, or mathematical assumptions and key control for homomorphic encryption.

Common constructions

Common building blocks

- Homomorphic encryption.
- Functional encryption.
- Multi-party computation.
- Zero-knowledge proofs for input validity.

Concrete compositions

Composition	Revealed value	Main caution
Homomorphic encryption plus threshold decryption	Aggregate or limited computation result	Trustees and output leakage define the confidence model.
MPC computation	Function output from private inputs	Abort behavior and the revealed output can leak sensitive information.
Functional encryption	Function value authorized by a function key	Key issuer trust and repeated-query leakage are central.
ZK proof plus public computation	Proof that a hidden input satisfies a function predicate	The predicate may still reveal sensitive facts.

Use cases

- restricted analytics
- private classification
- private tallying

Composition patterns

- Repeated function outputs can reconstruct inputs.
- Access control must govern who can ask which function.

Common adjacent concepts: Functional Encryption, Homomorphic Encryption, MPC.

Failure modes and anti-patterns

- Differencing attacks across multiple outputs.
- Functions that encode private values directly.
- Missing access control around who can ask which function.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: expert-only. Parameter sensitivity: scheme-dependent.

Related concepts

- Functional Encryption
- Homomorphic Encryption
- MPC

Further reading

- Functional encryption
- Multi-party computation

Make Receipts Useless

One-sentence intuition

This pattern tries to stop users from proving to a coercer how they acted.

Where it sits in the taxonomy

- Level: design-pattern
- Parent category: design-pattern
- Related concepts: Coercion-Resistant Voting, Electronic Voting, Mixnets

Problem it solves

Some protocols fail if participants can later prove how they acted to a coercer, buyer, or outside observer. Receipt-freeness patterns try to remove or weaken that proof while preserving enough verifiability for the system to accept legitimate actions.

Mental model

Think of a voting booth where any receipt a voter brings outside can be plausibly faked or superseded.

Minimal example

A voting system lets a voter cast again so an earlier coerced vote and its receipt no longer prove the final counted choice.

Security properties

- receipt freeness
- coercion resistance
- deniability

What it does not provide

- Protection against all real-world coercion.
- Safety on compromised devices.
- A simple add-on to ordinary privacy systems.

Assumptions

The system must define the coercion model, control what public evidence is created, and account for user interfaces, logs, screenshots, credential recovery, and repeated participation.

Post-quantum posture

Not applicable to the pattern by itself. Concrete posture depends on the voting, credential, encryption, proof, and tallying mechanisms used.

Confidence model

Confidence usually combines public verifiability for tally integrity with protocol features that make user-held evidence deniable, fakeable, revocable, or superseded by later actions.

Common constructions

Common approaches

- Deniable or fakeable transcripts.
- Re-voting where only the last vote counts.
- Controlled credential recovery or revocation.
- Mixnets or tallying designs that avoid public vote receipts.

Concrete compositions

Composition	Typical role	Main caution
Re-voting with last vote counts	Reducing value of early coerced receipts	Does not help if coercion happens after the final opportunity to vote.
Fakeable credential or transcript	Let users simulate evidence for any choice	Hard to make convincing without weakening auditability.
Mixnet tallying without per-voter receipts	Hide ballot-to-voter linkage	Device compromise or check-in metadata can still create receipts.
Coercion-resistant credential recovery	Let voters invalidate coerced credentials	Registration and recovery channels become part of the threat model.

Use cases

- coercion resistant voting
- private signaling
- sensitive choice systems

Composition patterns

- Receipt-freeness must include operational artifacts such as logs and screenshots.
- Ballot secrecy alone is weaker than coercion resistance.

Common adjacent concepts: Coercion-Resistant Voting, Electronic Voting, Mixnets.

Failure modes and anti-patterns

- User interfaces expose receipts.

- Logs or screenshots become proofs.
- Coercers demand credentials before the action.
- Small groups make choices inferable from outcomes.

Maturity and deployment

Classified as emerging. This label describes the concept category, not a blanket endorsement of every construction or implementation. Implementation risk: expert-only. Parameter sensitivity: high.

Related concepts

- Coercion-Resistant Voting
- Electronic Voting
- Mixnets

Further reading

- Electronic voting
- Coercion-resistant voting

Glossary

Glossary

Authenticated encryption

A symmetric encryption interface that provides plaintext confidentiality and lets receivers reject modified ciphertexts, often while authenticating public associated data.

Related: Authenticated encryption, Symmetric encryption, Message authentication codes.

Anonymity

The property that an actor is hidden within a set of possible actors.

Related: Security goals, Anonymous membership, Mixnets.

Binding

For commitments, the property that a committer cannot open one commitment to two different values.

Related: Commitments, Pedersen commitments, Homomorphic commitments.

Commitment

A primitive that hides a value while binding the committer to that value for a later opening.

Related: Commitments, Delayed reveal, Range proofs.

Completeness

For proof systems, honest proofs for true statements should verify.

Related: Zero-knowledge proofs, SNARKs, STARKs, and Bulletproofs.

Confidence model

The source of confidence for a cryptographic guarantee, such as a hardness assumption, public verification, a trusted issuer, one honest party, or a threshold of honest parties.

Related: Confidence models, Assumptions, Trusted setup.

Discrete logarithm

A mathematical problem where exponentiation is easy in a selected group but recovering the exponent should be hard.

Related: Discrete logarithm, Digital signatures, Pedersen commitments.

Factoring

The problem of recovering prime factors from a composite number.

Related: Factoring and RSA, Public-key encryption, Post-quantum posture.

Hiding

For commitments, the property that the commitment does not reveal the committed value before opening.

Related: Commitments, Pedersen commitments.

Lattice

A structured high-dimensional grid used as the substrate for many post-quantum schemes.

Related: Lattices, Homomorphic encryption, Key encapsulation and exchange.

Metadata leak

Information revealed outside the protected cryptographic payload, such as timing, size, participation, or public inputs.

Related: Metadata leakage, Composability, Threat-model checklist.

Multi-party computation

A protocol family where parties jointly compute a function without revealing their private inputs beyond the output.

Related: MPC, Secure aggregation, Reveal only a function.

Nullifier

A public value used to detect repeated anonymous actions within a context.

Related: Nullifiers, Anti-double-use nullifiers, Anonymous airdrop.

Oblivious pseudorandom function

A two-party protocol where a client learns a keyed pseudorandom function output for its private input without learning the key and without revealing the input to the server.

Related: Oblivious pseudorandom functions, Private set intersection, Discrete logarithm.

Pairing

A special map between algebraic groups that makes some hidden exponent relationships publicly checkable.

Related: Pairings, SNARKs, STARKs, and Bulletproofs, Trusted setup.

Post-quantum posture

An editorial classification of whether a construction is quantum-vulnerable, plausibly post-quantum, dependent on instantiation, unknown, or not applicable.

Related: Post-quantum posture, Lattices, Discrete logarithm.

Private set intersection

A protocol family that lets parties learn the overlap, size of overlap, or approved function of their private sets while limiting disclosure about non-matching elements.

Related: Private set intersection, Oblivious pseudorandom functions, MPC.

Random oracle model

An idealized proof model that treats a hash function as a public random function.

Related: Random oracle model, Hash functions, Zero-knowledge proofs.

Receipt-freeness

A property where a participant cannot create convincing evidence of how they acted.

Related: Security goals, Make receipts useless, Coercion-resistant voting.

Soundness

For proof systems, false statements should not verify except with negligible probability.

Related: Zero-knowledge proofs, Range proofs, Membership proofs.

Trusted setup

A parameter-generation process whose hidden trapdoor must not survive for the system's claims to hold.

Related: Trusted setup, SNARKs, STARKs, and Bulletproofs, Confidence models.

Unlinkability

The property that two actions cannot be linked to the same actor under the stated model.

Related: Security goals, Anonymous credentials, Mixnets.

Witness

Private information that makes a public proof statement true.

Related: Zero-knowledge proofs, Range proofs, Membership proofs.

Zero-knowledge

The property that a proof reveals no information beyond the truth of the statement, under the proof system's model.

Related: Zero-knowledge proofs, SNARKs, STARKs, and Bulletproofs, Random oracle model.

Appendices

Source Policy and Topic References

The atlas does not maintain a single canonical reading list for all readers. References should be topic-dependent: each concept, protocol, pattern, and case study should include a `Further reading` section close to the claims it supports.

The structured source registry is `book/data/references.yml`. Concept cards and concrete instances refer to sources by registry ID so the book can later expose source filters, freshness checks, and dependency maps.

Editorial rule

- Put beginner-friendly orientation in the page body.
- Put source-backed technical depth in the page's `Further reading` section.
- Prefer online references with stable URLs.
- Prefer standards, peer-reviewed papers, recognized preprints, textbooks, and official documentation.
- Do not invent citations.
- Mark uncertain or fast-moving claims with a verification TODO instead of presenting them as settled.

Machine-readable references

The structured registry records:

- reference ID;
- title, authors, year, source type, URL, and optional DOI;
- `used_by` topics.

The content validator checks that every concept-card reference and concrete-instance reference points to a known registry ID.

Topic-local examples

- Secure Channels links to TLS 1.3, HPKE, Noise, and Signal specifications.
- Polynomial Commitments links to KZG, FRI, and Halo references.
- Private Payments links to e-cash and Zerocash sources.
- Privacy-Preserving Revocation links to credential and vector-commitment sources.

Maintenance rule

When a topic is updated, update its local `Further reading` section and the structured reference registry together. If a source is useful only for one page, it still belongs in the registry when a concept card or concrete instance needs to cite it by ID.

Notation

This appendix records lightweight notation used across the atlas.

Notation	Meaning
`m`	Message or value
`r`	Randomness
`C`	Commitment
`pk`	Public key
`sk`	Secret key
`Hash(x)`	Hash of input `x`
`context`	Domain-separated application or protocol context

Conventions

Examples are conceptual unless a page explicitly states that it is implementation-oriented.

Maturity Scale

Maturity labels describe deployment confidence at a high level. They are not security guarantees.

Label	Meaning
deployed	Used in production systems with meaningful operational history
mature	Well studied and commonly implemented, though maybe specialized
emerging	Actively used or piloted, but trade-offs are still settling
research	Mostly evaluated in papers, prototypes, or limited settings
theoretical	Mainly conceptual or impractical today
not-applicable	The label does not apply to this page

Rule

Maturity should be evaluated for a concrete construction and implementation, not only for a broad topic name.

Editorial State, Coverage, and Backlog

This appendix states what "current" means for the atlas, how coverage depth is labeled, what changed in recent editorial passes, and which work remains.

Status model

Status	Meaning in this book
current	Reviewed against the atlas template, taxonomy, caveat, post-quantum, confidence-model, and source expectations for the current scope. It does not mean exhaustive or production guidance.
needs-review	Known to be stale, shallow, or in a fast-moving area that needs a focused update before readers should treat it as current.
outdated	Contains claims that are known to be superseded or misleading.
draft	Structurally incomplete or not yet reviewed against the atlas editorial checklist.

Coverage-depth model

`status` and `coverage_depth` answer different questions. A page can be `status: current` because it has been reviewed, while still being `coverage_depth: grouped-first-pass` because it intentionally summarizes several concepts.

Coverage depth	Meaning
standalone	A concept, protocol, pattern, or case study has its own template-complete page and concept card when applicable.
grouped-first-pass	Several related concepts are covered in one reviewed overview page. This is acceptable for breadth, but not a substitute for standalone depth.
routing-overview	A formerly grouped page that now mostly routes readers to standalone pages while retaining a compact comparison map.
overview	A landing or orientation page for a section.
reference	Appendix-style reference material.
not-applicable	Pages where coverage depth is not meaningful.

Current maturity assessment

The taxonomy is good for the book's mission. It separates goals, assumptions, primitives, proof systems, protocols, systems, and design patterns, which prevents the common mistake of treating a named tool as a complete system design.

The concrete instance layer is the right way to place algorithms and schemes. It avoids turning names such as `AES-GCM`, `Ed25519`, `Groth16`, or `TLS 1.3` into a ninth peer taxonomy level. Those names are tracked in `book/data/instances.yml` with machine-readable `instance_of` relationships to parent concept-card IDs.

One taxonomy tension remains: some objects are both building blocks and interactive protocols. Examples include oblivious transfer and oblivious pseudorandom functions. The current rule is:

- use `protocol` when the guarantee comes from interaction between parties;
- cross-link it as a building block where it is composed into MPC, PSI, credentials, or token systems;
- avoid creating a separate "protocol primitive" level unless the atlas later needs a more formal ontology.

Coverage added in recent passes

Area	Added concept	Why it matters
Basic primitives	Authenticated encryption	Modern systems usually need confidentiality and integrity together; bare encryption is easy to misuse.
Protocols	Oblivious pseudorandom functions	OPRFs are a common building block for password hardening, tokens, credentials, and PSI.
Protocols	Private set intersection	PSI is one of the core privacy-preserving computation protocols missing from the initial protocol set.
Security goals	Additional security goals	Covers verifiability, auditability, accountability, forward secrecy, and deniability.
Assumptions and substrates	Additional assumptions and substrates	Covers elliptic curves, finite-field groups, code-based assumptions, hash-to-curve, common reference strings, and proof-model distinctions.
Basic primitives	Primitive engineering concepts	Covers PRFs, PRPs, password hashing, nonce-misuse-resistant encryption, and key-committing encryption.
Structured primitives	Advanced structured primitives	Covers VRFs, blind signatures, polynomial/vector commitments, verifiable encryption, and e-cash primitives.
Proof systems	Proof-system components	Covers polynomial commitments, FRI, folding, recursive proofs, lookup arguments, sumcheck, and arithmetization.
Protocols	Additional protocol families	Covers oblivious transfer, PIR, PAKE, secure channels, anonymous tokens, blind-signature credentials, and private-payment protocols.
Systems	Systems and Applications Overview	Routes readers to private payments, ZK rollups, secure messaging, identity wallets, encrypted mempools, private ML analytics, voting systems, and anonymous airdrops.
Design patterns	Operational design patterns	Covers encrypt-then-prove, threshold issuance, privacy-preserving revocation, domain separation, transcript binding, and key rotation.
Protocols	Secure Channels	Promotes TLS, HPKE, Noise, and Signal-style channel assumptions into standalone treatment.
Protocols	Oblivious Transfer	Separates a core MPC building block from the broader protocol-family overview.
Protocols	Private Information Retrieval	Clarifies query privacy, non-collusion, and database freshness assumptions.
Protocols	Password-Authenticated Key Exchange	Distinguishes PAKE from password hashing and ordinary secure channels.

Area	Added concept	Why it matters
Structured primitives	Polynomial Commitments	Pulls out a central proof-system and rollup dependency with setup and post-quantum caveats.
Structured primitives	Vector Commitments	Gives indexed commitments and authenticated state roots a proper taxonomy location.
Systems	Private Payments	Promotes note/nullifier, e-cash, wallet, and metadata risks into a full case study.
Systems	ZK Rollups	Separates validity proofs from data availability, sequencing, and governance assumptions.
Design patterns	Domain Separation	Makes context labeling explicit as a reusable implementation pattern.
Design patterns	Transcript Binding	Makes transcript completeness a standalone composition rule.
Design patterns	Privacy-Preserving Revocation	Gives revocation a privacy-aware operational pattern instead of a footnote inside credentials.
Data model	Concrete Algorithms and Schemes	Adds machine-readable concrete-instance data via `book/data/instances.yml`.
Assumptions and substrates	Elliptic Curves	Promotes curve assumptions, validation hazards, and post-quantum posture into standalone treatment.
Assumptions and substrates	Code-Based Assumptions	Adds a standalone page for code-based post-quantum assumptions and deployment caveats.
Assumptions and substrates	Common Reference Strings	Separates setup material, toxic-waste, and ceremony confidence models from proof-system pages.
Basic primitives	Password Hashing	Separates offline guessing resistance from KDFs, PAKEs, and password storage operations.
Basic primitives	Key-Committing Encryption	Adds a standalone page for ciphertext/key binding and message-franking style ambiguity risks.
Structured primitives	Blind Signatures	Promotes blind issuance and unlinkability caveats into standalone treatment.
Structured primitives	Verifiable Random Functions	Adds a standalone VRF page with withholding, grinding, and randomness caveats.
Structured primitives	Verifiable Encryption	Gives encrypt-and-prove workflows a primitive-level treatment.
Structured primitives	E-cash Primitives	Adds issuer, coin, double-spend, and redemption building blocks for private payments.
Proof systems	FRI	Adds transparent low-degree testing as a standalone proof-system component.
Proof systems	Folding Schemes	Adds incremental proof composition and accumulator caveats.
Proof systems	Arithmetization	Makes statement encoding and underconstraint bugs explicit.
Proof systems	Sumcheck	Adds a standalone page for polynomial sum claims and GKR-style components.
Proof systems	Recursive Proofs	Separates recursion, verifier circuits, and public-input binding risks.
Proof systems	Lookup Arguments	Adds lookup tables, multiplicity, and table-versioning caveats.
Protocols	Anonymous Tokens	Adds issuance/redemption unlinkability and metadata boundaries.

Area	Added concept	Why it matters
Protocols	Blind-Signature Credentials	Separates blind issuance from full anonymous credential guarantees.
Systems	Secure Messaging	Adds a system page for channels, ratchets, devices, backups, and metadata.
Systems	Identity Wallets	Adds a system page for credentials, selective disclosure, issuers, and revocation.
Systems	Encrypted Mempools	Adds a system page for encrypted ordering, threshold release, and MEV boundaries.
Systems	Private Machine-Learning Analytics	Adds a system page for secure aggregation, MPC, HE, and differential privacy composition.
Design patterns	Key Rotation and Migration	Adds a standalone operational pattern for cryptographic agility and PQ migration.
Design patterns	Encrypt-then-prove	Adds a standalone pattern for proving statements about encrypted data.
Design patterns	Threshold Issuance	Adds a standalone pattern for distributed authorization and issuer quorum risks.
Data model	Generated Comparison Matrices	Adds generated reader-facing Markdown from machine-readable comparison YAML.
Data model	Source freshness clusters	Adds `book/data/source-freshness.yml` and validation for fast-moving reference review windows.

Coverage status

The concepts previously listed as highest-value gaps now have standalone first-pass pages. The grouped overview pages remain as routing maps and are marked `needs-review` with `coverage_depth: routing-overview` so they are not confused with complete standalone coverage.

Taxonomy area	Missing or shallow concepts	Why they matter
Security goals	promoted with routing overview	Confidentiality, integrity, authenticity, anonymity, unlinkability, forward secrecy, verifiability, auditability, accountability, deniability, non-repudiation, availability, and censorship resistance now have standalone pages. Receipt-freeness and coercion resistance remain grouped system-level refinements.
Assumptions and substrates	promoted with routing overview	Elliptic curves, code-based assumptions, and common reference strings now have standalone pages; finite-field groups and hash-to-curve remain future candidates.
Basic primitives	promoted with routing overview	Password hashing and key-committing encryption now have standalone pages; PRFs, PRPs, and misuse-resistant encryption remain candidates if depth is needed.
Structured primitives	promoted with routing overview	Blind signatures, VRFs, verifiable encryption, and e-cash primitives now have standalone pages.
Proof systems	promoted with routing overview	FRI, folding schemes, arithmetization, sumcheck, recursive proofs, and lookup arguments now have standalone pages.

Taxonomy area	Missing or shallow concepts	Why they matter
Protocols	promoted with routing overview	Anonymous tokens and blind-signature credentials now have standalone pages; private-payment protocol details are handled in the private-payments case study.
Systems	promoted with routing overview	Secure messaging, identity wallets, encrypted mempools, and private ML analytics now have standalone pages.
Design patterns	promoted with routing overview	Key rotation and migration, encrypt-then-prove, and threshold issuance now have standalone pages.

Source-depth status

The source registry now covers the main historical and standards references for existing pages, plus deeper clusters for ZK proof-system families, implementation stacks, threshold signing and DKG, anonymous credentials, private payments, secure channels, encrypted mempools, identity wallets, post-quantum migration, FHE, and MPC.

Remaining source work should focus on depth and freshness rather than breadth alone:

- add implementation-specific references only when they explain deployed behavior, parameter choices, or failure modes;
- revisit fast-moving draft specifications on the review windows in `book/data/source-freshness.yml`;
- keep source clusters topic-local so readers can distinguish foundational papers, standards, and deployed project documentation.

Claim-level citation rule

Use inline source links immediately after high-risk claims about post-quantum posture, standards status, protocol maturity, deployment cautions, performance trade-offs, implementation risk, setup assumptions, metadata leakage, revocation leakage, and circuit correctness.

Keep `Further reading` sections for background orientation. Do not rely on a page-level bibliography alone when a specific sentence or table row makes a nontrivial technical, maturity, standards, or deployment claim.

Current limitations and editorial priorities

The atlas has a strong taxonomy and data model, but several product and editorial limitations keep it from reading like a finished web book:

Limitation	Why it matters	Direction
Concept card discovery still lacks generated guided paths.	Filters and taxonomy sections make the card set easier to browse, but new readers still need goal-oriented routes through the material.	Add curated entry points for common reader goals, then consider query-generated paths after retrieval, citation, and review controls are defined.
Coverage is broader than it is deep.	Many pages are accurate, template-complete summaries but do not yet provide the worked examples, narrative progression, or design comparisons needed for durable understanding.	Develop a smaller set of flagship chapters with worked scenarios, deeper diagrams, and explicit comparison of competing approaches.
Claim-level sourcing is now started but uneven.	High-risk pages have inline citations for selected claims, but the whole atlas has not been audited sentence by sentence.	Expand claim-level citations for nontrivial technical, maturity, deployment, standards, and performance claims.
Expert review is now explicit but mostly empty.	Separate review metadata prevents `current` from implying expert cryptographic review, but the atlas still needs named human review for high-risk chapters.	Record expert reviewers only when named review actually happens; otherwise keep `expert.status: not-reviewed`.
Machine-oriented labels appear in reader-facing concept cards.	Values such as `public-verifiability` or inconsistent category capitalization make the interface feel database-first and reduce readability.	Add a presentation-label layer while retaining stable machine identifiers underneath.
Warning coverage is partly route-targeted.	Automatic warnings now cover common dangerous topics, but page-specific caution text still benefits from editorial refinement.	Keep the automatic `WarningBox` layer and add targeted local warnings for nuanced failure modes when useful.
The visual identity is functional but unfinished.	The layout is clean, but the placeholder logo and generic documentation styling do not yet give the atlas a distinctive editorial identity.	Replace placeholder branding after information architecture and chapter depth stabilize.
Sidebar hierarchy has historically been flatter than the concept hierarchy.	A child page displayed beside its parent obscures dependency and specialization relationships, making a large book harder to navigate.	Use linked sidebar categories for direct parent-child relationships and routing overviews, while preserving taxonomy boundaries for cross-level dependencies.

Completed backlog items in this pass

1. Promoted grouped coverage into standalone pages for elliptic curves, code-based assumptions, common reference strings, password hashing, key-committing encryption, blind signatures, VRFs, verifiable encryption, e-cash primitives, FRI, folding schemes, arithmetization, sumcheck, recursion, lookup arguments, anonymous tokens, blind-signature credentials, secure messaging, identity wallets, encrypted mempools, private ML analytics, key rotation, encrypt-then-prove, and threshold issuance.
2. Expanded `book/data/instances.yml` with additional curves, code-based schemes, blind/VRF schemes, protocol suites, proof-system constructions, status-list revocation, and migration-relevant concrete entries.
3. Added generated reader-facing rendering for comparison-matrix YAML.
4. Added source-freshness review windows for fast-moving reference clusters.
5. Added topic-local URLs to the grouped routing pages that still had title-only further-reading entries.
6. Added deeper source coverage for ZK proof-system families, threshold signing and DKG, anonymous credentials, private payments, secure channels, encrypted mempools, identity wallets, and post-

quantum migration.

7. Expanded `book/data/instances.yml` with legacy schemes, additional parameter families, pairing-friendly curves, ZK-friendly hashes, FHE schemes, MPC protocol families, deployed credential profiles, and deployed private-payment and encrypted-mempool profiles.
8. Promoted high-value security goals into standalone pages and concept cards for confidentiality, integrity, authenticity, anonymity, unlinkability, forward secrecy, and verifiability.
9. Added richer relationship edge types beyond `instance_of`, including `uses`, `requires`, `breaks-if`, and `commonly-composed-with`, with validation against concept-card IDs, instance IDs, aliases, and document slugs.
10. Cleaned older `Further reading` sections so title-only citations are replaced with topic-local URLs where a stable source URL was available.
11. Added generated reader-facing and machine-facing relationship graph exports: `book/docs/appendices/generated-relationship-graph.md` and `book/static/data/relationship-graph.json`.
12. Expanded `book/data/instances.yml` with PQ parameter profiles, less common curves, ZK implementation stacks, FHE libraries, MPC frameworks, wallet/credential deployment profiles, and encrypted-mempool prototypes.
13. Promoted auditability, accountability, deniability, non-repudiation, availability, and censorship resistance into standalone security-goal pages with concept cards.
14. Added implementation-specific references for halo2, gnark, arkworks, Circom, Zcash Foundation FROST, drand, OpenID4VC, EUDI ARF, AnonCreds, Shutter, OQS, Microsoft SEAL, OpenFHE, Concrete ML, MP-SPDZ, EMP, and FRESCO.
15. Added per-page `source_review` notes and review windows to fast-moving pages that depend on draft standards, ecosystem specifications, proving stacks, PQ migration tooling, FHE/MPC libraries, or encrypted-mempool prototypes.
16. Audited every sidebar entry and added linked parent-child categories where the relationship is direct or represented by a routing overview, without moving cross-level dependencies into the wrong taxonomy section.
17. Promoted Security Goals to a top-level sidebar category so all eight primary taxonomy levels have consistent navigation placement.
18. Renamed the Case Studies navigation level to Systems and Applications, moved it before Design Patterns to match the taxonomy, migrated its pages to natural `/docs/systems-and-applications/` routes, and removed the redundant additional-systems grouping.
19. Added data-driven homepage concept-card filters for search, taxonomy, maturity, post-quantum posture, implementation risk, and the three review dimensions.
20. Added separate structural, source, and expert review metadata to page frontmatter, validation, generated concept-card data, and reader-facing document pages.
21. Added automatic `WarningBox` rendering for expert-only, high-risk, research-stage, trusted-setup-sensitive, and quantum-vulnerable pages, plus converted existing local warning blocks to the shared component.
22. Replaced the flat homepage card wall with taxonomy sections, counted filter options, and progressive per-section disclosure.
23. Added targeted automatic warning coverage for trusted setup, nonce misuse, unauthenticated encryption, signature context binding, revocation leakage, metadata leakage, underconstrained ZK

circuits, and educational-not-production guidance.

24. Started claim-level citation coverage for high-risk claims on encryption, signatures, key exchange, secure channels, ZK proof systems, private payments, credentials, identity wallets, and metadata leakage pages.

Remaining editorial backlog

This is the authoritative maintenance backlog. Do not duplicate the active list in `TODO.md`.

1. Add curated reading paths for common reader goals, such as evaluating a ZK system, designing a secure channel, comparing private-computation approaches, or reviewing a private-payment design.
2. Develop 10 to 15 flagship chapters with worked examples, stronger narrative progression, more diagrams, and comparisons between plausible design choices.
3. Continue expanding claim-level citations or structured source references for nontrivial technical, deployment, maturity, and performance claims.
4. Normalize reader-facing labels and capitalization without changing stable concept-card IDs or relationship keys.
5. Replace the placeholder logo and refine the visual identity after the navigation and flagship chapter work stabilizes.
6. Maintain sidebar parent-child nesting when a page has a direct conceptual parent or a routing overview, but do not move a concept across taxonomy levels merely because it depends on a concept in another section.
7. Add a formal JSON schema for `static/data/relationship-graph.json` if external tools begin consuming the graph snapshot.
8. Add generated graph slices for common reader questions, such as "what breaks this guarantee?", "what does this scheme require?", and "which concepts use this primitive?".
9. Continue expanding `book/data/instances.yml` selectively with additional deployed libraries, wallet profiles, proof-system backends, PQ migration profiles, and legacy schemes only when readers are likely to encounter them.
10. Promote receipt-freeness, coercion resistance, fairness, or liveness into standalone pages if the atlas adds a larger voting, governance, or consensus-goals section.
11. Add versioned audit-status notes for implementation stacks where audits, release trains, or API stability materially change deployment advice.
12. Continue source-freshness reviews on the six-month windows and mark pages `needs-review` when draft standards or ecosystem profiles move.

Timestamped editorial changelog

Date	Change
2026-06-16	Added taxonomy-sectioned concept-card browsing, hardened filter option rendering, targeted automatic warning topics, and first-pass claim-level citations on high-risk pages.
2026-06-16	Added concept-card filters, split structural/source/expert review metadata, reader-facing review status panels, and automatic `WarningBox` risk callouts.
2026-06-05	Aligned the web-book navigation with the taxonomy by replacing Case Studies with Systems and Applications before Design Patterns and moving system pages to natural routes.

Date	Change
2026-06-05	Promoted Security Goals from the Taxonomy orientation category to its own top-level sidebar category.
2026-06-05	Added the current product and editorial limitations to the authoritative backlog, including discovery, chapter depth, claim-level sourcing, review semantics, reader-facing labels, warnings, and visual identity.
2026-06-05	Reorganized the sidebar around direct parent-child relationships and routing overviews while preserving taxonomy boundaries and existing page URLs.
2026-06-04	Added AI-generation disclosure to homepage, intro, footer, and PDF cover.
2026-06-04	Added PDF table-of-contents page numbers and clickable concept-card navigation.
2026-06-04	Added first-pass grouped coverage for missing security goals, assumptions, primitive engineering concepts, structured primitives, proof-system components, protocol families, systems, and operational design patterns.
2026-06-04	Promoted secure channels, oblivious transfer, private information retrieval, password-authenticated key exchange, polynomial commitments, vector commitments, private payments, ZK rollups, domain separation, transcript binding, and privacy-preserving revocation to standalone pages with concept cards.
2026-06-04	Added machine-readable concrete-instance data in `book/data/instances.yml` and validation for `instance_of` and reference IDs.
2026-06-04	Added generated diagram sources for secure-channel handshakes, ZK-rollup data flow, private-payment note/nullifier flow, and privacy-preserving revocation.
2026-06-04	Promoted the remaining grouped backlog topics into standalone pages with concept cards.
2026-06-04	Added `source-freshness.yml` review clusters for ZK, threshold signing, anonymous credentials, private payments, secure channels, identity wallets, and post-quantum migration.
2026-06-04	Added generated reader-facing comparison matrices from `book/data/comparison-matrices/*.yml`.
2026-06-04	Expanded concrete instances with curves, code-based KEMs, blind signatures, VRFs, FROST, MLS, Privacy Pass, PLONK, Nova, Halo, GKR-style sumcheck, and status-list revocation.
2026-06-04	Promoted confidentiality, integrity, authenticity, anonymity, unlinkability, forward secrecy, and verifiability into standalone security-goal pages with concept cards.
2026-06-04	Expanded source coverage for ZK proof-system families, threshold signing and DKG, anonymous credentials, private payments, secure channels, encrypted mempools, identity wallets, FHE/MPC, and post-quantum migration.
2026-06-04	Expanded concrete instances with legacy schemes, pairing-friendly curves, ZK-friendly hashes, FHE schemes, MPC protocol families, deployed credential profiles, private-payment profiles, and encrypted-mempool profiles.
2026-06-04	Added relationship edge types for uses, requirements, break conditions, and common compositions, with validator checks against pages, concept cards, instances, and aliases.
2026-06-04	Added generated relationship graph exports as reader-facing Markdown and machine-facing JSON.
2026-06-04	Expanded concrete instances with PQ parameter profiles, BLS12-377/BW6-761/Jubjub, halo2, gnark, arkworks, Circom, Microsoft SEAL, OpenFHE, Concrete ML, MP-SPDZ, EMP, FRESCO, OpenID4VC, EUDI, ISO mdoc, AnonCreds, DIDComm, Shutter, and OQS entries.
2026-06-04	Promoted auditability, accountability, deniability, non-repudiation, availability, and censorship resistance into standalone security-goal pages with concept cards.

Date	Change
2026-06-04	Added per-page source-review windows to fast-moving ZK, threshold, credential, payment, channel, mempool, wallet, PQ, FHE, MPC, and private-ML pages.

Maintenance rule

For fast-moving areas such as post-quantum cryptography, zero-knowledge proof systems, fully homomorphic encryption, MPC frameworks, anonymous credentials, and blockchain privacy, re-check source freshness at least every six months. If a page has not been reviewed in that window, mark it `needs-review` rather than leaving it `current`.

Bottom line

The taxonomy is sound. The atlas is mature as an educational map, the highest-priority gaps have standalone pages, and concrete instances plus relationship edges are machine-readable and generated into reader-facing views. The biggest remaining work is turning that breadth into a guided web-book experience through better discovery, deeper flagship chapters, claim-level sourcing, clearer review semantics, and disciplined source freshness.

Comparison Matrices

Comparison Matrices

These matrices are editorial shortcuts. They compare where to look first, not which construction to deploy.

The manually maintained overview below is intentionally short. The reader-facing generated rendering of the full machine-readable YAML matrix data is Generated Comparison Matrices.

Basic primitives

Primitive	Primary goal	Does not provide	Common risk
Hash functions	Fixed-length digest with preimage and collision resistance	Encryption, authentication, hiding low-entropy secrets	Missing domain separation or obsolete algorithms
Symmetric encryption	Confidentiality under a shared secret key	Key distribution or authentication by itself	Nonce misuse or unauthenticated ciphertexts
MACs	Shared-key integrity and authenticity	Public verifiability or confidentiality	Replay, key reuse, timing-leaky comparisons
Digital signatures	Public verifiability of message origin and integrity	Confidentiality or signer understanding	Ambiguous signing contexts or nonce reuse
Public-key encryption	Recipient-controlled confidentiality	Sender authentication or metadata privacy	Key misbinding or unsafe raw encryption
Secret sharing	Threshold reconstruction and confidentiality below threshold	Share authentication or governance	Correlated share custody or unclear recovery rules

Privacy protocols

Protocol	Main privacy goal	Confidence model	Metadata leaks
Anonymous credentials	Selective disclosure or unlinkable authorization	Trusted issuer plus holder secret	Issuer-verifier collusion, rare attributes, revocation checks
Nullifiers	One anonymous action per context	Holder secret plus public duplicate registry	Timing, transaction fees, reused contexts
Secure aggregation	Aggregate-only disclosure	Honest-majority or non-collusion depending on protocol	Participation, dropout, cohort size
MPC	Joint computation without revealing inputs	Adversary threshold and abort model	Participant set, circuit shape, aborts
Mixnets	Sender-recipient unlinkability through shuffling	At least one honest mix	Timing, batch membership, message size

Systems and patterns

Item	Level	Primary goal	Deployment caution
Anonymous membership	Design pattern	Prove eligibility without revealing member identity	Needs anti-double-use design when repeated action matters
Private aggregation	Design pattern	Reveal aggregate rather than individual inputs	Small groups and differencing attacks dominate many failures
Delayed reveal	Design pattern	Commit now and open later	Define deadlines, challenges, and abort handling
Private DAO voting	System	Private eligible voting with public tally confidence	Not coercion-resistant by default
Anonymous airdrop	System	One private claim per eligible user	Public ledger metadata can defeat cryptographic anonymity

Data files

The machine-readable versions live in:

- `book/data/comparison-matrices/proof-systems.yml`
- `book/data/comparison-matrices/primitives.yml`
- `book/data/comparison-matrices/privacy-protocols.yml`
- `book/data/comparison-matrices/system-patterns.yml`

Regenerate the reader-facing appendix with `npm run generate:comparison-matrices` from `book/`, or with `npm run generate` when updating all generated book assets.

Generated Comparison Matrices

This page is generated from `book/data/comparison-matrices/*.yml`. Edit the YAML data, not this Markdown file.

Basic primitive comparison

Comparison of common primitive roles, non-goals, and implementation risks.

Concept	Primary Goal	Does Not Provide	Common Risk	Post Quantum Posture	Implementation Risk
Hash functions	fixed-length digest with preimage and collision resistance	encryption, authentication, or hiding for low-entropy secrets	missing domain separation or obsolete algorithms	plausible with adequate output length	medium
Symmetric encryption	confidentiality under a shared secret key	key distribution or authentication by itself	nonce misuse or unauthenticated ciphertexts	plausible with conservative key sizes	high
Authenticated encryption	confidentiality plus ciphertext and associated-data integrity	public verifiability, identity, or metadata privacy	nonce reuse, missing associated-data context, or tag-check bypass	plausible with conservative parameters	high
MACs	shared-key integrity and authenticity	public verifiability or confidentiality	replay, key reuse, or timing-leaky comparisons	plausible	medium
Digital signatures	public verifiability of message origin and integrity	confidentiality or signer understanding	ambiguous signing contexts or nonce reuse	depends on scheme	high
Public-key encryption	recipient-controlled confidentiality	sender authentication or metadata privacy	key misbinding or unsafe raw encryption	depends on scheme	high
Password hashing	make offline guessing of low-entropy secrets expensive	high entropy, online rate limiting, or password-authenticated key exchange	fast unsalted hashes, weak reset flows, or stale work factors	plausible with current memory-hard parameters	medium
Key-committing encryption	bind ciphertext validity to the intended key or key commitment	sender attribution, deniability, or metadata privacy by itself	using non-committing AEAD where key ambiguity matters	depends on the underlying encryption and authentication layers	high
Secret sharing	threshold reconstruction and confidentiality below threshold	share authentication or governance	correlated share custody or unclear recovery rules	plausible for the information-theoretic core	medium

Source: `book/data/comparison-matrices/primitives.yml`.

Privacy protocol comparison

Comparison of common privacy protocols and their metadata boundaries.

Concept	Main Privacy Goal	Confidence Model	Metadata Leaks	Common Failure	Maturity
Anonymous credentials	selective disclosure or unlinkable authorization	trusted issuer plus holder secret	issuer-verifier collusion, rare attributes, revocation checks	revocation or verifier context tracks users	emerging
Anonymous tokens	unlink issuance and redemption of authorization tokens	issuer policy plus blind signature, OPRF, or accumulator assumptions	issuance timing, redemption timing, transport identifiers	token privacy defeated by browser, network, or verifier logs	emerging
Blind-signature credentials	authorize a credential without showing the issuer the eventual presentation token	trusted issuer plus blind-signature unforgeability and holder-secret policy	issuance metadata, presentation timing, revocation checks	blind issuance treated as full anonymous credentials	emerging
Nullifiers	one anonymous action per context	holder secret plus public duplicate registry	timing, transaction fees, reused contexts	nullifier accepted without eligibility proof	emerging
OPRFs	client learns keyed PRF output without revealing input	mathematical assumption plus server-key secrecy	request timing, server identity, request volume	missing domain separation or invalid group-element handling	emerging
Private set intersection	reveal only set overlap, cardinality, or approved function	depends on semi-honest, malicious, or helper-server model	set size, output, timing, abort behavior	repeated queries, low-entropy elements, or wrong adversary model	mature
Secure aggregation	aggregate-only disclosure	honest-majority or non-collusion depending on protocol	participation, dropout, cohort size	small cohorts or repeated queries reveal individuals	mature
MPC	joint computation without revealing inputs	adversary threshold and abort model	participant set, circuit shape, aborts	output or abort pattern leaks inputs	mature
Mixnets	sender-recipient unlinkability through shuffling	at least one honest mix	timing, batch membership, message size	small batches or all mixes collude	mature
Secure messaging	message-content confidentiality with forward secrecy and participant authentication	endpoint secrets, ratchet state, key directories, and server-delivery assumptions	contact graph, device list, timing, IP addresses	endpoint compromise or plaintext backups defeat protocol guarantees	deployed

Source: `book/data/comparison-matrices/privacy-protocols.yml`.

Proof-system family comparison

High-level comparison for selecting which proof-system page to read next.

Concept	Setup	Common Strength	Common Risk	Post Quantum Posture	Maturity
SNARKs	varies; many deployed families use trusted or universal setup	very small proofs and fast verification	setup and pairing assumptions can dominate the confidence model	often vulnerable for pairing-based systems	deployed to emerging by family
STARKs	transparent in common families	hash-based transparency and scalable proving	larger proofs and careful parameter choices	plausible when hash choices and parameters are appropriate	emerging
Bulletproofs	no trusted setup	compact range proofs	discrete-logarithm assumptions and heavier verification for large statements	vulnerable in common discrete-logarithm constructions	mature
FRI	transparent	scalable low-degree testing for STARK-style systems	parameter, field, query, and hash choices drive soundness and size	plausible when instantiated with conservative hashes and parameters	emerging
Folding schemes	construction-specific	incrementally combine repeated computation claims	fast-moving security analysis and accumulator-finalization details	depends on commitment and proof-system assumptions	research to emerging
Recursive proofs	inherited from the inner and outer proof systems	aggregate or chain proofs by verifying proofs inside proofs	transcript binding, verification-key binding, and field-cycle constraints	depends on every recursive layer	emerging
Lookup arguments	inherited from the arithmetization and commitment scheme	efficient range, table, and opcode checks	table versioning, multiplicities, and leaked lookup values	depends on the enclosing proof system	emerging
Sumcheck	transparent as an interactive protocol component	reduces large polynomial sum claims to small checks	wrong degree bounds, field sizes, or Fiat-Shamir transcript binding	plausible when paired with post-quantum commitments and hashes	mature component
Arithmetization	not a setup model; it defines the statement representation	turns computation into checkable constraints or polynomial identities	underconstrained circuits or traces prove the wrong program	not applicable by itself	mature concept, implementation-specific

Source: `book/data/comparison-matrices/proof-systems.yml`.

System and pattern comparison

Comparison of design patterns and system-level privacy compositions.

Concept	Level	Primary Goal	Inherited Risk	Metadata Boundary	Deployment Caution
Anonymous membership	design pattern	prove eligibility without revealing member identity	issuer, set, proof system, and anonymity set	timing, verifier identity, group size	needs anti-double-use or rate-limit design when repeated action matters
Private aggregation	design pattern	reveal aggregate rather than individual inputs	aggregation mechanism, cohort size, validity checks	participation, output value, repeated releases	small groups and differencing attacks dominate many failures
Encrypt-then-prove	design pattern	prove validity of encrypted data without revealing the plaintext	encryption, proof statement, recipient-key binding, and transcript context	proof timing, recipient key, public inputs	bind the proof to the exact ciphertext, key, and application context
Threshold issuance	design pattern	require multiple issuers to authorize tokens, credentials, or signatures	DKG or share provisioning, quorum policy, issuer availability	issuer participation and issuance timing	threshold control is not trustlessness when operators collude
Key rotation and migration	design pattern	replace keys or algorithms without silent downgrade or continuity loss	trust roots, inventory accuracy, client enforcement, version policy	supported versions and migration timing	dual-stack periods often preserve the weakest accepted algorithm
Delayed reveal	design pattern	commit now and open later	commitment binding, timing, abort policy	commitment and reveal times	define deadlines, challenges, and abort handling
Private DAO voting	system	private eligible voting with public tally confidence	credentials, nullifiers, ballot encryption, proofs, tallying	wallet funding, submission timing, public discussion	not coercion-resistant by default
Secure messaging	system	protect message contents and session keys across devices and compromise windows	secure channels, ratchets, key directories, backups, endpoint security	contact graph, delivery timing, device list	endpoint compromise and plaintext backup policy often dominate
Identity wallets	system	hold credentials and present claims with selective disclosure	issuer trust, holder-device secrets, verifier policy, revocation mechanism	presentation timing, verifier identity, status checks	rare attributes and online revocation can re-identify holders
Encrypted mempools	system	hide transaction contents before ordering or reveal	threshold encryption, sequencing, censorship resistance, key release	sender network path, size, timing, inclusion status	encryption cannot prevent censorship or metadata-based ordering alone
Private machine-learning analytics	system	learn aggregates, model updates, or metrics with reduced exposure of raw inputs	secure aggregation, MPC, HE, differential privacy, cohort policy	participation, cohort membership, output releases	outputs, repeated queries, and small cohorts can defeat cryptographic input privacy

Concept	Level	Primary Goal	Inherited Risk	Metadata Boundary	Deployment Caution
Anonymous airdrop	system	one private claim per eligible user	eligibility source, nullifiers, wallet funding, withdrawal privacy	claim timing, fees, exchange flows	public ledger metadata can defeat cryptographic anonymity

Source: `book/data/comparison-matrices/system-patterns.yml`.

Generated Relationship Graph

This page is generated from `book/data/relationships.yml`. Edit the YAML relationship data, concept cards, and instance registry rather than this Markdown file.

Machine-readable snapshot: relationship-graph.json.

Graph size: 120 referenced nodes and 111 directed edges.

Direct Dependencies

`requires` and `uses` edges show dependencies that should be reviewed before changing a concept, protocol, system, or concrete instance.

Source	Relation	Target	Notes
Pedersen commitment	Requires	Discrete Logarithm	Binding depends on the infeasibility of finding the generator relation.
Range Proofs	Requires	Commitments	Range proofs usually constrain a hidden value inside a commitment or ciphertext.
Authenticity	Requires	Transcript Binding	Authenticating incomplete context can bind the right key to the wrong session or statement.
Forward Secrecy	Requires	Key Encapsulation and Exchange	Secure channels need fresh key-establishment material to make later long-term key compromise less damaging.
Zcash Orchard shielded protocol	Uses	Halo-style recursion	Orchard uses Halo-style proving to avoid the trusted setup model used by earlier Zcash pools.
Zcash Sapling shielded protocol	Uses	Private Payments	Sapling is a concrete shielded-payment protocol suite.
BBS Data Integrity credentials	Uses	Anonymous Credential	BBS-based credential profiles support selective disclosure and unlinkable derived proofs under pairing assumptions.
W3C Data Integrity ECDSA credentials	Uses	Authenticity	ECDSA data integrity credentials primarily provide issuer authenticity and integrity, not anonymity.
W3C Data Integrity EdDSA credentials	Uses	Authenticity	EdDSA data integrity credentials primarily provide issuer authenticity and integrity, not anonymity.
LUCID encrypted mempool proposal	Uses	Delayed Reveal	Encrypted mempools use commit-before-reveal or delayed-decryption sequencing rules.
Ferveo threshold-decrypted mempool	Uses	Threshold Cryptography	Threshold decryption distributes reveal power across a committee.
Threshold BLS signatures	Requires	Pairings	Threshold BLS inherits both threshold-share assumptions and pairing-friendly curve assumptions.
MASCOT	Uses	Oblivious Transfer	MASCOT uses OT-based preprocessing for maliciously secure arithmetic MPC.
Accountability	Requires	Authenticity	Misbehavior cannot be attributed if actions are not bound to accountable roles, keys, or credentials.

Source	Relation	Target	Notes
Non-Repudiation	Requires	Digital Signature	Non-repudiation commonly depends on signatures plus operational key-control evidence.
Censorship Resistance	Requires	Availability	A valid action needs at least one live inclusion path before censorship resistance can be meaningful.
BLS12-377	Requires	Pairings	BLS12-377 is used as a pairing-friendly curve in proof-system stacks.
BW6-761	Requires	Pairings	BW6-761 is often discussed as an outer curve in recursive proof cycles.
SLH-DSA SHA2 parameter sets	Uses	Hash Function	SLH-DSA SHA2 profiles rely on hash-based security assumptions and large signatures.
SLH-DSA SHAKE parameter sets	Uses	Hash Function	SLH-DSA SHAKE profiles rely on SHAKE-based hash assumptions and large signatures.
halo2 proving system stack	Uses	Recursive Proofs	halo2 is commonly associated with recursive proof-system engineering.
gnark	Uses	Arithmetization	gnark circuit definitions must compile to constraints that exactly capture the intended statement.
arkworks	Uses	SNARKs, STARKs, and Bulletproofs	arkworks provides implementation components for multiple SNARK constructions.
Circom	Uses	Arithmetization	Circom circuits need explicit constraint review to avoid underconstrained statements.
Microsoft SEAL	Uses	Homomorphic Encryption	Microsoft SEAL exposes concrete FHE schemes whose parameter choices remain expert-sensitive.
OpenFHE	Uses	Homomorphic Encryption	OpenFHE supports several FHE families; guarantees depend on the selected scheme and parameters.
MP-SPDZ framework	Uses	Multi-Party Computation	MP-SPDZ exposes many MPC protocols with different adversary models.
EMP toolkit	Uses	Oblivious Transfer	EMP packages often depend on OT and garbled-circuit assumptions.
FRESCO	Uses	Multi-Party Computation	FRESCO is a framework layer; protocol selection determines the security model.
Hyperledger AnonCreds v1	Uses	Anonymous Credential	AnonCreds uses credential-specific ZK proofs and revocation registries.
Shutter encrypted mempool	Uses	Threshold Cryptography	Shutter-style encrypted mempools use threshold encryption and key-release committees.

Break Conditions and Inherited Risks

`breaks-if`, `breaks-if-missing`, `breaks-if-misused`, `weakens-if`, `inherits-risk-from`, and `unsafe-instance-of` edges show conditions that can defeat or materially weaken a guarantee.

Source	Relation	Target	Notes
Digital Signature	Inherits risk from	Discrete Logarithm	ECDSA, EdDSA, and Schnorr-style signatures inherit discrete-logarithm quantum vulnerability.

Source	Relation	Target	Notes
Public-Key Encryption	Inherits risk from	Factoring and RSA	RSA-based encryption inherits factoring and padding assumptions.
Homomorphic Encryption	Inherits risk from	Lattices	Many modern FHE families rely on lattice assumptions and parameter selection.
SNARKs, STARKs, and Bulletproofs	Inherits risk from	Pairings	Pairing-based SNARKs inherit pairing and elliptic-curve assumptions.
SNARKs, STARKs, and Bulletproofs	Inherits risk from	Trusted Setup	Some proof systems require setup ceremonies or structured reference strings.
Zero-Knowledge Proof	Inherits risk from	Random Oracle Model	Fiat-Shamir-style non-interactive proofs often use random-oracle-model reasoning.
Confidentiality	Breaks if missing	Authenticated Encryption	Confidentiality claims often fail operationally when encryption is used without authentication or plaintext is logged.
Anonymity	Breaks if	Metadata Leakage	Timing, network, amount, and rare-attribute metadata can defeat anonymity even when proofs verify.
Unlinkability	Breaks if misused	Nullifier	Nullifiers deliberately create linkability inside a context and must not be reused across contexts.
Verifiability	Breaks if missing	Transcript Binding	Verification can be meaningless if public inputs, verifier keys, or context are omitted.
Encrypted Mempools	Breaks if	Metadata Leakage	Size, timing, sender path, and censorship behavior can reveal transaction strategy even when payloads are encrypted.
RSAES-PKCS1-v1_5 encryption	Unsafe instance of	Public-Key Encryption	Legacy RSA encryption can fail through padding-oracle behavior and should be isolated in migration inventories.
SHA-1	Unsafe instance of	Hash Function	SHA-1 collision resistance is not adequate for new cryptographic uses.
MD5	Unsafe instance of	Hash Function	MD5 is retained only for legacy recognition and non-adversarial checksum context.
Auditability	Breaks if	Metadata Leakage	Over-collected audit logs can defeat privacy goals even when evidence integrity is strong.
Deniability	Breaks if misused	Digital Signature	Publicly verifiable signatures on message content can create durable third-party evidence.
Availability	Weakens if	Trusted Setup	Setup or recovery ceremonies can become availability bottlenecks if no replacement path exists.
Censorship Resistance	Breaks if	Metadata Leakage	Sender, fee, size, and timing metadata can leave enough information to censor targeted users.

Composition and Usage

`commonly-composed-with`, `composes-with`, `implements-pattern`, and `used-in` edges show common composition paths and placement relationships.

Source	Relation	Target	Notes
Commitments	Composes with	Zero-Knowledge Proof	ZK proofs often prove statements about committed values without opening them.
Nullifier	Implements pattern	Anti-Double-Use Nullifiers	Nullifiers are the public duplicate-detection tag used by the pattern.
Anonymous Airdrop	Composes with	Nullifier	Airdrops use nullifiers to reject duplicate claims.
Private DAO Voting	Composes with	Anonymous Membership	Voters may prove eligibility without revealing which member they are.
Private DAO Voting	Composes with	Private Aggregation	The tally should reveal an aggregate rather than individual ballots.
Homomorphic Encryption	Used in	Private Aggregation	Additive homomorphic encryption is a common private tallying mechanism.
Secure Aggregation	Implements pattern	Private Aggregation	Secure aggregation is a protocol family for aggregating many client inputs.
Mixnets	Used in	Electronic Voting	Mixnets can break the link between submitted encrypted ballots and decrypted ballots.
Authenticated Encryption	Composes with	Symmetric Encryption	AEAD schemes combine encryption with integrity and associated-data authentication.
Authenticated Encryption	Composes with	Message Authentication Codes	AEAD schemes often use MAC-like authentication internally or replace ad hoc encryption-plus-MAC composition.
Oblivious Pseudorandom Functions	Used in	Private Set Intersection	OPRF-based PSI is a common construction family.
Private Set Intersection	Composes with	Multi-Party Computation	PSI can be built from general MPC techniques or specialized protocols depending on the deployment model.
Secure Channels	Composes with	Transcript Binding	Secure channels bind identities, negotiation, and key exchange into the derived traffic keys.
Secure Channels	Composes with	Authenticated Encryption	Secure channels usually protect application records with AEAD once handshake keys are derived.
Password-Authenticated Key Exchange	Composes with	Secure Channels	PAKEs can establish or authenticate a session key that is then used by a secure channel.
Oblivious Transfer	Used in	Multi-Party Computation	OT is a core building block for many MPC and garbled-circuit protocols.
Polynomial Commitments	Used in	ZK Rollups	Many rollup proof systems use polynomial commitments to bind witness or trace polynomials.
Vector Commitments	Used in	Privacy-Preserving Revocation	Revocation systems can use vector commitments or authenticated status lists for compact status proofs.
Private Payments	Composes with	Nullifier	Private payments often use nullifiers or serial numbers to prevent duplicate spending.
ZK Rollups	Composes with	Transcript Binding	Rollup proofs must bind public inputs, state roots, verifier keys, and batch data to the proof.

Source	Relation	Target	Notes
Confidentiality	Commonly composed with	Integrity	Most deployed confidentiality mechanisms also need tamper detection.
Integrity	Commonly composed with	Authenticity	Systems usually need both tamper detection and origin binding.
Anonymity	Commonly composed with	Unlinkability	Anonymous systems often need unlinkability across repeated actions, but the goals are distinct.
Unlinkability	Commonly composed with	Domain Separation	Context-bound tags prevent linkability across unrelated applications.
Forward Secrecy	Commonly composed with	Secure Channels	Forward secrecy is usually realized at the protocol layer.
Verifiability	Commonly composed with	Integrity	A verifier needs tamper-evident inputs and authentic verification parameters.
Gennaro-Jarecki-Krawczyk-Rabin DKG	Used in	Threshold Cryptography	DKG is a setup protocol for dealerless threshold keys.
Poseidon	Used in	Arithmetization	ZK-friendly hashes are chosen to reduce circuit or constraint cost.
MiMC	Used in	Arithmetization	MiMC is an example of a low-multiplicative-complexity hash used in some proof circuits.
Rescue-Prime	Used in	Arithmetization	Rescue-Prime is a proof-system-oriented hash family with scheme-specific parameters.
CKKS homomorphic encryption	Used in	Private Machine Learning Analytics	Approximate homomorphic arithmetic is common in privacy-preserving analytics and inference discussions.
BFV homomorphic encryption	Used in	Private Machine Learning Analytics	Exact homomorphic arithmetic can support bounded private analytics workloads.
SPDZ protocol family	Used in	Multi-Party Computation	SPDZ is a concrete actively secure arithmetic MPC family.
Auditability	Commonly composed with	Verifiability	Audit trails are stronger when reviewers can independently verify records, proofs, or logs.
Accountability	Commonly composed with	Auditability	Accountability usually needs durable evidence and a review process.
Deniability	Commonly composed with	Forward Secrecy	Key evolution and erasure reduce the value of later transcript compromise.
Non-Repudiation	Commonly composed with	Accountability	Signed evidence must feed a dispute or accountability process to have practical effect.
Availability	Commonly composed with	Threshold Cryptography	Threshold designs can remove single points of failure if quorum independence assumptions hold.
Censorship Resistance	Commonly composed with	Encrypted Mempools	Encrypted mempools can reduce content-based censorship before reveal, but only under committee and inclusion assumptions.
Jubjub	Used in	Zcash Sapling shielded protocol	Jubjub appears in Zcash Sapling-related circuit and note components.

Source	Relation	Target	Notes
ML-KEM-512	Used in	Key Encapsulation and Exchange	ML-KEM parameter profiles should be selected according to policy and protocol constraints.
ML-KEM-768	Used in	Key Encapsulation and Exchange	ML-KEM-768 is a common general-purpose migration profile.
ML-KEM-1024	Used in	Key Encapsulation and Exchange	Larger KEM parameters increase integration pressure and should be inventory-driven.
ML-DSA-44	Used in	Digital Signature	ML-DSA parameter profiles affect certificate, protocol, and hardware constraints.
ML-DSA-65	Used in	Digital Signature	ML-DSA-65 is a common general-purpose post-quantum signature migration profile.
ML-DSA-87	Used in	Digital Signature	Higher signature parameters require compatibility review before deployment.
HQC-128	Used in	Code-Based Assumptions	HQC profiles are migration-planning entries until the backup KEM standard is final.
HQC-192	Used in	Code-Based Assumptions	Higher HQC profiles provide code-based algorithm diversity with larger integration costs.
HQC-256	Used in	Code-Based Assumptions	HQC-256 is retained as a high-category planning profile pending final standardization.
Concrete ML	Commonly composed with	Private Machine Learning Analytics	FHE-based ML tooling still needs output, model, and repeated-query leakage review.
OpenID for Verifiable Credential Issuance	Used in	Identity Wallets	OID4VCI describes credential issuance flows, not presentation privacy by itself.
OpenID for Verifiable Presentations	Used in	Identity Wallets	OID4VP describes wallet-to-verifier presentation flows across credential formats.
EUDI Wallet Architecture and Reference Framework	Commonly composed with	Identity Wallets	EUDI ARF profiles wallet roles, trust lists, and secure-device assumptions.
ISO mdoc / mobile driving licence	Used in	Identity Wallets	ISO mdoc and mobile driving licence profiles are deployed credential-wallet formats.
DIDComm Messaging v2	Commonly composed with	Identity Wallets	DIDComm is a wallet-agent messaging layer and should not be mistaken for credential privacy.
Shutter encrypted mempool	Commonly composed with	Censorship Resistance	Threshold encryption can reduce pre-reveal content-based censorship but not all exclusion paths.
Open Quantum Safe tooling	Used in	Key Rotation and Migration	OQS tooling is useful for experimentation and interoperability, not a deployment certificate.

Adjacency Index

Outgoing edges grouped by source. This table is useful when reviewing what a concept, concrete scheme, or page depends on or composes with.

Source	Outgoing relationships
Accountability	Commonly composed with -> Auditability Requires -> Authenticity
Hyperledger AnonCreds v1	Uses -> Anonymous Credential
Anonymity	Breaks if -> Metadata Leakage Commonly composed with -> Unlinkability
Anonymous Airdrop	Composes with -> Nullifier
arkworks	Uses -> SNARKs, STARKs, and Bulletproofs
Auditability	Breaks if -> Metadata Leakage Commonly composed with -> Verifiability
Authenticated Encryption	Composes with -> Message Authentication Codes Composes with -> Symmetric Encryption
Authenticity	Requires -> Transcript Binding
Availability	Commonly composed with -> Threshold Cryptography Weakens if -> Trusted Setup
BBS Data Integrity credentials	Uses -> Anonymous Credential
BFV homomorphic encryption	Used in -> Private Machine Learning Analytics
BLS12-377	Requires -> Pairings
BW6-761	Requires -> Pairings
Censorship Resistance	Breaks if -> Metadata Leakage Commonly composed with -> Encrypted Mempools Requires -> Availability
Circom	Uses -> Arithmetization
CKKS homomorphic encryption	Used in -> Private Machine Learning Analytics
Commitments	Composes with -> Zero-Knowledge Proof
Concrete ML	Commonly composed with -> Private Machine Learning Analytics
Confidentiality	Breaks if missing -> Authenticated Encryption Commonly composed with -> Integrity
Deniability	Breaks if misused -> Digital Signature Commonly composed with -> Forward Secrecy Contrasts with -> Non-Repudiation
DIDComm Messaging v2	Commonly composed with -> Identity Wallets
Digital Signature	Inherits risk from -> Discrete Logarithm
Domain Separation	Strengthens -> Transcript Binding
EMP toolkit	Uses -> Oblivious Transfer
Encrypted Mempools	Breaks if -> Metadata Leakage
EUDI Wallet Architecture and Reference Framework	Commonly composed with -> Identity Wallets
Ferveo threshold-decrypted mempool	Uses -> Threshold Cryptography

Source	Outgoing relationships
Forward Secrecy	Commonly composed with -> Secure Channels Requires -> Key Encapsulation and Exchange
FRESCO	Uses -> Multi-Party Computation
Gennaro-Jarecki-Krawczyk-Rabin DKG	Used in -> Threshold Cryptography
gnark	Uses -> Arithmetization
halo2 proving system stack	Uses -> Recursive Proofs
Homomorphic Encryption	Inherits risk from -> Lattices Used in -> Private Aggregation
HQC-128	Used in -> Code-Based Assumptions
HQC-192	Used in -> Code-Based Assumptions
HQC-256	Used in -> Code-Based Assumptions
Integrity	Commonly composed with -> Authenticity
ISO mdoc / mobile driving licence	Used in -> Identity Wallets
Jubjub	Used in -> Zcash Sapling shielded protocol
LUCID encrypted mempool proposal	Uses -> Delayed Reveal
Make Receipts Useless	Strengthens -> Electronic Voting
MASCOT	Uses -> Oblivious Transfer
MD5	Unsafe instance of -> Hash Function
Microsoft SEAL	Uses -> Homomorphic Encryption
MiMC	Used in -> Arithmetization
Mixnets	Used in -> Electronic Voting
ML-DSA-44	Used in -> Digital Signature
ML-DSA-65	Used in -> Digital Signature
ML-DSA-87	Used in -> Digital Signature
ML-KEM-1024	Used in -> Key Encapsulation and Exchange
ML-KEM-512	Used in -> Key Encapsulation and Exchange
ML-KEM-768	Used in -> Key Encapsulation and Exchange
MP-SPDZ framework	Uses -> Multi-Party Computation
Non-Repudiation	Commonly composed with -> Accountability Requires -> Digital Signature
Nullifier	Implements pattern -> Anti-Double-Use Nullifiers
Oblivious Pseudorandom Functions	Used in -> Private Set Intersection
Oblivious Transfer	Used in -> Multi-Party Computation
OpenID for Verifiable Credential Issuance	Used in -> Identity Wallets
OpenID for Verifiable Presentations	Used in -> Identity Wallets

Source	Outgoing relationships
Open Quantum Safe tooling	Used in -> Key Rotation and Migration
OpenFHE	Uses -> Homomorphic Encryption
Password-Authenticated Key Exchange	Composes with -> Secure Channels
Pedersen commitment	Requires -> Discrete Logarithm
Polynomial Commitments	Used in -> ZK Rollups
Poseidon	Used in -> Arithmetization
Private DAO Voting	Composes with -> Anonymous Membership Composes with -> Private Aggregation
Private Information Retrieval	Contrasts with -> Private Set Intersection
Private Payments	Composes with -> Nullifier
Private Set Intersection	Composes with -> Multi-Party Computation
Public-Key Encryption	Inherits risk from -> Factoring and RSA
Range Proofs	Requires -> Commitments
Rescue-Prime	Used in -> Arithmetization
RSAES-PKCS1-v1_5 encryption	Unsafe instance of -> Public-Key Encryption
Selective Disclosure JWT	Contrasts with -> Anonymous Credential
Secure Aggregation	Implements pattern -> Private Aggregation
Secure Channels	Composes with -> Authenticated Encryption Composes with -> Transcript Binding
SHA-1	Unsafe instance of -> Hash Function
Shutter encrypted mempool	Commonly composed with -> Censorship Resistance Uses -> Threshold Cryptography
SLH-DSA SHA2 parameter sets	Uses -> Hash Function
SLH-DSA SHAKE parameter sets	Uses -> Hash Function
SNARKs, STARKs, and Bulletproofs	Inherits risk from -> Pairings Inherits risk from -> Trusted Setup
SPDZ protocol family	Used in -> Multi-Party Computation
Threshold BLS signatures	Requires -> Pairings
Unlinkability	Breaks if misused -> Nullifier Commonly composed with -> Domain Separation
W3C Data Integrity ECDSA credentials	Uses -> Authenticity
W3C Data Integrity EdDSA credentials	Uses -> Authenticity
Vector Commitments	Used in -> Privacy-Preserving Revocation
Verifiability	Breaks if missing -> Transcript Binding Commonly composed with -> Integrity
Zcash Orchard shielded protocol	Uses -> Halo-style recursion

Source	Outgoing relationships
Zcash Sapling shielded protocol	Uses -> Private Payments
Zero-Knowledge Proof	Inherits risk from -> Random Oracle Model
ZK Rollups	Composes with -> Transcript Binding

Concrete Algorithms and Schemes

This appendix explains how concrete algorithms, schemes, parameter families, and named protocol suites fit into the taxonomy. The machine-readable source of truth is `book/data/instances.yml`, where each entry declares `instance_of` relationships to parent concept-card IDs.

Where algorithms sit in the taxonomy

Concrete algorithms inherit the taxonomy level of the concept they instantiate. They are not a separate top-level level; they sit in a cross-cutting instance layer attached to a parent concept.

Use this mental format:

```
Concrete instance of: [parent concept]
```

Examples: Ed25519 is a concrete instance of digital signatures, Groth16 is a concrete instance of SNARK-style proof systems, and TLS 1.3 is a concrete instance of secure-channel protocols. The registry also tracks concrete entries such as P-256, Curve25519, BLS12-381, BN254, BLS12-377, BW6-761, Jubjub, SHA-1, MD5, Classic McEliece, HQC profiles, ML-KEM and ML-DSA parameter profiles, RSA blind signatures, ECVRF, FROST, threshold BLS, MLS, Privacy Pass, PLONK-style proof systems, Nova-style folding, Halo-style recursion, ZK proving stacks, ZK-friendly hashes, FHE schemes and libraries, MPC protocol and framework families, W3C/OpenID/AnonCreds/EUDI credential profiles, Zcash-style payment profiles, encrypted-mempool profiles, and W3C Bitstring Status Lists.

Example	Taxonomy placement	Why
SHA-256	Basic primitive: hash functions	It is a concrete hash algorithm.
AES-GCM	Basic primitive: symmetric encryption / authenticated encryption	It is a concrete encryption mode and authentication composition.
HMAC-SHA-256	Basic primitive: message authentication codes	It is a concrete MAC construction.
Ed25519	Basic primitive: digital signatures	It is a concrete signature scheme.
X25519	Basic primitive: key exchange	It is a concrete Diffie-Hellman function over Curve25519.
ML-KEM	Basic primitive: key encapsulation	It is a concrete post-quantum key-encapsulation mechanism.
Groth16	Proof system	It is a concrete succinct proof system.
Pedersen commitment	Basic primitive: commitments	It is a concrete commitment scheme.
KZG commitment	Structured primitive: polynomial/vector commitment	It is a concrete commitment scheme with pairing and setup assumptions.
TLS 1.3	Protocol: secure channels	It is a concrete protocol suite combining key exchange, authentication, transcript binding, and traffic protection.
OPAQUE	Protocol: password-authenticated key exchange	It is a concrete PAKE suite with OPRF, envelope, and authenticated key-exchange components.

This distinction matters because a name like `SHA-256` does not explain its goal, assumptions, misuse cases, or composition role. The concept page should explain the primitive; the instance entry should explain the concrete trade-offs.

Why most algorithms are not first-class pages

The book is organized by concepts, not as an algorithm catalog. That keeps the taxonomy readable, but readers still encounter names such as `SHA-256`, `AES-GCM`, `Ed25519`, `X25519`, `BLS12-381`, `Groth16`, or `ML-KEM` before they understand where those names sit.

For that reason, concrete algorithms should usually appear first as comparison tables on their parent concept pages. A dedicated page is useful only when the named scheme has distinct assumptions, failure modes, deployment status, or composition risks that would overload the parent page.

Machine-readable instance registry

The instance registry lives in `book/data/instances.yml`. Each entry includes:

- `id` and `name`;
- `kind`, such as algorithm, scheme, mode, curve, standard, or protocol suite;
- `instance_of`, which points to one or more concept-card IDs;
- taxonomy level, maturity, post-quantum posture, setup posture, assumptions, uses, cautions, and references.

Validation checks that every instance ID is unique, every parent in `instance_of` exists as a concept card, and every reference ID exists in `book/data/references.yml`.

Use an `instance_of` relationship rather than a new `level` value. That preserves the main taxonomy while allowing filters such as "show concrete instances of digital signatures" or "show post-quantum instances of key encapsulation".

Concrete algorithms should be added when they do at least one of the following:

- appear frequently in real systems;
- change the trust assumptions or post-quantum posture;
- have important misuse hazards;
- are common in blockchain, zero-knowledge, privacy, or applied security systems;
- are legacy names readers still need to recognize.

Expansion candidates by category

The registry is intentionally selective. The tables below are expansion candidates; some entries are already present in `book/data/instances.yml` and others remain useful future additions.

Assumptions and substrates

These are mostly parameter families, groups, curves, or problem families rather than standalone algorithms.

Area	Include first	Include later or mention as caution
Finite-field discrete logarithm	FFDHE groups, safe-prime groups	Small or custom groups
Elliptic curves	P-256, P-384, P-521, Curve25519, Curve448, secp256k1, Jubjub	Legacy binary curves
Pairing-friendly curves	BLS12-381, BN254, BLS12-377, BW6-761	MNT curves and specialized cycle choices
RSA substrate	RSA modulus sizes, public exponent conventions	Multi-prime RSA, raw RSA
Lattice assumptions	Learning with errors (LWE), module-LWE, short integer solution (SIS), module-SIS, NTRU lattices	Ring-LWE variants, parameter-set caveats
Hash-to-curve	RFC 9380 suites	Ad hoc hash-to-curve mappings

Basic primitives

Primitive page	Include first	Include later or mention as caution
Hash functions	SHA-256, SHA-384, SHA-512, SHA3-256, SHAKE128, SHAKE256, BLAKE2, BLAKE3	SHA-1 and MD5 as broken or legacy context
ZK-friendly hashes	Poseidon, Rescue, MiMC, Griffin	Pedersen hash, domain-specific circuit costs
Symmetric encryption and AEAD	AES-GCM, ChaCha20-Poly1305, XChaCha20-Poly1305, AES-GCM-SIV, AES-SIV	AES-CBC as legacy; AES-ECB as an anti-pattern
Block ciphers	AES-128, AES-192, AES-256	DES and 3DES as legacy context
Message authentication codes	HMAC-SHA-256, HMAC-SHA-512, CMAC-AES, GMAC, Poly1305, KMAC	CBC-MAC misuse outside its narrow setting
Key derivation and password hashing	HKDF, PBKDF2, scrypt, Argon2id	Raw hashes as password storage anti-patterns
Randomness and nonces	HMAC_DRBG, Hash_DRBG, CTR_DRBG, ChaCha20-based CSPRNGs, operating-system CSPRNG interfaces	Reused nonces, userland entropy mixers
Commitments	Hash commitments, Pedersen commitments, Merkle commitments	Low-entropy committed values without salt or hiding
Digital signatures	Ed25519, Ed448, ECDSA P-256, ECDSA secp256k1, RSA-PSS, Schnorr/BIP-340, BLS signatures, ML-DSA-44/65/87, SLH-DSA SHA2/SHAKE profiles	RSA PKCS #1 v1.5 signatures, DSA, Falcon/FN-DSA as standardization and deployment profiles evolve
Public-key encryption and KEMs	RSA-OAEP, HPKE, X25519, X448, ML-KEM-512/768/1024	RSAES-PKCS1-v1_5, ECIES variants, HQC-128/192/256 while standardization is still in progress
Secret sharing	Shamir secret sharing, additive secret sharing, Feldman VSS, Pedersen VSS	Naive share splitting

Structured primitives

Primitive page	Include first	Include later or mention as caution
Homomorphic encryption	BFV, BGV, CKKS, TFHE/FHEW, Microsoft SEAL, OpenFHE, Concrete ML	Parameter-selection examples and bootstrapping costs
Homomorphic commitments	Pedersen vector commitments, KZG commitments, inner-product-argument commitments	Trusted-setup and pairing-specific caveats
Threshold cryptography	FROST, threshold BLS, threshold ECDSA families, distributed key generation (DKG)	Implementation-specific MPC signing protocols
Accumulators and authenticated data structures	Merkle trees, sparse Merkle trees, RSA accumulators, bilinear accumulators, Verkle/KZG vector commitments	Dynamic accumulator update costs
Timelocks and VDFs	Wesolowski VDF, Pietrzak VDF, timelock encryption from repeated squaring	Centralized delay services
Functional encryption	Identity-based encryption, attribute-based encryption, inner-product functional encryption	General functional encryption as mostly theoretical

Proof systems

Proof-system page	Include first	Include later or mention as caution
Interactive proof building blocks	Sigma protocols, Schnorr identification, Fiat-Shamir transform	Rewinding assumptions and transcript ambiguity
SNARK families	Groth16, PLONK, Marlin, Sonic-style universal setup systems, halo2, gnark, arkworks, Circom	Scheme-specific arithmetization details and audit posture
Transparent proof systems	STARKs, FRI, DEEP-FRI	Parameter and hash choices
Inner-product systems	Bulletproofs, Halo-style accumulation, Nova-style folding	Proof-size and verification trade-offs
Range proofs	Bulletproof range proofs, Pedersen commitment plus range proof patterns	Bit-decomposition pitfalls
Membership proofs	Merkle inclusion proofs, sparse Merkle proofs, RSA accumulator witnesses, KZG opening proofs	Non-membership proofs and update witnesses
Lookup and set arguments	Plookup-style lookups, permutation arguments, grand-product arguments	Circuit-specific soundness caveats

Protocols

Protocol page	Include first	Include later or mention as caution
Key establishment and secure channels	TLS 1.3, HPKE, Noise patterns, Signal X3DH and Double Ratchet	Legacy TLS and static key exchange
Multi-party computation	Yao garbled circuits, GMW, BGW, SPDZ, MASCOT, MP-SPDZ, EMP, FRESCO, oblivious transfer extension	Fairness and abort-model variants
Secure aggregation	Bonawitz-style secure aggregation, Prio/Prio+	Small-cohort leakage and dropout handling

Protocol page	Include first	Include later or mention as caution
Oblivious pseudorandom functions	RFC 9497 OPRF/VOPRF/POPRF suites	Prime-order-group suites are quantum-vulnerable; application context binding matters.
Private set intersection	Diffie-Hellman PSI, OPRF-based PSI, circuit PSI	Cardinality-only PSI and malicious-security upgrades
Anonymous credentials	CL signatures / Idemix, BBS+ signatures, selective-disclosure JWT/VC patterns, ZK credential systems, OID4VCI, OID4VP, AnonCreds	Revocation, rare-attribute leakage, and wallet/verifier metadata
Mixnets	Chaumian mixnets, Sphinx packet format, Loopix-style mixnets	Timing and active tagging attacks
Electronic voting	Helios-style encrypted tallying, mixnet tallying, homomorphic tallying, coercion-resistant protocols	Receipt-freeness claims without a coercion model

Design patterns and systems

Pattern or system page	Include first	Include later or mention as caution
Commit-reveal	Hash commit-reveal, Pedersen commit-reveal, VDF-assisted delayed reveal	Abort and last-mover advantage
Anonymous membership	Semaphore-style Merkle membership, accumulator-based membership, nullifier-based rate limiting	Small anonymity sets
Anti-double-use nullifiers	Hash nullifiers, serial-number e-cash patterns, context-bound nullifiers	Cross-context linkability
Private aggregation	Secret-shared aggregation, Prio-style validation, differential privacy composition	Differencing attacks
Private payments	Zerocoin, Zerocash, Zcash Sapling/Orchard-style note systems	Ledger metadata leakage
Identity wallets	W3C VC Data Integrity profiles, SD-JWT VC, OID4VCI, OID4VP, EUDI ARF, ISO mdoc/mDL, DIDComm	Verifier correlation, device attestation, and status-check leakage
Encrypted mempools	LUCID, Ferveo, Shutter-style encrypted mempools	Reveal timing, committee availability, censorship fallback, and deployment maturity
ZK rollups	Groth16-based rollups, PLONK-ish rollups, STARK-based rollups	Data availability and prover centralization

Editorial policy

Algorithm coverage should be practical rather than exhaustive. Prefer:

- one concept page for the primitive or protocol;
- a comparison table for common concrete schemes;
- a dedicated page only when the scheme has distinct assumptions, failure modes, deployment status, or composition risks.

Legacy or broken algorithms can be included when readers need to recognize them, but they should be labeled as legacy, deprecated, or unsafe for new systems.

Further reading

- NIST, FIPS 180-4: Secure Hash Standard.
- NIST, FIPS 202: SHA-3 Standard.
- NIST, FIPS 197: Advanced Encryption Standard.
- NIST, FIPS 186-5: Digital Signature Standard.
- NIST, Post-Quantum Cryptography standardization.
- RFC 5869, HMAC-based Extract-and-Expand Key Derivation Function.
- RFC 7748, Elliptic Curves for Security.
- RFC 8032, Edwards-Curve Digital Signature Algorithm.
- RFC 8439, ChaCha20 and Poly1305 for IETF Protocols.
- RFC 9180, Hybrid Public Key Encryption.
- RFC 9380, Hashing to Elliptic Curves.

Diagram Authoring

Diagrams in the atlas should be source-controlled as structured YAML and rendered into static assets.

Policy

- Edit diagram sources in `book/data/diagrams/*.yml`.
- Validate diagram YAML against `book/schemas/diagram.schema.json`.
- Generate SVG assets with `npm run generate:diagrams`.
- Use generated SVG files from `book/static/img/diagrams/` in pages.
- Keep generated Mermaid files in `book/static/diagrams/` when a simple flowchart view is useful for review.
- Do not hand-edit generated SVG or Mermaid outputs.

When to add a diagram

Prefer a diagram when the page explains:

- taxonomy or category boundaries;
- dependency chains between assumptions, primitives, proof systems, protocols, and systems;
- protocol flows with multiple participants or public artifacts;
- composition paths where a component contributes only one narrow guarantee;
- failure modes caused by metadata, setup, trust, or omitted checks.

Source format

Each YAML file defines:

- a stable `id`;
- a human-readable `title` and `description`;
- a grid layout;
- typed nodes with labels, details, and positions;
- directed edges;
- output paths for generated SVG and optional Mermaid.

Example:

```yaml
id: example-flow
title: "Example flow"
description: A short explanation for readers and screen readers.
layout:
  width: 960
  height: 480
  columns: 3
  rows: 2
  node_width: 244
  node_height: 72
outputs:
  svg: static/img/diagrams/example-flow.svg
  mermaid: static/diagrams/example-flow.mmd
nodes:
  - id: input
    label: Input
    detail: Public data
    kind: actor
    column: 1
    row: 1
  - id: proof
    label: Proof
    detail: Verifiable artifact
    kind: proof
    column: 2
    row: 1
edges:
  - from: input
    to: proof
```

Current diagram sources

- `book/data/diagrams/assumption-stack.yml`
- `book/data/diagrams/nullifier-flow.yml`
- `book/data/diagrams/private-voting-composition.yml`
- `book/data/diagrams/taxonomy-flow.yml`
- `book/data/diagrams/zero-knowledge-proof-flow.yml`

Post-Quantum Posture

Post-quantum posture describes how a primitive, protocol, or system is expected to behave against an adversary with a large cryptographically relevant quantum computer.

This label is not a deployment recommendation. It is an editorial signal that tells the reader which parts of a design are likely to need replacement or closer review during post-quantum migration.

Labels

Label	Meaning
vulnerable	The usual construction is broken or seriously weakened by known quantum algorithms.
plausible	The construction family is commonly treated as a post-quantum candidate, assuming appropriate parameters and implementation.
depends	The posture depends on the concrete instantiation, parameter set, or supporting primitives.
unknown	The posture is not established enough for this atlas to classify it.
not-applicable	The page describes a goal, pattern, or organizational model rather than a cryptographic construction.

Quick matrix

Concept family	Working posture	Notes
Hash functions	plausible	Depends on output length and security target.
Symmetric encryption and MACs	plausible	Depends on key sizes and concrete security margins.
RSA and finite-field discrete logarithm	vulnerable	Affects RSA encryption, RSA signatures, and related assumptions.
Elliptic-curve discrete logarithm	vulnerable	Affects ECDSA, EdDSA, Schnorr-style signatures, and many commitments.
Pedersen commitments	vulnerable	Usually discrete-logarithm based.
Pairing-based SNARKs	vulnerable	Pairings rely on elliptic-curve discrete-logarithm style assumptions.
STARK-style proof systems	plausible	Often hash-based and transparent, but the full system still depends on parameters and hash choices.
Bulletproof-style range proofs	vulnerable	Usually discrete-logarithm based.
Lattice-based encryption and FHE	plausible	Depends on scheme, parameters, and implementation.
Functional encryption	depends	The posture is scheme-specific and often research-stage.
Secret sharing core	plausible	Information-theoretic sharing can be post-quantum, while authentication and transport layers may not be.

Concept family	Working posture	Notes
Nullifiers	depends	Usually hash-based, but the surrounding credential or proof system may not be post-quantum.
VDFs and timelocks	depends	Many constructions rely on assumptions whose post-quantum posture is construction-specific.
Design patterns	not-applicable	The pattern inherits posture from the primitives and protocols used.

How to use this label

When a page says "depends," ask:

- Which concrete scheme is being used?
- Which mathematical assumption does it rely on?
- Is any setup, signature, encryption, or proof layer quantum-vulnerable?
- Are parameters sized for a post-quantum security target?
- Is the claim backed by a standard, peer-reviewed analysis, or only by early research?

Current standards context

As of June 4, 2026, NIST's principal post-quantum standards are FIPS 203 for ML-KEM, FIPS 204 for ML-DSA, and FIPS 205 for SLH-DSA. NIST also states that Falcon/FN-DSA and HQC were selected for ongoing standardization, but those are not the same as the three finalized FIPS standards.

This atlas should still classify broad concepts conservatively, because a concept such as "signature" or "encryption" can be instantiated with either quantum-vulnerable or post-quantum schemes.

Migration source-depth notes

Migration should be tracked as an operational program, not only as an algorithm replacement. Inventory classical RSA, finite-field, elliptic-curve, pairing, and hybrid dependencies; separate key establishment from authentication; and treat threshold, credential, proof-system, and hardware-validation profiles as independent migration workstreams.

Further reading

- NIST: Post-Quantum Cryptography FIPS Approved
- NIST CSRC: Post-Quantum Cryptography Project
- NIST, NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption
- NIST, Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process
- NIST, Status Report on the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process
- NIST NCCoE, Migration to Post-Quantum Cryptography
- NIST NCCoE: Frequently Asked Questions about Post-Quantum Cryptography

- CISA, NSA, and NIST, Quantum-Readiness: Migration to Post-Quantum Cryptography.
- Open Quantum Safe, project documentation.
- Open Quantum Safe, TLS integrations.
- Shor, Algorithms for Quantum Computation; Discrete Logarithms and Factoring.
- Grover, A Fast Quantum Mechanical Algorithm for Database Search.
- NIST FIPS 203, FIPS 204, and FIPS 205.

Confidence Models

A confidence model states where the reader's confidence is supposed to come from.

In cryptographic systems, confidence may come from a mathematical assumption, public verification, a quorum of independent parties, at least one honest participant, an honest majority, a trusted issuer, or operational controls. These are not interchangeable.

Common models

Model	Meaning	Typical failure
mathematical-assumption	Security rests mainly on a hardness assumption and implementation correctness.	The assumption is broken, parameters are weak, or implementation leaks secrets.
public-verifiability	Anyone can check the relevant proof, transcript, or commitment.	The public statement is incomplete or the verifier checks the wrong property.
trusted-setup	A setup phase must be generated honestly or securely destroyed.	Toxic waste, biased parameters, or unverifiable setup.
trusted-issuer	One issuer or authority controls eligibility, credentials, or identity binding.	The issuer misissues, logs, censors, or deanonymizes users.
one-honest-party	Privacy or correctness holds if at least one party in a set behaves honestly.	All parties collude, or the honest party is excluded.
t-of-n-threshold	A property fails only when at least `t` out of `n` parties collude or are unavailable.	Enough parties collude, lose keys, or are coerced.
honest-majority	Security requires more honest than corrupt participants.	The adversary controls the majority or network scheduling violates the model.
non-collusion	Multiple services are assumed not to share data or keys.	Services collude, merge, are subpoenaed together, or share infrastructure.
client-side-secret	A user secret or device key anchors the guarantee.	The secret is stolen, shared, backed up insecurely, or coerced from the user.
external-timing	Timing or delay assumptions are part of the guarantee.	Hardware advantage, network latency, or denial of service changes the effective timing.

Quick matrix

Concept or protocol	Typical confidence model	What to ask
Hash commitment	mathematical-assumption	Is the message high entropy or properly randomized?
Pedersen commitment	mathematical-assumption plus public parameters	Who generated the generators, and can anyone know their discrete-log relation?
Pairing-based SNARK	trusted-setup or universal setup, depending on construction	What happens if setup toxic waste exists?
STARK-style proof	public-verifiability and transparent setup	Are hash assumptions and public inputs appropriate?
Bulletproof-style range proof	mathematical-assumption and public-verifiability	Is the proof bound to the correct commitment and range?

Concept or protocol	Typical confidence model	What to ask
Threshold decryption	t-of-n-threshold	How many trustees can collude before privacy fails?
Mixnet	one-honest-party plus batching	Is at least one mix honest, and is the anonymity set large enough?
MPC	honest-majority, dishonest-majority, or threshold model	Which corruption model does the protocol actually support?
Secure aggregation	threshold, dropout, and collusion model	How many clients or helper servers may collude?
Anonymous credentials	trusted-issuer plus holder secret	Can the issuer or verifier link presentations?
Nullifiers	client-side-secret plus public registry	Does the nullifier prove eligibility, or only non-reuse?
E-voting	threshold trustees, public bulletin board, and audit process	Which authorities can collude, censor, or create receipts?

How to write this in pages

Prefer concrete statements:

> Privacy holds if fewer than `t` of `n` trustees collude, the proof system statement is correct, and metadata outside the tally is handled separately.

Avoid vague statements:

> The system is decentralized and therefore trustless.

Further reading

- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.
- Shamir, How to Share a Secret.
- Benaloh, Verifiable Secret-Ballot Elections.

Metadata Leakage

Metadata leakage is information revealed outside the protected cryptographic payload.

Common leak classes

Leak	Examples	Typical mitigation
Identity	public keys, accounts, issuer records	anonymous credentials, mixnets, fresh identifiers
Timing	submission time, reveal time, response delay	batching, delays, cover traffic
Size	ciphertext length, proof size, message count	padding, fixed-size messages
Participation	who joined, dropped out, or abstained	cohort thresholds, aggregation policy
Context	election id, app id, public inputs	domain separation, minimal public inputs
Output	final aggregate, function result, tally	cohort-size rules, differential privacy, query limits

How to use this appendix

For every primitive, protocol, or system page, separate the cryptographic statement from the metadata statement. A zero-knowledge proof may hide a witness under the proof-system definition (Goldwasser, Micali, and Rackoff), while public inputs, timing, routes, and communication patterns can still identify the user; traffic-analysis risks were already central in early anonymity-system work (Chaum 1981).

Further reading

- Chaum, Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms.
- Goldwasser, Micali, and Rackoff, The Knowledge Complexity of Interactive Proof Systems.
- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.

Threat-Model Checklist

Use this checklist before selecting primitives.

Questions

- What is the exact security goal: confidentiality, integrity, anonymity, unlinkability, verifiability, or coercion resistance?
- Which actors can be malicious, curious, offline, coerced, or compromised?
- Which parties must remain honest, non-colluding, available, or publicly verifiable?
- What metadata remains public even if cryptographic payloads are hidden?
- What setup, issuer, threshold, timing, or network assumptions are required?
- What happens on abort, dropout, lost keys, stale state, or disputed outputs?
- Which claims are mathematical, and which are operational or governance claims?
- Which components are quantum-vulnerable, and which merely inherit posture from another layer?

Output

A useful threat model should produce three lists: guarantees, non-goals, and leaks. If a claim cannot be placed in one of those lists, it is probably too vague to guide design.

Further reading

- Katz and Lindell, Introduction to Modern Cryptography.
- Canetti, Universally Composable Security; A New Paradigm for Cryptographic Protocols.